



© 2025 ANSYS, Inc. or its affiliated companies
Unauthorized use, distribution, or duplication is prohibited.

Ansys Dynamic Reporting User's Guide



ANSYS, Inc.
Southpointe
2600 Ansys Drive
Canonsburg, PA 15317
ansysinfo@ansys.com
<http://www.ansys.com>
(T) 724-746-3304
(F) 724-514-9494

Release 2025 R1
January 2025

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001:2015 companies.
--

Copyright and Trademark Information

© 2025 ANSYS, Inc. Unauthorized use, distribution or duplication is prohibited.

ANSYS, Ansys Workbench, AUTODYN, CFX, FLUENT and any and all ANSYS, Inc. brand, product, service and feature names, logos and slogans are registered trademarks or trademarks of ANSYS, Inc. or its subsidiaries located in the United States or other countries. ICEM CFD is a trademark used by ANSYS, Inc. under license. CFX is a trademark of Sony Corporation in Japan. All other brand, product, service and feature names or trademarks are the property of their respective owners. FLEXlm and FLEXnet are trademarks of Flexera Software LLC.

Disclaimer Notice

THIS ANSYS SOFTWARE PRODUCT AND PROGRAM DOCUMENTATION INCLUDE TRADE SECRETS AND ARE CONFIDENTIAL AND PROPRIETARY PRODUCTS OF ANSYS, INC., ITS SUBSIDIARIES, OR LICENSORS. The software products and documentation are furnished by ANSYS, Inc., its subsidiaries, or affiliates under a software license agreement that contains provisions concerning non-disclosure, copying, length and nature of use, compliance with exporting laws, warranties, disclaimers, limitations of liability, and remedies, and other provisions. The software products and documentation may be used, disclosed, transferred, or copied only in accordance with the terms and conditions of that software license agreement.

ANSYS, Inc. and ANSYS Europe, Ltd. are UL registered ISO 9001: 2015 companies.

U.S. Government Rights

For U.S. Government users, except as specifically granted by the ANSYS, Inc. software license agreement, the use, duplication, or disclosure by the United States Government is subject to restrictions stated in the ANSYS, Inc. software license agreement and FAR 12.212 (for non-DOD licenses).

Third-Party Software

See the [legal information](#) in the product help files for the complete Legal Notice for ANSYS proprietary software and third-party software. If you are unable to access the Legal Notice, contact ANSYS, Inc.

Published in the U.S.A.

Table of Contents

Overview of Ansys Dynamic Reporting	1
Ansys Dynamic Reporting Components	3
Database	4
Local and Network Server	4
Ansys Dynamic Reporting Reports	4
Data Analysis	4
Data Display	5
Ansys Dynamic Reporting Architecture	5
Ansys Dynamic Reporting Server Architecture	5
Data Operations	8
Embedded Remote Sessions	9
Ansys Dynamic Reporting Security	10
System Requirements	11
ADR Server	11
Web Browsers	12
Ansys Installation and Licensing	12
Launching Ansys Dynamic Reporting	13
Ansys Dynamic Reporting Server Launcher	13
User Interface Launcher	13
Command Line Launcher	18
Additional Launching Mechanisms	18
Launching Using Ansys Dynamic Reporting Template Editor	18
Launching Using EnSight	21
Launching From the Database	23
Accessing the Web Interface	23
Changing a Server/Database Password	24
Connecting to a Running Server	27
Shutting Down a Server	28
Web Interface Overview	29
Logging Into the Web Interface	30
Enabling Dark Mode	31
Web Interface Menus	37
Querying the Database	39
Querying for Report Items	39
Querying for Sessions	42
Querying for Datasets	45
Querying for Report Templates	48
Modifying Tags of Selected Report Items	51
Deleting Selected Report Items	54
Deleting Orphaned Sessions and Datasets	55
Uploading and Managing Files	57
Uploading Files into Data Items	57
Using the Media Browser	58
Listing Files	60
Managing Existing Files	60
Uploading Files	62
Managing Folders	64
Creating a New Folder	66
Adding Files to a Folder	68

Managing Existing Folders	68
Managing Folder Permissions	69
Generating Reports	73
Report Generation Overview	73
Creating and Editing Templates	74
Starting the Ansys Dynamic Reporting Template Editor	75
Creating a New Template	76
Editing Templates	79
Copying Data Items and Report Templates Between Databases	79
Exporting a Template to Python API	80
Report Components and Properties	82
Common Report Properties	82
Property Macro Substitution	83
Explicit Anchors	84
Macro Formatting Filters	84
Formatting Filter Chains	85
LaTeX Support	85
AsciiMath Support	86
Message of the Day	86
Indexing Options	88
Data Items	91
Animation	92
File	93
HTML	93
Image	94
None	94
Scene	95
String	100
Table	100
Table Display Options	100
Table Properties	106
Tree	120
Conditional Formatting	124
Formatting Types	124
Formatting String	125
Using the Conditional Formatting Editor	126
Setting Expressions	130
Layout Templates	133
Common Layout Properties	135
Using Custom Web Components to Override the Report Style	139
Columns	140
Panel	141
Table of Contents	142
Tabs	144
Boxes	145
Carousel	149
Slider	150
Image Controls	153
Slider Configuration Options	160
Slider Properties	160
Slider Events	165

Click and Drag Settings	167
Comparison Sliders	168
Image Comparison	168
Deep Image Comparison	173
Image Comparison Keyboard Shortcuts	178
Iterator	179
Linked Report	183
Page Header	185
Page Footer	185
Tag Properties	188
Data Filter	190
PowerPoint (PPTX)	190
User Defined Content Block	190
Generator Templates	192
Common Generator Properties	193
Table Merge	197
Table Reduction	203
Table Row/Column Filter	209
Table Value Filter	214
Table Row/Column Sort	218
SQL Query	222
Tree Merge	225
Items Comparison	230
Items Comparison Configuration Options	231
Items Comparison Examples	235
Statistical Analysis	242
Generating Reports Using the Web Interface	246
Configuring a Remote Session	249
Creating the Configuration File	250
Creating the Session Lifecycle Python Script	251
Access Control Lists	255
Understanding Access Control Lists	255
Enabling Access Control Lists	258
Creating Access Control Lists in Ansys Dynamic Reporting	258
Creating ACLs in the Admin Interface	258
Accessing the Admin Interface	259
Adding a Group	260
Changing Permissions in a Group	261
Deleting Groups	262
Adding a User	263
Changing a User	264
Deleting Users	266
Creating ACLs in the User Interface	267
Managing Item Categories	267
Assigning and Removing Categories for Multiple Items	273
Assigning Categories During Upload	280
Editing Categories in the Item Detail Page	283
Sharing an Item from the Item Detail Page	284
Creating Access Control Lists in EnSight	285
Creating Access Control Lists in Ansys Dynamic Reporting Template Editor	286
Creating Access Control Lists in Ansys Dynamic Reporting Data Import	290

Scripting Tools	291
Scripting APIs	291
EnSight Python Ansys Dynamic Reporting API	292
API Modules	292
API Objects	300
EnSight Command Language Ansys Dynamic Reporting API	304
Command Line Tools	304
Ansys Dynamic Reporting Launcher Command Line	304
Ansys Dynamic Reporting Command Line Utility	309
Copy (cp)	310
Import (import)	311
List (ls)	315
Delete (rm)	316
HTML Export (htmlexport)	317
PDF Export (pdfexport)	318
Tagging (tag)	319
Query Expressions	320
Troubleshooting	325
Multiple Reports in Parallel	325
Compatibility Between Versions	326
Known Issues and Limitations	327

Overview of Ansys Dynamic Reporting

Ansys Dynamic Reporting (ADR) is the report-generation, analytics, and data-storage framework integrated into multiple Ansys products. It is a web-oriented collection of tools that allows you to bring all of your data together in one place, organize it, and create compelling reports for yourself, your engineers, your colleagues, and your customers.

Organized

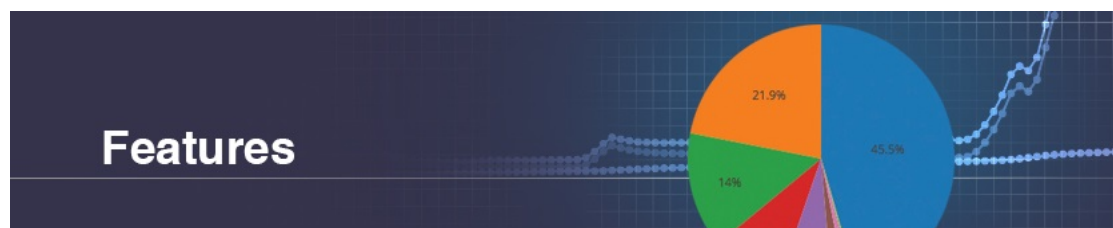
Data is stored in ways that make sense for your workflows, projects, users, and more. You no longer need to worry about where you saved a file: Ansys Dynamic Reporting has you covered. With a rich metadata infrastructure, Ansys Dynamic Reporting allows you to tag, search, group, collect, and present your data in new ways. Ansys Dynamic Reporting understands your data, from structured tables to full 3D models and can present them in intuitive ways. And your analysis data, whether images from EnSight or plots and queries from post-processing, are connected back to their original source datasets and analysis sessions, allowing you to quickly return to the data for further exploration.

Efficient

Your data can automatically flow into the Ansys Dynamic Reporting database or be manually uploaded, as needed, with a single click or command. Our REST APIs (including the PyAnsys project [PyDynamic-rReporting](#)) allow you to automate your workflows and smooth the process of centralizing your rich data. Save hundreds of wasted hours navigating around complex file structures and let our framework do the work for you.

Productive

Ansys Dynamic Reporting reports do not need to be reformatted to meet the needs of different audiences and can be customized to fit your objectives. Plus, with interactive charts, graphs, tables, and 3D models, your audience will be fully equipped to understand your data. Interactive on-screen organization layouts like tabs, sliders, carousels and automatic tables of contents, figures, and tables mean that your reports will be beautiful and informative.



Using the Ansys Dynamic Reporting system makes it possible to explore your data and create high-productivity tools:



Live Reports

Reports are saved data searches. When new material matches the report settings, it will automatically be included in the report.



Live 3D Models

Pan, zoom, rotate, step through time, and more. No plugin required. All major browsers supported.



Rich Plots

Present data as a table or a plot. Switch back and forth between them easily. Zoom in to the details or zoom out for the context. Interactively probe your plots to explore the details of the data.



Tables of Contents

Generate complex reports that automatically include a table of contents for easy navigation. Generate lists of tables and figures to quickly jump to important elements of your report.



Live Object Sliders

Sliders allow you to visualize a plethora of images, animations, or 3D scenes all in an organized, compact and interactive interface.



Application Integration

One click and EnSight and Fluent content is saved to your database, from saving images and plots to pushing out 3D models. Quickly set custom tags to precisely characterize your visualizations and analyses.

Related topics:[Ansys Dynamic Reporting Components](#)[Ansys Dynamic Reporting Architecture](#)[System Requirements](#)[Ansys Installation and Licensing](#)

Ansys Dynamic Reporting Components

Ansys Dynamic Reporting is a framework that allows engineers to store their data coming from multiple sources, keep it organized, and create reports that automatically update when the data changes. While Ansys Dynamic Reporting is designed to work in concert with EnSight, the data that is stored can come from potentially anywhere.

The Ansys Dynamic Reporting framework is based upon a database for data storage and a service that analyzes the data and generates reports.

The framework enables you to store the results of simulations or other post-processing assets into one or more Ansys Dynamic Reporting databases. These databases keep track of the provenance of the input data items, which allows the data items to be organized and easily searched. The system allows for the direct display of data items and for the generation of multi-item reports using report templates. These reports can generate composite displays from multiple items, including the incorporation of items from different simulations.

Ansys Dynamic Reporting databases have a rich data model, allowing you to store a wide range of data types: text, formatted HTML, tables of information, numbers, images, movies, 3D models, trees of information, and arbitrary files. All data items are linked with their sources, allowing you to process collections of data together and to dynamically construct new collections based on higher-level sets of rules. Powerful APIs allow you to implement custom workflows, tailored to the way you work.

Ansys Dynamic Reporting is web-centric, providing an intuitive, browser-based mechanism for interaction with data items and the generation and sharing of reports. Charts are interactive, tables can be sorted, and 3D models can be manipulated. Once a report template is designed, you can log into the Ansys Dynamic Reporting web front end and start generating reports, filtering the data down to ones that are the most important today.

Related topics:[Database](#)[Local and Network Server](#)[Ansys Dynamic Reporting Reports](#)[Data Analysis](#)[Data Display](#)

Database

A database is simply an organized collection of data. When you push data into Ansys Dynamic Reporting, it is stored in the database along with information about where the data comes from and how it relates to other data. This allows you to store the data there and access it in an efficient way through database queries. The database is like a black box for the user: you place the data there and get it out from there when you need it. The data is stored in an organized, efficient way, but the details and optimization are handled directly by Ansys Dynamic Reporting.

Local and Network Server

Once your data is stored in the database, you need a way to visualize it. To do so, you will need to start a server, a program that interfaces with the database. There are two types of servers in Ansys Dynamic Reporting: local and network server. A local server is one that is accessible only from the machine it runs on, generally your local desktop. A network server is one that can be accessed by any user on the same network as the machine it runs on. So, if you are working on your data on a single machine and don't need to share your reports, a local server is all you need. If you are working with other engineers, or want to be able to show your reports to other people on the same network, then you will want to start a network server.

Ansys Dynamic Reporting Reports

Ansys Dynamic Reporting reports are the result of sending a filtered set of data items through a report template. Data items are chosen with a database query, allowing flexibility in the selection of data to process. Report templates encode the visual appearance of the report, including options for processing the data items that are sent in. Templates can be restricted to a single set of known data or be designed to be executed repeatedly, handling a wide range of expected data. But a template can be changed at any time, allowing it to be augmented with new analyses and information as your needs evolve.

Ansys Dynamic Reporting templates are organized conceptually as trees. Sub-templates are strung together to handle different subsets of data items, often used to visually lay out data on a page. For example, a column layout template may be used to display data in a single column, while a tab layout template could be used to put different collections of data in separate tabs. Any template may have other sub-templates under it to process data. For example, a tab layout template could have other templates under each of the tabs to lay out the information that lands on each tab.

To provide this hierarchical template system of processing data, each template can restrict the set of data items that it processes. Thus, the top-level template is handed all the requested data items, and then each sub-template successively filters the data, ultimately displaying only the data pertinent to its part of the report. It is also possible for any template to perform an independent search of the database for specific data items.

Data Analysis

Ansys Dynamic Reporting provides *generators*, a special kind of report template that can change the data that is passed to it. In this way, complex data processing can be applied to data items. For example, table merges, collapses, and expansions can be done through generators. Generators play a central role in data aggregation and reduction. Powerful operations like "extract the ten best datasets from this collection" allow you to design reports that efficiently convey important information. Gen-

erators can reduce data into statistical descriptions (for example, counts, mean, min, max) and present their results as data items that can be further manipulated and displayed by additional generators and templates.

Data Display

Ansys Dynamic Reporting harnesses the power of HTML5, JavaScript, D3, and WebGL to graphically present your data, allowing you to quickly see relationships between data values and find clusters of information. All graphs and charts are interactive, providing the ability to pan, zoom, probe, and restrict your data. Modern display styles like image carousels and slider displays allow for interactive exploration of collections of parameters (for example, variables, time, user-defined). All of these are supported in the context of an indexing system capable of generating interactive tables of content, lists of figures, and lists of tables.

Ansys Dynamic Reporting Architecture

Conceptually, Ansys Dynamic Reporting consists of a core server engine that combines a modern SQL database with a bulk data storage mechanism. The database stores both metadata for the data items and the report templates. The Ansys Dynamic Reporting system is accessed via a REST API that has been wrapped in Python for convenience and simplicity. This API allows for items in the database to be added, removed, and edited. This interface has been integrated into EnSight, Fluent and other Ansys applications as an extension to simplify the capture of new data elements directly from within the application.

The Python API can be used directly from the interpreter embedded in the application or from a standalone Python instance. The Python interface provides access to Ansys Dynamic Reporting from custom, non-Ansys applications. A desktop user interface tool, the Ansys Dynamic Reporting [Template Editor](#) (p. 75), provides interactive editing of report templates and supports complex server operations, including generating new databases, local server control (start, stop, and so on.), and copying data items and templates between servers.

Related topics:

[Ansys Dynamic Reporting Server Architecture](#)

[Data Operations](#)

[Embedded Remote Sessions](#)

[Ansys Dynamic Reporting Security](#)

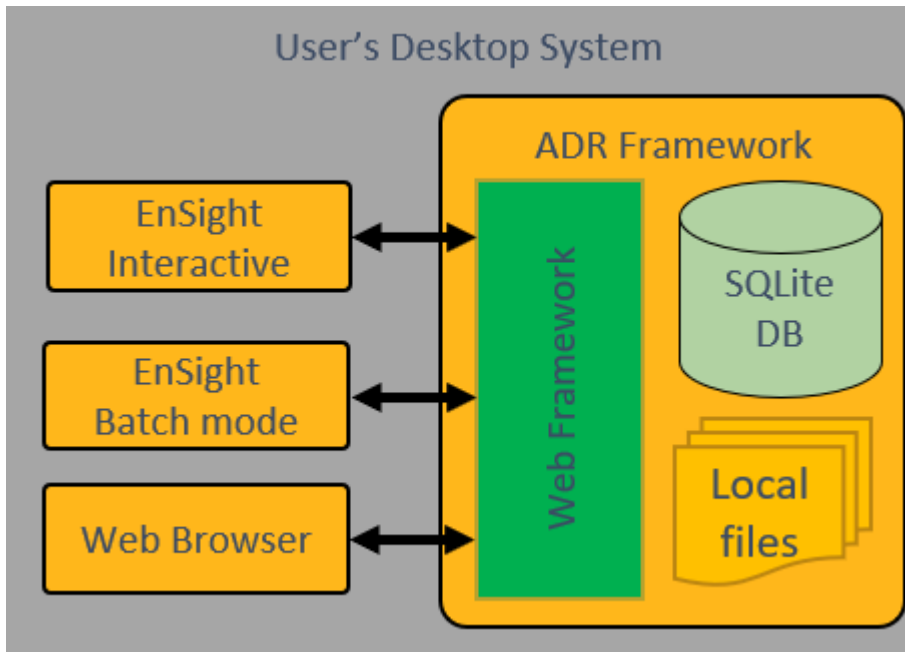
Ansys Dynamic Reporting Server Architecture

Key to the Ansys Dynamic Reporting system is the ADR server.

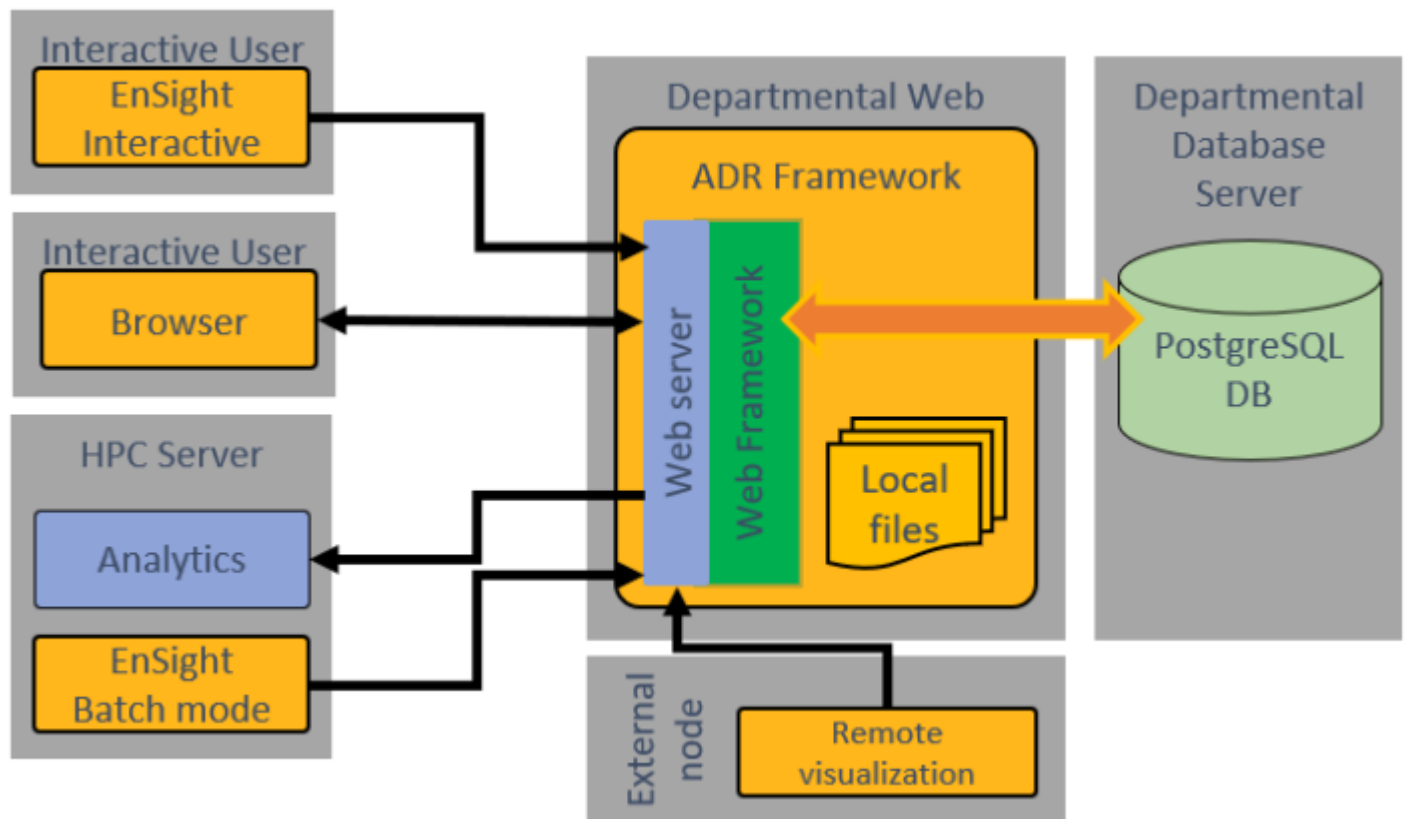
An ADR server is software that allows you to store, access, and analyze objects created in EnSight (or other applications) that help you collect, catalog, analyze data and easily create reports of cumulative information. These objects, called data items, can have a variety of types, from a (x,y) list of values to image files to 3D geometry. The data items are collected into databases, which allows for easy storage and post-collection analysis. An ADR server also provides a mechanism to select, process and display collections of items in customized report formats delivered via HTML.

The server provides the web interface to the database and report generation infrastructure. It processes the REST API for external applications, providing an nginx-based load-balanced service that can be integrated behind a departmental or externally facing server using simple HTTP proxy interfaces.

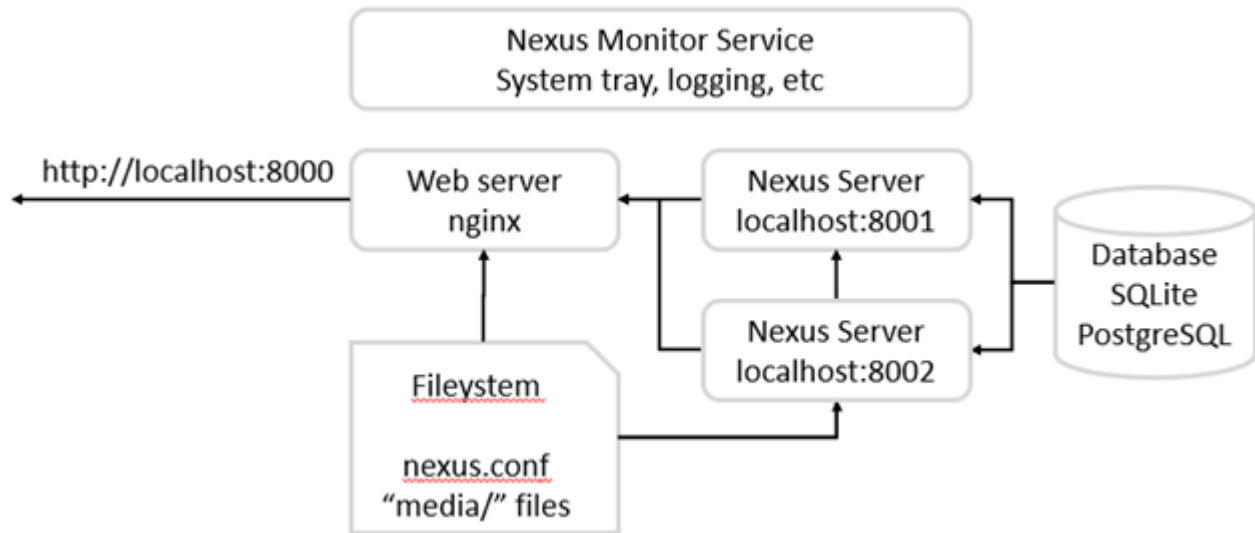
The server can be configured to run locally on a desktop in a configuration with the following structure:



It can also be configured to run on a remote server (such as a departmental resource) or deployed in a cloud setting:

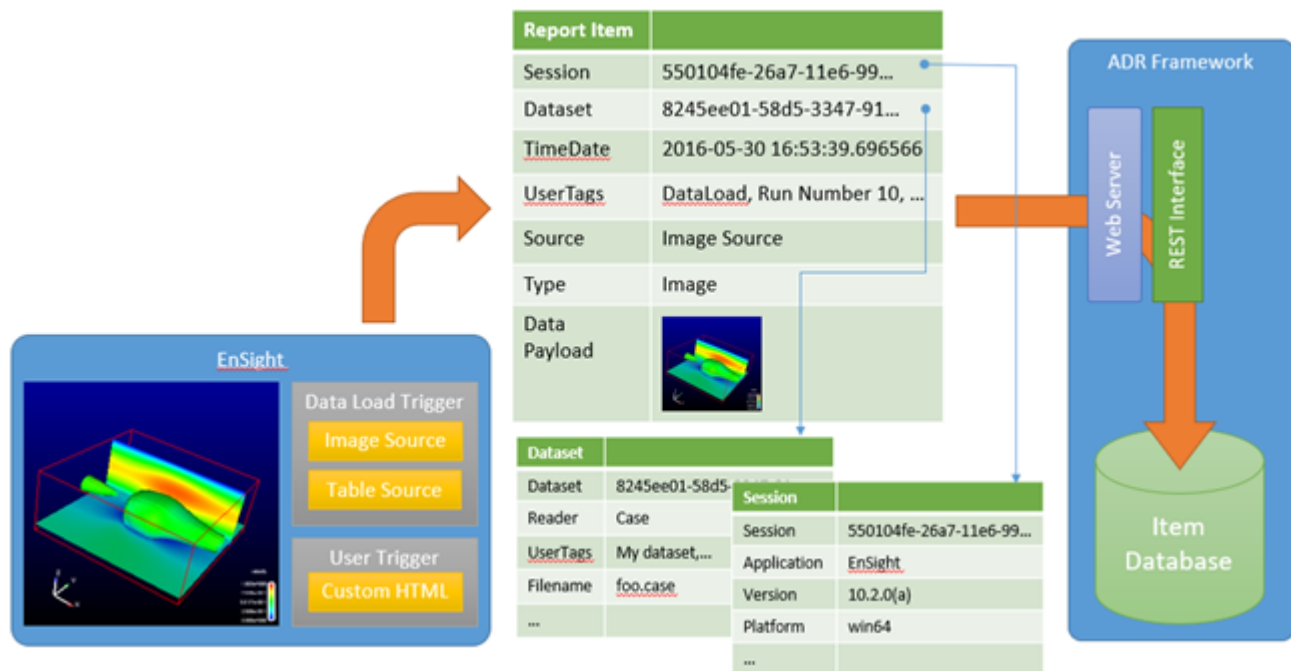


The external URL interface is supplied by an instance of the nginx web server (nginx is included in the Ansys Dynamic Reporting distribution). The nginx server is used to serve up media files for Ansys Dynamic Reporting and serves as a load balancing interface for the actual ADR server instances. There can be more ADR server instances processing the various Ansys Dynamic Reporting web requests. For example, with two instances, two reports may be generated at the same time. Each of these servers talk to the nginx server over their own ports. All Ansys Dynamic Reporting metadata is stored in an SQL database. Currently SQLite and PostgreSQL (12 and above) are supported and the ADR servers communicate with those databases. Ansys Dynamic Reporting stores bulk data (images, movies, and so on) in the local file system. The ADR servers and nginx both need to be able to access this file system (nginx needs only read-only access). Finally, there is a process that serves to monitor the health of the ADR server. This service provides the mechanisms needed to shut down the server, handle system tray operations, logging and other status functions.

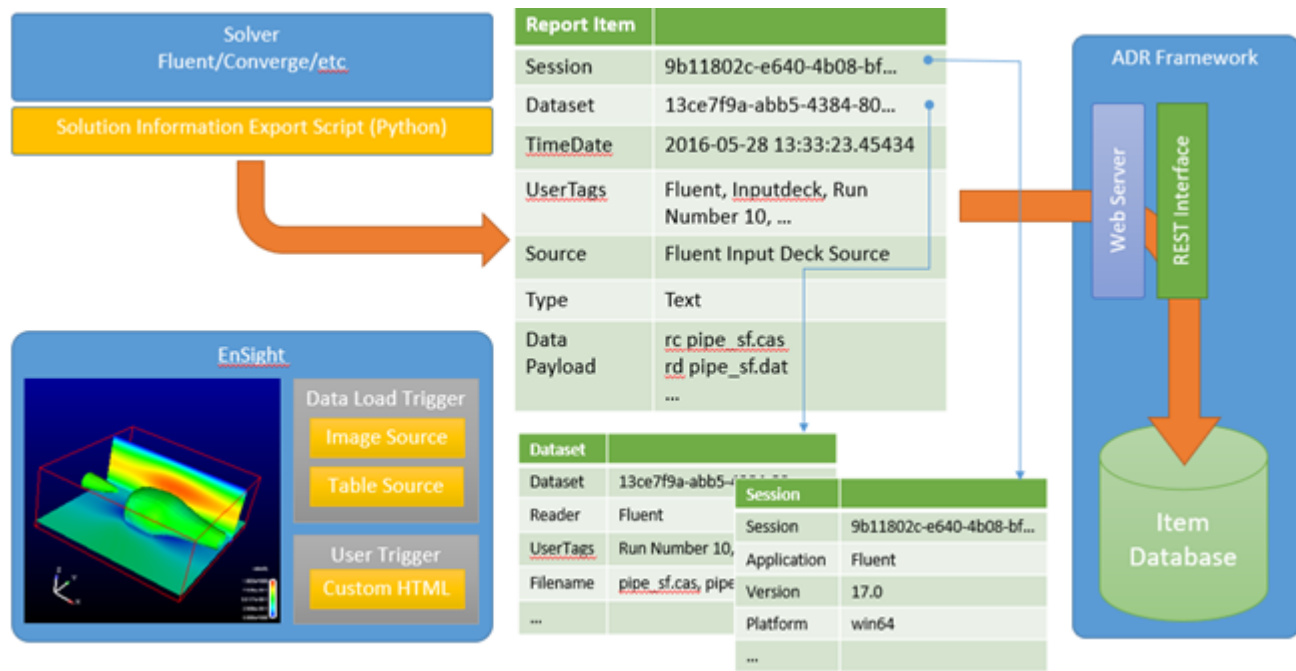


Data Operations

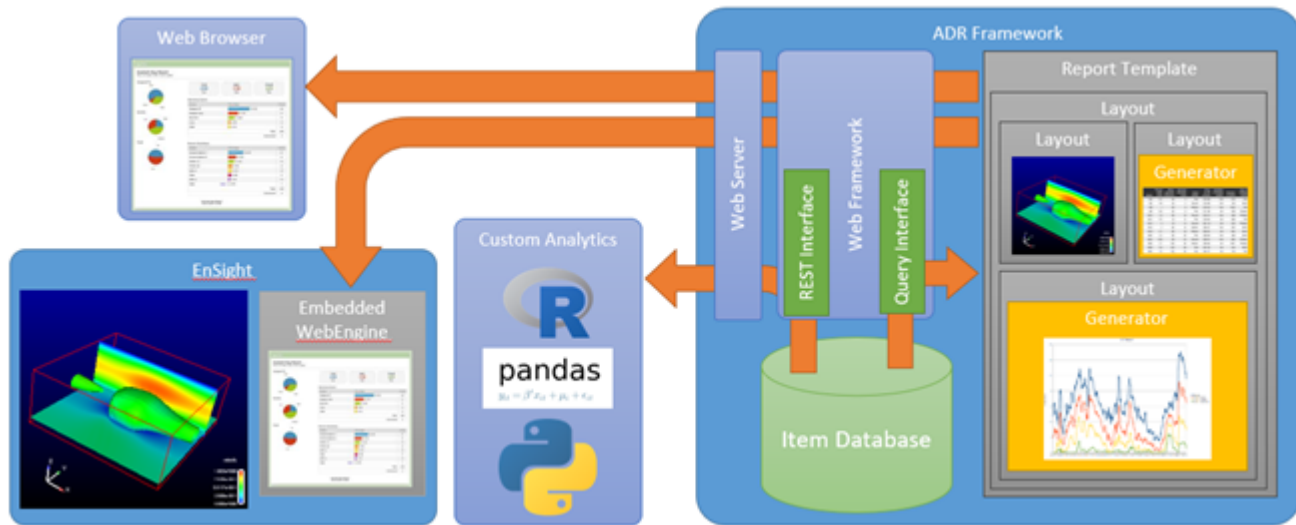
You can push data into the database from within EnSight:



Or from other applications:



The resulting database supports query operations and report generation:



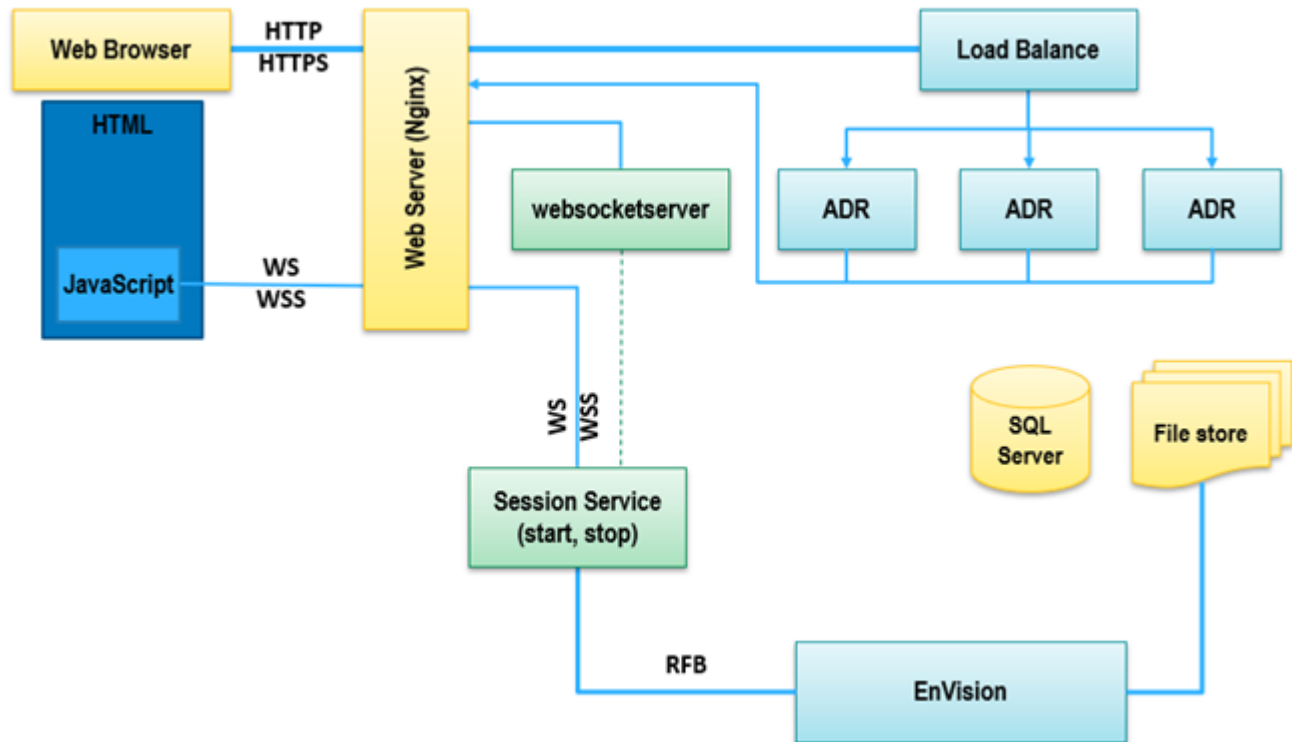
Embedded Remote Sessions

The Ansys Dynamic Reporting infrastructure provides a simple framework that enables remote sessions. More accurately, Ansys Dynamic Reporting provides a means to authenticate and authorize access to a pre-configured resource pool. Access is provided via a web-server based VNC client embedded in the ADR web framework.

Before the remote service is launched, the site administrator must design a collection of sessions and the mechanisms to start and stop a session. In the simplest form, this can be a pair of shell scripts that start and stop a VNC server desktop session based on parameters dynamically captured by ADR (for example, the username, session screen dimensions, and index into a collection of sessions). This is detailed in [Configuring a Remote Session](#) (p. 249).

The framework reads this configuration and presents the authenticated and authorized user a selection of available sessions they can use. If a session is selected from the list, a web page with an embedded VNC viewer which routes communication over a web-socket connection is opened and the session started. When the page is closed, the session is closed. Additionally, built-in sessions allow for local invocations of EnSight or EnVision to be displayed embedded in a report.

Architecturally, this service adds an additional component, [websocketserver](#) to Ansys Dynamic Reporting.



A WebSocket connection is established through NGINX and WebSocket server and a Session process launched to provide SSL encryption and WS protocol encapsulation of the VNC server stream produced by the EnVision instance. The HTML/JavaScript VNC client served to the web browser via NGINX is the consumer of the VNC stream tunneled via the WebSocket protocol through NGINX. That client then displays the remote desktop/applications in a frame in the web browser.

For the simple case of EnVision and EnSight viewing of *.evsn and *.ens file items, the WebSocket server has session support built-in. The launcher user interface or command line can be used to enable and specify the number of concurrent sessions of each built-in type that are allowed.

Ansys Dynamic Reporting Security

In general, the ADR system relies on common security solutions for remote web services.

Authentication

While ADR supports user logins within the ADR framework itself, the more common approach is to have the site configure the NGINX server to proxy the authentication. This allows the site to include its own authentication system in the web server without modification to the ADR system. If ADR sees

an authenticated user coming in from NGINX, it will honor that authentication, provided a username match can be made to a user in the ADR administration system.

Authorization

Authorization in ADR is controlled by the user administration system in ADR and is generally handled by the group system. The current implementation allows any authenticated user to be able to use both the GUI and REST APIs to manipulate the data in the database. The administration system can select what users are allowed to do (read/write and so on). Access to the Session system is controlled by the group permissions. A user must be a member of the *RemoteSession* group to be able access web pages and other resources related to the remote session system.

Encryption

Ansys Dynamic Reporting makes no explicit attempt to encrypt data itself. Thus, encryption is left to the web server and web socket systems in addition to any storage system encryption. It is strongly recommended that web access is limited to SSL connections and all outward facing aspects of ADR support this. On the backend, the media store and the database store are asked to provide their own solutions for stored data encryption.

System Requirements

In general, the Ansys Dynamic Reporting data import can be run on any system capable of running the EnSight client or simple Python applications. The key requirement is that the system have HTTP access to the system running the ADR server.

Related topics:

[ADR Server](#)

[Web Browsers](#)

ADR Server

An ADR server can be run in two different ways.

Local Server

A local ADR server runs on the same system as the EnSight client and is not accessible from other systems. It can require a significant amount of disk space and in rare situations RAM. Disk space usage scales with the number of images and geometry objects stored in the database. An example database holding approximately 1000 items, including 10 movies and 4 3D geometry objects may use ~300MB of disk space. When idle, the RAM use is around 50MB, but can increase greatly depending on the complexity of the report operations. 1GB of RAM is suggested for a large server.

Remote Server

A remote ADR server listens on the network to serve requests from other machines. Remote servers benefit most by running multiple server processes to load-balance many simultaneous requests. They frequently have significantly more RAM and disk space due to the need to hold and serve data from more than one user. The extra data load is often facilitated by employing a (remote or local) PostgreSQL database. Because of this, there are increased options for performance optimization.

A local server can be turned into a remote server by enabling network connectivity to the machine by an appropriate choice of hostname and port in the [Ansys Dynamic Reporting Server Launcher](#) (p. 13).

Web Browsers

The ADR server has been tested with the following browsers. The system uses advanced HTML5 elements, so the most recent versions of some of these browsers may be required:

- Apple Safari
- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Note:

Windows Internet Explorer is no longer supported, upgrade to Microsoft Edge.

Ansys Installation and Licensing

Ansys Installation

You can access Ansys Dynamic Reporting through the standard Ansys unified installer. By default, Ansys Dynamic Reporting is included in both the EnSight and Fluent installations, or it can be installed as a standalone package. The Ansys Dynamic Reporting tool launchers and scripts are located in the Ansys installation directory `v251\CEI\bin`.

Ansys Dynamic Reporting Licensing

From release 2020 R1 and later, using the Ansys Dynamic Reporting tools, including launching and maintaining ADR servers, requires no additional licensing from Ansys.

Example Ansys Dynamic Reporting Database

A database that contains all of the data and reports used when generating this documentation is included in the distribution. The database and associated media directory are included in `v251\CEI\nexus251\examples\documentation_database\documentation_examples.zip`. To use the database, extract the contents of the zip file into an empty directory and [launch a new ADR server](#) (p. 13) using the extracted `bd.sqlite3` file as the source.

Launching Ansys Dynamic Reporting

There are multiple ways to create and launch an Ansys Dynamic Reporting server and Ansys Dynamic Reporting web interface.

Any server launched locally will only be accessible on the machine it is launched on as the server will reject external access as a security measure. You will also need a username and password to be able to modify the database. No username or password are required to display items in the database directly or via reports.

Related topics:

[Ansys Dynamic Reporting Server Launcher](#)

[Additional Launching Mechanisms](#)

[Accessing the Web Interface](#)

[Changing a Server/Database Password](#)

[Connecting to a Running Server](#)

[Shutting Down a Server](#)

Ansys Dynamic Reporting Server Launcher

The Ansys Dynamic Reporting Server Launcher is the first place to start when working with standalone ADR servers. This tool is capable of creating and configuring a Ansys Dynamic Reporting server to support both local and remote access. It can configure the use of local SQLite or remote PostgreSQL databases, set up formal ACL control, run parallel servers, and so on.

There are two methods to use the Ansys Dynamic Reporting Server Launcher:

[User Interface Launcher](#)

[Command Line Launcher](#)

User Interface Launcher

The user interface launcher works by creating a server configuration file inside of the Ansys Dynamic Reporting database directory named `nexus.conf`. This tool allows you to interactively edit this file and then launch a server using the information stored in the configuration file.

Running the Launcher

To run the user interface launcher

1. Open a command prompt/shell, and navigate to `C:\Program Files\ANSYS Inc\v251\CEI\bin\` on Windows or `/ansys_inc/v251/CEI/bin/` on Linux.

2. Type **adr_launcher**.

The following dialog is displayed.

The screenshot shows the 'adr_launcher' dialog box with the following sections and fields:

- File Help Language** (Menu bar)
- Current status**
 - Server:
 - Database directory:
 - Database: sqlite:db.sqlite3
- Basic options**
 - Hostname: 127.0.0.1 (Note: 'localhost' & '127.0.0.1' may limit remote access.)
 - Server name: ADR Database
 - Port: 0
- Advanced options** (Expanded)
 - Number of parallel servers to launch: 1
 - Base port number for dynamic allocation: 8000 (Random base port checked)
 - Maximum item upload size (MB): 5000
 - ☒ Allow remote (non-localhost) access
 - ☐ Allow ADR webpage iframe embedding
 - ☐ Enable per-item ACL support
 - ☐ Verbose logging
- PostgreSQL server** (Selected tab)
 - ☐ Use PostgreSQL (Test connection button)
 - PostgreSQL server hostname: 127.0.0.1
 - PostgreSQL server port: 5432
 - PostgreSQL database name: nexus_database
 - Username: (empty field)
 - Password: (empty field)
- Embedded sessions** (Unselected tab)

Selecting a Database to Configure

After opening the launcher, select the database whose configuration you want to edit.

1. If you do not already have a database created, from the menu bar select **File** → **Create local database**.
2. To select the database configuration, from the menu bar select **File** → **Select local database**.

The dialog is updated to reflect the target database.

File Help Language

Current status

Server: STOPPED

Database directory:

Database: sqlite:db.sqlite3

Basic options

Hostname 127.0.0.1 'localhost' & '127.0.0.1' may limit remote access.

Server name ADR Database

Port 0

☐ **Advanced options**

Number of parallel servers to launch 1

Base port number for dynamic allocation 8000 ☒ Random base port

Maximum item upload size (MB) 5000

☒ Allow remote (non-localhost) access

☐ Allow ADR webpage iframe embedding

☐ Enable per-item ACL support

☐ Verbose logging

PostgreSQL server Embedded sessions

☐ Use PostgreSQL Test connection

PostgreSQL server hostname 127.0.0.1

PostgreSQL server port 5432

PostgreSQL database name nexus_database

Username

Password

The current status of the server (if any) is displayed at the top of the dialog. Once a database is selected, you can edit the various configuration options. Common options are always available. Select the **Advanced options** check box for more settings.

Tabs are provided for the PostgreSQL and [embedded remote session \(p. 9\)](#) options. To enable a remote session, on the **Embedded sessions** tab, select the **Enable external session support** check box.

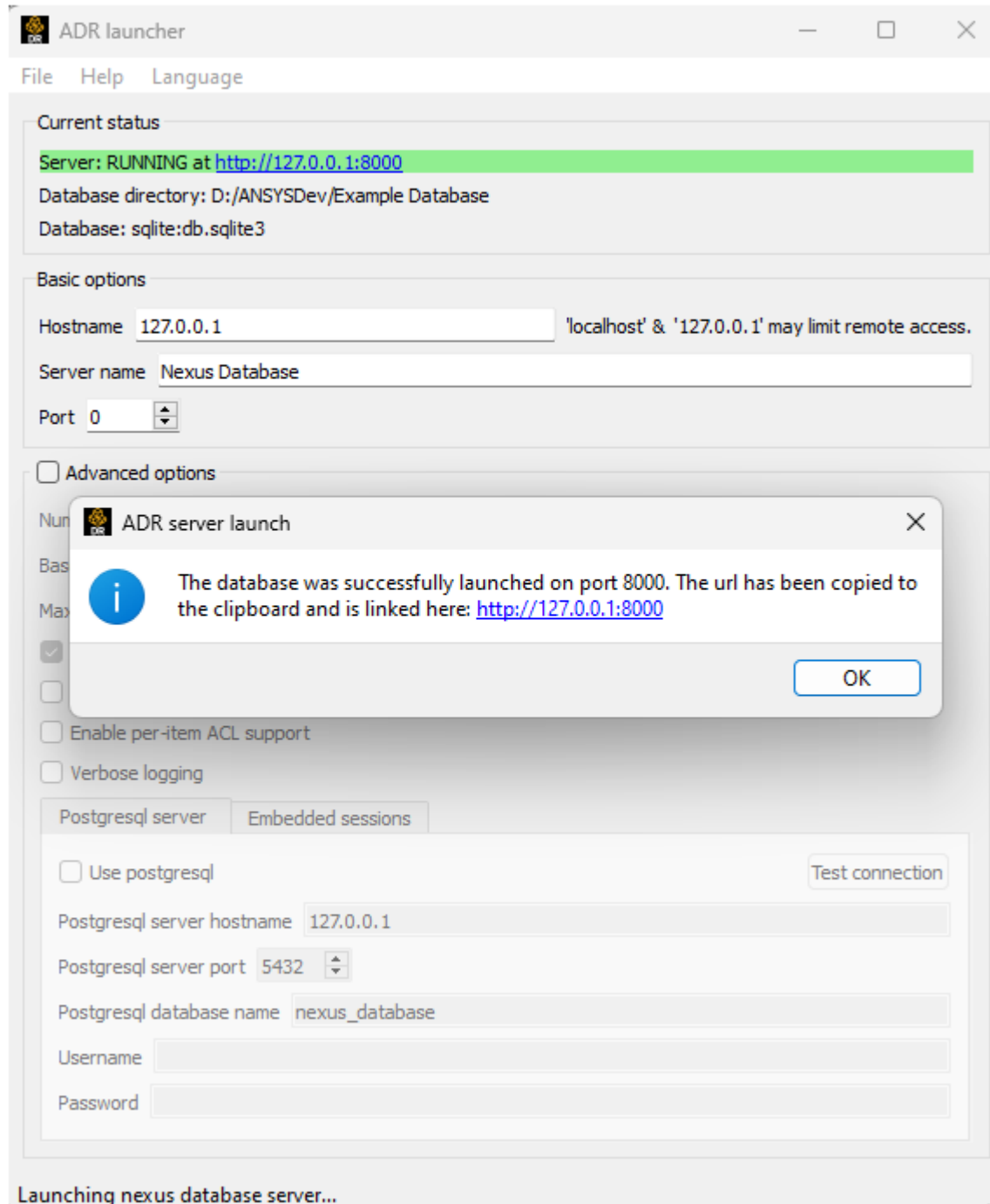
Specific features of each option are described in more detail in [Ansys Dynamic Reporting Launcher Command Line](#) (p. 304).

3. To save the configuration settings, from the menu bar select **File** → **Save configuration**.

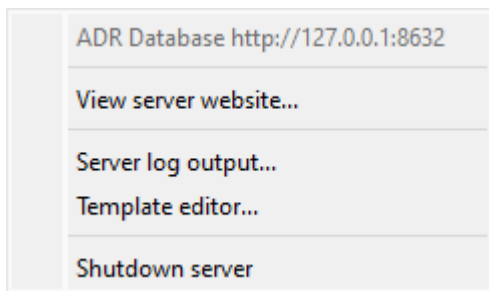
Launch the Server

To launch server, from the menu bar select **File** → **Launch server**.

The application launches a server and then presents the following dialog:



Clicking the dialog link opens the server in a browser window. The server status is updated, along with the URL for the server. The File menu option to shut down the server is now available.



If your desktop supports it, an icon is placed in your system tray and you can control the server from the system tray menu as well.

From the File menu, you can also review any server log output (if the Verbose option is checked in the configuration), view the website in a browser and launch the Ansys Dynamic Reporting template editor on the database.

Command Line Launcher

All of the functionality of the launcher is available from the command line and can be used in scripts. The command line options (and configuration details) are documented in [Ansys Dynamic Reporting Launcher Command Line](#) (p. 304).

Additional Launching Mechanisms

In most cases, the [Ansys Dynamic Reporting Server Launcher](#) (p. 13) is the best method to start/stop a server. However, there are multiple ways to launch a local server from within other applications.

The following launching options are available:

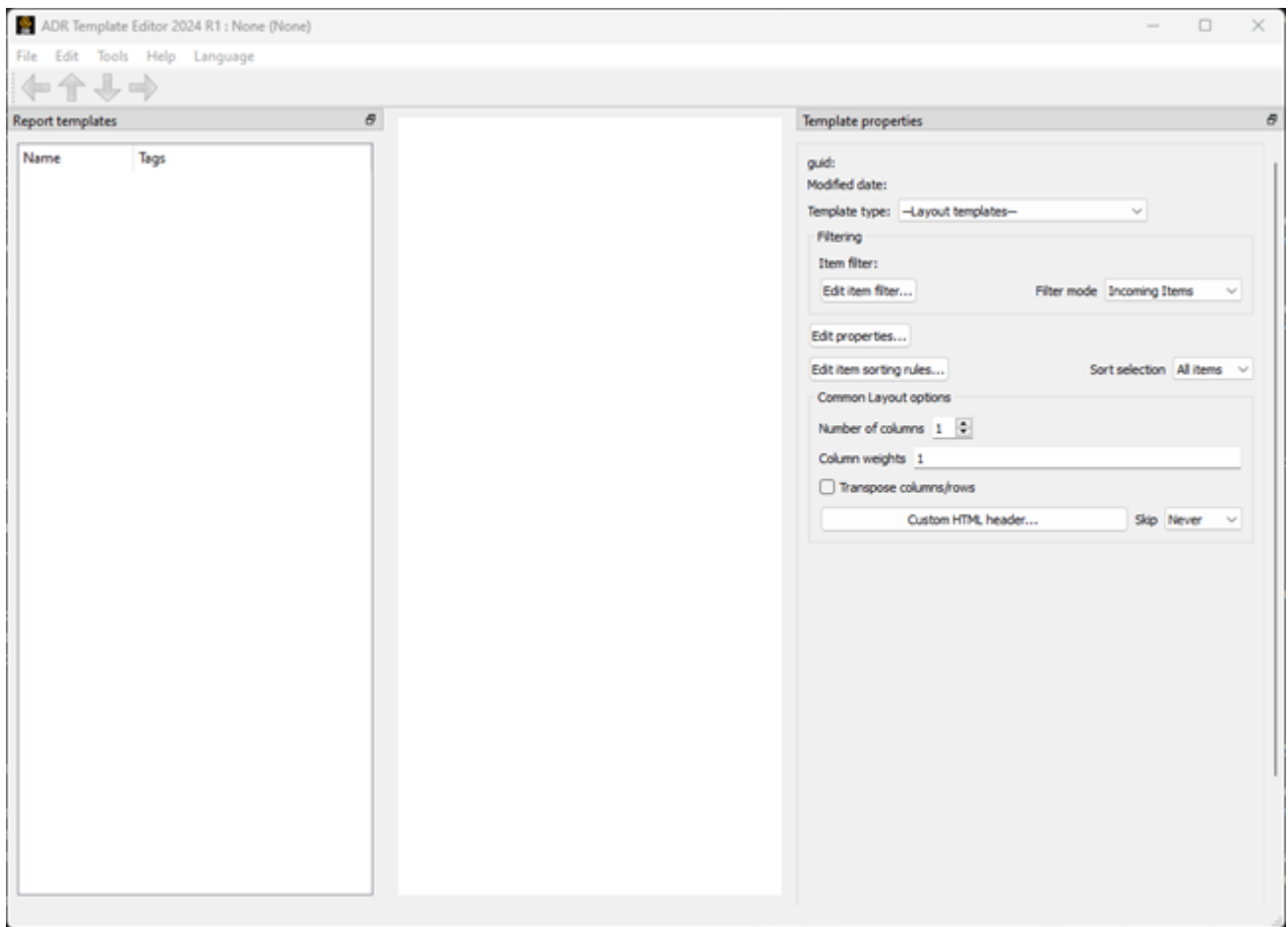
- [Launching Using Ansys Dynamic Reporting Template Editor](#)
- [Launching Using EnSight](#)
- [Launching From the Database](#)

Launching Using Ansys Dynamic Reporting Template Editor

You can use the Template Editor to launch a server.

1. Open a command prompt/shell, and navigate to `<Ansys installation path>\CEI\bin`.
2. Type **adr_template_editor**.

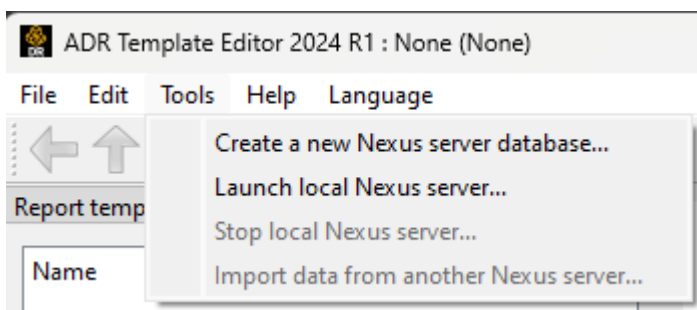
The Template Editor opens.



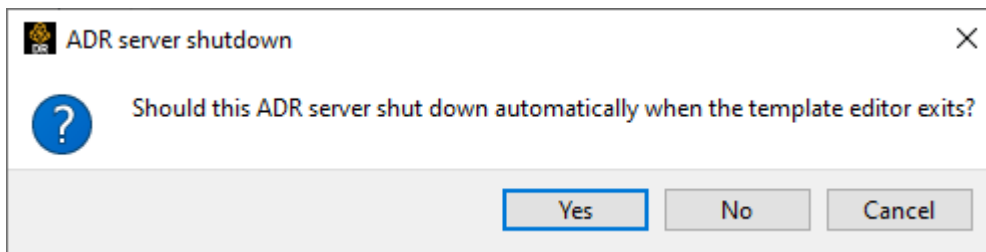
3. If you need to create a new database before launching the server:
 - a. On the menu bar, select **Tools** → **Create a new Nexus server database**.
 - b. In the dialog, browse your directories to the location where you want the empty database to be created.
 - c. Click **Select Folder**.

The directory is updated to include a media directory, a `manage.py` file, a `view_report.nexdb` file and a `db.sqlite3` file.

4. On the menu bar, select **Tools** → **Launch local Nexus server**.

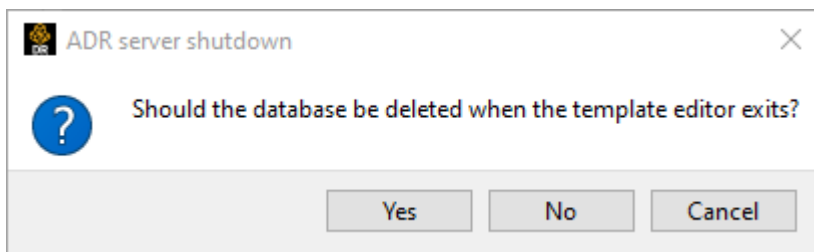


The following dialog opens:

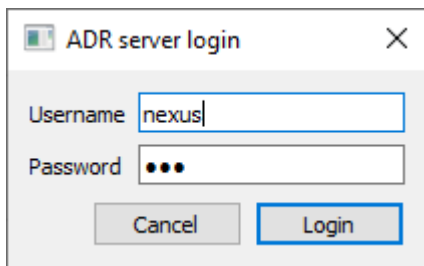


5. The dialog asks if the server should be shut down when you quit the Ansys Dynamic Reporting Template Editor. If you answer No, the server will remain running and accessible by your web browser even after you quit the application. Click **No**.

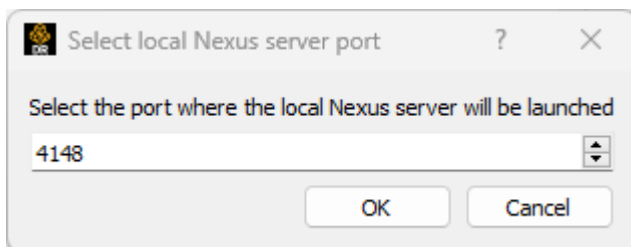
The following dialog opens:



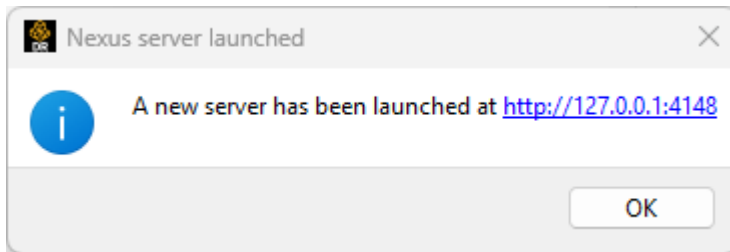
6. The dialog asks if the database should be deleted when the template editor exits. Click **No**.
7. In the **ADR server login**, leave the default username and password and click **Login**.



8. Navigate to, and select, the `db.sqlite3` file.
9. Click **Open**.
10. In the dialog, click **OK** to use a random open port.



The following dialog opens:

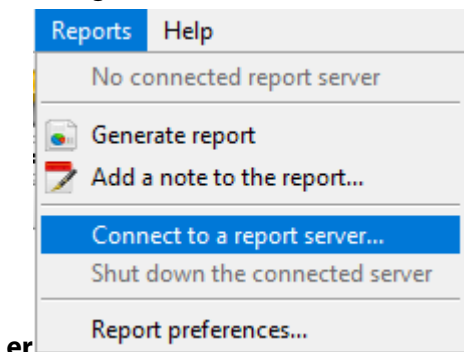


The server is now up and running.

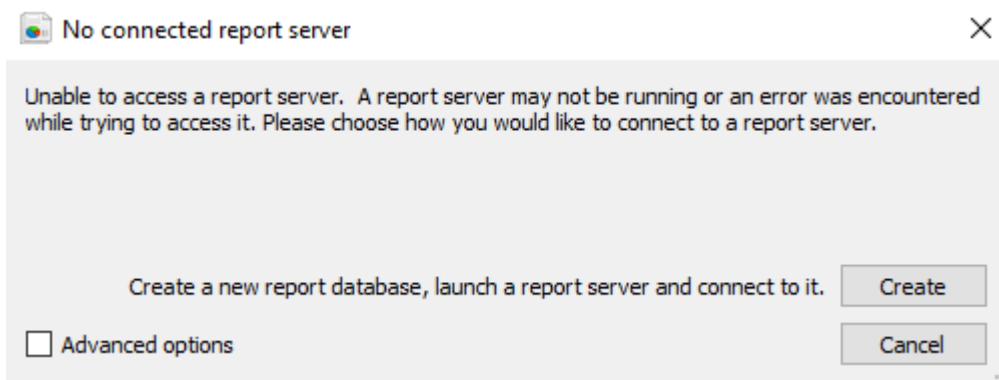
Launching Using EnSight

You can use EnSight to launch a server.

1. In EnSight, from the menu bar, select **Reports** → **Connect to a report serv-**

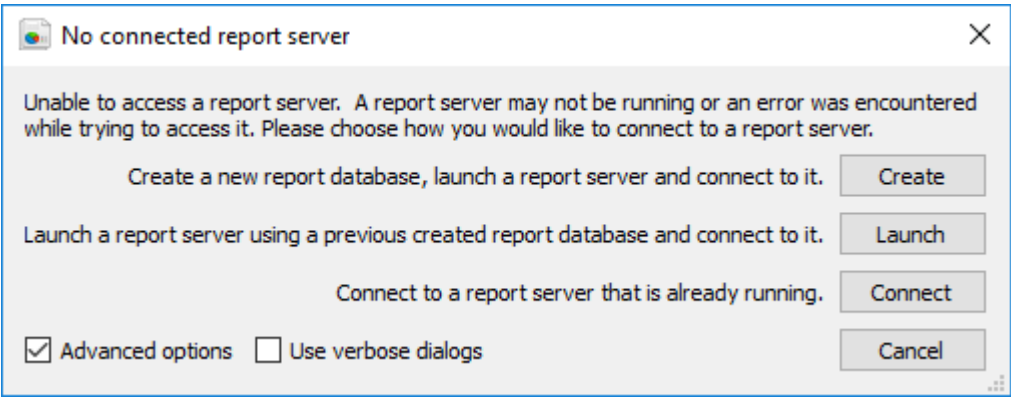


The following dialog opens.

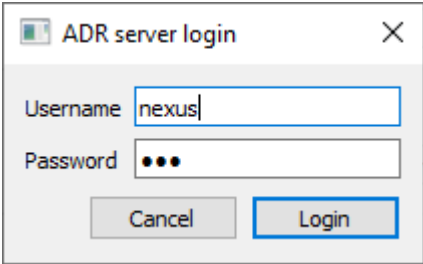


2. Select **Advanced options**.

The following dialog is displayed.



3. Select one of the following options:
- **Create**
Create a new database and launch a new server.
 - **Launch**
Launch a new server connected to an existing database.
 - **Connect**
Connect to a server that is already running.
4. Complete the following steps, based on the option you chose.

Option	Steps
Create	1. In the dialog, browse your directories to the location where you want the empty database to be created. 2. Click Select Folder. The directory is updated to include a media directory, a manage.py file, a view_report.nexdb file and a db.sqlite3 file.
Launch	1. In the Neus server login, leave the default username and password and click Login.  2. Navigate to, and select, the db.sqlite3 file.

Option	Steps
	3. Click Open .
Connect	Select one of the following options: - In the dialog, enter the Server URL, Username, and Password, then click OK. - Click Select local server, enter the port number for the server, and click OK.

Launching From the Database

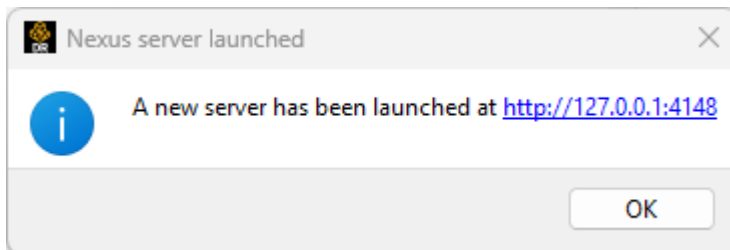
This option can only be used if you already have a database and want to start the Nexus service on top of it.

1. Navigate to the database directory.
2. Double-click the `view_report.nexdb` file.

The server is launched automatically.

Accessing the Web Interface

After a server is launched, the following dialog is displayed:



Clicking the link opens a browser window, displaying the web interface to the server.

Ansys

DYNAMIC REPORTING

Reports - Query - Information -

Sign In

Ansys Dynamic Reporting

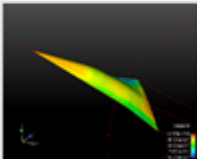
Organized. Productive. Open.

Preserve your simulation results data where it is organized, productive and flexible. Generate reports automatically. Combine custom searches and analytics to provide critical decision support. Share results and analytics with colleagues. Perfect for consortia groups, parameter and longitudinal studies.



Reminder: scheduled maintance for this server is Monday mornings from 8AM to Noon.

This is an example Message Of The Day with an image!



Organized

Data is uniquely tagged by dataset, run, date, unique ID and more so that anything can be found easily.

Learn more »

Productive

Data flows automatically into the system. No manual babysitting required. Reports can be run automatically or manually.

Learn more »

Open

Use it with data from EnSight or any other application.

Learn more »

ADR © 2017-2024 ANSYS, Inc.

Changing a Server/Database Password

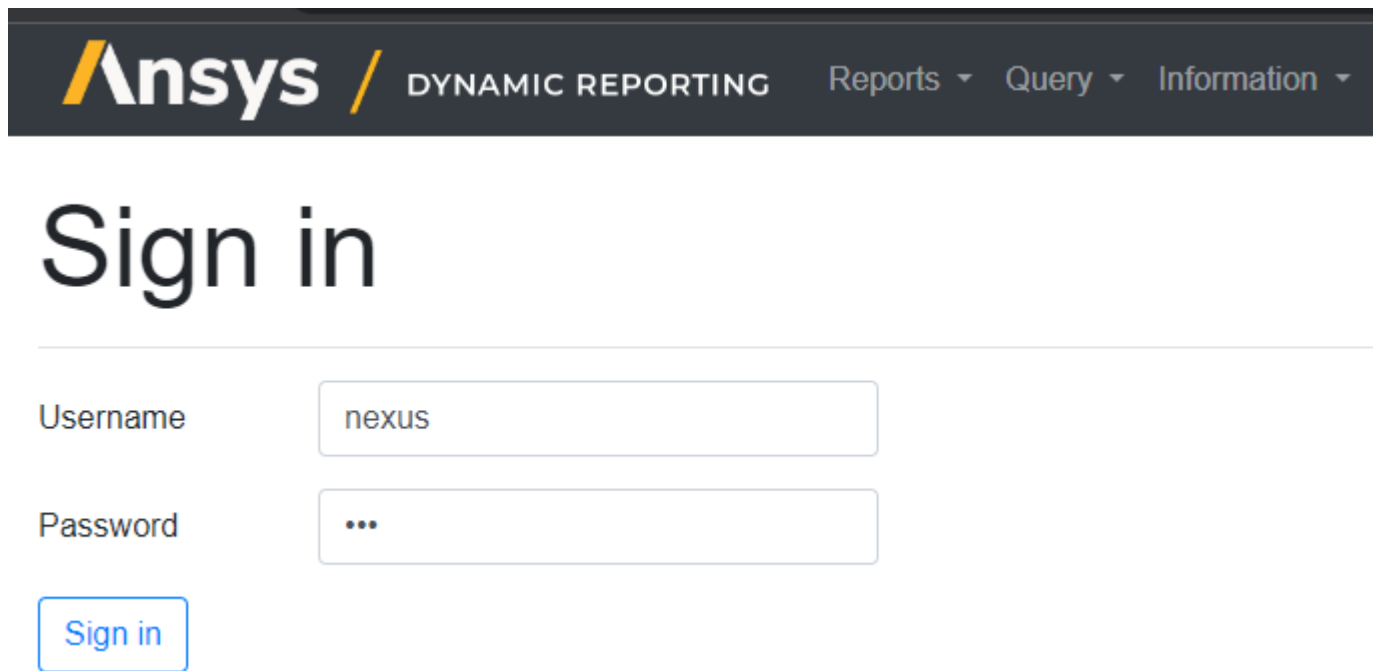
When a database is initially created, it is assigned an admin user with the username of *nexus* and the password *cei*. This can be changed by options in the server itself.

The [User administration page \(p. 258\)](#) allows you to add more users, modify existing users, and so on. This information is saved in the database and is used by any server that is launched with the database listed in the **Information** → **About** page.

Important:

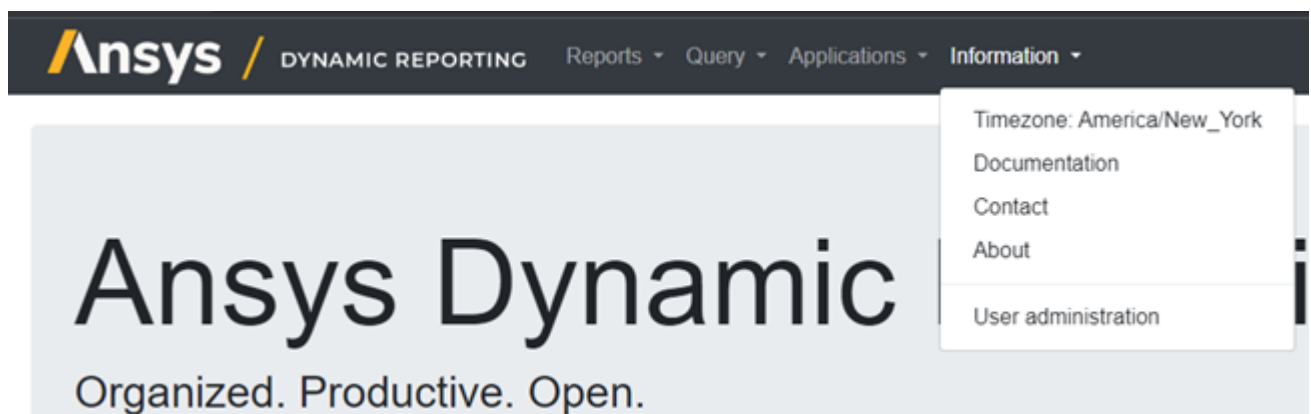
Operations performed on the administration page control access to your server and the data it contains. Ansys is not able to recover passwords you change so proceed with caution.

1. On the menu bar of the Ansys Dynamic Reporting web interface, click **Sign in**.
2. Enter the *nexus* **Username** and *cei* **Password**.



The screenshot shows the top navigation bar of the Ansys Dynamic Reporting interface. It includes the Ansys logo, the text 'DYNAMIC REPORTING', and three dropdown menus: 'Reports', 'Query', and 'Information'. Below the navigation bar is a large 'Sign in' heading. Underneath, there are two input fields: 'Username' with the value 'nexus' and 'Password' with three dots indicating a masked password. A blue 'Sign in' button is positioned below the password field.

3. Click **Sign in**.
4. From the menu bar, select **Information** → **User administration**.



The following page opens. The interface may vary a bit, depending on which version of Ansys Dynamic Reporting you are running.

Site Administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	Change
Users	+ Add	Change

5. On the **Site Administration** page, next to **Groups**, click **Change**.
6. From the list, click **nexus**.

Select user to change

The **Change user** page opens.

7. To change the user name, enter a new one into the **Username** field.
8. To change the password:
 - a. Click the words **this form**.
 - b. Enter the new password in the fields.

Change password: nexus

Enter a new password for the user **nexus**.

Password:

Password (again):

Enter the same password as before, for verification.

CHANGE PASSWORD

c. Click **Change Password**.

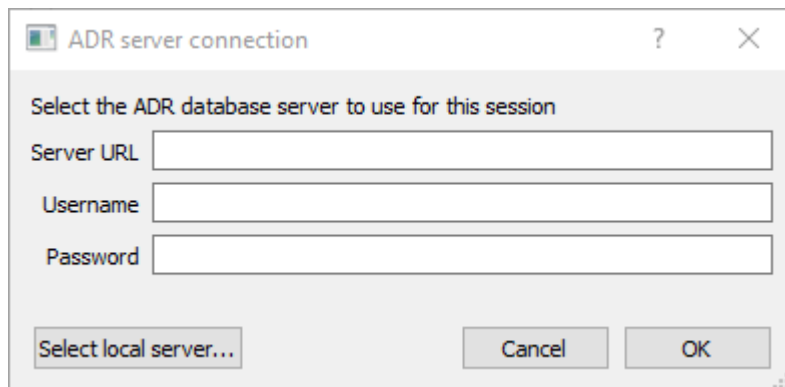
9. Click **Save**.

Connecting to a Running Server

You can use the Template Editor to connect to a running server.

1. Open a command prompt/shell, and navigate to `<Ansys installation path>\CEI\bin`.
2. Type **adr_template_editor**.
3. From the menu bar, select **File** → **Connect to a Nexus server**.

The following dialog is displayed:



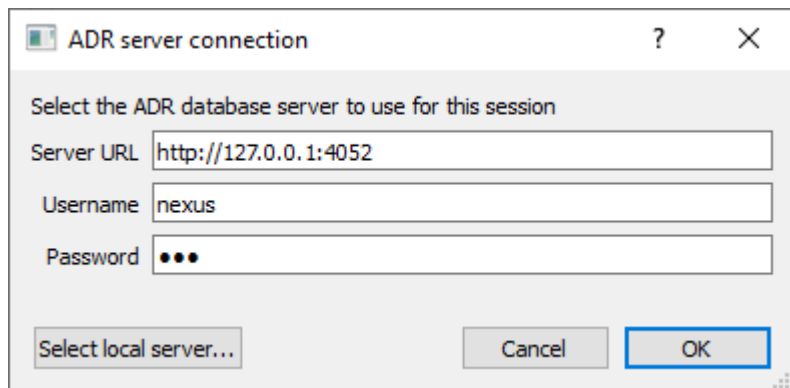
The dialog box is titled "ADR server connection" and contains the following elements:

- A title bar with a question mark icon and a close button (X).
- A label: "Select the ADR database server to use for this session".
- Three input fields: "Server URL", "Username", and "Password".
- A button labeled "Select local server..." at the bottom left.
- Buttons labeled "Cancel" and "OK" at the bottom right.

4. Enter the information manually to connect to a remote server, or click **Select local server**.

- Set the port number to the number used when launching the running server and click **OK**.

Username and password will be filled automatically with the default *nexus* and *cei*.



- Click **OK**.

Note:

You can also connect to a different server using the **File** → **Disconnect from current Nexus server** and **File** → **Connect to a Nexus server** menu options.

Shutting Down a Server

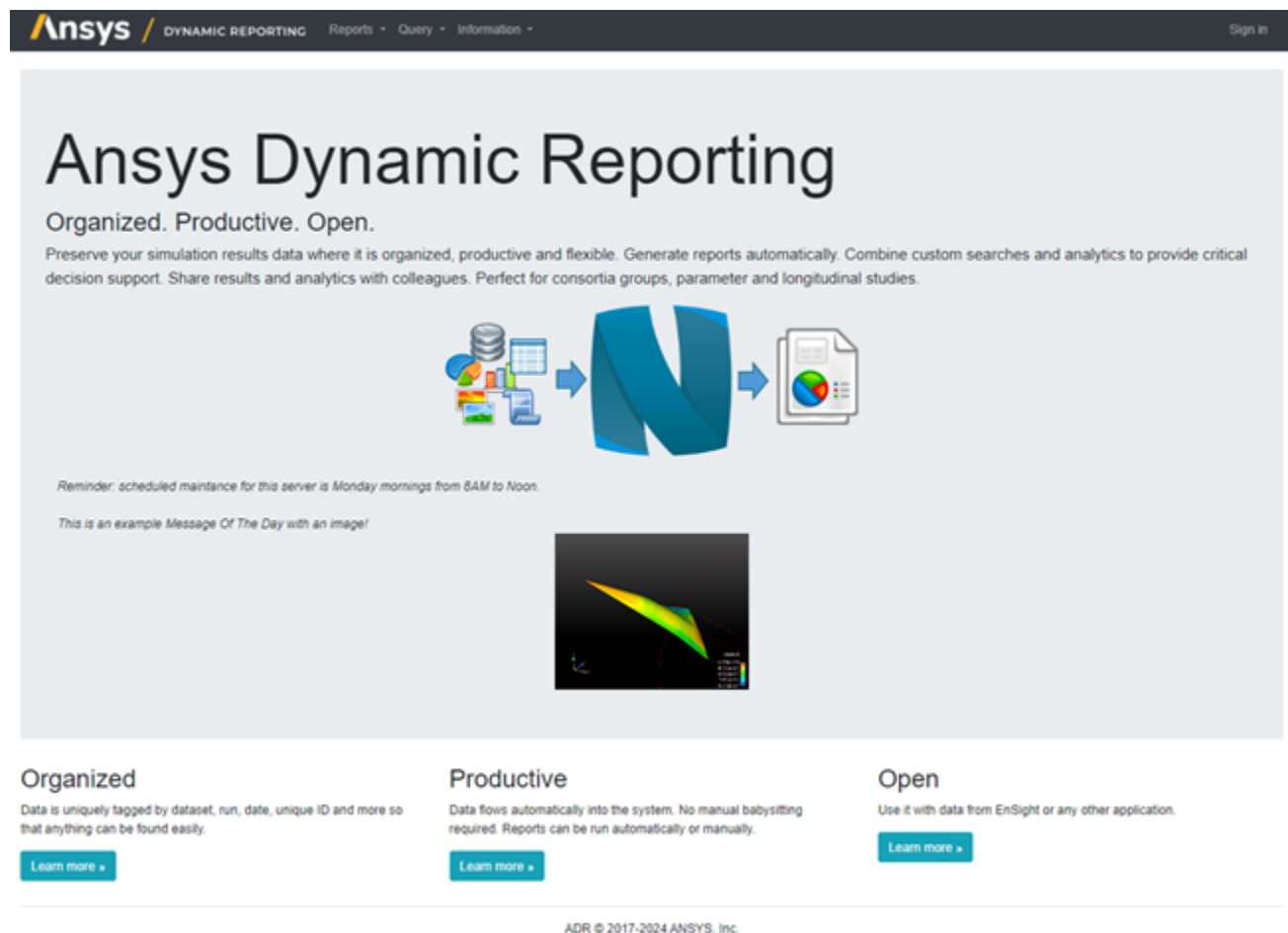
Unless explicitly shut down, a server will continue to run in the background on most systems. On Windows, closing the server console window will kill the server, but on other platforms the server may continue to run. Ensure that you manually shut down servers that you no longer need. When you are done using the server, you can stop it in the Template Editor by selecting **Tools** → **Stop local Nexus server** from the menu bar. Note that this will shut down only the server you are currently connected to.

Web Interface Overview

You can access the Ansys Dynamic Reporting database and generate reports in the web interface. This interface also allows you some level of control over the database content.

To access the web interface, click the URL link provided when [launching \(p. 13\)](#) Ansys Dynamic Reporting, or open a web browser and navigate to the to the hostname and port number that corresponds to the ADR service (for example, localhost : 8000).

The web interface is displayed.



Related topics:

- [Logging Into the Web Interface](#)
- [Enabling Dark Mode](#)
- [Web Interface Menus](#)

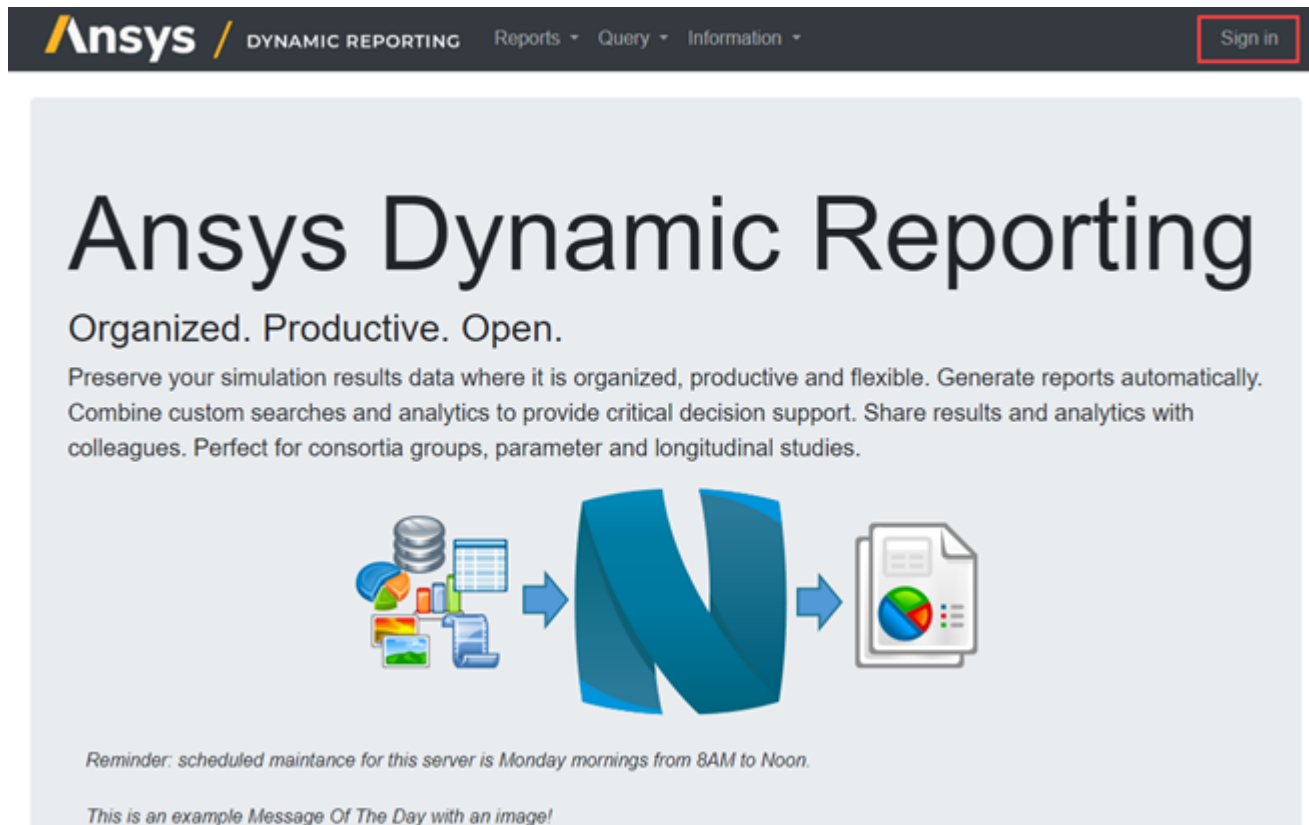
Logging Into the Web Interface

The web interface allows you visualize the content of the database and generate reports. Operations that modify the database itself are allowed only if you are logged in with the correct credentials. These credentials are shared with the [scripting API](#) (p. 291).

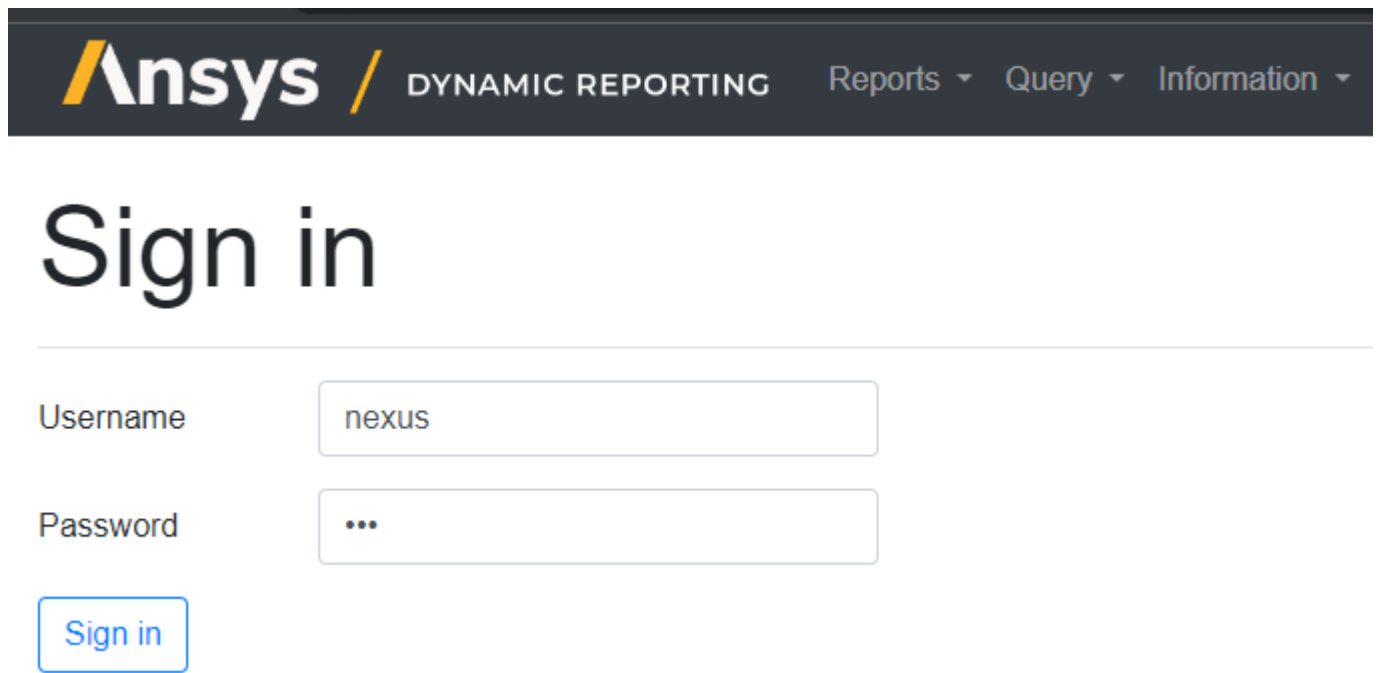
For more details on users and groups, see [Access Control Lists](#) (p. 255).

To log into the web interface:

1. [Launch Ansys Dynamic Reporting](#) (p. 13) and open the web interface (p. 23).
2. On the menu bar, click **Sign in**.



3. Enter a **Username** and **Password**.



- Click **Sign in**.

Enabling Dark Mode

You can enable dark mode in two ways:

- External Python API
- Web user interface

External Python API

Dark Mode can be enabled/disabled temporarily per request using the [external Python API](#) available from the template editor. For example, to export a report as HTML in dark mode:

```
from ansys.dynamicreporting.core.utils import report_remote_server, report_objects

source = report_remote_server.Server(url="http://localhost:9000", username="nexus", password="cei")
source.validate()
source.export_report_as_html("e548be92-4052-11eb-aacb-2f34187eafe8",
"C:/Users/me/Documents/htmlexport", query={'colormode': 'dark'})
```

The exported HTML will now be in dark mode. This is achieved by `query={'colormode': 'dark'}`. The default is light mode which can also be set by `query={'colormode': 'light'}`.

Setting this query produces a report URL equivalent to:

```
http://localhost:9000/reports/report_display/
?view=e548be92-4052-11eb-aacb-2f34187eafe8&colormode=dark
```

The `colormode=dark` query parameter is what temporarily enables dark mode only for that session. This is not saved for future requests and has to be added manually for each request.

If you are developing an external client that uses Ansys Dynamic Reporting, you may also add this query parameter manually to any URL that you are requesting.

Web Interface

You can enable and disable dark mode from anywhere within the web interface using the menu bar. To enable dark mode:

1. [Log into the web interface \(p. 30\)](#).
2. In the top right corner of the menu bar, click **Hello *username*!** and select **Dark Mode**.



Almost every part of the web interface now appears dark, including report templates and items within templates.

For example:



Parametric Report

Analyst	hkanchi
Date	10/5/2021 07:42 AM

Table of Contents

- 1 Design Points Table
- 2 Base Case Simulation Setup
 - 2.1 Physics
 - 2.1.1 Models
 - 2.1.2 Material Properties
 - 2.1.3 Cell Zone Conditions
 - 2.1.4 Boundary Conditions
 - 2.1.5 Reference Values
 - 2.2 Solver Settings
- 3 Mesh
- 4 Contours
- 5 Contours Slider
- 6 Contours Comparison
- 7 Scenes Slider

Design Points Table

	parameter_1 [m s ⁻¹]	report-def-0-op [kg m ⁻¹ s ⁻²]	para3 [m s ⁻¹]	Write Data	Status
DP0	2	8.650266	3	True	Updated : Not Converged
DP1	2.5	13.3949	3.75	False	Updated : Not Converged
DP2	3	19.18703	4.5	False	Updated : Not Converged

Base Case Simulation Setup

Physics

Models

Model 1	Settings
Space	3D
Time	Steady
Viscous	SST k-omega turbulence model

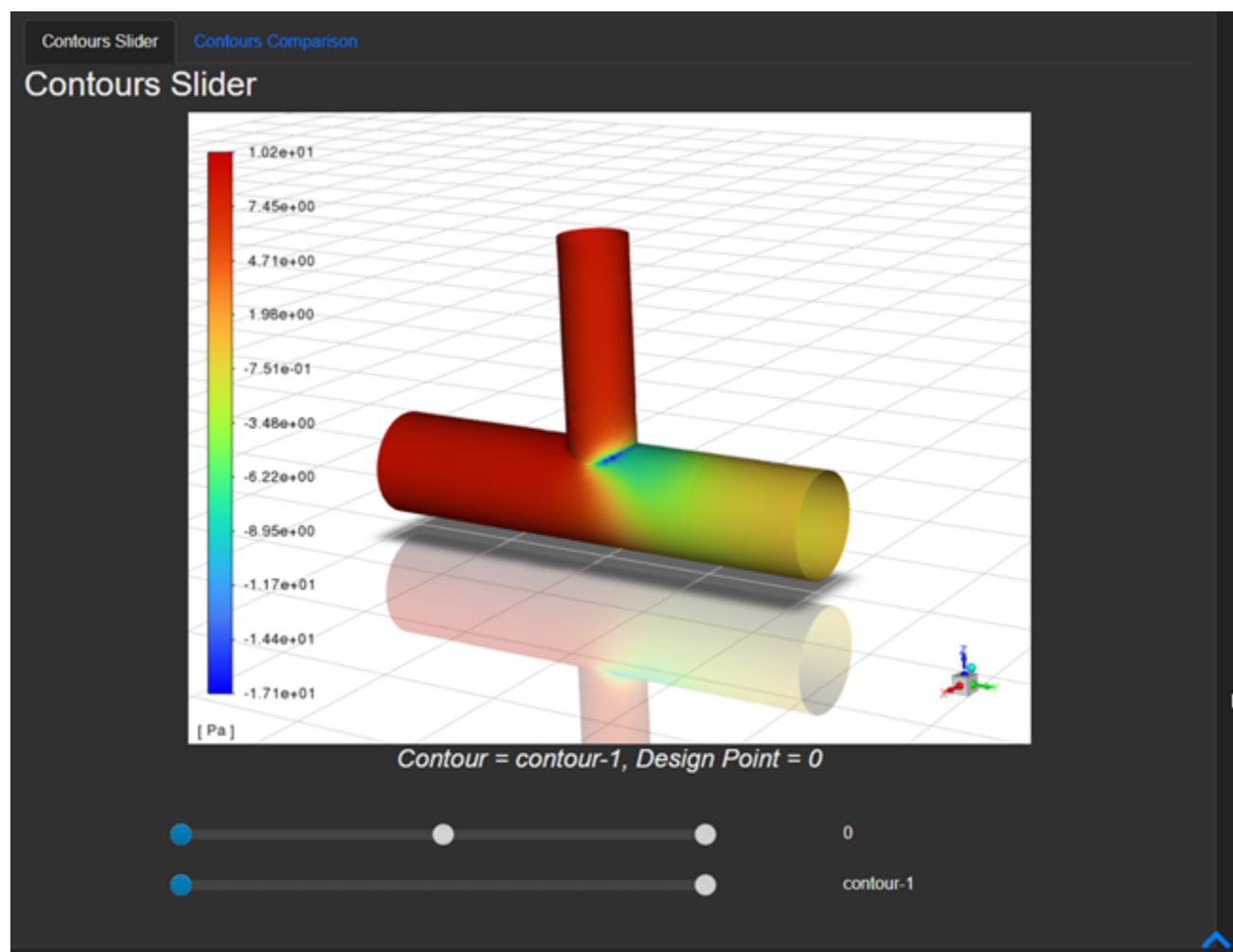
Material Properties

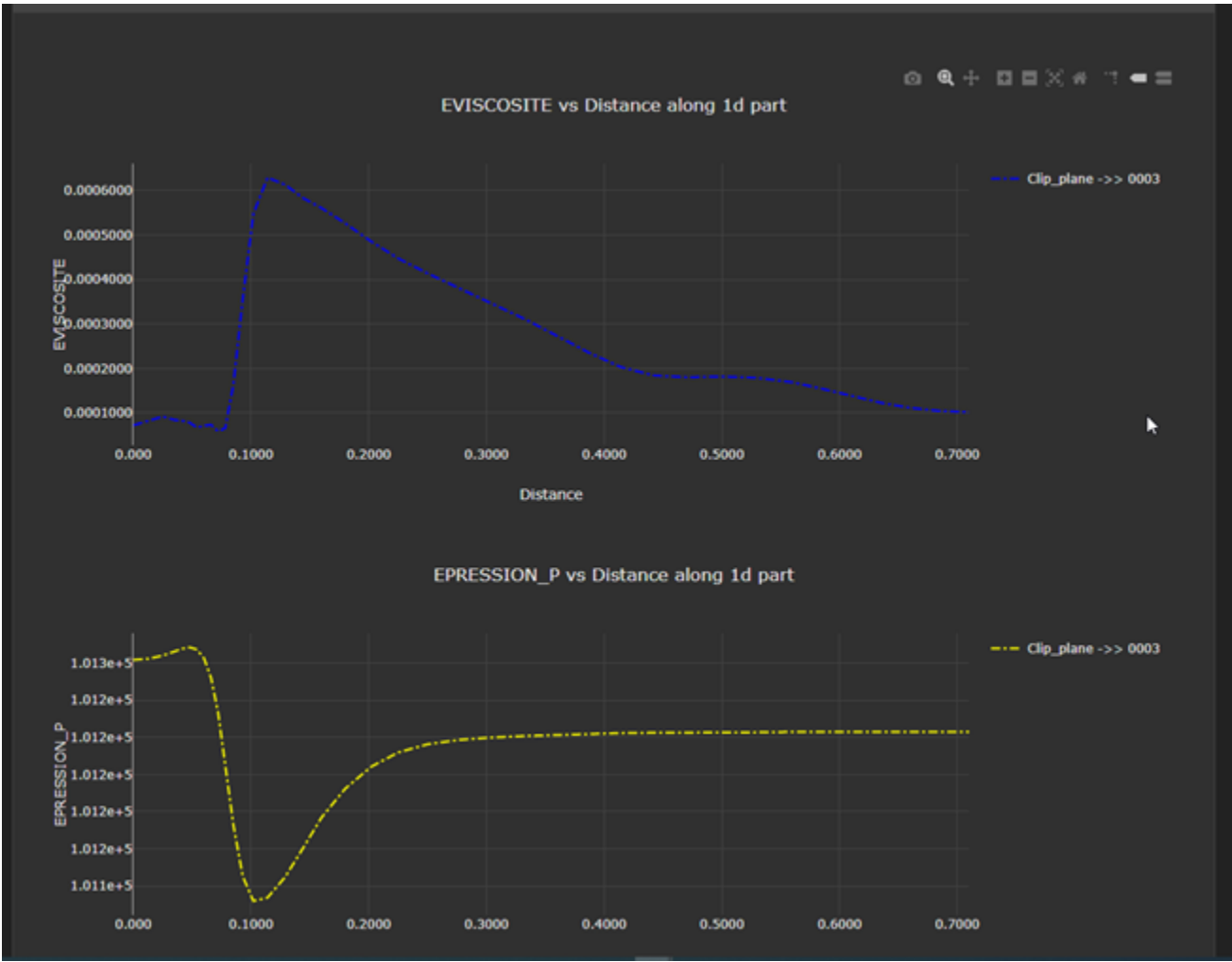
Reference Values

Area	1 m ²
Density	1.225 kg/m ³
Enthalpy	0 J/kg
Length	1 m
Pressure	0 Pa
Temperature	288.16 K
Velocity	1 m/s
Viscosity	1.7894e-05 kg/(m s)
Ratio of Specific Heats	1.4
Yplus for Heat Tran. Coef.	300
Reference Zone	fluid

Solver Settings

+ -	
— Equations	
Flow	True
Turbulence	True
— Numerics	
Absolute Velocity Formulation	True
— Pseudo Time Explicit Relaxation Factors	
Density	1
Body Forces	1
Turbulent Kinetic Energy	0.75





Web Interface Menus

The following menu options are available in the web interface.

Reports Menu

Option	Description
Generate report	Opens the Report Generation (p. 246) page.

Query Menu

In this menu, the first four options always appear. The next three are visible only when you are logged into the web interface. The last two only appear when you are logged in and [enable Access Control Lists \(p. 258\)](#).

Option	Description
Report items	Opens the Report Item Search (p. 39) page.

Option	Description
Sessions	Opens the Sessions Search (p. 42) page.
Datasets	Opens the Datasets Search (p. 45) page.
Report templates	Opens the Report Templates Search (p. 48) page.
Modify tags of selected report items	Opens the Modify Report Item Tags (p. 51) page.
Delete selected report items	Opens the Delete Report Items (p. 54) page.
Remove orphaned sessions/datasets	Automatically deletes orphaned sessions and datasets (p. 55).
Manage item categories	Opens the Manage item categories (p. 267) page.
Assign categories to selected report items	Opens the Assign Categories to Report Items (p. 273) page.

Applications Menu

This menu only appears when you are logged into the web interface.

Option	Description
Upload files into data items	Opens the Upload desktop items into data files (p. 57) page.
Media browser	Opens the Media browser (p. 58) page.

Information Menu

In this menu, the first four options always appear. The last option is visible only when you are logged into the web interface as a superuser (admin).

Option	Description
Timezone	Allows you to select a timezone for date/time display.
Documentation	Opens this documentation.
Contact	Displays contact information for questions or comments about Ansys Dynamic Reporting.
About	Displays a page with information about the ADR server you are currently visualizing. It provides information on the database (location, content) and on the version of Ansys Dynamic Reporting that is being used to run this server.
User administration	Opens the admin interface (p. 258) .

Querying the Database

The Ansys Dynamic Reporting web interface allows you to extract the data from the database. Every item placed in the database is assigned three things:

- Item (the data file itself)
- Dataset (the data it comes from)
- Session (the analysis session during which it was produced)

You can use these (or combinations of them) to filter the database and recall the items via queries.

You can also query for report templates.

Related topics:

[Querying for Report Items](#)

[Querying for Sessions](#)

[Querying for Datasets](#)

[Querying for Report Templates](#)

[Modifying Tags of Selected Report Items](#)

[Deleting Selected Report Items](#)

[Deleting Orphaned Sessions and Datasets](#)

Querying for Report Items

Use the **Report Item Search** page to find and view report items.

1. From the menu bar, select **Query** → **Report Items**.

The **Report Item Search** page opens.

Ansys

DYNAMIC REPORTING

Reports ▾

Query ▾

Applications ▾

Information ▾

Hello, nexus! ▾

Report Item Search

Search Filters

	Field	Comparison	Value(s)		
and ▾	Item name ▾	Contains any of ▾		+	-
				+	-

Preview Query

Run Query

2. In the **Search Filters**, enter the query to run against the database to extract the list of items. An example query is shown in the following image.

Report Item Search

Search Filters

	Field	Comparison	Value(s)		
and ▾	Item types ▾	Include ▾	File HTML Image Movie	+	-
				+	-

3. To preview the query, click **Preview Query**.

The **Query Preview** dialog opens. You can copy the query string for other purposes, like use with the Python API, by clicking .

Query Preview

×

A|i_type|cont|image;



Done

4. Click **Run Query**.

A list of items that satisfy the query is displayed.

Report Item List

Back to query

157 item(s) found

Show 10 entries

Search:

Name	Type	Date	Source	Tags
2DImage	image	2016-07-07 9:39 am	image_source	
Image Source	image	2016-07-11 10:01 am	image_source	
Image Source	image	2016-07-11 10:02 am	image_source	
Image Source	image	2016-07-14 9:25 am	image_source	
Image Source	image	2016-07-14 9:26 am	image_source	
Image Source	image	2016-07-14 9:26 am	image_source	
Image Source	image	2016-07-14 9:34 am	image_source	

- 5. To see details of a specific item, click the item link.

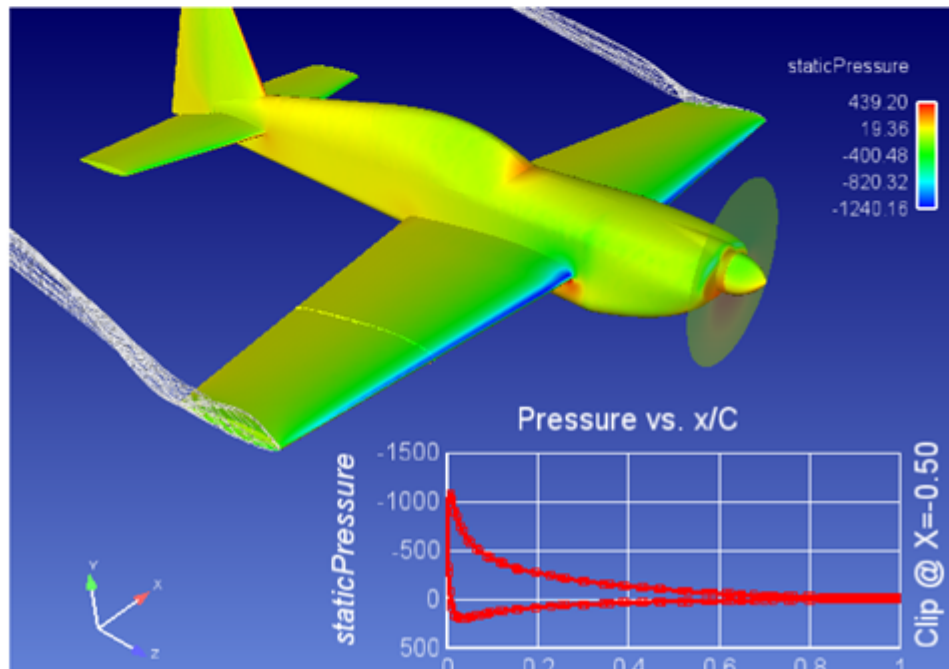
The item details page is displayed.

Report Item

Metadata [Click to edit fields](#)

Name	2DImage
Date	July 7, 2016, 9:39 a.m.
Source	image_source
Sequence number	1
Tags	
Item ID	4c50ba8e-4448-11e6-9c03-6fc4ae1aebd8
Session ID	c3829700-4446-11e6-bca5-eb6fb852637d
Dataset ID	c6d3ccc2-7558-31d0-8490-9058952c29dc
Type	image
Categories	None
You can only change categories you own.	

Details



Querying for Sessions

Use the **Sessions Search** page to find and view sessions.

1. From the menu bar, select **Query** → **Sessions**.

The **Session Search** page opens.

Report Session List

Back to query

53 session(s) found

Show 10 entries

Search:

Application	Version	Date	Tags
EnSight	10.2.0(a)	2016-06-30 1:49 pm	
EnSight	10.2.0(a)	2016-07-03 6:32 pm	
EnSight	10.2.0(a)	2016-07-04 1:12 pm	
EnSight	10.2.0(a)	2016-07-05 4:01 pm	
EnSight	10.2.0(a)	2016-07-05 4:32 pm	
EnSight	10.2.0(a)	2016-07-07 9:36 am	
EnSight	10.2.0(a)	2016-07-11 10:00 am	
EnSight	10.2.0(a)	2016-07-14 9:24 am	
EnSight	10.2.0(a)	2016-07-19 9:31 am	
EnSight	10.2.0(a)	2016-08-08 9:36 am	

Previous

1

2

3

4

5

6

Next

5. To see details of a specific session, click the session link.

The session details page is displayed.

Session

Metadata

Application	EnSight
Application version	10.2.0(a)
Date	June 30, 2016, 1:49 p.m.
Hostname	iliad
Host platform	win64
Tags	
Session ID	ba4ed22c-3eea-11e6-8dba-8bfffef931899

7 Referencing Items

- [HTML Source, html_source, 06/30/2016 1:49 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:50 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:52 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:52 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:54 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:54 p.m.](#)
- [Query Source, query_source, 06/30/2016 1:54 p.m.](#)

Querying for Datasets

Use the **Dataset Search** page to find and view datasets.

1. From the menu bar, select **Query** → **Datasets**.

The **Dataset Search** page opens.

Anslys

DYNAMIC REPORTING

Reports

Query

Applications

Information

Hello, nexu!

Dataset Search

Search Filters

	Field	Comparison	Value(s)	
and	Dataset filename	Contains any of		+ -

Preview Query

Run Query

2. In the **Search Filters**, enter the query to run against the database to extract the list of items. An example query is shown in the following image.

Dataset Search

Search Filters

	Field	Comparison	Value(s)	
and	Dataset format	Equal	EnSight Case	+ -

3. To preview the query, click **Preview Query**.

The **Query Preview** dialog opens. You can copy the query string for other purposes, like use with the Python API, by clicking .

Query Preview

A\d_fmt|eq|EnSight Case;



Done

4. Click **Run Query**.

A list of items that satisfy the query is displayed.

Report Dataset List

[Back to query](#)

15 dataset(s) found

Show	10	entries	Search:	
Filename	Format	Tags		
crash.case	EnSight Case			
cube.case	EnSight Case	crank_angle=60 timestep=30		
cube.case	EnSight Case			
extra300_RC_Plane_nodal.case	EnSight Case			
breakfast_set_obj.case	EnSight Case			
ensightb.case	EnSight Case			
Barcelona Pavillion.case	EnSight Case			
extra300_RC_Plane_nodal.case	EnSight Case			
breakfast_set_obj.case	EnSight Case			
crash.case	EnSight Case			
Previous 1 2 Next				

5. To see details of a specific dataset, click the dataset link.

The dataset details page is displayed.

Dataset

Metadata

Filename	crash.case
Format	EnSight Case
Directory name	
Number of parts	15
Number of elements	6597
Tags	
Dataset ID	b11cee5b-e467-363a-94ed-621abffae37b

1 Referencing Item

- [test, commandline, 10/19/2016 10:27 a.m.](#)

Querying for Report Templates

Use the **Report Template Search** page to find and view report templates.

1. From the menu bar, select **Query** → **Report templates**.

The **Report Template Search** page opens.

Ansys / DYNAMIC REPORTING Reports ▾ Query ▾ Applications ▾ Information ▾ Hello, nexus! ▾

Report Template Search

Search Filters

	Field	Comparison	Value(s)	
and ▾	Template visible ▾	Equal ▾	Toplevel Child	+ - + -

Preview Query Run Query

- In the **Search Filters**, enter the query to run against the database to extract the list of items. An example query is shown in the following image.

Report Template Search

Search Filters				
	Field	Comparison	Value(s)	
and	Template name	Contains any of	B	

- To preview the query, click **Preview Query**.

The **Query Preview** dialog opens. You can copy the query string for other purposes, like use with the Python API, by clicking .

Query Preview



A|t_name|cont|B;



Done

- Click **Run Query**.

A list of items that satisfy the query is displayed.

Report Template List

[Back to query](#)

22 report template(s) found

Show 10 entries

Search:

Name	Type	Date	Tags	# of children
box_images	panel	2016-10-20 3:14 pm	images	0
box_movies	panel	2016-10-21 9:32 am	movies	0
Merged Table	Generator:tablemerge	2017-02-17 3:42 pm		1
Box Template	box	2017-03-06 10:33 pm	boxes	3
box_plot	basic	2017-03-06 10:33 pm		0
Tab layout	Layout:tabs	2017-03-09 2:23 pm		2
Raw Table	Layout:basic	2017-03-09 2:24 pm		0
Filtered Table	Layout:basic	2017-03-09 3:03 pm		0
Raw Table	Layout:panel	2017-03-09 4:19 pm		0
Filtered Table	Layout:panel	2017-03-09 4:19 pm		0

[Previous](#)
[1](#)
[2](#)
[3](#)
[Next](#)

- To see details of a specific report template, click the template link.

The report template details page is displayed.

Report Template

Metadata

Name	box_images
Type	panel
Date	Oct. 20, 2016, 3:14 p.m.
Visibility	Child
Item filter	A[ji_type]cont[image;
Parameters	{"properties": {"image_link": "1"}, "column_count": 2, "column_widths": [1.0, 1.0]}
Tags	images
Template ID	781e4011-8efc-11e6-85c2-e434faa90175
Parent	Box Template

Children

- No children for this report template

Modifying Tags of Selected Report Items

When you are logged into the web interface, you can search for report item tags and modify them.

1. From the menu bar, select **Query** → **Modify tags of selected report items**.

The **Modify Report Item Tags: Search** page opens.

Ansys / DYNAMIC REPORTING Reports ▾ Query ▾ Applications ▾ Information ▾ Hello, nexus! ▾

Modify Report Item Tags: Search

Search Filters

	Field	Comparison	Value(s)	
and ▾	Item name ▾	Contains any of ▾		+ -
				+ -

Preview Query Run Query

2. In the **Search Filters**, enter the query to run against the database to extract the list of items. If you want to modify the tags on all the items in the database, leave the fields empty.
3. Click **Run Query**.

A list of items that satisfy the query is displayed. Multiple tags on each item are separated by spaces.

Select tag modification operation

Operation
Add a tag

Tag name

☐ Tag value

Add the tag ". Overwrite if the data item already contains the tag name: ".

Back to query
Reset selection
Modify selected...
Modify all...

243 item(s) found
0 item(s) selected

Show 10 entries
Search:

☐	Name	Type	Date	Source	Tags	Modified Tags Preview
☐	Image Source	image	2016-10-06 5:27 pm	image_source	dd small image	dd small image
☐	Image Source	image	2016-10-12 3:13 pm	image_source	crank_angle=60 timestep=30	crank_angle=60 times
☐	Text Source	string	2016-10-12 3:14 pm	text_source	crank_angle=60 timestep=30	crank_angle=60 times
☐	HTML Source	html	2016-10-12 3:14 pm	html_source	crank_angle=60 timestep=30	crank_angle=60 times
☐	Bob	table	2016-10-12 3:19 pm	query_source	crank_angle=60 timestep=30 Hello float	crank_angle=60 times

- To change specific tags in the query list, select the check boxes next to the report item names. To modify all tags in the query, leave the check boxes cleared.

- From the **Operations** list, select one of the following options:

- **Add a tag**

Adds a new tag to either the selected items or all items in the list. The tag will be added only if it does not already exist.

- **Remove a tag**

Removes a tag from either the selected items or all items in the list. If **Tag value** is toggled on and a value is entered, then only tags with that particular value will be removed.

- **Replace a tag**

Replaces an existing tag with another one. Applies to either the selected items or all items in the list.

- In the **Tag name** field, enter the name of the tag to modify.
- To modify a tag based on the value it contains, select the **Tag value** check box and enter a value in the field.

8. Check the setting description string and **Modified Tags Preview** column to ensure that the modifications you have entered are correct before committing the tag changes.

Select tag modification operation

Operation: Remove a tag Tag name: crank_angle ☐ Tag value

Remove the tag 'crank_angle' from the data item if it exists, potentially with any value: 'crank_angle="';

Back to query Reset selection Modify selected... Modify all...

243 item(s) found
0 item(s) selected

Show 10 entries Search:

☐	Name	Type	Date	Source	Tags	Modified Tags Preview
☐	Image Source	image	2016-10-06 5:27 pm	image_source	dd small image	dd= small= image=
☐	Image Source	image	2016-10-12 3:13 pm	image_source	crank_angle=60 timestep=30	timestep=30
☐	Text Source	string	2016-10-12 3:14 pm	text_source	crank_angle=60 timestep=30	timestep=30
☐	HTML Source	html	2016-10-12 3:14 pm	html_source	crank_angle=60 timestep=30	timestep=30
☐	Bob	table	2016-10-12 3:19 pm	query_source	crank_angle=60 timestep=30 Hello float	timestep=30 Hello= float=

9. To commit the changes to the items selected in step 4, click **Modify selected**. To commit the changes to all items in the query, click **Modify all**.

Important:

There is no ability to undo tag modifications.

Examples

In this example database, there are two items with the following tags.

Pulley1 3D	scene	03/14/2019 15:59	analysis=mechanical params=pulley1 type=3D
Pulley2 3D	scene	03/14/2019 16:01	analysis=mechanical params=pulley2 type=3D

To remove all tags for *params=*, regardless of its value, select the operation **Remove a tag**, set the **Tag name** to **params** and do not toggle on **Tag value**.

To remove only the *params=pulley1* tag but not the *params=pulley2* tag, select the operation **Remove a tag**, set the **Tag name** to **params**, and toggle on **Tag value**. In this field, enter **pulley1**.

To change the tag *analysis=mechanical* to be *analysis=Mech*, select the operation **Replace a tag**, set the **Tag name** to **analysis**, and toggle on **Tag value**. In this field, enter **Mech**. This operation will take all of the *analysis=** tags (regardless of their value), and replace them with *analysis=Mech*.

Deleting Selected Report Items

When you are logged into the web interface, you can search for report items and delete them.

1. From the menu bar, select **Query** → **Delete selected report items**.

The **Delete Report Items: Search** page opens.

	Field	Comparison	Value(s)	
and	Item name	Contains any of		+ -

Preview Query Run Query

2. In the **Search Filters**, enter the query to run against the database to extract the list of items. An example query is shown in the following image.

Delete Report Items: Search

	Field	Comparison	Value(s)	
and	Item types	Include	File HTML Image Movie News	+ -

3. Click **Run Query**.

A list of items that satisfy the query is displayed.

Review report items to delete

[Back to query](#)
[Reset selection](#)
[Delete selected...](#)
[Delete all...](#)

158 item(s) found
0 item(s) selected

Show entries
Search:

☐	Name	Type	Date	Source	Tags
☐	2DImage	image	2016-07-07 9:39 am	image_source	
☐	Image Source	image	2016-07-11 10:01 am	image_source	
☐	Image Source	image	2016-07-11 10:02 am	image_source	
☐	Image Source	image	2016-07-14 9:25 am	image_source	
☐	Image Source	image	2016-07-14 9:26 am	image_source	
☐	Image Source	image	2016-07-14 9:26 am	image_source	
☐	Image Source	image	2016-07-14 9:34 am	image_source	

- To delete selected items, select the check boxes next to those items and click **Delete selected**. To delete all items in the query, click **Delete all**.
- In the confirmation dialog, click **Continue**.

Confirmation



Are you sure you want to proceed with deletion of 158 returned item(s)?

Continue

Deleting Orphaned Sessions and Datasets

When you are logged into the web interface, you can automatically detect sessions and datasets in the database that have no items associated to them, and delete them. This can be needed to clean up the database after you have pushed and deleted items manually.

From the menu bar, select **Query** → **Delete orphaned sessions/datasets**.

The **Database Cleanup Operations** page is displayed, providing information on how many orphaned sessions and datasets have been removed.

Uploading and Managing Files

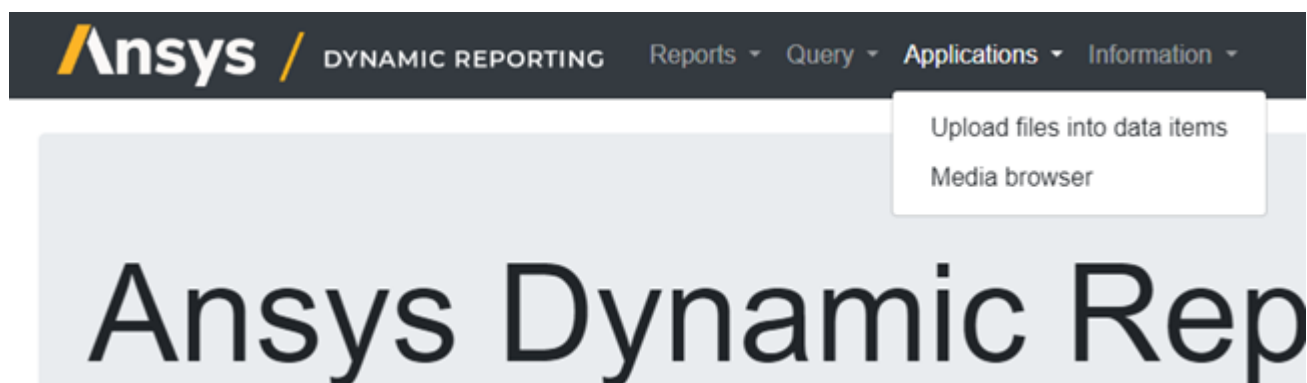
When you are logged into the web interface, there are two methods you can use to upload and manage files:

- [Directly uploading files \(p. 57\)](#)
- [Media browser \(p. 58\)](#)

Uploading Files into Data Items

This option allows you to upload a file and directly place it into the Ansys Dynamic Reporting database.

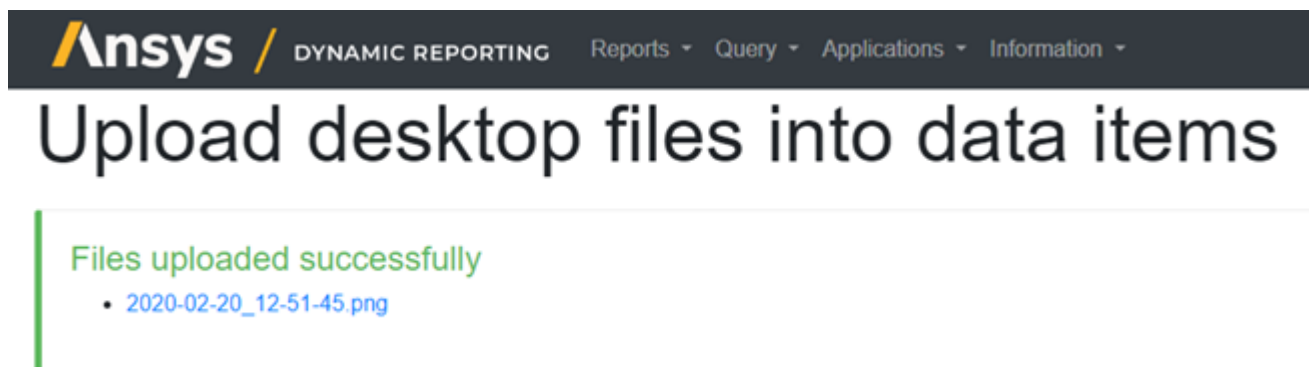
1. Ensure you are [logged into the web interface \(p. 30\)](#).
2. From the menu bar, select **Applications** → **Upload files into data items**.



The **Upload desktop files into data items** page is displayed.

3. Click **Browse** and choose something to upload. Supported file formats are displayed on the right.
4. Click **Upload**.

A notification is displayed.



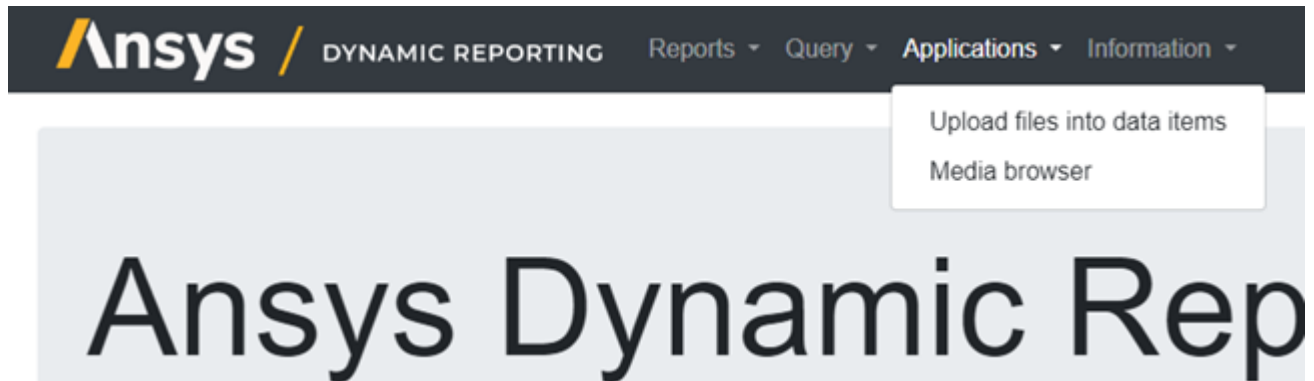
Using the Media Browser

The Media Browser is an alternative to uploading physical media files through the web interface. Although it currently only supports local storage, it is designed with the future integration of various cloud storage providers in mind.

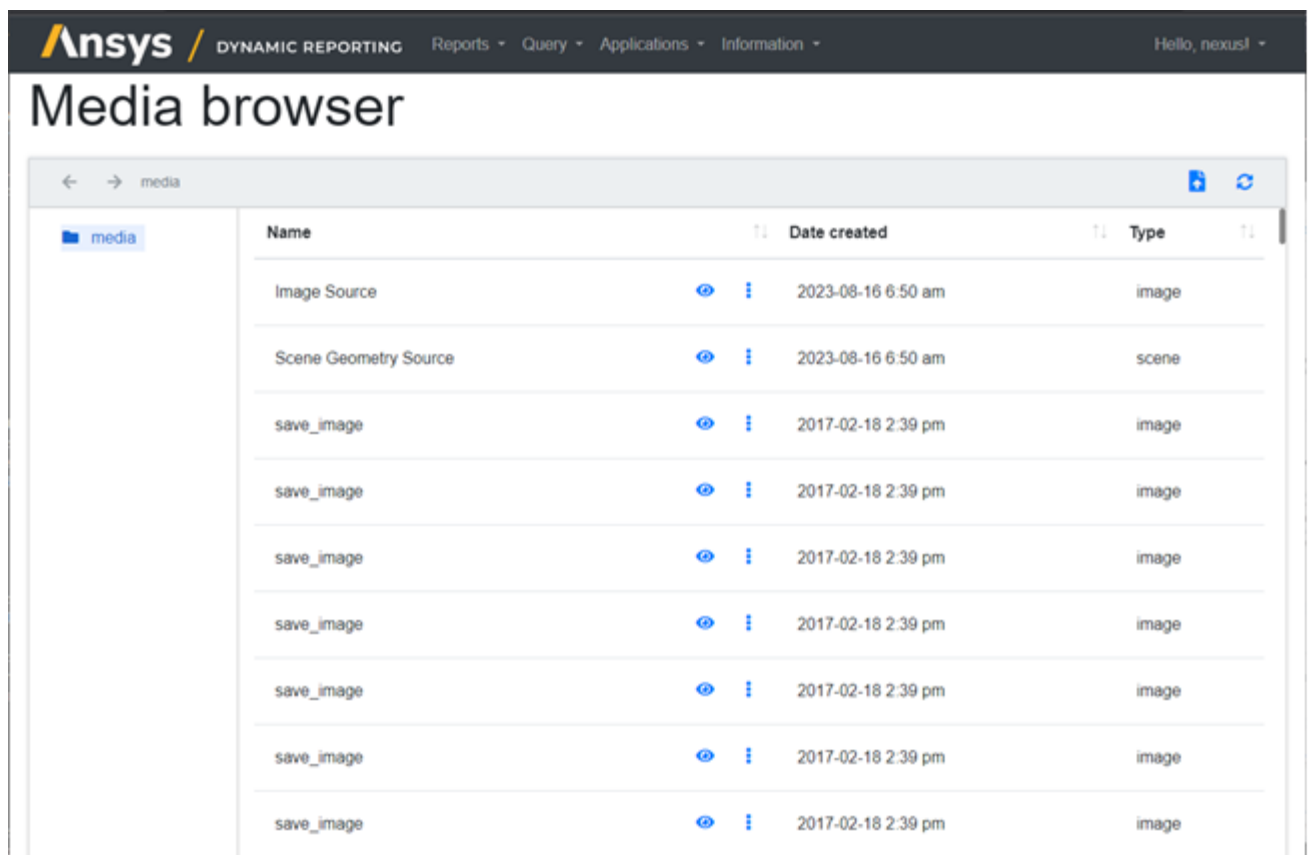
To open the Media Browser:

1. Ensure you are [logged into the web interface \(p. 30\)](#).

- From the menu bar, select **Applications** → **Media Browser**.



The **Media browser** page is displayed.

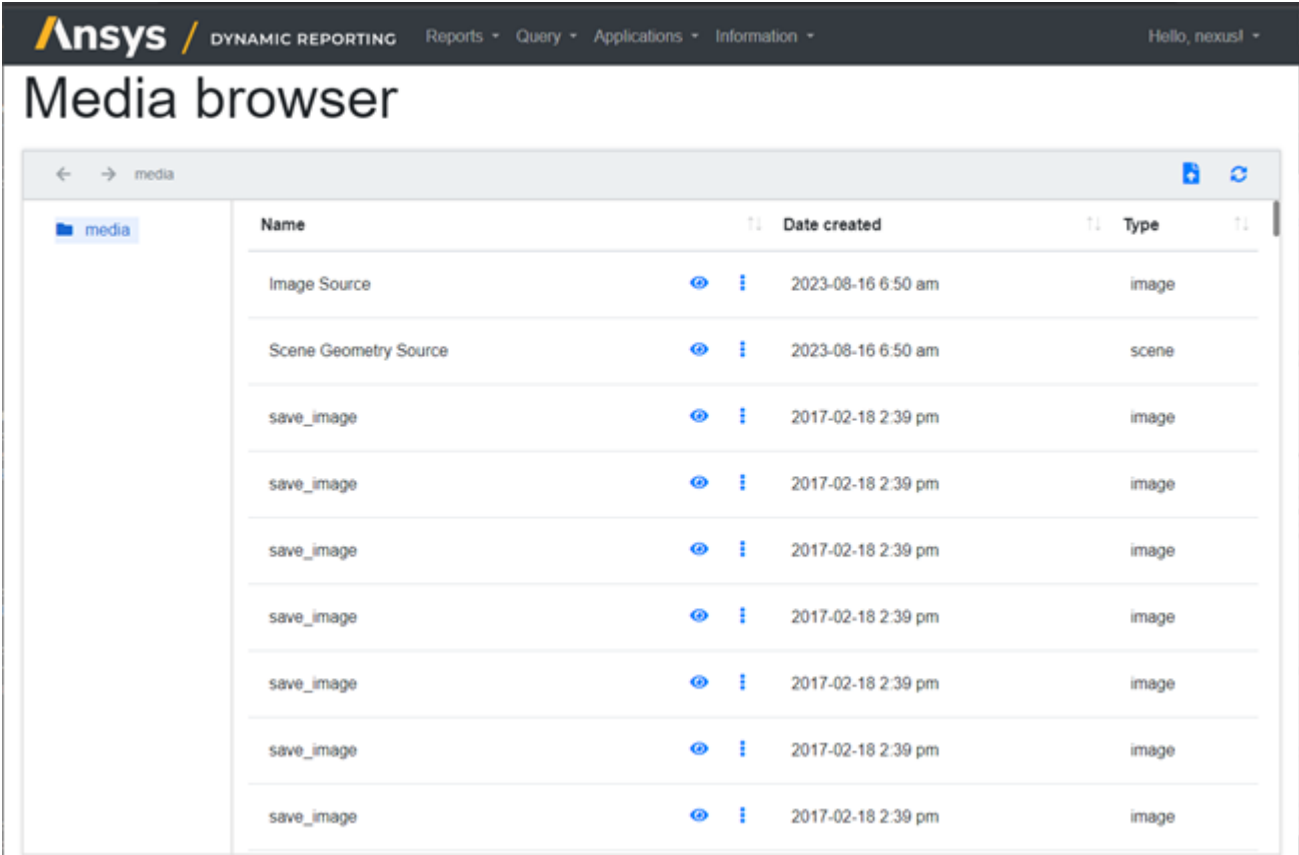


Related topics:

- [Listing Files](#)
- [Managing Existing Files](#)
- [Uploading Files](#)
- [Managing Folders](#)

Listing Files

The most basic feature provided by the Media Browser is the file listing. It acts as a single point to view all stored physical media files. Currently, only local storage is supported which means these files are loaded from the path specified by the CEI_NEXUS_LOCAL_MEDIA_DIR environment variable, which is the root **media** folder. By default, all available media files are listed under the media folder.



Click the **Refresh** () button in the top-right corner to display the most recent list of files.

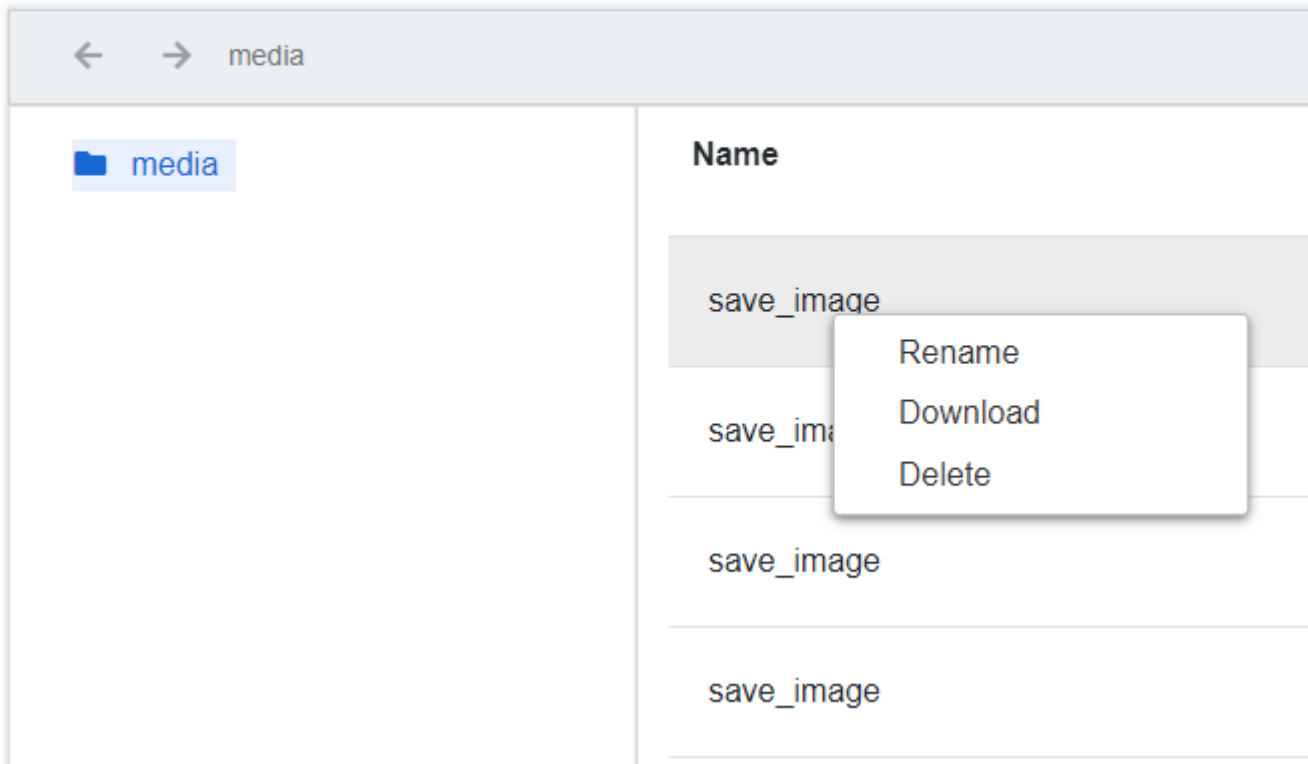
To view the details of a file, click the **View** () button next to the file name. The details page opens in a new tab.

Managing Existing Files

Media files can be renamed, downloaded or deleted using the context menu.

1. To open the menu, do one of the following:
 - Right-click a file name.

Media browser



- Click the  menu to the right of the file name.



2. Select one of the following options:

- **Rename**

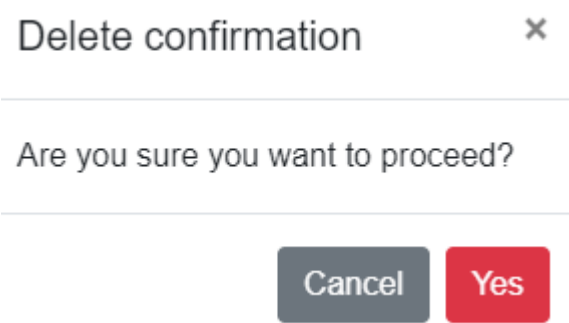
Renames the file. Type a new name in the field and press **Enter** to save the change.

- **Download**

Downloads the file onto your local machine.

• **Delete**

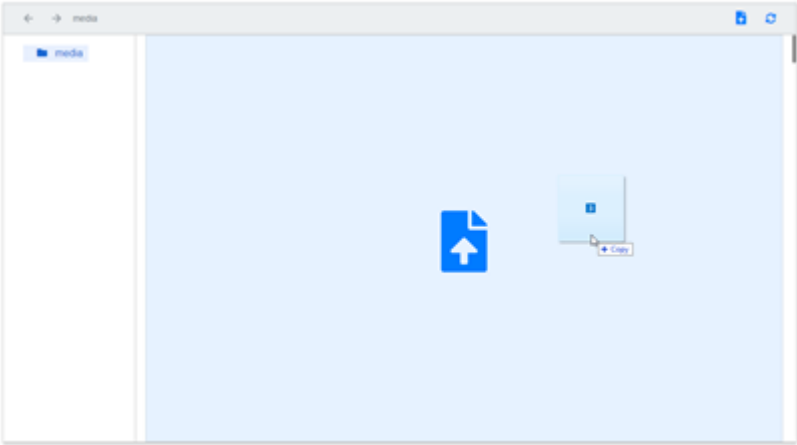
Deletes the file from the physical storage and refreshes the file list. Select **Yes** in the confirmation dialog to proceed.



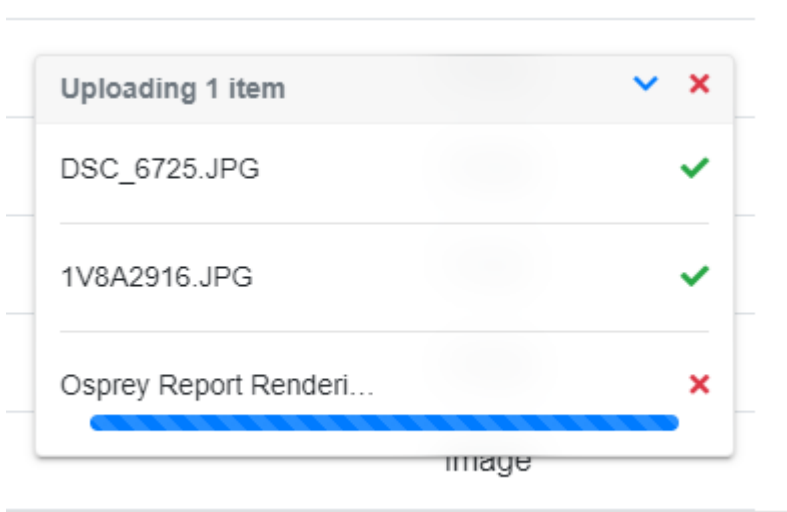
Uploading Files

Just like any other cloud storage provider, the Media Browser provides the ability to upload media files. The destination of these uploads is wherever you choose. Currently, only local storage is supported which means these files go to the path specified by the CEI_NEXUS_LOCAL_MEDIA_DIR environment variable, which is the root **media** folder.

- 1. To upload one or more files, complete one of the following procedures:

Option	Description
Manual	 1. Click the Upload () button in the top-right corner. 2. In the dialog, navigate to the files you want to upload, select them, and click Open.
Drag-and-drop	Drag the files to upload onto the right pane containing the file list. 

A dialog opens, showing the upload progress.

**Note:**

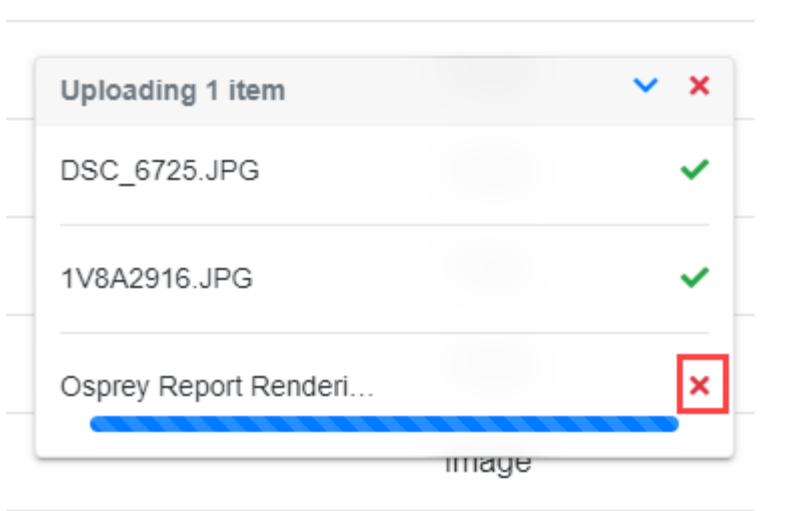
The default maximum upload size is 5000 MB. Use the Ansys Dynamic Reporting Server launcher [user interface](#) (p. 13), [command line tool](#) (p. 309), or the [launch_local_database_server\(\)](#) Python API to change the maximum size.

The NEXUS_MAX_UPLOAD_SIZE environment variable is no longer supported.

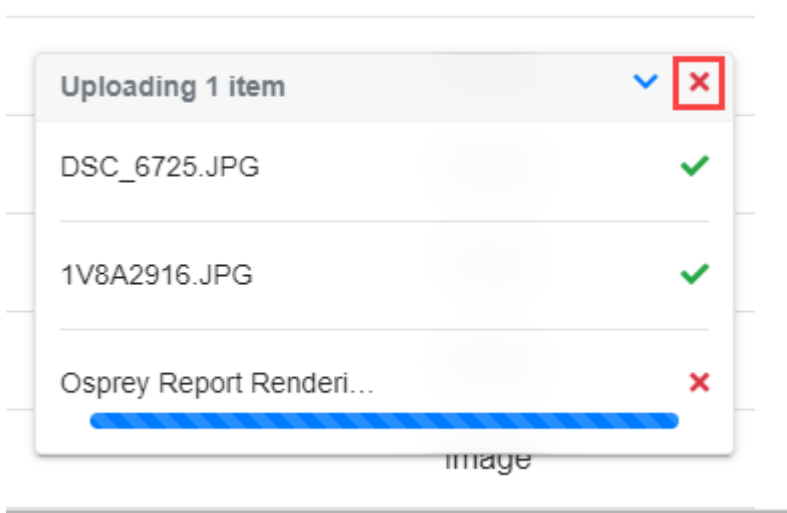
2. You can upload more files as the current upload is in process. To do so, repeat the previous step.

The order in which you add the files does not matter. Files are uploaded asynchronously and the smaller files take less time than the larger ones. As the files finish uploading, the file list will refresh and the newly uploaded files will appear in the list.

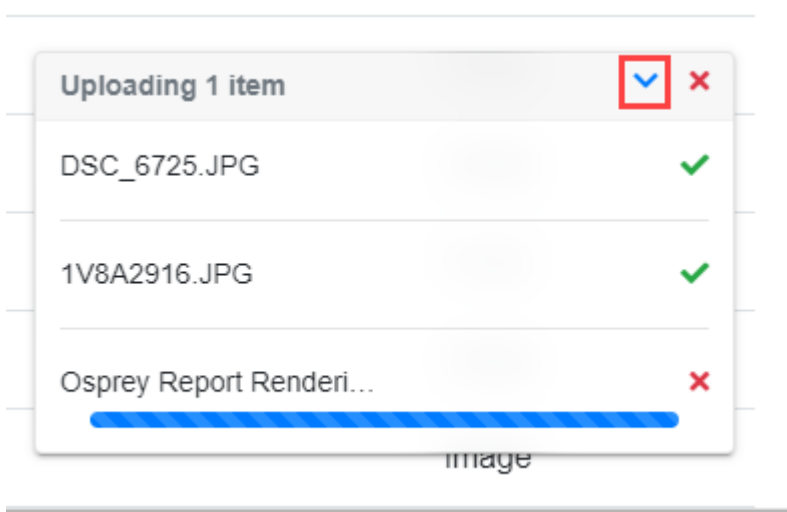
3. To cancel a file upload in progress, click the red X beside the file name.



4. To cancel all of the file uploads in progress, click the red X on the top-right corner of the dialog



- To minimize the dialog, click the blue arrow to collapse or expand it.



Managing Folders

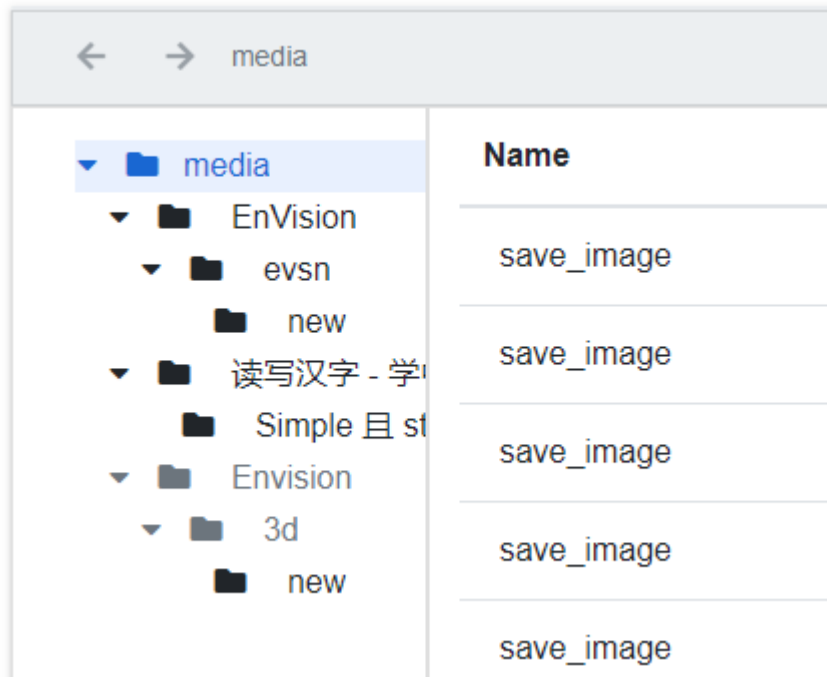
Folders in the Media Browser are a logical way to organize files and behave differently than physical folders on your hard drive. In order to enable folders, you must enable [Access Control Lists](#) (p. 258). When enabled, any list of files that is displayed is filtered by user permissions.

Unlike physical folders, creating a folder will not create one in your physical storage. Instead, the existing concept of [item categories](#) (p. 267) is reused. Each folder is an item category that is stored in the database.

When you click a folder, all the files stored in it will be displayed (all the items linked to that item category will be listed). Since an item can belong to more than one category, a file can also be linked to more than one folder.

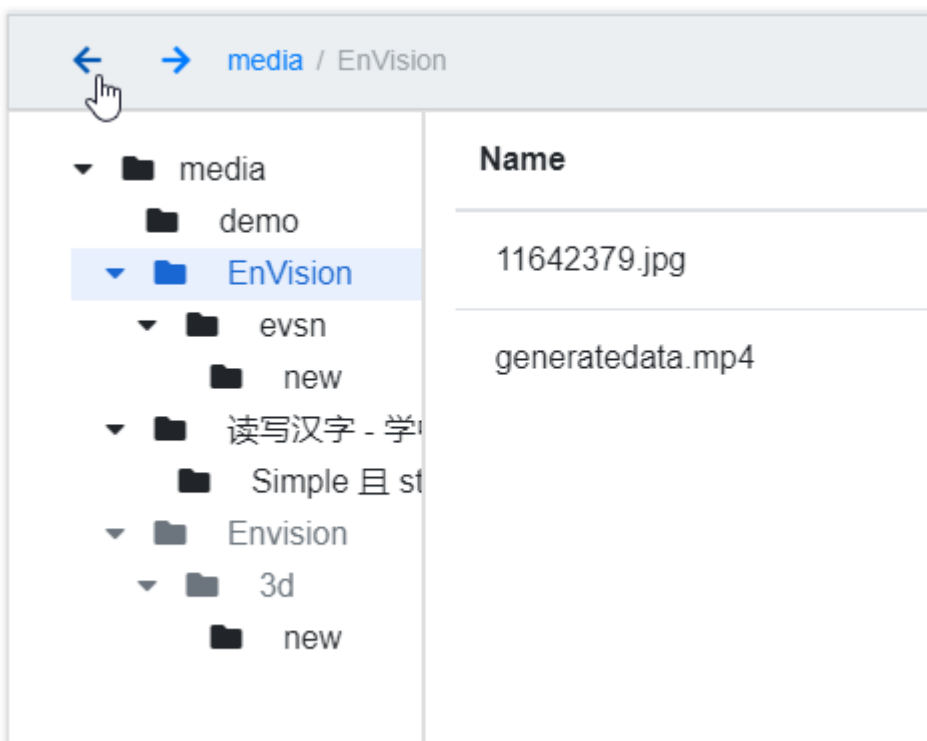
Instead of listing all available media files, the **media** root folder will only list files that do not belong to a folder (do not have an item category assigned to them).

Expand the folder tree by clicking on the caret next to the **media** root folder.



Click any folder to view its contents.

Just like any other file explorer, your history of navigation is tracked and saved. You may go back or forward in your history using the **Forward** and **Back** buttons.



To jump to a folder in the current path, click the breadcrumb link.

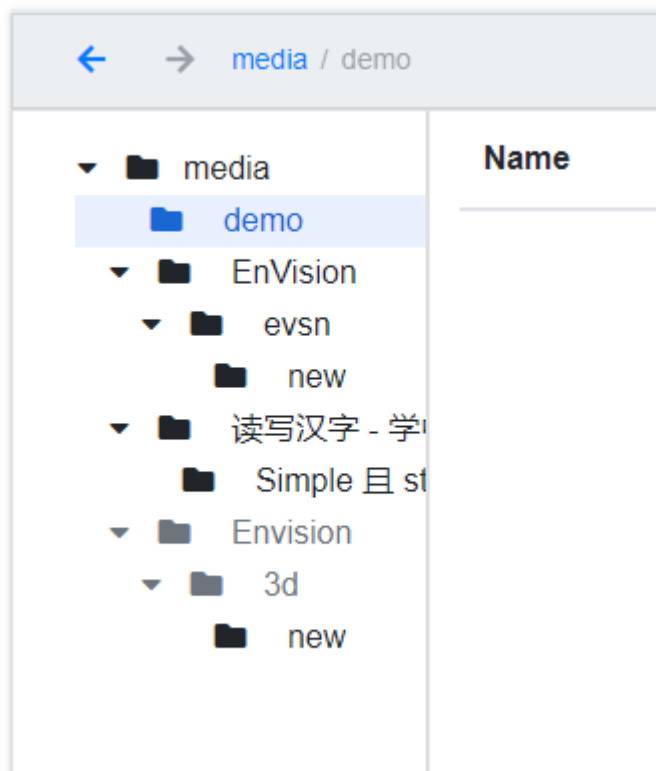


Folder names must adhere to the following naming conventions

- Allowed characters are a-z, A-Z, 0-9, '.', '_', and '-'
- They may contain an optional / or space in between
- International strings are allowed

If the name is invalid, you will see a warning when the folder tree is loaded and the invalid ones will be removed from the tree.

Slashes in a folder name are treated as a special case. A folder (or category) name such as **Envision/3d/new** will be parsed into a tree and displayed as shown in the image below. The actual folder that is displayed is **new** (a leaf of the tree) which corresponds to the category **Envision/3d/new**. Since this leaf is the actual folder, the remaining folders **Envision** and **3d** are non-existent and are grayed out. To make these into actual folders, you will have to create two more: **EnVision** and **EnVision/3d**. Folders may also be grayed out if you do not have proper permissions on them.

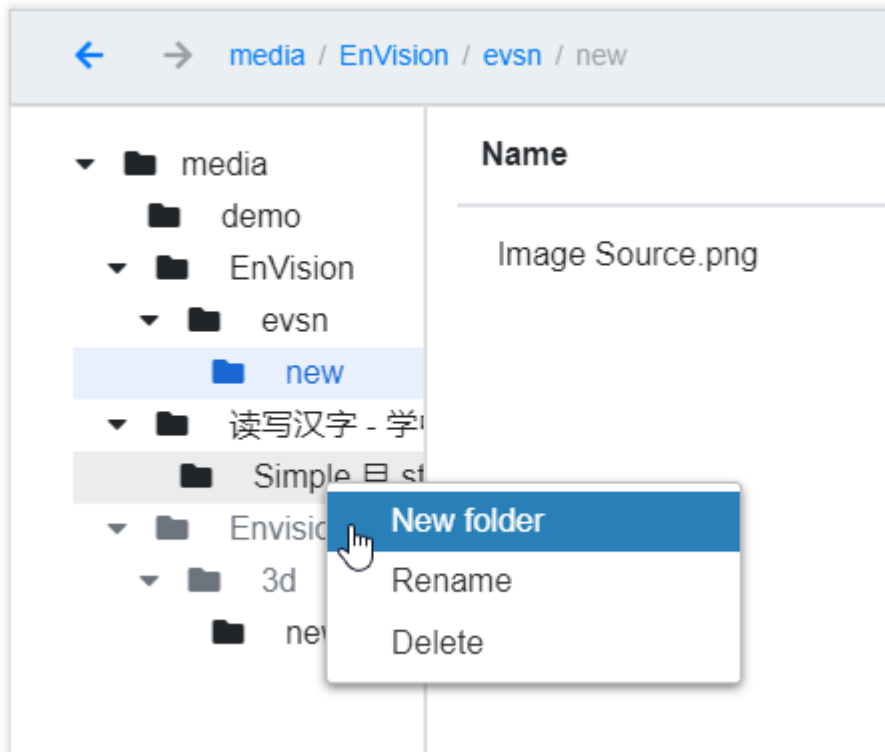


Creating a New Folder

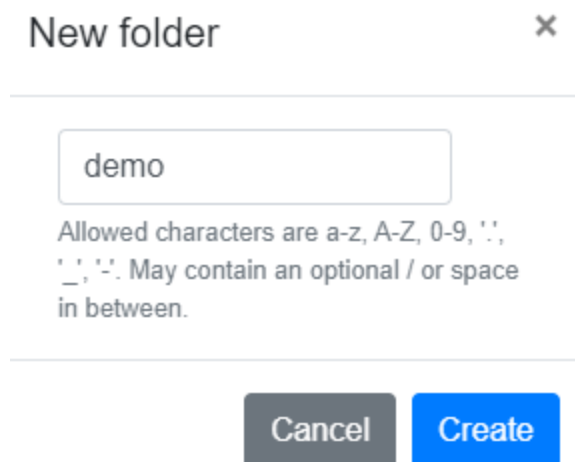
To create a new folder:

1. Perform one of the following options:

- In the top-right corner click **New folder** ().
- To insert a new child folder, right-click a folder and select **New folder** from the context menu.

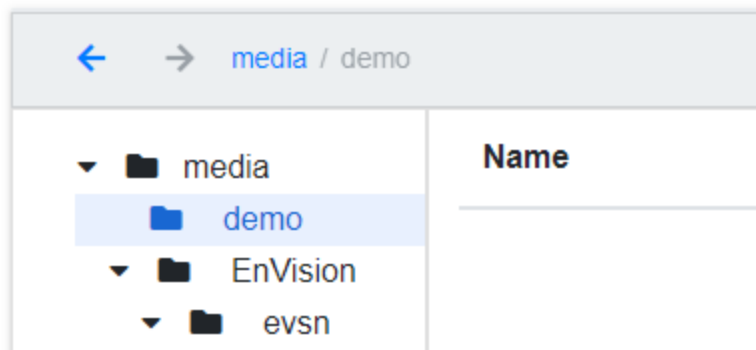


2. In the **New folder** dialog, enter the folder name.



3. Click **Create**.

A new folder is either created at the top most level, as a child to the root **media** folder, or inserted below the selected folder, depending on which option you selected in step 1.



Adding Files to a Folder

To add files to an existing folder:

1. Navigate to the folder by clicking it in the folder tree, using the **Forward** and **Back** buttons, or clicking a breadcrumb link.

2. Click the **Upload** () button in the top-right corner.

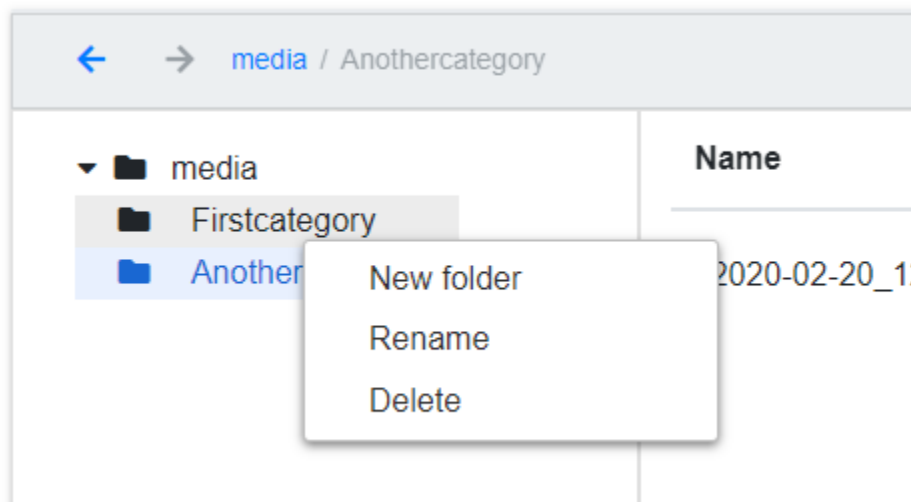
3. In the dialog, navigate to the files you want to upload, select them, and click **Open**.

The uploaded files are linked to the currently clicked/highlighted folder, known as the **current working folder**. They are given the permissions of the parent folder (the folder they are uploaded to).

Managing Existing Folders

Folders can be renamed or deleted using the context menu.

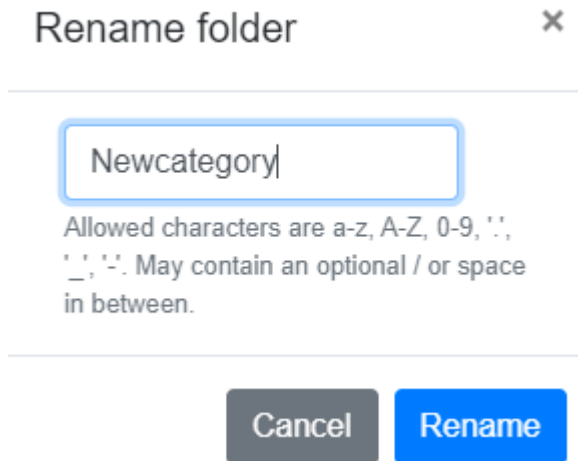
1. Right-click the folder to open the context menu.



2. Select one of the following options:

- **Rename**

Renames the folder. Type a new name in the field and click **Rename** to save the change.



Rename folder ×

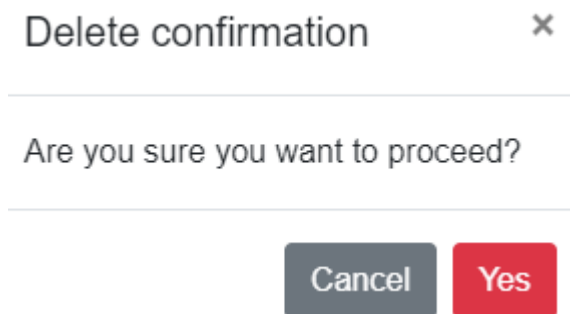
Newcategory|

Allowed characters are a-z, A-Z, 0-9, '.', '_', '-'. May contain an optional / or space in between.

Cancel Rename

- **Delete**

Deletes the selected folder and its child folders. A deleted folder's files are moved up to the root **media** folder and are not physically deleted.. Select **Yes** in the confirmation dialog to proceed.



Delete confirmation ×

Are you sure you want to proceed?

Cancel Yes

Managing Folder Permissions

Since folders are just item categories, all four types of permissions (*own*, *view*, *change*, and *delete*) are all available for folders as well. These can be edited from the [Manage Item Categories page \(p. 267\)](#). Using these, you can simulate folder permissions.

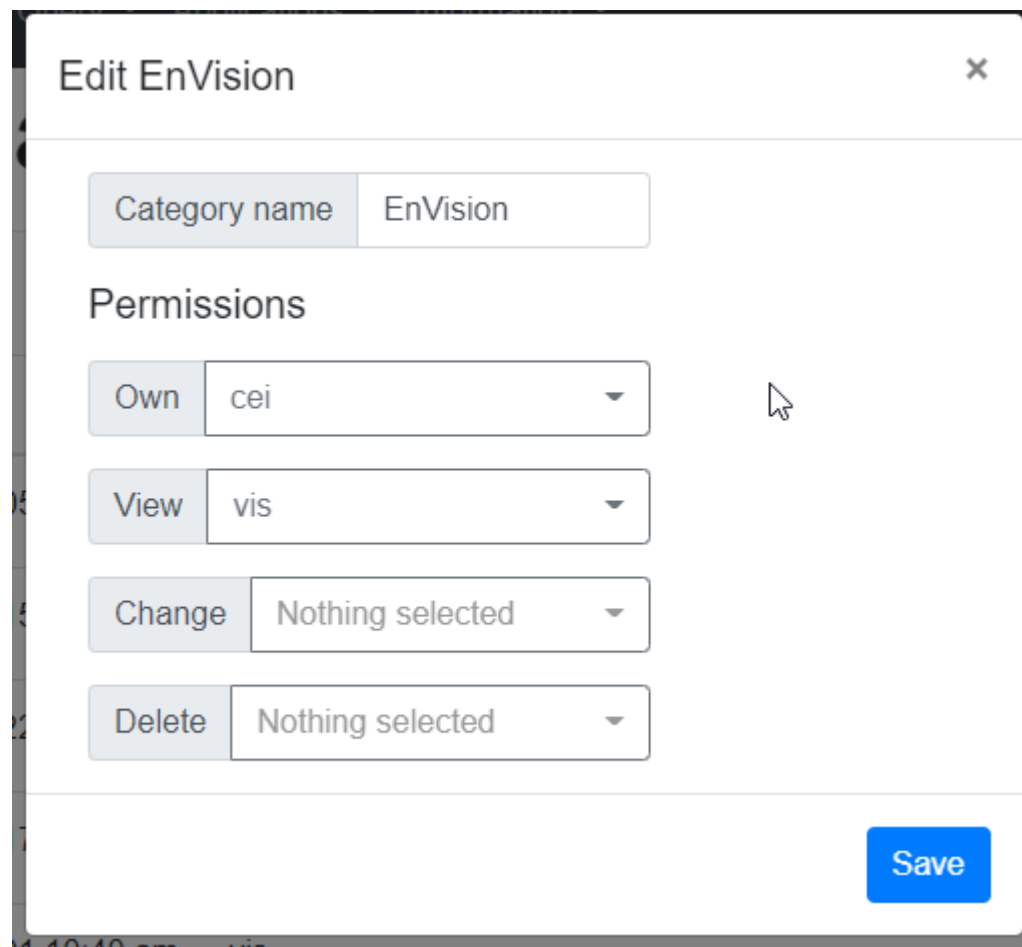
For example, if you want to give another user **read** permissions on folder **demo**, you would give **view** permissions to their group for the **demo** category. For *write/upload* and *rename* permissions, use the *change* permission. For *delete* permissions, use the *delete* permission. *Own* permissions can be used to give all three permissions at once.

One major difference between folder and item categories is that generally with categories, you can assign items to a category only if you have *own* permissions on the category, but for the Media Browser, you can upload files to a folder (assign items to a category) if you have either *change* or *own* permissions on the folder. This is to simulate write permissions of a physical folder.

If you do not have certain permissions on a file or folder, the context menu option will be grayed out to prevent the operation.

Example

On the **Manage item categories**, give view-only permissions to the group **vis** for the **EnVision** category.

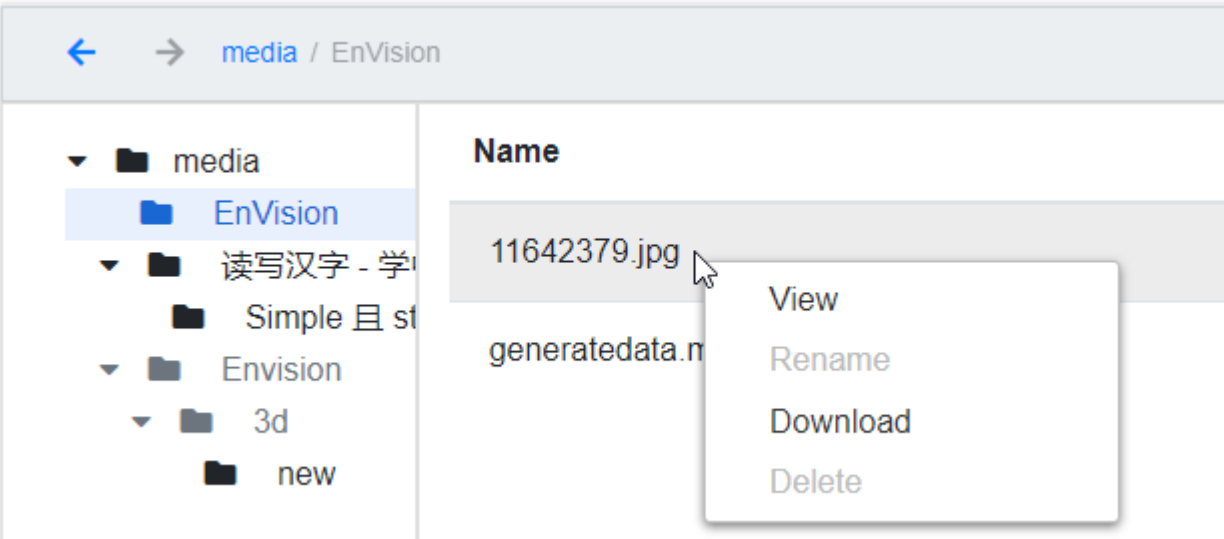
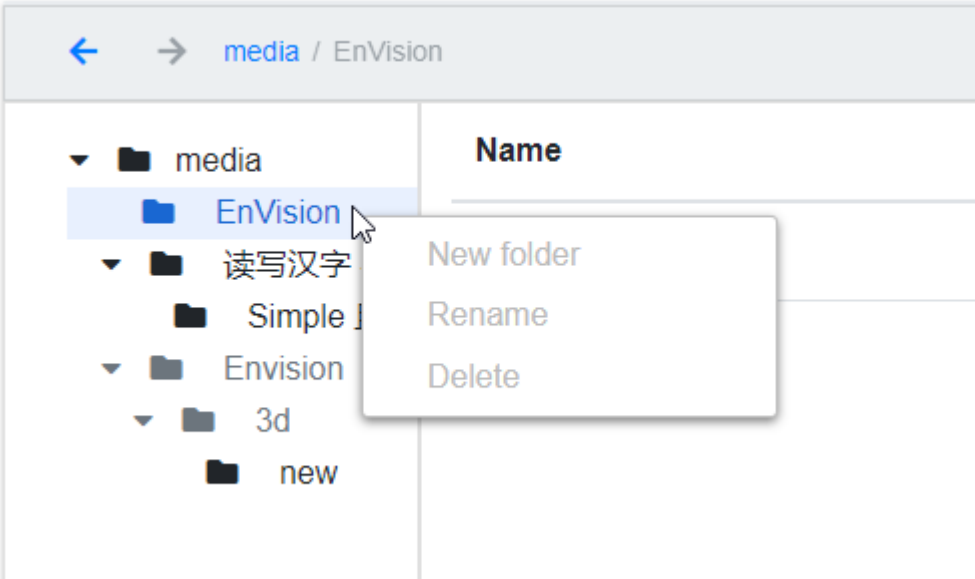


The screenshot shows a dialog box titled "Edit EnVision" with a close button (X) in the top right corner. Inside the dialog, there is a section for "Category name" with a text input field containing "EnVision". Below this is a section titled "Permissions" with four rows of controls:

- Own**: A dropdown menu with "cei" selected.
- View**: A dropdown menu with "vis" selected.
- Change**: A dropdown menu with "Nothing selected" displayed.
- Delete**: A dropdown menu with "Nothing selected" displayed.

A blue "Save" button is located at the bottom right of the dialog box.

As a result, all permissions except *read* are unavailable for both the **EnVision** folder as well as all files linked to that folder when logged in as the user.



Generating Reports

A report is generated by hierarchical selection of a set of data items that are passed through a hierarchy of report templates. You select a root template, and a collection of filters, that then selects the items in the database and displays them in the report. The database is queried and the list of items is passed to the root template. That template may perform additional filtering, data processing or generate specific layout constructs before passing the items on to its child templates. If a template has no children, it displays the items using the properties, context, and layout provided by the parent templates.

Related topics:

- [Report Generation Overview](#)
- [Creating and Editing Templates](#)
- [Report Components and Properties](#)
- [Data Items](#)
- [Layout Templates](#)
- [Generator Templates](#)
- [Generating Reports Using the Web Interface](#)

Report Generation Overview

Understanding the following key points allows you to fully use the Ansys Dynamic Reporting report generation system.

- Reports are generated from collections of [data items \(p. 91\)](#).
- Report templates define how a report is generated and laid out ([generator \(p. 192\)](#) templates and [layout \(p. 133\)](#) templates)
- Report templates filter, sort, or arrange lists of data items.
- Use the Ansys Dynamic Reporting Template Editor to [create and edit \(p. 74\)](#) report templates
- Data items display themselves within boundaries specified by the layout templates.
- Reports are generated using the [web interface \(p. 29\)](#) and display in web pages.

The following workflow defines the common report generation steps:

1. [Launch \(p. 13\)](#) Ansys Dynamic Reporting, either with a new or existing database.
2. Use one of the following methods to place items in the database:
 - [EnSight Python Ansys Dynamic Reporting API \(p. 292\)](#)

- [External Python API](#)
 - [Upload files \(p. 57\)](#)
3. Review the various items in the database in a web browser.
 4. Use the Ansys Dynamic Reporting Template Editor to [define \(p. 74\)](#) a hierarchical collection of templates.
 5. [Generate \(p. 246\)](#) specific reports, displaying (and interacting with) the resulting content in a web browser.

Creating and Editing Templates

At the root of every report is a top level report template. This template is handed a collection of data Items which it processes itself or hands off to child templates. At each template the following operations are performed:

1. Filter the incoming items, potentially removing some items from the list or perform a direct query of the database, merging the new items with the incoming items or replacing them.
2. Sort the items using various data item fields.
3. After sorting, you can choose to restrict the item list down to only the first or last item (by default, all items are retained).
4. Set some properties that should be applied to all of the items.
5. Begin a layout for how items should be displayed. This will generate an HTML block that organizes and lays out the HTML generated in steps 6 and/or 7.
6. If the report template has child report templates, pass the filtered item list to each of the report template children in order. Each child layout displays its results inside of the parent layout.
7. If the report template has no report template children, draw each of the items in the current layout. Each data item provides an HTML representation of itself that is placed sequentially within the block generated in step 5 according to the template layout rule (for example, two columns).

The sequence noted above is applied recursively (step 6) until all of the child report templates have been visited. The resulting HTML is displayed in the web browser.

There are two very distinct classes of templates, the [layout \(p. 133\)](#) and [generator \(p. 192\)](#) template. There are no restrictions on parent/child relationships between layout and generator templates, other than the fact that the hierarchy must represent a non-looping tree of templates (an acyclic graph).

Related topics:

- [Starting the Ansys Dynamic Reporting Template Editor](#)
- [Creating a New Template](#)
- [Editing Templates](#)
- [Copying Data Items and Report Templates Between Databases](#)
- [Exporting a Template to Python API](#)

Starting the Ansys Dynamic Reporting Template Editor

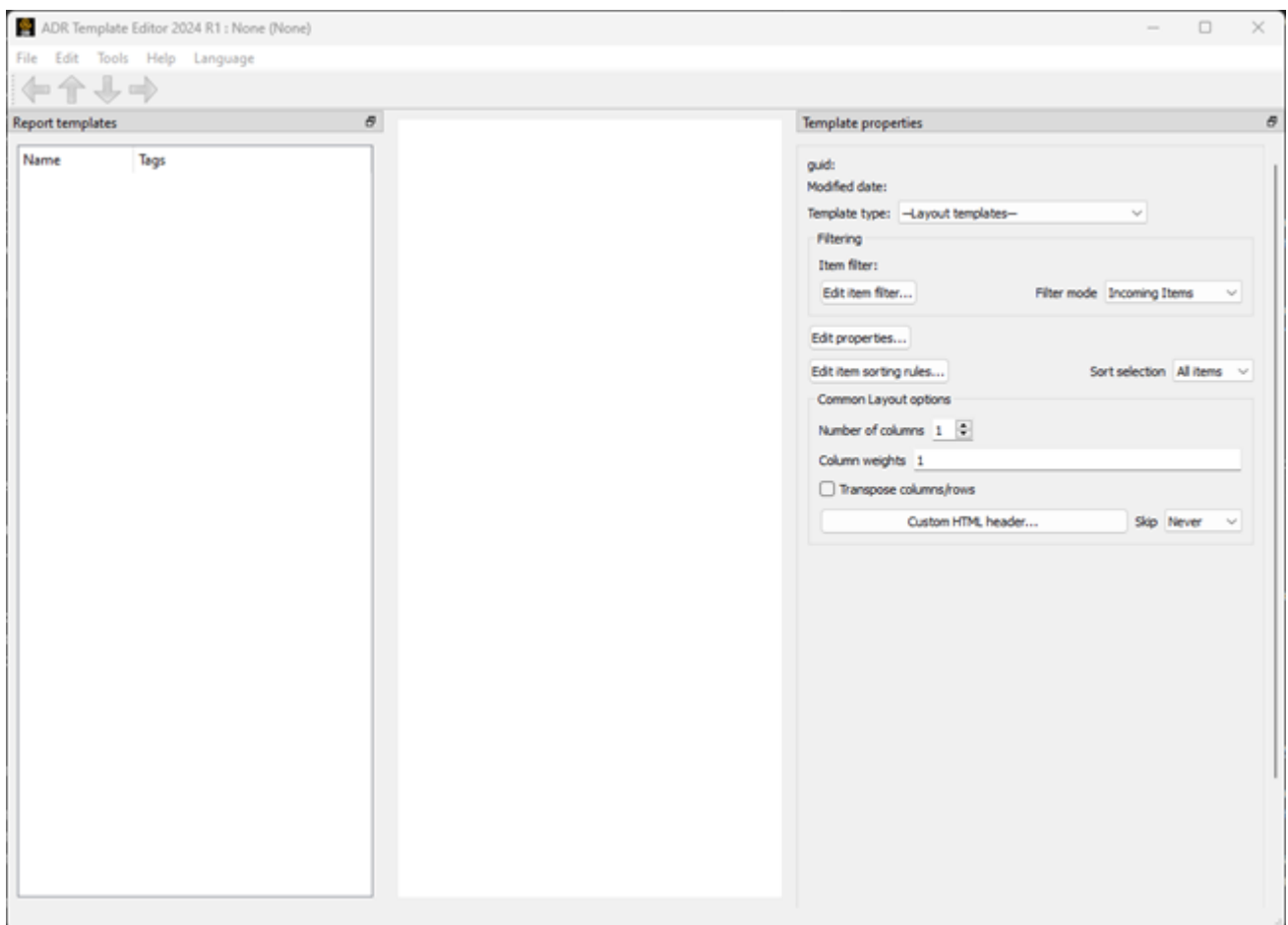
The Template Editor allows you to perform a number of operations. These include:

- [Creating \(p. 18\)](#) a new database
- [Starting \(p. 18\)](#) and stopping a local database
- [Creating \(p. 76\)](#) report templates
- [Editing \(p. 79\)](#) report templates
- [Copying \(p. 79\)](#) report templates and data items between servers

To start the Template Editor:

1. Open a command prompt/shell, and navigate to `<Ansys installation path>\CEI\bin`.
2. Type **adr_template_editor**.

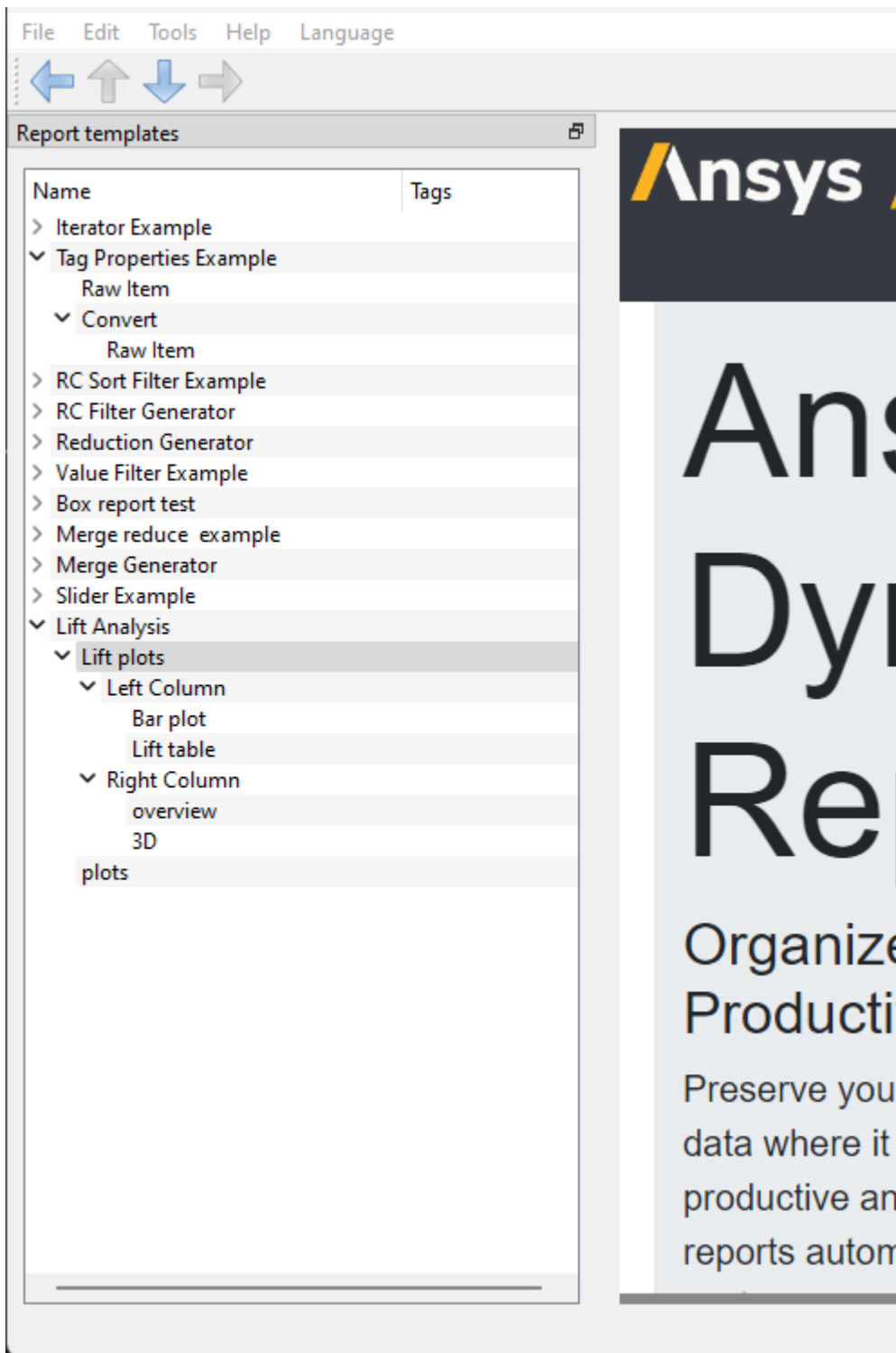
The template editor opens.



You can now use the **File** menu to connect to a running server. Or you can use the **Tools** menu to launch a new server. Once a connection has been established, you can create/edit report templates and copy data/templates from another database into this database.

Creating a New Template

The most common operation performed in the Template Editor is to create and edit report templates. When a connection is made with a database, a list of all of the (top-level) report templates and their associated tags appear in the report template editor left-hand pane.



To create a new template:

1. Determine where in the **Report templates** list you want the new template to be placed. If you want a top-level template, do not select anything. Otherwise, expand the template list and select the template you want to create a child template for.
2. From the menu bar, select **File** → **New report template**.

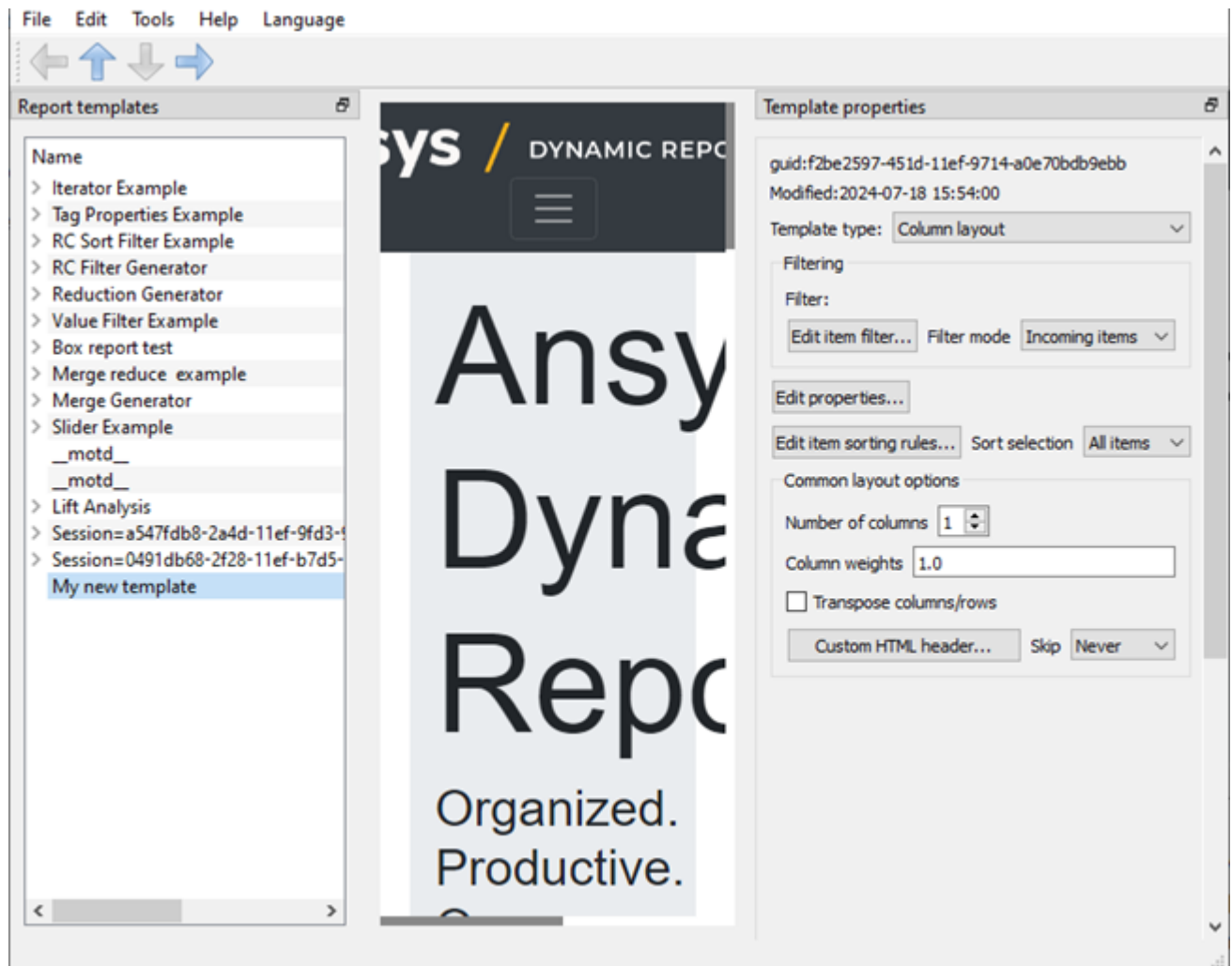
Your new template is added to the list as **New Template**. It is located either at the top level and not indented (a parent template) or indented under the template you selected previously (a child template).

3. If required, use the navigation arrows, or click-and-drag, to move the template to another location in the list.



4. Double-click **New Template** and type the template name. Press **Enter** to save the change.
5. In the **Template properties** pane, select the template type and set the properties as required.

For more details on each template type and properties, see the [layout \(p. 133\)](#) and [generator \(p. 192\)](#) template descriptions.



6. To save the changes, from the menu bar, select **File** → **Save templates**.

- For top-level templates, to view the report generated by this template, from the menu bar, select **File** → **View selected report**.

Editing Templates

You can edit existing templates from the **Report templates** list. You can:

- Use the navigation arrows, or click-and-drag, to move the template to another location in the list.



- Double-click the template name and enter a new one. Press **Enter** to save the change.
- In the **Template properties** pane, change the template type and/or properties as required.

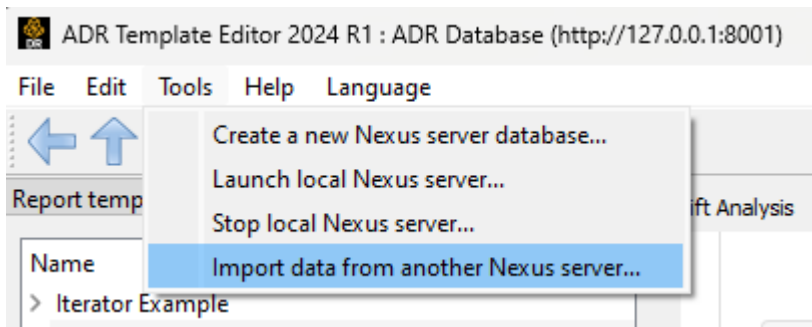
For more details on each template type and properties, see the [layout](#) (p. 133) and [generator](#) (p. 192) template descriptions.

To save the changes, from the menu bar, select **File** → **Save templates**.

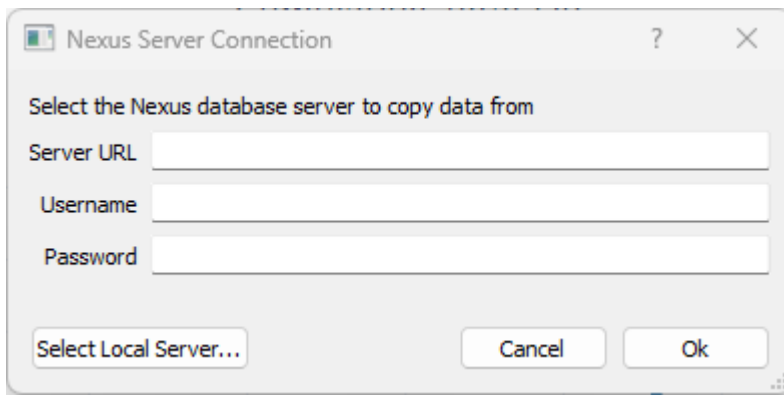
Copying Data Items and Report Templates Between Databases

Once you establish a connection to an ADR server, you can copy data items and report templates from another ADR server into the server the template editor is connected to (the server listed in the window title bar).

- From the menu bar, select **Tools** → **Import data from another Nexus server**.

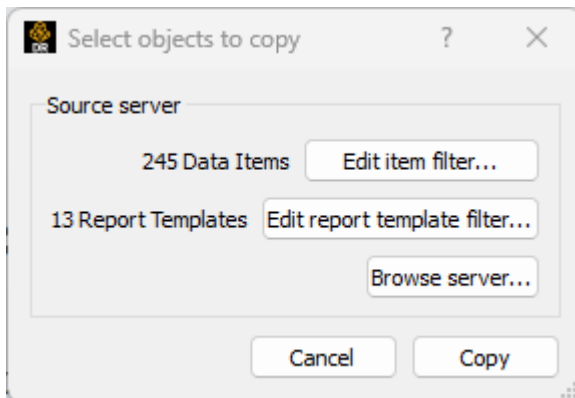


The connection dialog is displayed.



2. Enter the connection information for the source server (the server currently connected to the template editor is always the destination for the data).

The source server is contacted and by default, all data items and report templates on the source server are selected. The details are displayed in a dialog.



3. If required, click **Edit item filter** and/or **Edit report template filter** to specify data item and report template filters, narrowing the collection of items to be copied.
4. Click **Browse server** to open a web browser pointed at the source server, allowing you to review and explore potential selections.
5. Once you have set all the required filters and finalized your selection. click **Copy**.

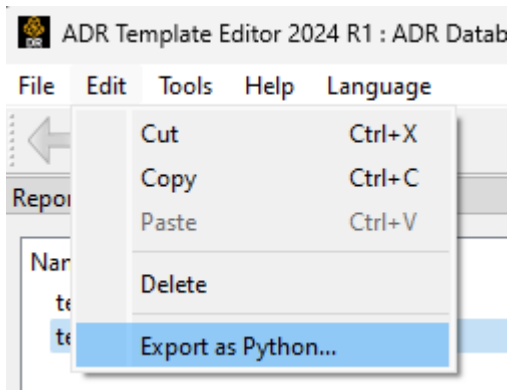
This operation can take a significant amount of time and progress dialogs illustrate the various stages and completion status.

When the operation is complete, the selected data items and/or report templates exist in both databases. If identical items/templates exist in the destination database, they are overwritten by the imported items.

Exporting a Template to Python API

You can create and/or modify a report template using the [External Python API](#). The Template Editor allows you to automatically generate the Python commands that create an existing template. You can use this feature to create Python scripts that automatically generate report templates.

1. In the **Report templates** list, select a template.
2. From the menu bar, select **Edit** → **Export as Python**.



3. In the dialog, browse to a Python file to export the commands into, select the file, and click **Save**.

The commands are exported into the selected file.

Note:

The Python commands to generate only the selected template are generated. When you select a template that is a child template, the generated script describes how to create its root template and all its children.

The resulting file will contain the following:

- Section to connect to the running server (assumes a running server)

```
import ansys.dynamicreporting.core as adr
adr_service = adr.Service(ansys_installation=r"C:\Program Files\Ansys Inc\v251")
adr_service.connect(url="http://127.0.0.1:8000", username="nexus", password="cei")
server = adr_service.serverobj
```

- Creation of the root template

```
template_0=server.create_template(name="Top Level", parent=None,
report_type="Layout:panel")
server.put_objects(template_0)
```

- Creation of child templates

```
template_1=server.create_template(name="Chapter 1", parent=template_0,
report_type="Layout:basic")
template_1.params={'column_count': 2, "column_widths": [1.0, 1.0]}
server.put_objects(template_1)
server.put_objects(template_0)

template_4=server.create_template(name="Table", parent=template_1,
report_type="Generator:tablemerge")
template_4.params={'merge_params': {"column_labels_as_ids": 1,
"transpose_output": 0, "force_numeric": 1,
"merge_type": "row", "column_merge": "all",
"column_id_row": "0", "collision_tag": ""},
```

```
        "unknown_value": "nan", "table_name": "merged table",
        "source_rows": "\\\"Time|merge\\\"",
        "properties": {"plot": "table"}}'
template_4.set_filter("A|i_type|cont|table;")
server.put_objects(template_4)
server.put_objects(template_1)
server.put_objects(template_0)

[...]
```

Note:

Every time a child template is created, it is pushed together with all its parent templates.

Report Components and Properties

Ansys Dynamic Reporting supports many different properties and components in the reports, such as indexing and macros.

Related topics:

- [Common Report Properties](#)
- [Property Macro Substitution](#)
- [LaTeX Support](#)
- [AsciiMath Support](#)
- [Message of the Day](#)
- [Indexing Options](#)

Common Report Properties

When an item is displayed, it is always in the context of a report template. The report template supports a number of properties that control how the item should be presented in a report. These properties are collectively referred to as the display context. Each data item type supports a different set of properties that can be used to control the specific display of a data item. For more details, see [Data Items](#) (p. 91)

Context properties are inherited through the report template hierarchy. For example, if a top level layout template set the table Item property `title` to **HELLO**, all table items whose visual representation is generated by that template or a child template will have their `title` property set to **HELLO**, potentially overriding the specific value placed on the table data Item. This can be overridden by a child template which might set the `title` property to **GOODBYE**. In any case, contexts are inherited from parent report templates and changes in the context only effect children below the current template.

There are a number of common report properties that are specified implicitly for every report. These include:

Property	Value
<code>date_date</code>	The current date in the system locale representation.

Property	Value
date_datetime	The current date and time in the system locale representation.
date_iso	The current date in ISO 8601 representation.
date_year	The current year.
pagebreak	Defaults to 0. If set to 1, a hint is included in the generated HTML that suggests that the browser try not to put a page break within the content inside of the template. In practice, this tends to result in a page break being inserted before the layout content when the HTML is printed, if the content would have extended into the next page.
page_col_pixel_width	1/12 of a page in pixels. Core columns are laid out in 1/12 page increments.
page_dpi	An estimate of the number of pixels per inch of the output page.
page_number	The current report page number.
page_width	The width of the page in inches.
template_name	The name of the current template being processed

The following properties can be useful when debugging templates. These provide the report designer with additional insight into the operation of template generation, property contexts and so on.

Property	Value
item_context_debug	If set to 1, the system inserts the GUID, the current list of properties, and the item tags into the HTML output whenever an item is rendered. This can be useful when tracking down context/property management issues.
template_timing_debug	If set to 1, the system reports timing information collected when report templates are executed. The output includes: the total time spent running the template (includes filtering, sorting, and children), the amount of time spent executing data item filters in this specific template, and the amount of time spent executing the data item sort in this specific template.

Property Macro Substitution

Many data items and report templates types support macro substitution using properties. For example, if the property Hello is set to **How are you?** in the report context, and an HTML item contains the text `<h1>Hello is set to: {{Hello}}</h1>`, the resulting page will render Hello is set to: How are you? with heading 1 formatting.

Macros can also be used in properties themselves and in that situation, the tags on the current item can also be included in the macro. For example, if the [table item property \(p. 106\)](#) plot_title is set to: My {{my_tag}} Can Use Macros and the table item being rendered has the tag my_tag='Cool Title' set on it, the title for the plot is My Cool Title Can Use Macros.

Tags on the item's associated session and dataset objects can be used. To access the value of a session tag, use the syntax: {{s/session_tag_name}}. For a dataset tag, the syntax is {{d/data-set_tag_name}}. For example, if the dataset associated with an data item has the tags variable=pressure X=1, the macro string {{d/variable}} {{d/X}} returns pressure 1.

Fields on data items, sessions, and datasets can also be accessed. The syntax `{{if/data_item_field}}`, `{{sf/session_item_field}}` and `{{df/data-set_item_field}}` are used to access these fields. The following fields are supported:

Item	Field
Data	if/date
	if/name
	if/sequence
	if/source
Session	sf/applica- tion
	sf/date
	sf/hostname
	sf/platform
	sf/version
Dataset	df/dirname
	df/filename
	df/format

Note:

Only one level of macro expansion is supported.

Related topics:

[Explicit Anchors](#)

[Macro Formatting Filters](#)

[Formatting Filter Chains](#)

Explicit Anchors

Ansys Dynamic Reporting allows you to embed links between different portions of the report using embedded explicit anchors and links in the report. This is performed using the `nexus_anchor` and `nexus_link` macros. The `nexus_anchor` macro specifies a named link target. One or more `nexus_link` macros can point to the same named target (the name options to the macros make this link):

```
{{name|nexus_anchor}}  
{{name|nexus_link:"link text"}}
```

Macro Formatting Filters

The macro substitution engine supports generic formatting. Formatting filters are specified using the pipe character (`|`). For example:

```
{{name|floatformat:1}}
```

This attempts to convert the name tag to numeric form and if successful, format the number using the `floatformat` filter. In addition to the built-in Django filters, there are some filters provided by Ansys Dynamic Reporting itself. These include:

- **|simple_sprintf:"{name}"**: Apply simple numeric formatting. If the input value is not a valid number, the value is passed through verbatim. The name string can have the following values:

Format name	Description
floatdotX	X is [0-9] which is the equivalent of printf "%0.Xf".
scientific	The equivalent of printf "%e".
scientificX	X is [0-9] which is the equivalent of printf "%0.Xe".
sigfigsX	X is [0-9] which is the equivalent of printf "%.Xg".

For example, to use scientific notation with four digits after the decimal:

```
{{name|simple_sprintf:"scientific4"}}
```

- **|numeric_comp:"{value},{less},{equal},{greater}"**: Convert the input value to numeric form and compare it to the numeric representation of value. If the input is less than the value, return the less string. If the input is equal to the value, return the equal string. If the input is greater than the value, return the greater string. If any of the less, equal, or greater values are '_', then use the original value.
- **|lexical_comp:"{value},{less},{equal},{greater}"**: Operates the same as numeric_comp, but does not convert the input nor value to numeric form, so all comparisons are done as strings instead of numeric values.

Formatting Filter Chains

Formatting filters can be chained for more complex formatting. You can follow one formatting filter with another. The output of the previous serving as the input for the next. For example:

```
{{value|numeric_comp:"0.,Too low,_,_"|simple_sprintf:"floatdot2"}}
```

This example uses the non-numeric pass-through feature of `simple_sprintf` to customize the display of a number. If the number is greater than or equal to 0.0, the number is passed on to the next filter, otherwise the string `Too low` is passed on to `simple_sprintf`. For a number like 3.452453, the output would be 3.45. For a number like -1.24, the output would be `Too low`.

LaTeX Support

LaTeX formatted strings are supported in many places in Ansys Dynamic Reporting.

General Syntax

In common text, you can use the inline notation by wrapping the LaTeX with the `\` (and `\`) character sequences. For example, a custom HTML header that uses this string:

```
<h2>Inline LaTeX:  $\sqrt{(n - \text{t}|T - \text{early})}$ , see!</h2>
```

is displayed in a panel template header as:

Inline LaTeX: $\sqrt{(n_c(t|T_{\text{early}}))}$, see!

Plot Syntax

Most [table properties](#) (p. 106) that support text for display (`plot_title`, `ytitle`, and so on) also support the use of LaTeX styling for the creation of math formulas. If the table item is displayed as a table, use the general LaTeX syntax. If the LaTeX is part of a chart (pie, line, bar, and so on), the syntax is a little different. The plotter LaTeX syntax is fairly simple, wrap the LaTeX text with **\$** characters. For example, a `plot_title` property with the value:

```
:math:``\alpha_{1c} = 352 \pm 11 \text{ km s}^{-1}``
```

is displayed in the plot as:

$$\alpha_{1c} = 352 \pm 11 \text{ km s}^{-1}$$

AsciiMath Support

Ansys Dynamic Reporting supports displaying equations formatted using the [AsciiMath](#) style. In common text, you can use the inline notation by wrapping the AsciiMath expression with the ``` characters. For example, a custom HTML header that uses this string:

```
<h2>Inline ASCIIMath: ``sum_{i=1}^n i``</h2>
```

is displayed in a panel template header as:

Inline ASCIIMath: $\sum_{i=1}^n i$

Message of the Day

Often there is some reason you would like to inform Ansys Dynamic Reporting users of recent or upcoming changes to an ADR server. To do this, create a message of the day report. In Ansys Dynamic Reporting, there is one reserved report name. If you create a report template and give it the name `__motd__` (two underscores, motd, and two more underscores), that report template is rendered in the Ansys Dynamic Reporting banner, right before the **Learn more** button. This report is not available through the **Reports** menu.

You can have multiple message of the day reports and they are all displayed. These reports do not have items selected beforehand so to display data item content, use the filter options to start a new query directly from the database.

Example

The following front page was generated with a pair of message of the day reports:

Ansys / DYNAMIC REPORTING Reports Query Information Sign in

Ansys Dynamic Reporting

Organized. Productive. Open.

Preserve your simulation results data where it is organized, productive and flexible. Generate reports automatically. Combine custom searches and analytics to provide critical decision support. Share results and analytics with colleagues. Perfect for consortia groups, parameter and longitudinal studies.

A specific report

- + Name 0
- + Name 1 Image Source
- + Name 2 m A grstirroad
- + Name 3 02/13/17 19:25:41
- + Name 4 False

Organized

Data is uniquely tagged by dataset, run, date, unique ID and more so that anything can be found easily.

[Learn more >](#)

Productive

Data flows automatically into the system. No manual babysitting required. Reports can be run automatically or manually.

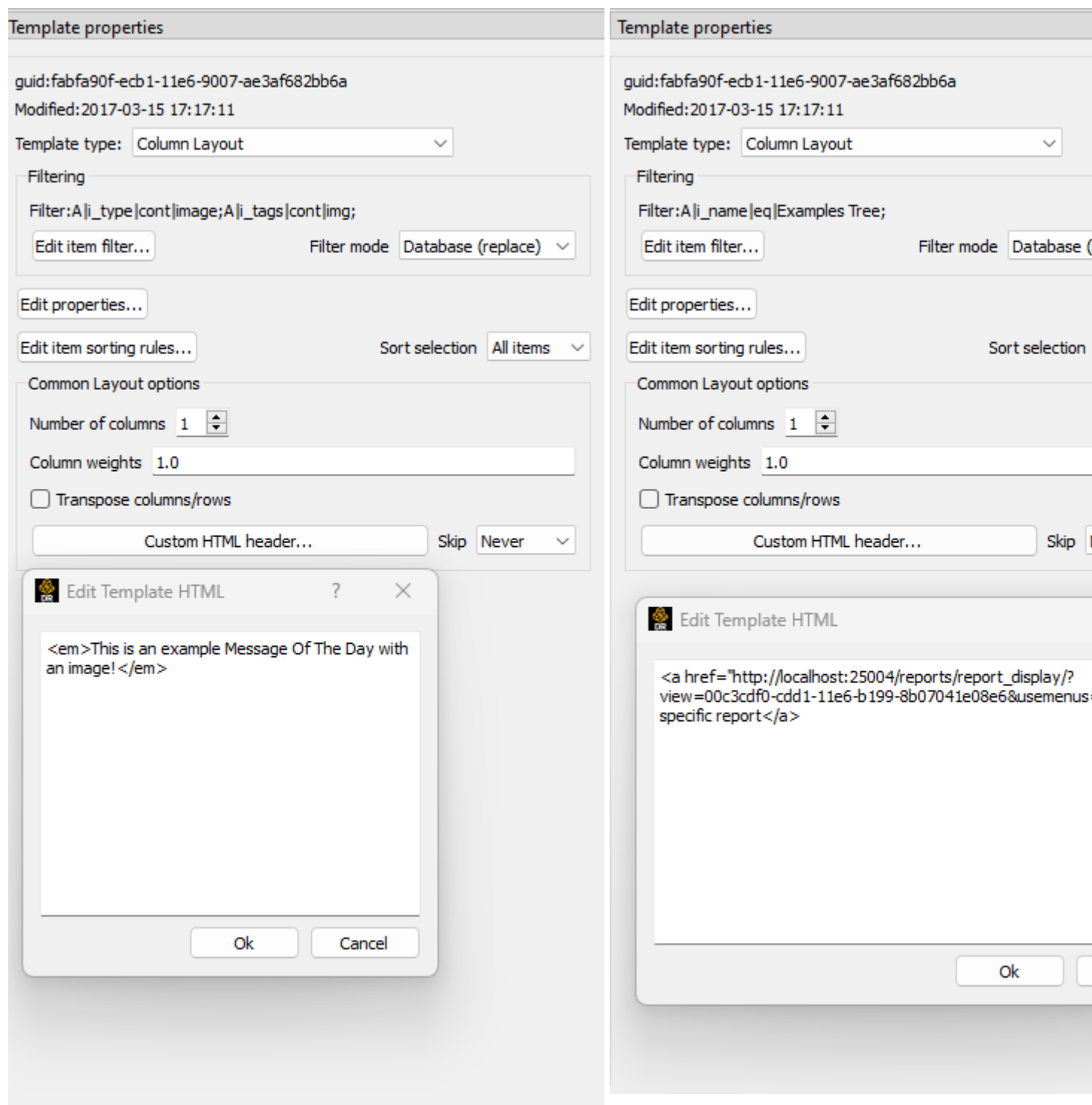
[Learn more >](#)

Open

Use it with data from EnSight or any other application.

[Learn more >](#)

The two reports were configured as follows:



Notice the one on the left does a filter for tree items using the database (replace) mode to bring data items into the report.

Indexing Options

Ansys Dynamic Reporting provides basic support for the dynamic generation of report index features. These features include a table of contents, lists of figures, and lists of tables. The entries for the table

of contents and the lists are collected as the report is generated and can be inserted into the final, output report using the [Table of Contents \(p. 142\)](#) layout template. Entries can be generated for layout templates and for data items.

Properties

The following properties are used to generate the entries for the table of contents and lists. As with all properties, they are inherited by the children of the layout they are applied to. For example, set `TOCItem` on an upper-level template and disable it on lower-level templates for hierarchical generation of table of contents entries.

Property	Description
<code>TOCItem</code>	Enables or disables generating table of content entries. Values for this property are: • 0 Disable the generation of entries. • 1 Generate an entry for this layout (is inherited by children of this layout). • 2 Generate entries for the children of this layout (both layout children and any data items displayed by this layout).
<code>TOCLevel</code>	For table of contents entries, the value of this property determines the indent level of <code>TOCItem</code> entries. Entries at the same <code>TOCLevel</code> appear at the same indent level in the table of contents (with the same number of decimals in the numbering). The <code>TOCLevel</code> is not the absolute level of indent, rather it controls if the indent moves up or down. If a new <code>TOCItem</code> entry is generated with level greater than the previous level, it appears one level deeper than the previous level, even if the number is two or more greater than the previous entry. If a new <code>TOCItem</code> entry is generated with a level less than the previous level, it appears at the level it specifies.
<code>TOCFigure</code>	Enables or disables generating list of figures entries. Values for this property are: • 0 Disable the generation of entries. • 1 Generate an entry for this layout (is inherited by children of this layout). • 2 Generate entries for the children of this layout (both layout children and any data items displayed by this layout).
<code>TOCTable</code>	Enables or disables generating list of tables entries. Values for this property are:

Property	Description
	• 0 Disable the generation of entries. • 1 Generate an entry for this layout (is inherited by children of this layout). • 2 Generate entries for the children of this layout (both layout children and any data items displayed by this layout).
TOCName	Specifies a name for generated entities of any kind. By default, the name displayed with the entity is the name field of the template or the item. This property allows a specific name to be used instead.

Macros

When generating entries, you can use macros when creating custom HTML headers or properties for things like plot titles and so on. These macros expand to the current number for the item, figure, and table lists. The available macros are:

- `{{toc_item_number}}`: The current item number string (for example, 1.2, 1.2.3, 1)
- `{{toc_figure_number}}`: The current figure number
- `{{toc_table_number}}`: The current table number

In a custom HTML header they can be used like this:

```
<h2>{{toc_item_number}}Data</h2>
```

or in a table property to dynamically label the table (plots) within a layout:

Property	Value
plot_title	Table {{toc_table_number}}: Temperature vs Distance

Exceptions

There are several exceptions from the basic inheritance rules. Generator layouts cannot be added to the lists. Some report layouts do not allow their children (layouts or items) to generate entities. The [Carousel](#) (p. 149), [Tabs](#) (p. 144), and [Slider](#) (p. 150) layouts are three examples. This is because they do not support links pointing to their specific children.

Data Items

A data item is a simple piece of information stored in the data payload of the item and the associated metadata. The data payload can be one of these types:

- [Animation \(p. 92\)](#)

Video in an MP4 movie file.

- [File \(p. 93\)](#)

A generic file

- [HTML \(p. 93\)](#)

Raw HTML source (in a `<div>` block).

- [Image \(p. 94\)](#)

A PNG image.

- [None \(p. 94\)](#)

An empty item (no data payload) that can be use for tagging or other metadata purposes.

- [Scene \(p. 95\)](#)

A scene of 3D geometry.

- [String \(p. 100\)](#)

A UTF-8 string.

- [Table \(p. 100\)](#)

A 2D array of doubles with row/column headers. Can be displayed as a table or a plot (line, bar, pie, and so on).

- [Tree \(p. 120\)](#)

A collection of name/value pairs organized as a tree

All data items can generate the HTML snippets necessary to display themselves.

In addition to the data payload, all data items include a common collection of metadata attributes. These include:

- The session Item describes the application that stored the data item
 - Hostname (50 characters)
 - Session tags (No limit)
 - Application name (40 characters)

- date/time
- Platform (50 characters)
- Version (20 characters)
- The dataset item describes the data source of the item
 - Filename (250 characters)
 - Directory name (250 characters)
 - Data format (50 characters)
 - Dataset tags (No limit)
- Date/Time
- Specific item tags (No limit)
- The name of the source of the item (80 characters)
- A sequence number (for sorting and so on)
- The name of the data item (80 characters)

Animation

The animation data item contains a video clip. It is a wrapper around an MP4 encoded movie file. Ansys Dynamic Reporting does not transcode the video, it must be converted to an H.264 MP4 container suitable for web display before being pushed into Ansys Dynamic Reporting.

Various web portal configurations (NGINX/Apache) may put limits on the size of the files that can be uploaded. Generally, files of several hundred MB are allowed, provided that the maximum size of a REST request allows them.

Display Options

This item is displayed in the browser's native movie display plug-in. Individual browsers scale the playback window in different ways. There is a `width` property that allows a report to try to specify the width of the display window in pixels and ask the browser to scale while preserving the aspect ratio. The `controls` property specifies if the animation should play by default and if playback controls should be visible.

Properties

Property	Description	
con-trols	Value	Operation
	0	No controls displayed
	1	Display controls

Property	Description	
	2	Autoplay animation, no controls displayed
	3	Display controls and autoplay
width	Maximum width of the MP4 frame in pixels	

File

The file data item allows you to upload and download generic files to the Ansys Dynamic Reporting data storage system. Ansys Dynamic Reporting makes no attempt to look into the uploaded files.

Various web portal configurations (NGINX/Apache) may put limits on the size of the files that can be uploaded. Generally, files of several hundred MB are allowed, provided that the maximum size of a REST request allows them.

Display Options

This item payload is not displayed. The item itself appears as a link in the report which allows the file to be downloaded through the browser.

Properties

No properties apply specifically to file items.

3D File Format Support

The file Item can display specific 3D file formats using the same proxy image/3D viewer mechanism used with [scene \(p. 95\)](#) Items.

The file Item is also able to display EnVision Scenario files (*.evsn) files using the embedded EnVision viewer, subject to the same prerequisites outlined in the scene item section.

HTML

The HTML data item stores HTML formatted text.

Only simple HTML text should be used here. Ansys Dynamic Reporting takes this string and passes it directly to the HTML interpreter to build the report. Complex HTML that tries to control the appearance of the page (calls to external CSS modules and so on), is not turned off and it might conflict with how the report page is structured and displayed. Ensure the HTML passed in this field is correctly formatted and does not interfere with the rest of the page.

Display Options

Context translation is performed on the raw HTML and the result is inserted directly into the report output.

Properties

No properties apply specifically to HTML items.

Image

The image data item contains a 2D image. Internally, all images are stored in PNG format. Metadata tags are inserted into the files that include the Globally Unique Identifier (GUID) of the data item object. Even if the image is exported from Ansys Dynamic Reporting, you can determine the original source of the imagery.

Various web portal configurations (NGINX/Apache) may put limits on the size of the images that can be uploaded. Generally, images of several hundred MB are allowed, provided that the maximum size of a REST request allows them.

Display Options

These items are displayed in the report, inline with other data items as an image. There are properties that allow the output display width and height in pixels. The browser is responsible for scaling the imagery. The `image_link` property can be used to make the image responsive to mouse clicks. Mouse clicks can be configured to display the image at full resolution outside of the context of the report. This can be in the same browser tab the report is displayed in or a new browser tab. The image can also be a link to the data item details web page.

Properties

Property	Description	
height	Height of the image in pixels	
im-image_link	Value	Operation
	0	Image is not clickable (default)
	1	Click the image to go to a page with only the image
	2	Click the image to open the image in a new tab
	3	Click the image to open the image data item details page in a new tab
width	Width of the image in pixels	

None

The none data item contains no data. It has the common item features (metadata, tags, and so on) but no data payload. It can be used as a token item or a source of properties.

Display Options

This item is not displayed.

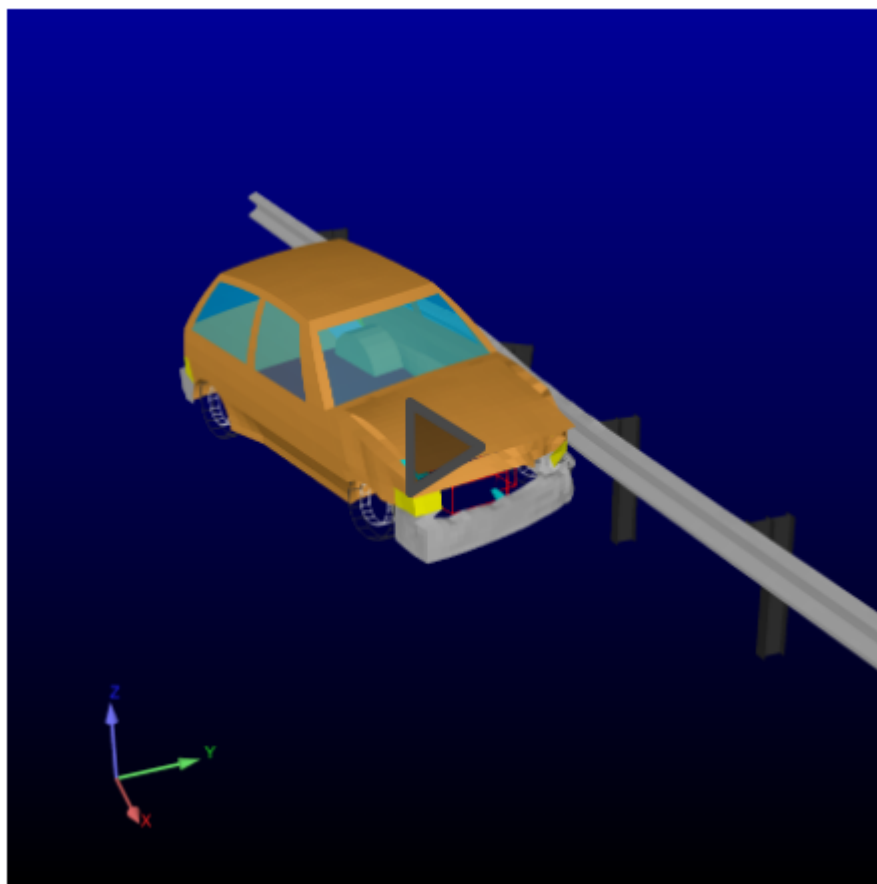
Properties

No properties apply specifically to none items.

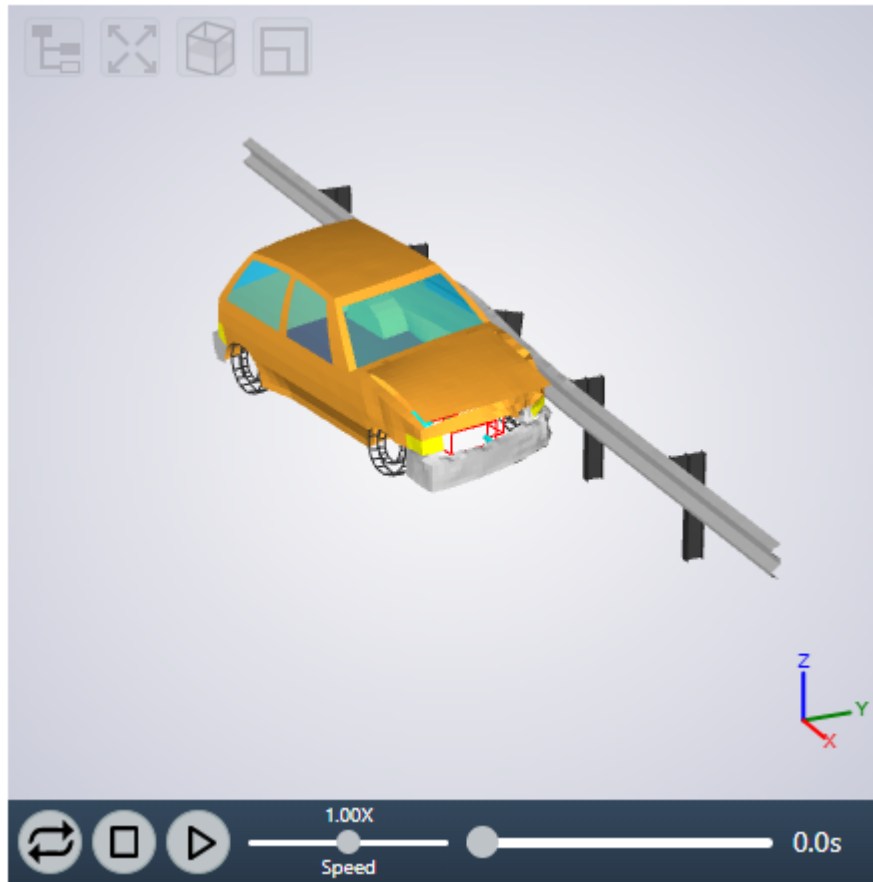
Scene

The scene data item contains a scene of 3D geometry that is rendered interactively in the browser using WebGL. The scene consists of a collection of parts that can be animated over time. Parts are a collection of shaded polygon and line models that support basic coloring attributes, including the ability to be colored by a variable.

An example scene:

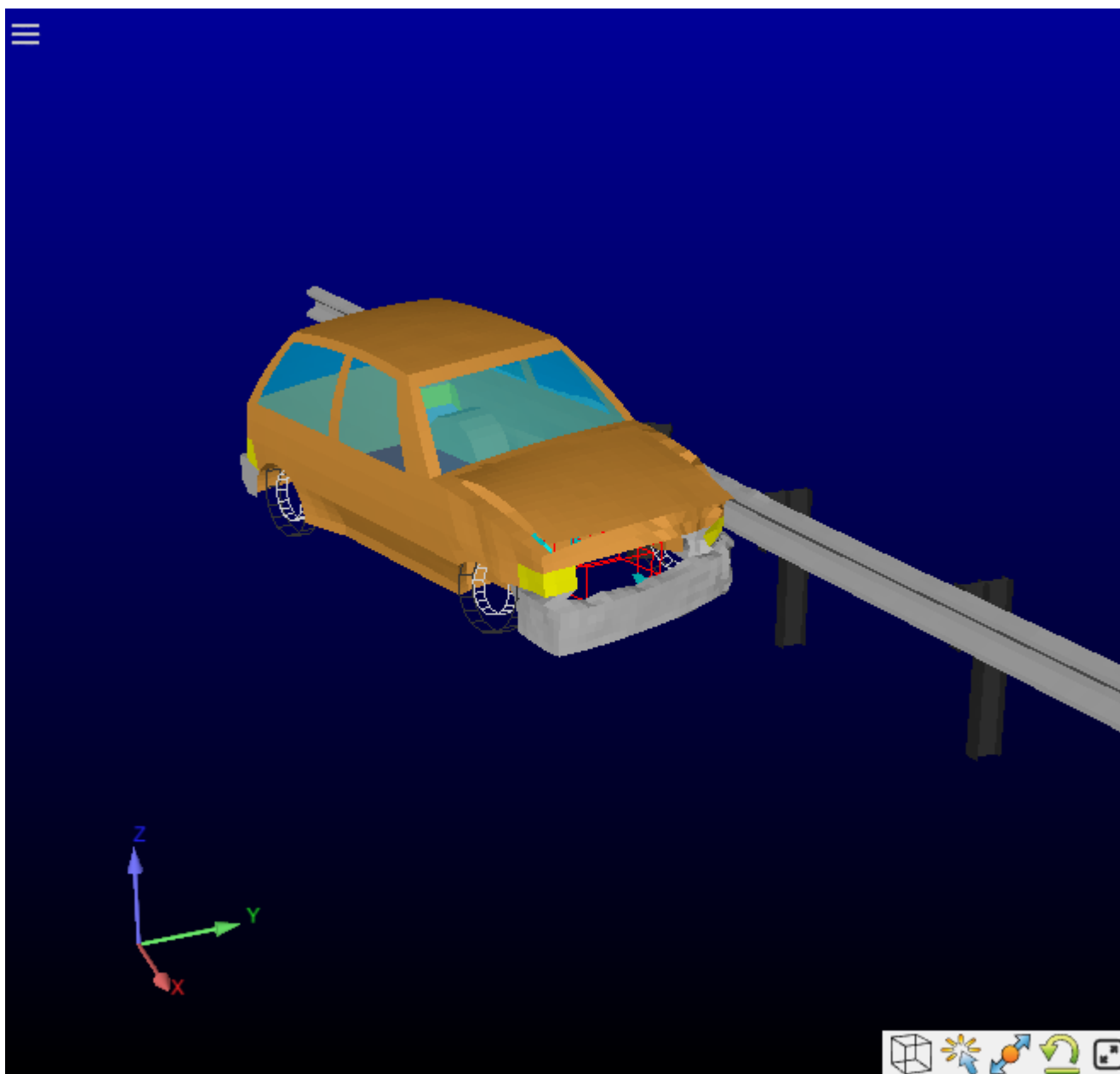


By default, the proxy image for the 3D scene is displayed. If EnSight is the source of the 3D scene, this is the same image that is visible when the scene is exported. Clicking the scene activates it.



You can use the options in the top-left corner of the view to display the part tree, re-center/size the scene, clip the scene, and transition to/from full browser window mode.

The embedded EnVision viewer can also be used to view some types of 3D scenes. This viewer looks like the following:



Use the  menu in the top-left corner to hide/show parts and change variable displays and the controls in the lower-right corner for element outlines, part highlighting, re-centering, revert, and full screen mode.

Various web proxy configurations may put limits on the size of the geometry files that can be uploaded. Generally, files of several hundred MB are allowed, provided that the maximum size of a REST request allows them.

Properties

Property	Description
enable_exploded_view	Enables an interactive exploded view for all geometry formats except GLB and EnSight scenes.

Property	Description	
	Value	Operation
	0	Disable the exploded view (default).
	1	Enable the exploded view.
height	Height in pixels of the 3D rendering frame. The default is the height of the proxy image (if any).	
width	Width in pixels of the 3D rendering frame. The default is the width of the proxy image (if any).	
scene_translate	An optional, three value vector representing an offset applied to the vertices of a GLB file. This can be used to place the geometry for simpler viewing. The default centers the geometry around the view origin. Example: [1.0,0.0,0.3].	
scene_scale	An optional scaling factor applied to vertices before the translation is applied.	
	Value	Operation
	0.0	The scale is set to the length of the largest axis (default).
use_remote_viewer	-1.0	No scaling is applied.
	Sets which viewer displays some geometry items. See 3D Viewers (p. 98) for more details.	
	Value	Operation
vr	0	Uses the WebGL rendering engine (default).
	1	Uses the embedded EnVision viewer.
	Control access to the virtual reality user interface for scenes containing GLB format geometry. See Virtual Reality Support (p. 99) .	
	Value	Operation
	disable	Never enable VR support.
	ifavailable	Enable VR options if the browser supports VR.
	enable	Always enable VR options.

3D Viewers

The following file formats can be uploaded into Ansys Dynamic Reporting as a scene item:

- STL
- PLY
- EnSight Scene Geometry Source
- SCDOC
- DSCO
- AVZ
- GLB

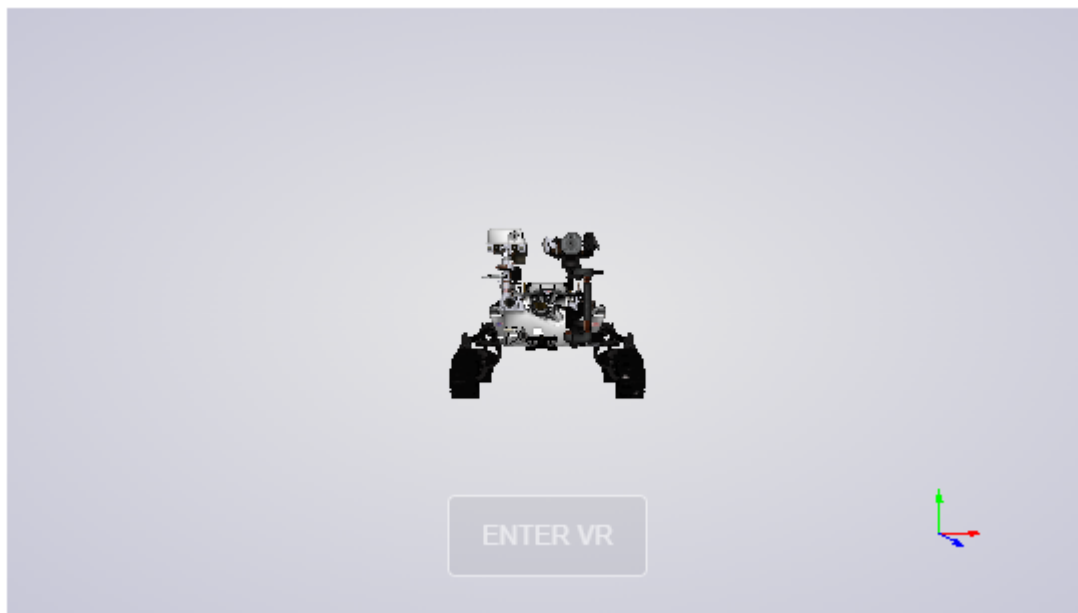
The first three formats (STL, PLY, EnSight Scene Geometry Source) can be displayed using WebGL or using the embedded EnVision viewer. The other formats can only be displayed with the WebGL viewer. Use the `use_remote_viewer` to enable the EnVision viewer, if it is available.

Note:

To use the embedded EnVision viewer, it must be enabled in the ADR server launcher [configuration setup \(p. 13\)](#) and you must be logged into the web interface.

Virtual Reality Support

Ansys Dynamic Reporting includes basic virtual reality support using WebXR. It is only supported when displaying scenes with GLB format geometry files and it only supports viewing. Interaction with the 3D scenes is not currently supported. When enabled, the 3D interactive scene has an **ENTER VR** button overlaid.



Click the button to enter virtual reality mode. Leaving the mode depends on the specific device being used.

To enable virtual reality:

1. Upload a scene item with a GLB file payload.
2. Set the `vr` property to `enable` or `ifavailable` for the template the item is displayed in.
3. Enable WebXR in the browser the Ansys Dynamic Reporting report is being displayed in. For example, the built-in web browser in an Meta Oculus Quest display has WebXR enabled automatically. You can install the WebXR emulator from the Chrome web store to enable WebXR without a physical VR device.
4. View the page containing the scene item.

- Optionally, use the `scene_translate` and `scene_scale` properties to adjust the initial view of the 3D scene.

String

The string data item stores a simple ASCII string.

Display Options

Context translation is performed on the raw string and the result is displayed in the output report as a paragraph with selected justification.

Properties

Property	Description
justification	The justification of the text in its column. Values are: - center (default) - left - right - justify

Table

The table data item contains a 2D array of elements arranged as rows and columns. These elements can be single or double precision floating point values or fixed length strings.

Related topics:

[Table Display Options](#)

[Table Properties](#)

Table Display Options

The table item can be displayed as a physical table as text with optional column sorting or as an interactive plot. The value of the `plot` [property \(p. 106\)](#) controls the specific type of display that is generated.

Table View

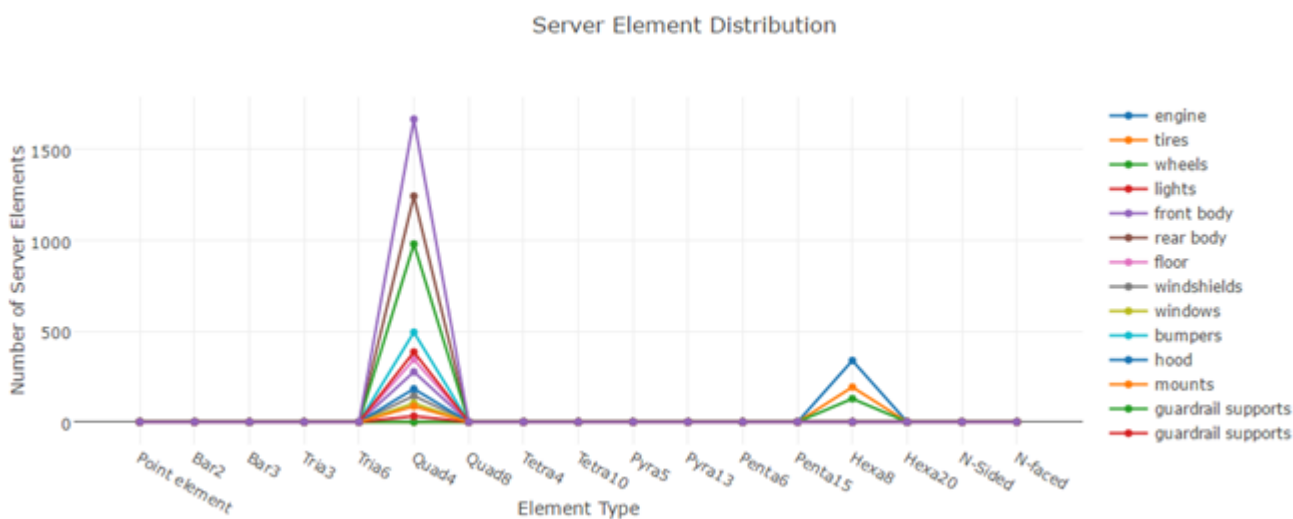
A table data item displayed as a table might look like this:

Server Element Distribution																	
	Point element	Bar2	Bar3	Tria3	Tria6	Quad4	Quad8	Tetra4	Tetra10	Pyra5	Pyra13	Penta6	Penta15	Hexa8	Hexa20	N-Sided	N-faced
default_region	0	0	0	0	0	0	0	0	0	0	0	0	0	8432	0	0	0
leftWall	0	0	0	0	0	92	0	0	0	0	0	0	0	0	0	0	0
rightWall	0	0	0	0	0	92	0	0	0	0	0	0	0	0	0	0	0
lowerWall	0	0	0	0	0	108	0	0	0	0	0	0	0	0	0	0	0
atmosphere	0	0	0	0	0	92	0	0	0	0	0	0	0	0	0	0	0
defaultFaces	0	0	0	0	0	16864	0	0	0	0	0	0	0	0	0	0	0

Tables have optional row and column headers that are presented as bold-face labels along the top and left sides of the table. Each table has an optional title that is displayed, centered, over the top of the table. Columns support sorting. Clicking individual columns sorts the rows according to the order of the values in that column. Values in the table can be strings or floating point numbers. The format [property \(p. 106\)](#) sets the default formatting for the entire table or for individual columns. A table value can be interpreted as a time value (the number of seconds since 1970-01-01T00:00:00.000000). In conjunction with the `date_XY` formatting option, these can be displayed in proper date/time formats.

Line Plot View

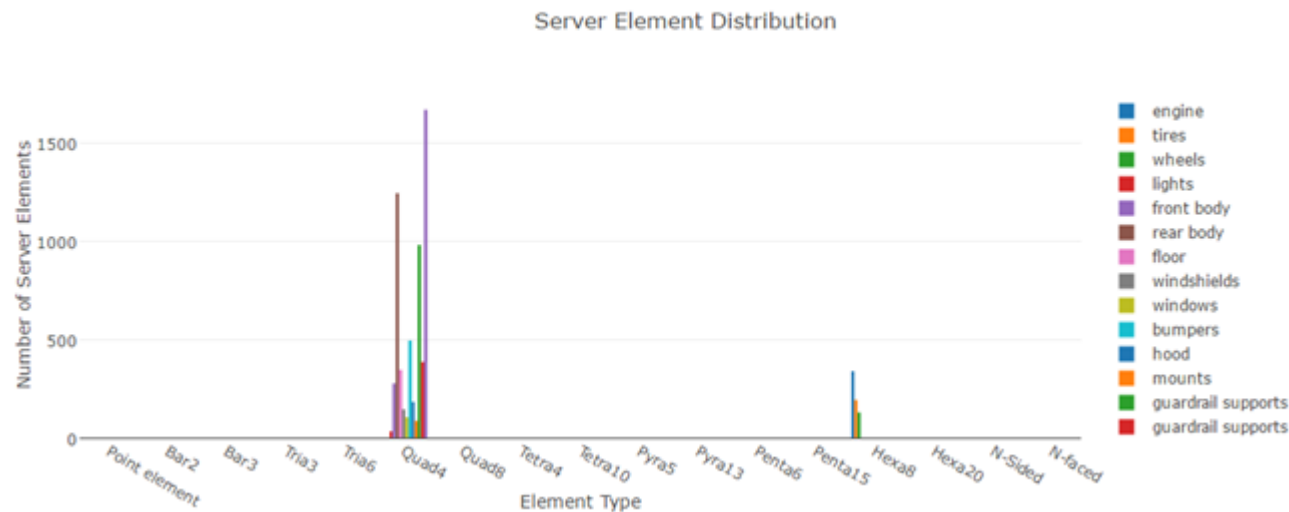
The table can be displayed as a line plot.



In line plot mode, rows in the table are mapped to lines in the line plot. A specific row or the column labels can be used as the common X axis for the data in each row. Not a Number (NaN) values in the row are not included in any plots (they are simply skipped). Line colors, widths, styles, thicknesses and symbols can all be controlled using properties. A line plot with symbols but with the `line_style` [property \(p. 106\)](#) of `none` results in a scatter plot.

Bar Graph View

The table can be displayed as a bar graph.



In bar plot mode, rows in the table are mapped to sets of bars in the bar plot. A specific row or the column labels can be used as the common X axis for the data in each row. Not a Number (NaN) values in the row are not included in any plots (they are simply skipped). Bar plots map the `line_color` property (p. 106) to the color of each bar set.

Histogram View

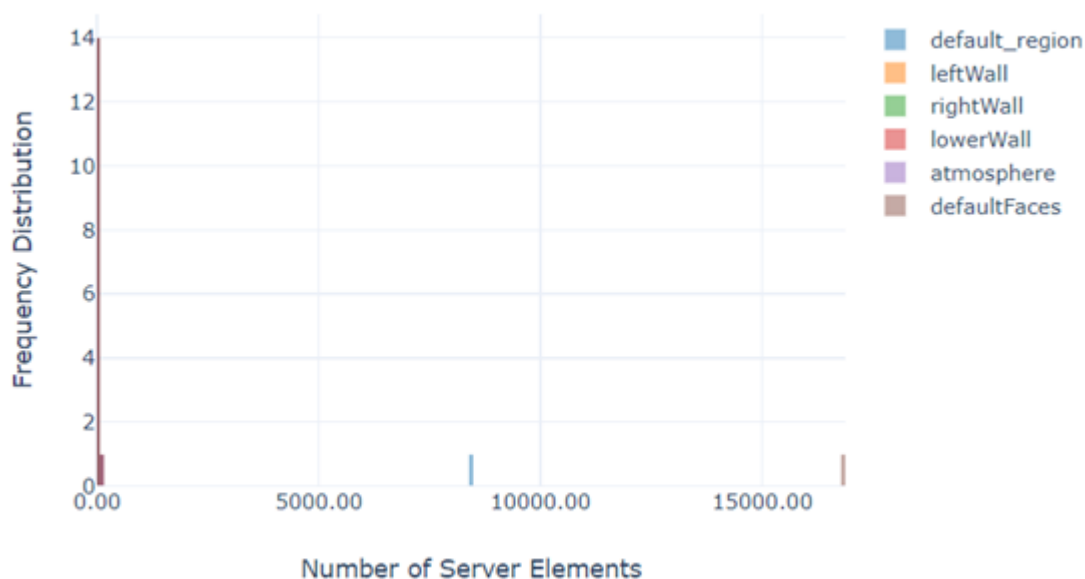
The table example can be displayed as a histogram plot categorized by its **Y labels** in two scenarios:

- **Render large table as histogram**

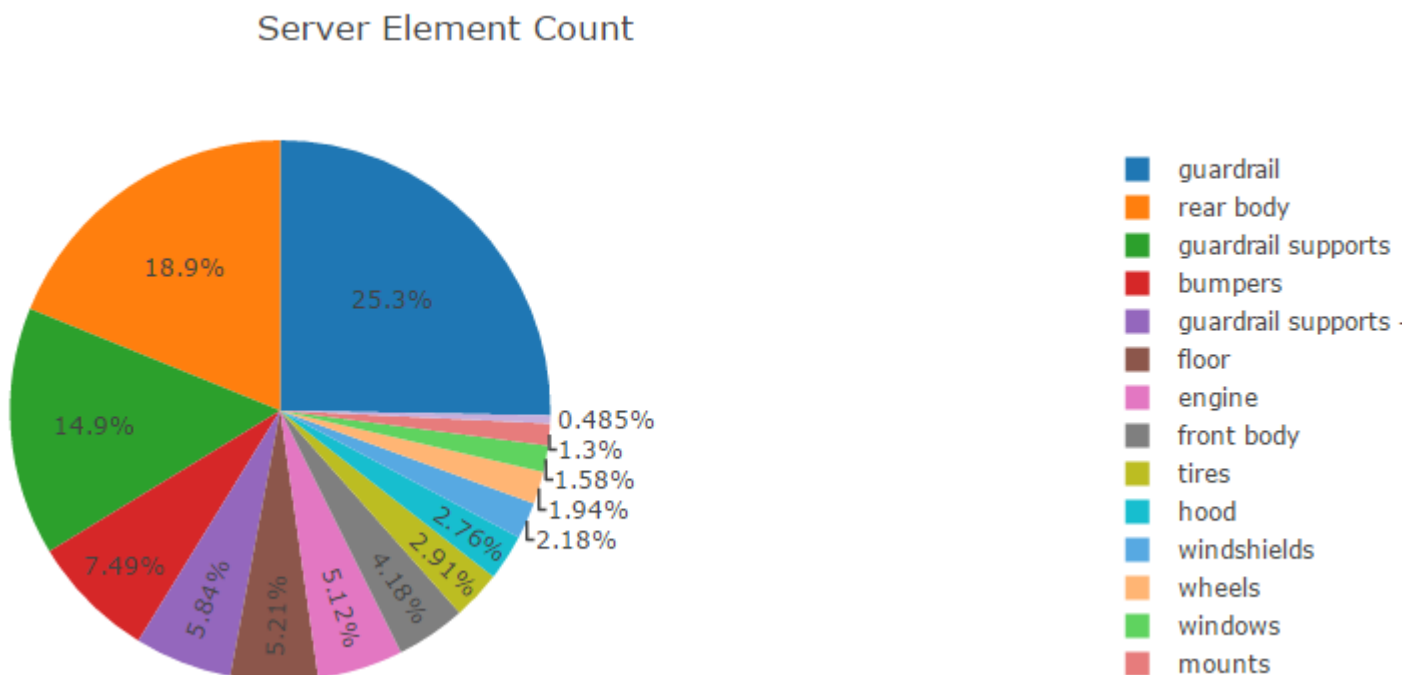
By default, Ansys Dynamic Reporting displays any data table as histogram if the table columns count is greater than the [rendering threshold](#) (p. 106). The initial threshold is 50 columns, but can be modified.

- **Specify the display option**

You can also specify the plot property as a histogram.



Pie Chart View



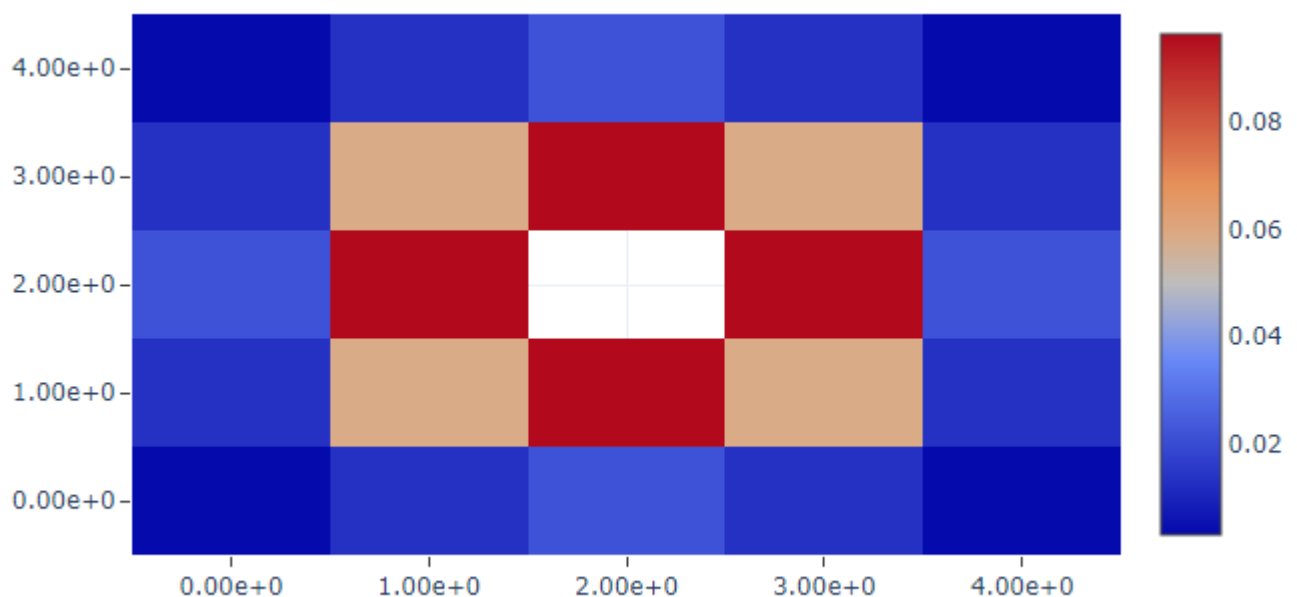
In pie chart mode, each row is displayed as a separate pie graph with a common legend. The colors of the pie wedges can be set using the `line_color` property (p. 106). The row label and the specific wedge value can be seen when you hover over an individual wedge.

Heatmap View

A 2D array of values can be displayed graphically as blocks colored by a palette. The following is an example table of values.

0.00291	0.01306	0.02153	0.01306	0.00291
0.01306	0.05854	0.09653	0.05854	0.01306
0.02153	0.09653	nan	0.09653	0.02153
0.01306	0.05854	0.09653	0.05854	0.01306
0.00291	0.01306	0.02153	0.01306	0.00291

If the plot `property` (p. 106) is set to `heatmap`, the table is displayed as:



The Not a Number (NaN) value is interpreted as a missing value. Grid lines can be added to the display using the `show_border` property. The `format`, `palette`, `palette_reverse`, `palette_show` and `palette_range` properties are all supported.

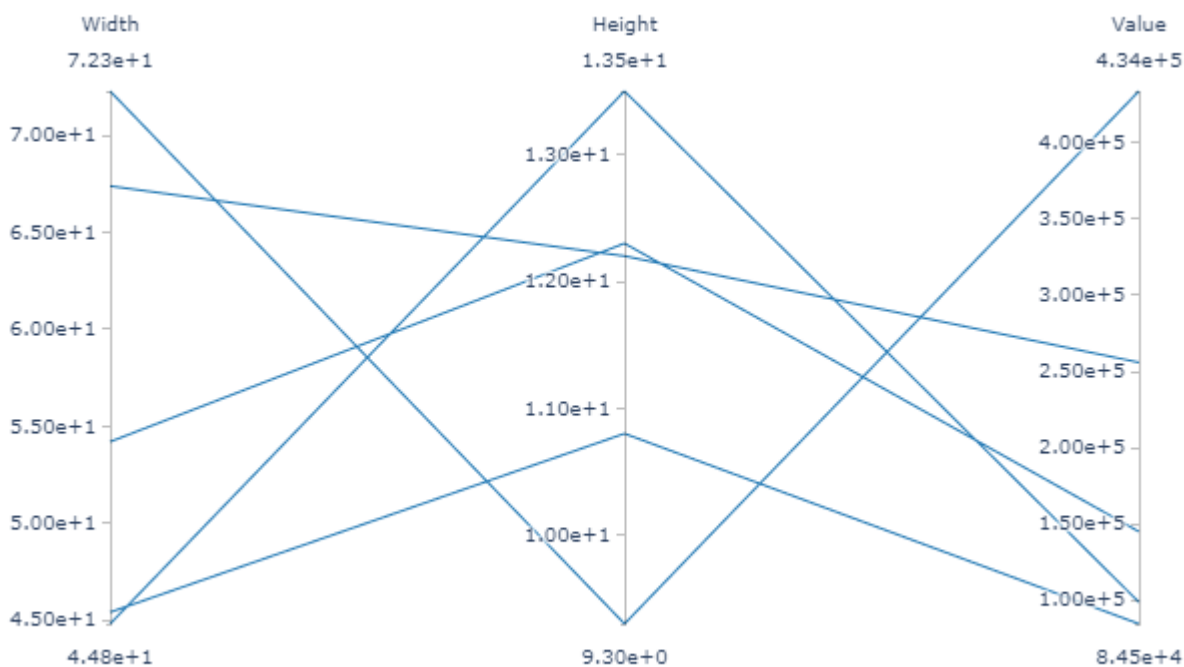
Parallel Coordinates Plot View

In a parallel coordinates plot, each row in the table is considered an observation. Each column in the table is a different observation. The following is an example table of five observations. Each observation consists of three values (width, height and value).

Width	Height	Value
54.2	12.3	1.45e5
72.3	9.3	4.34e5
45.4	10.8	8.45e4

Width	Height	Value
67.4	12.2	2.56e5
44.8	13.5	9.87e4

If the plot [property \(p. 106\)](#) is set to *parallel*, the table is displayed as:



One axis for each of the columns in the table with one line for each row in the table. The minimum and maximum values for each column can be set using the `column_minimum` and `column_maximum` properties. The `format` and `line_color` properties are also used in parallel coordinates plots.

Sankey Diagram View

A sankey diagram is a nodal flow diagram. It draws a plot of linked nodes, where each link has a specific weight or value. Links are directional, in that the flow from node A to node B is displayed separately from the flow from node B to A, and they can have different weights. Weights must be non-zero, positive floating point numbers.

The nodal network is represented as a table. The rows and columns are the nodes. Rows are the sources and columns are the targets. The value in the table is the weight of the link from the source to the target. Weights that are less than or equal to 0.0 are non-existent links and are not included in the diagram.

Note:

Labels for the nodes are assumed to be the same for the sources and targets. The column labels are used as the node labels, any supplied row labels are ignored.

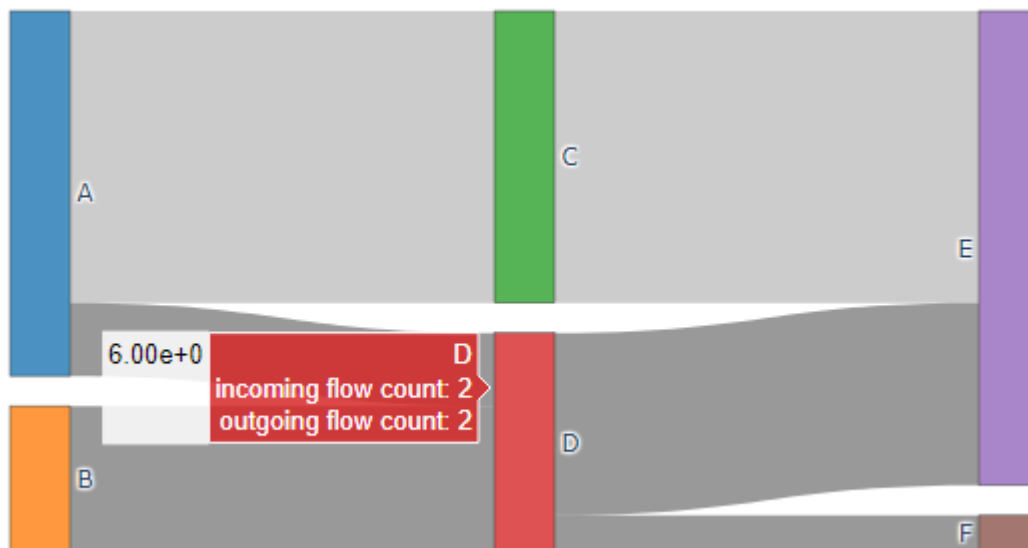
The following is an example graph with six nodes: A, B, C, D, E, F. Nodes A and B feed C and D while nodes C and D feed E and F with the following weights:

Source	Target	Weight
A	C	8
B	D	4
A	D	2
C	E	8
D	E	5
D	F	1

That graph is represented using this table (which is passed to Ansys Dynamic Reporting):

	A	B	C	D	E	F
A	0	0	8	2	0	0
B	0	0	0	4	0	0
C	0	0	0	0	8	0
D	0	0	0	0	5	1
E	0	0	0	0	0	0
F	0	0	0	0	0	0

If the plot [property \(p. 106\)](#) is set to *sankey*, the table is displayed as:



The `format` property is also used in sankey plots.

Table Properties

Properties can be scalars or lists. For items listed as arrays or scalars, an array can be used to specify a specific value for each instance in a plot or if a single value is specified, it is used for all instances.

These properties specify things like the color of a trace on a chart or the name of the marker to be used with each trace.

Multiple rows can be used as Y values for plot lines in the same plot, selected by the `yaxis` property. The value of that property can be a row name or the row index into the table (if a number). The `xaxis` property provides matching row names providing X values for each `yaxis` row. Other properties can provide values for each Y row as well. These can be scalars (used for all nodes of all row lines) or arrays. If arrays, individual values in the array are used for each Y axis line. If the value is simple value, it is used for all nodes in the line (for example, `#ff0000` used as a color). If the value is prefixed with an `@` character, it is interpreted as a row reference. A row reference can be either the index number of a row or the name of a row. For example when used with `line_marker_size`: `@0` selects the first row (row at index 0), `@Temperature` selects the row with the name *Temperature*, while `0` specifies the value *0.0*.

Property	Description	Values	Sub-values
<code>align_column</code>	Controls the justification of the text in the columns. If the number of columns is less than the number of justifications you have set, the sequence will repeat itself.	• left • right • center	N/A
<code>bar_gap</code>	Adjusts the histogram bin and bar chart's bar gap.	Integer or float type. The minimum value is 0 and the maximum value is 1. Any value that falls out of this range sets the gap as 0.	N/A
<code>bar_mode</code>	Sets up the histogram and bar chart's plot configuration for organizing the bins/bars.	• group (default) • stack • overlay	N/A
<code>column_minimum</code>	For a parallel coordinates plot, specify the minimum value for the range of each axis.	An array of float values, for example: <code>[0., 1e-10, -2.5]</code> .	N/A
<code>column_maximum</code>	For a parallel coordinates plot, specify the maximum value for the range of each axis.	An array of float values, for example: <code>[100., 2e-10, -1.0]</code> .	N/A
<code>format</code>	Formatting for values in a table or plot. It may be an array or a	• scientific Use scientific notation for the display of numbers.	Date Formats

Property	Description	Values	Sub-values
	scalar (scientific or [scientific, date_00]).	- floatdotX Display numbers in fixed notation using X digits to the right of the decimal. - sigfigsX Display numbers rounded to X significant figures. - date_XY Format the value as a date. If the value is a number, the value is interpreted as the number of seconds since midnight, Jan 1, 1970. X determines the format for the date. Y determines the format for the time.	'_' Do not include the date 0 2017-01-06 1 Jan 6, 2017 2 01/06/2017 3 6 Jan 2017 4 Locale specific display of date Time Formats '_' Do not include the time 0 13:20:00 1: 1:20PM 1 1:20PM

Property	Description	Values	Sub-values
			• 2 13:20:00.000000 • 3 Locale specific display of date and time • 4 Locale specific display of time
height	Height of the chart in pixels.	Numeric.	N/A
histo-gram_bin_size	Adjusts the histogram bin size.	Integer or float type. The value is used to determine how the histogram builds up the bins. For example, setting the value equal to 10 constructs histogram bins in intervals of [0-9], [10-19], and so on.	N/A
histogram_cumulative	Sets whether or not to cumulate the histogram.	• 0: Do not cumulate (default) • 1: Cumulate	N/A
histogram_normalize	Sets whether or not to normalize the histogram.	• 0: Do not normalize (default) • 1: Normalize	N/A
histo-gram_threshold	Sets the threshold column count for the data table. A number over this value renders the data table as a histogram.	Integer. Default value is 50.	N/A
item_justification	Controls the justification of tables in reports and detail	• left • right	N/A

Property	Description	Values	Sub-values
	pages. Overrides report-wide justification.	- center - none (default)	
labels_column	Column labels.	An array of strings, for example [column A, column B]	N/A
labels_row	Row labels.	An array of strings, for example [row 1, row 2]	N/A
legend_position	If a legend is shown, this property can be used to position the legend relative to the plot. For example, the value [1.0, 0.5] places the legend to the right of the plot, centered vertically while [0.0, 0.0] places it in the lower left corner of the plot.	[X, Y]. The domain of the plot is from 0,0 to 1,1. Numbers above and below 0,1 are valid.	N/A
line_color	Color of the plot line, set of bars, histogram, or pie arc segments. For example #ff0000 is red.	Array or scalar ([#808080, #ff0000, #00ffff] or #8f4f55) color or an array of row references. If a row reference is specified, the values in the row are mapped through the palette and palette_range properties to select the color.	N/A
line_error_bars	A vertical error bar is drawn centered on each marker location. The length of the error bars is selected using this property.	A single value, an array of values or an array of row references.	N/A
line_marker	The name of the marker to use.	Array or scalar. The suffix -open can be added to specify that a filled marker be drawn only in outline and -dot can be added to specify a dot be drawn in the center of the marker. The marker specification: square-open-dot results in the outline of a square drawn with a dot in the center.	- none: No marker is drawn - circle: Filled circle - square: Filled square

Property	Description	Values	Sub-values
			• cross: Filled plus symbol • x: Cross rotated 45 degrees • tri- angle: Filled triangle • pentagon: Filled pentagon • hexagon: Filled hexagon • octa- gon: Filled octagon • start : Filled five point star • dia- mond: Square rotated 45 degrees • as- ter- isk: Line drawn asterisk (*)

Property	Description	Values	Sub-values
			• hash: Line drawn hash symbol (#) • plus: Line drawn plus symbol (+) • times: Plus rotated 45 degrees
line_marker_auxX	Specifies up to 10 auxiliary values. The formatting is the same as line_marker_size and it declares extra values that can be accessed in the line_marker_text strings as v/auxX.	X can have the value [0-9], for example line_marker_aux0.	N/A
line_marker_opacity	The opacity of the marker from [0.0, 1.0].	Array, scalar, or row reference. The default opacity is 1.0.	N/A
line_marker_scale	If the line marker is specified as a row, this property is used to apply a linear transform to the value before it is used as the marker size.	[M, B] and the transform is final_size = M*input_size + B. The default is [1.0, 0.0].	N/A
line_marker_size	The size of the marker in points	Array, scalar, or row reference. If a row reference is specified, the values in the row are used as the size of the individual markers after mapping the values through the line_marker_scale transform.	N/A

Property	Description	Values	Sub-values
line_marker_text	If set, the value of this property is used to generate the strings that are displayed at a marker when the mouse hovers over a marker.	String that can include multiple macros. For example: '({{v/x}},{{v/y}}){{v/rowname}} color={{v/color}} size={{v/size}} error={{v/error}}'. Any property in the current context can be used in the string as well as several properties specific to this marker.	• v/x: X value for the marker • v/y: Y value for the marker • v/row-name: Name of the row • v/color: The color value for the marker • v/size: Marker size • v/error: Error bar length • v/auxX: A value specified by line_marker_auxX. For example, line_marker_aux3 is available as v/aux3.

Property	Description	Values	Sub-values
line_style	Selects the dashed line form	Array or a scalar.	• none: No line is drawn • solid: Solid line • dot: Dotted line • dash: short dashed lines • long-dash: Longer dashed lines • dash-dot: mixed short dashes and dots • long-dash-dot: Mixed long dashes and dots
line_width	Width of the plot line in pixels.	Array or a scalar (for example, [1,3,3] or 1)	N/A
marker_text_row-name	If the row for a given data value has a name, then by default, that name is appended to any marker text	• 0: Row name is never appended • 1: Row name is always appended	N/A

Property	Description	Values	Sub-values
	generated for hover display.		
nan_display	In the table format, the value displayed to represent a NaN value.	String. The default is NaN.	N/A
palette	Sets the name of the palette to use when <code>line_colors</code> is used to select marker colors by values. If a - is placed before the name, the palette order is reversed.	• Greys • YlGnBu • Greens • YlOrRd • Bluered • RdBu • Reds • Blues • Picnic • Rainbow • Portland • Jet • Hot • Blackbody • Earth • Electric • Viridis	N/A
palette_position	If a color palette has been selected and set to be shown, use this property to position the center of the palette color bar relative to the plot. For example, the value: [1.2, 0.5] places the color bar to the right of the plot	[X, Y]. The domain of the plot is from 0,0 to 1,1. Numbers above and below 0,1 are valid.	N/A

Property	Description	Values	Sub-values
	while [-0.2, 0.5] places it to the left of the plot.		
palette_range	Sets the range of the palette color mapping.	[min,max]. The min color value is mapped to the lowest color in the palette and max to the highest color in the palette.	N/A
palette_show	Hides or displays the palette color bar	0: Hides the color bar 1: Adds the color bar to the plot. Default when palette is specified.	N/A
palette_title	Displays a text title for the color bar.	String	N/A
plot	Changes the display mode for the table item.	- table: Display as table - bar: Display as bar chart - line: Display as line chart - pie: Display as pie chart - heatmap: Display as heatmap plot - parallel: Display as parallel coordinates chart - sankey: Display as sankey diagram - histogram: Display as a histogram	N/A
plot_margins	Adjusts the amount of spacing between the plot border (not including axis tick labels or titles) and the boundaries of its container. It can be used to provide more space for axis tick labels or to expand the plot to make more effective use of whitespace (when the legend is hidden).	[{left},{top},{right},{bottom}], where each of the four values is an integer number of pixels to use as the margin along the left, top, right and bottom of the plot respectively. Use default to leave a value set to its natural value. For example, [default, default, 5, default] adjusts the right side of the plot to more closely follow the right side of the plot container.	N/A

Property	Description	Values	Sub-values
plot_title	The title for the table if displayed as a chart.	String	N/A
plot_xaxis_type	Mapping for the X axis.	- linear: Linear mapping - log: Log mapping	N/A
plot_yaxis_type	Mapping for the Y axis.	- linear: Linear mapping - log: Log mapping	N/A
stacked	Enable or disable displaying multiple data values in bar chart mode as stacked bars.	0: Disabled 1: Enabled	N/A
show_border	Displays or hides the border around a plot	0: Border hidden (default) 1: Border displayed	N/A
show_legend	Displays or hides the plot legend.	0: Legend hidden 1: Legend displayed (default)	N/A
show_legend_border	Displays or hides the border around a plot legend	0: Border hidden (default) 1: Border displayed	N/A
table_cond_format	This property sets the conditional formatting for the cells. See Conditional Formatting (p. 124) for additional details.		
table_page	Controls the number of rows that are visible in each page of the table.	Default is 0, which turns the table pagination off. Any number larger than 1 sets the number of rows for each page. If the property is set to 1, the default value of 10 rows per page is used.	N/A
table_pagemenu	Controls the options displayed in the Show field, how many rows are visible for each page.	Array of values. For example, [10, 100, 500] displays only these three values as options. The default is [10, 25, 50, 100, -1], where -1 stands for <i>All</i> .	N/A
table_scrollx	Hides or displays the horizontal scroll bar.	0: Scroll bar hidden 1: Scroll bar displayed (default)	N/A

Property	Description	Values	Sub-values
table_scrolly	Controls the vertical scroll bar.	The height of the table that is visible in points (1 point = 1/72 inch). Default is 0, or no scroll bar.	N/A
table_search	Hides or displays the table search field, allowing you to search inside table values.	0: Search field hidden (default) 1: Search field displayed	N/A
table_sort	Controls if and how the table is interactively sorted.	- none: The table is not sortable. - all: All of the columns (including row labels) are sortable. - data: Only the data columns (not the row labels) are sortable. This is the default.	N/A
table_title	Title for the table if displayed as a table.	String	N/A
table_bordered	Displays or hides a border around the table.	0: Border hidden 1: Border displayed (default)	N/A
table_condensed	Minimizes the table's horizontal and vertical height, making it look compact. If both this property and table_wrap_content are set to 1, the table is not condensed. If you turn on wrapping, the text flows to the next row and it will lose the condensed look when that happens.	0: Off (default) 1: On	N/A
table_wrap_content	Wraps content to the next line	0: Off (default)	N/A

Property	Description	Values	Sub-values
	inside a table cell. Table headers (column labels and row labels) do not wrap because the label/header should determine the wrapping point of the rest of the rows. Only the content in the body of the table wraps.	• 1: On	
table_default_col_labels	Enable/disable the addition of default column labels.	• 0: Disabled • 1: Enabled (default)	N/A
title	Title used if plot_title or table_title is not specified.	String	N/A
width	Width of the chart in pixels.	Integer	N/A
xaxis	Row number(s) or name(s) to use as the X axes values for line charts.	A single value or an array of row references.	N/A
xaxis_format	If set, this conditional formatting (p. 124) string is used (instead of the one specified by the format property) to format the tick values of the X axis.		
xrange	The minimum and maximum values for the X axis.	Array of values, for example: [0.0, 10.0]	N/A
xtitle	Title to use for the chart X axis.	String	N/A
yaxis	Row number(s) or name(s) to use as the Y axes values for charts.	A single value or an array of row references. If no value is specified, all rows not specified in the xaxis property are used as yaxis rows.	N/A
yaxis_format	If set, this conditional formatting (p. 124) string is used (instead of the one specified by the format property) to format the tick values of the Y axis.		
yrange	The minimum and maximum values for the Y axis.	Array of values, for example: [0.0, 10.0]	N/A

Property	Description	Values	Sub-values
ytitle	Title to use for the chart Y axis.	String	N/A

Tree

The tree data item contains a tree of name/value pairs. These pairs are organized in an ordered tree structure. Each entry includes a *name* which is used for display purposes, a *key* that can be used for classification/comparison, and a *value*. The entry may optionally include display hints and a list of children entries (each of which can also have children). Data values in the tree can be boolean, integer, floats, date/times, strings, a data Item Globally Unique Identifier (GUID), or empty.

Display Options

The tree item is displayed as a two column table, where child trees can be nested as an indented collection of entries. The left column includes controls that enable the children of an entry to be hidden or displayed. To the left of those controls are the name values from the entries. In the right column is the entry value. The value supports various formatting options for the different types. The properties that control this are documented below.

- Name 0	
Child 0	25
Child 1	685.85
Child 2	False
Child 3	02/13/17 14:25:41
Child 4	iAtnmns rrd gao
+ Name 1	Image Source
- Name 2	
Child 0	Image Source
Child 1	
Child 2	893
Child 3	939.61
Child 4	True
+ Name 3	02/13/17 14:25:41
+ Name 4	False

+ Name 0	928
+ Name 1	655
+ Name 2	238
+ Name 3	151
+ Name 4	831
+ Name 5	939
- Name 6	609
Child 0	Test: 0
Child 1	Test: 6
Child 2	Test: 12
Child 3	Test: 18
Child 4	Test: 24
+ Name 7	194

The tree can be set up to display entries in collapsed or expanded form. In the latter, all of the children are visible.

Tree Item Hinting

Individual entries support [hints](#) that suggest the initial state of entries. Current hints include: state and header. The default visibility is controlled by the state hint (if no hint is given the entry is displayed expanded). The `tree_initial_state` property can be used to override the expansion hints, but it is applied to the entire tree. The header hint specifies that a row in the tree table is displayed as a header, with bold text and enhanced bottom border.

In addition to hints, every tree row include a key value which is not displayed (the name and value values are always displayed). The key value is a mechanism to tag an item with additional, hidden information that can be used to filter items or aggregate them using [generators](#) (p. 124).

Properties

Property	Description
item_justification	Controls the justification of trees in reports and detail pages. Overrides report-wide justification. Values are: - center - left - right By default, there is no justification.
table_cond_format	Sets the conditional formatting for the tree entries. See Conditional Formatting (p. 124) for more details.
tree_format_bool	This string is used when the value is a boolean. It takes the form {XXX}#{YYY} where XXX is the string used for the true value and YYY is the string used for the false value. The default is True#False.
tree_format_date	When the value is a date/time, this string is used to pick the formatting. See Table Properties (p. 106) for specifics. The default is date_44.
tree_format_float	For a float value, this string is used for formatting. See Table Properties (p. 106) for specifics. The default is floatdot2.
tree_format_guid	When the value is a GUID that maps to another valid data item, this value is used to select how the item is displayed. The values are: - link: Displays the item via a hyperlink to a new page (default). - inline: Attempts to display the item directly in the tree view structure.
tree_initial_state	Allows the tree to be entirely expanded or collapsed initially. Values are: - expanded (default) - collapsed
tree_bordered	Displays or hides the tree's border. Values are: 0: Border hidden 1: Border displayed (default)
tree_condensed	Minimizes the tree's horizontal and vertical height, making it look compact. Values are: 0: Off (default) 1: On

Property	Description
tree_global_toggle	Displays or hides the Expand All/Collapse All option for tree nodes. Values are: • 0: Off (default) • 1: On

Conditional Formatting

Conditional formatting is a property that sets how to color a cell/entry, based on a set of rules that you specify. Conditional formatting can be applied to [table \(p. 100\)](#) and [tree \(p. 120\)](#) data items.

Conditional formatting is specified as a string. This string consists of a list of rules. The list of rules is executed against every cell in the table or tree data item. The rules allow you to control of the color of the cell text, the background color of the cell, the bold font attribute, and other visual details. For trees, the conditional formatting can decide if a given node starts as open or closed. This formatting is applied when the HTML for the web page is generated.

Related topics:

[Formatting Types](#)

[Formatting String](#)

[Using the Conditional Formatting Editor](#)

[Setting Expressions](#)

Formatting Types

You can apply conditional formatting to table and tree data items.

Table Conditional Formatting

The target portion of a rule is based on row and column labels. If those labels are not defined for a table, the row/column index (zero-based) can be used to select the portion of the table the rule applies to.

Tree Conditional Formatting

To understand how conditional formatting is applied to tree objects, first you must understand node names and keys in tree objects (see the [External Python API](#) for tree item details). Tree formatting begins by creating a table based the nodes in the tree. The table row names are computed from the tree node names. Consider the tree formed from this code snippet:

```
c0 = dict(name='Child' key='leaf0', value=500)
c1 = dict(name='Child' key='leaf1', value=[1000, 2000])
dict(name='Top', key='root', value=[100, 200], children=[c0, c1])
```

The table generated from this tree looks like this:

Row label	Column 0	Column 1
Top/root	100	200
Top/root Child/leaf0	500	
Top/root Child/leaf1	1000	2000

Row labels are generated by concatenating the name and key for the tree node with the / character and the parent to the child using the | character. The naming convention makes it possible to write rule target expressions using the visible node names and the (hidden) node key value. Tree conditional formatting follows the rules for tables using the transformation from tree to table noted above.

Tree formatting has two additional formatting functions, `openparent()` and `openparents()`. These formatting functions allow the conditional formatting rule to set the initial state of the target nodes. The `openparent()` function ensures that the parent node to the target node is expanded. The `openparents()` function ensures that all of the parent nodes to the top of the tree are expanded. This can be useful in conjunction with the `tree_initial_state` property set to `collapsed`.

Formatting String

The conditional formatting string consists of a collection of rules. Rules are run sequentially. Each rule decides what cells (if any) it applies to using a target function. Each of the potential target cells has the comparison function run on the contents of the cell. If the function returns true, the formatting function in the rule is executed and the display attributes of the cell are computed. Note: once a formatting function is executed on a cell, that cell is excluded from processing by any other rules.

The formatting string is encoded as a collection of sub-strings, one for each rule, separated by the `&` character like this:

```
{Rule}&{Rule}&{Rule}...
```

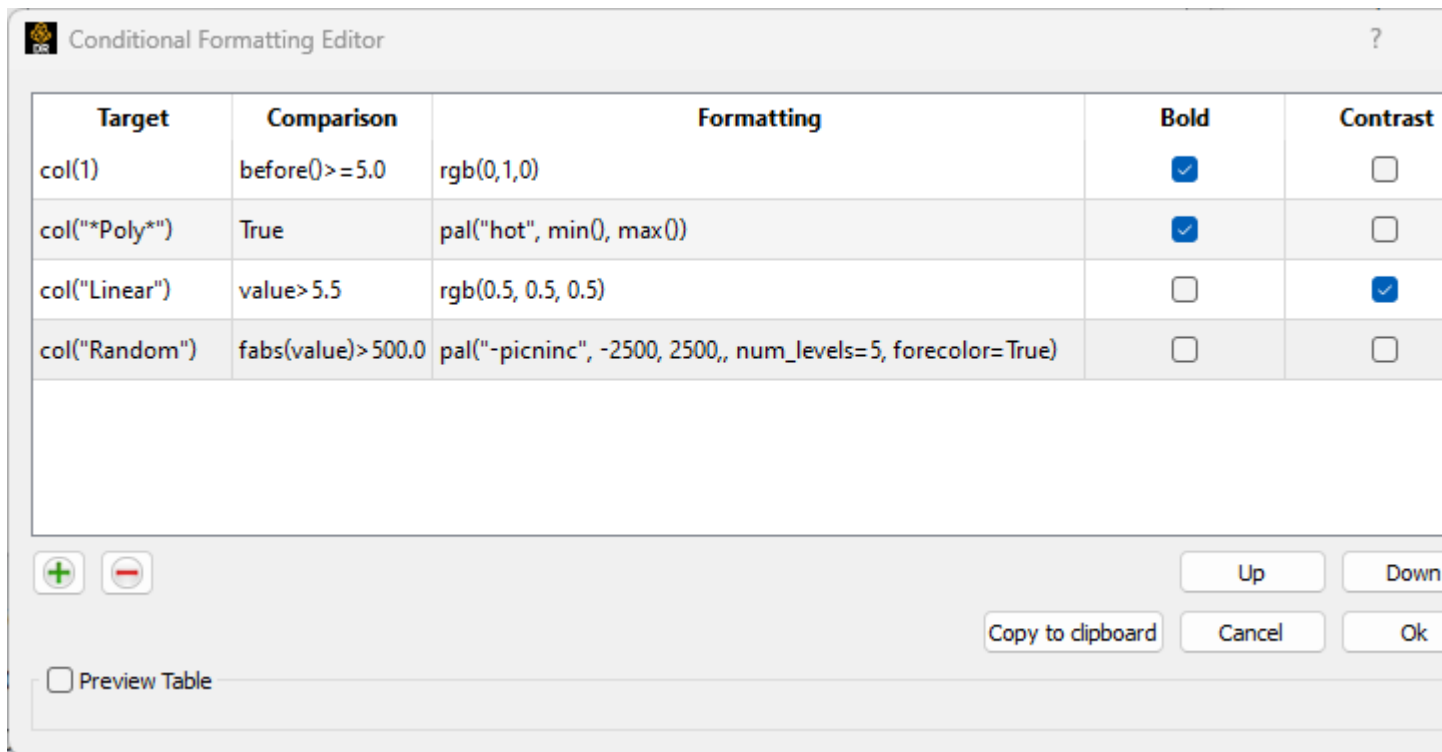
Each rule consists of three Python expressions: a target, the comparison and a formatting expression formatted, like this:

```
{Target expression}:{Comparison expression}|{Formatting expression}
```

The following is a valid conditional formatting string containing four rules:

```
col(1):before(>=5.0|bold()+rgb(0.,1.,0.)&col("\*Poly\*"):True|bold()+pal("hot",min(),max())&col("Linear"):value+contrast())&col("Random"):fabs(value)>500.|pal("-picnic",-2500, 2500, num_levels=3)+pal("picnic", -2500, 2500,
```

Here is the string displayed in the **Conditional Formatting Editor** dialog in the Ansys Dynamic Reporting Template Editor:



Setting the rules inside the Conditional Formatting Editor generates a string for the `table_cond_format` or `tree_cond_format` properties. You can set this property into a template using the [Template API](#) `set_property` call. you can also add it to a data item or tree directly using the [External Python API](#).

For a table:

```
rule = "col(0):value>1.|bold()"
d = dict(array=array, collbls=col_names, title="Table with embedded conditional formatting", table_cond_format=
item = server.create_item(name="Formatted Table", source="testing", sequence=0)
```

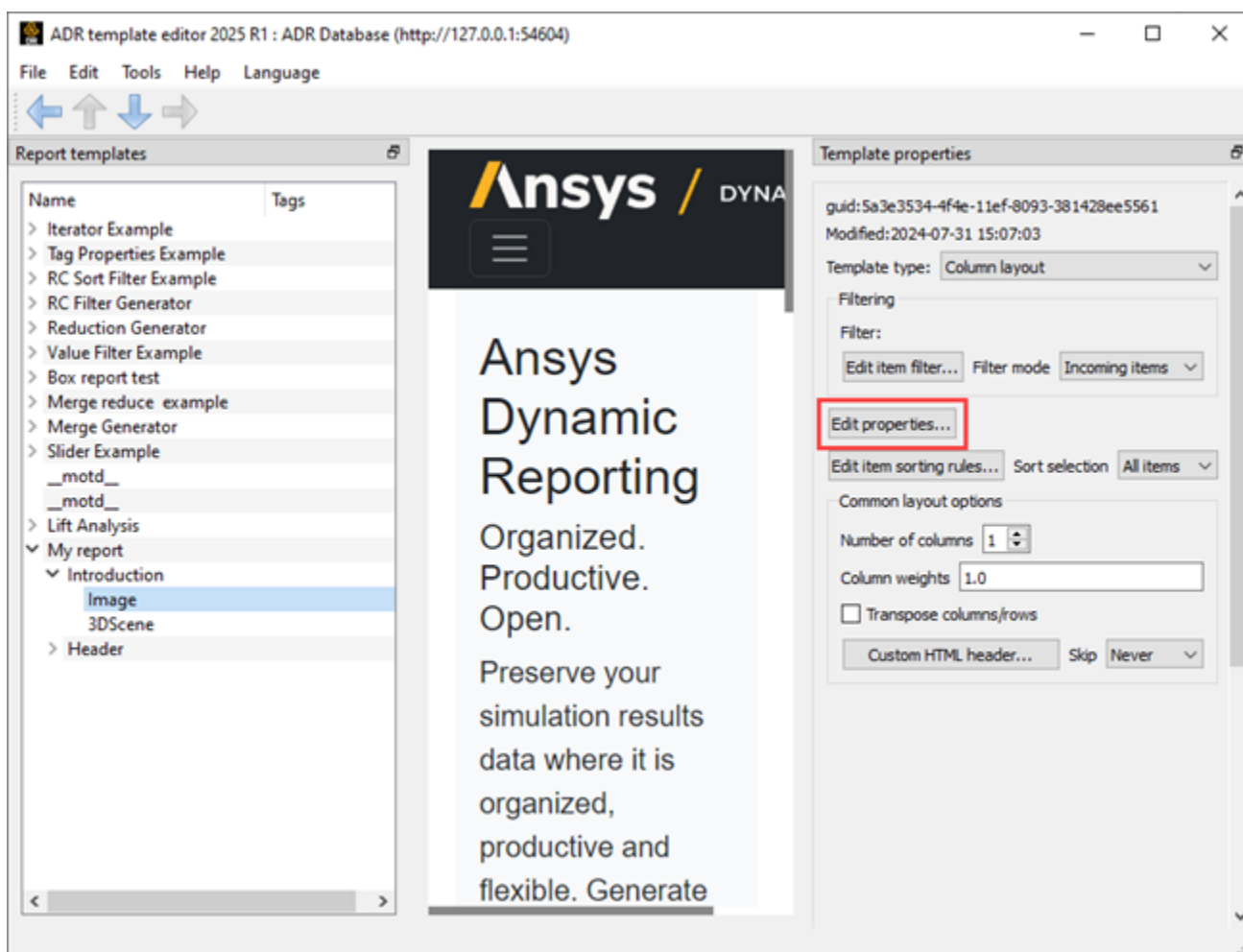
For a tree (assuming that the child1 and child2 nodes contain two columns of data):

```
rule = 'col(1):value>before()|pal("Picnic",0.,1.)+openparents()'
tree = dict(name='Top level', key='root', value=['', ''], children=[child1, child2], state='collapsed', tree_co
item = server.create_item(name="Formatted Tree", source="testing", sequence=0)
item.set_payload_tree([tree])**
```

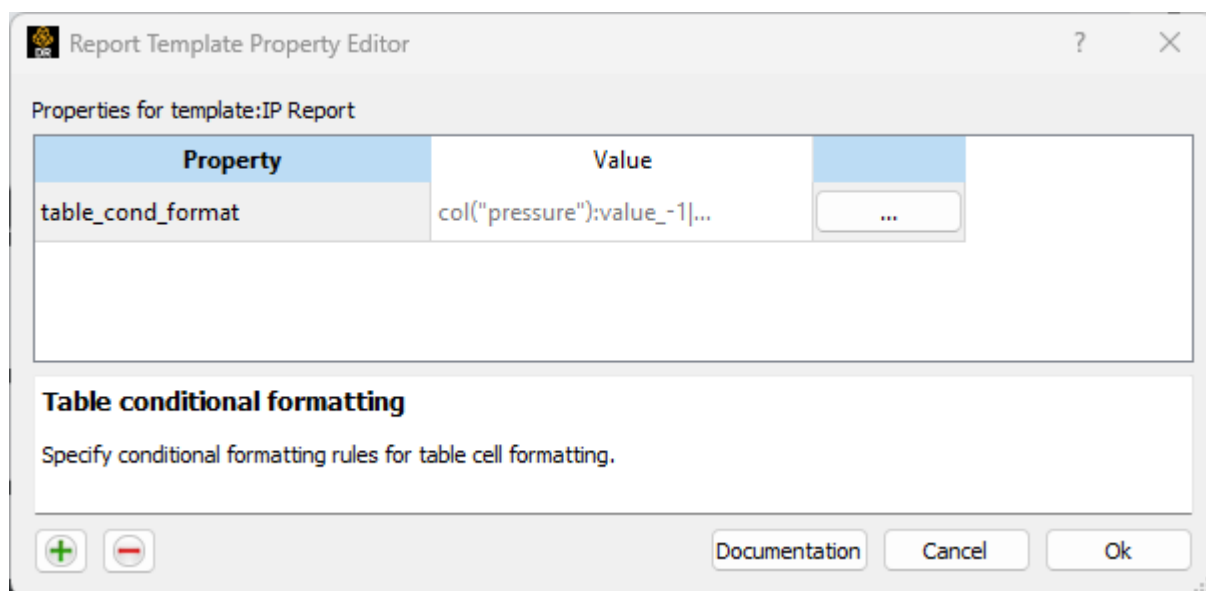
Using the Conditional Formatting Editor

You can access the **Conditional Formatting Editor** dialog when editing the report template properties.

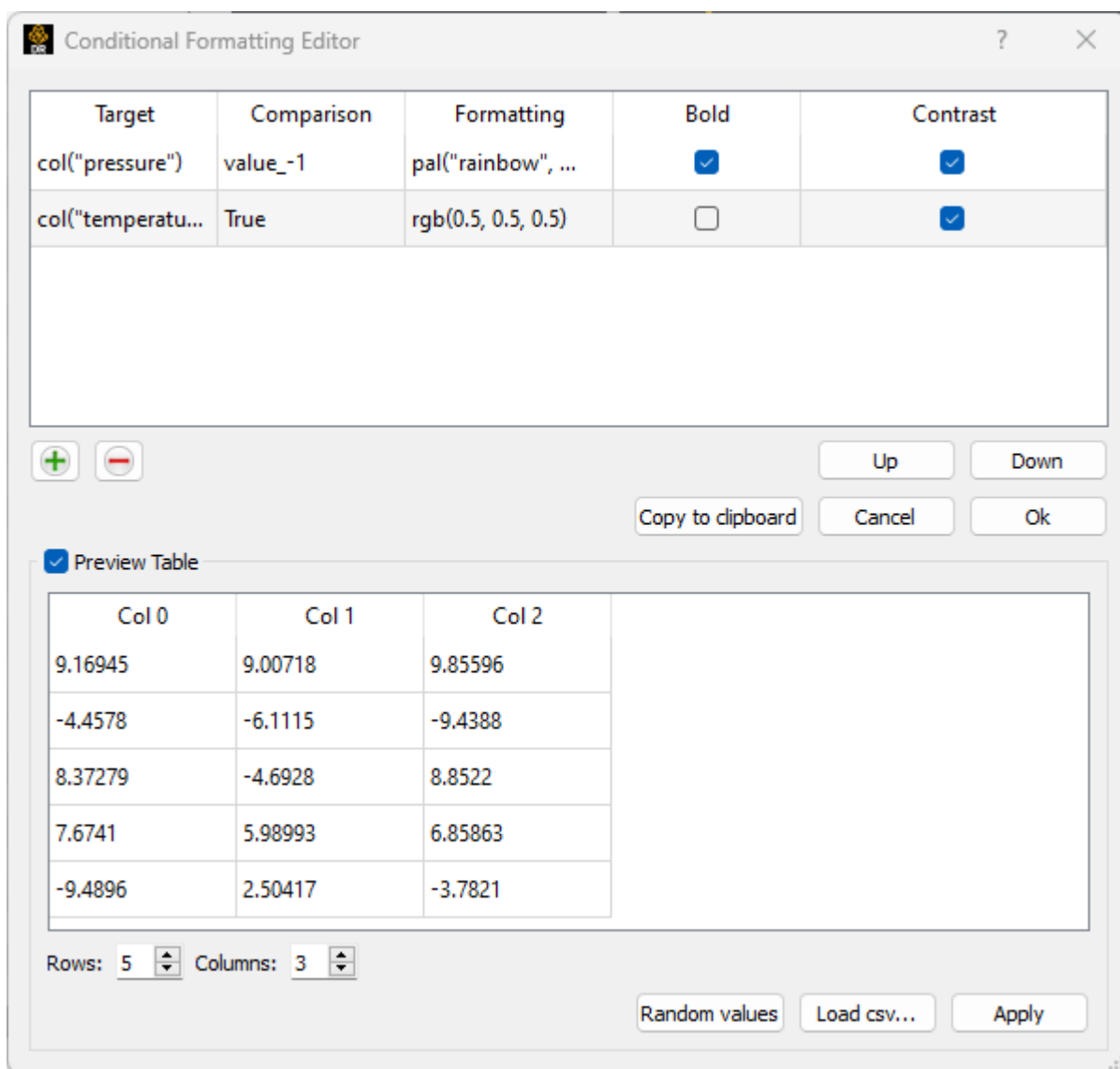
1. In the Ansys Dynamic Reporting Template Editor, select a report template and click **Edit properties**.



2. In the Report Template Property Editor dialog, enter **table_cond_format** or **tree_cond_format** into the **Property** cell.
3. Click .



The **Conditional Formatting Editor** dialog opens.



4. Use the **Add** () and **Remove** () buttons to add and remove rules.
5. For each rule, enter a target and comparison [expression](#) (p. 130).
6. Optionally, enable the **Bold** and **Contrast** options.

If **Bold** is enabled, the cell entry is displayed in bold format. If **Contrast** is enabled, the cell text color is white or black based on the selected cell background color (a light background selects a black font, while a dark background selects a white font).

7. Optionally, use the **Up** and **Down** buttons to reorder the rules.

Rules apply in order, from the top to the bottom one. If a cell comparison expression evaluates *True* for multiple rules, only the first rule for which the expression evaluated as *True* is applied.

8. To obtain the [formatting string](#) (p. 125), click **Copy to clipboard**.

9. To test the rules in a sample table:
 - a. Click **Preview Table**.
 - b. Create the preview table by either setting the number of rows and columns and click **Random values**, or import a CSV table by clicking **Load csv**.
 - c. Optionally, change the column headers by double-clicking them.
 - d. To apply the rules to the preview table, click **Apply**.

Note:

You must click **Apply** each time you make changes to the rules, to see the new results.

Setting Expressions

Use the following guidelines when setting expressions for conditional formatting.

Target Expressions

The target expression can use the following Python functions:

Function	Description
<code>row(Y)</code>	Select which rows to apply the rule to. Y can be either a number corresponding to the 0-based row index or the row name as a string. If the string name is used, <code>fnmatch()</code> compliant wildcard expressions can be used.
<code>col(X)</code>	Select which columns to apply the rule to. X can be either a number corresponding to the 0-based column index or the column name as a string. If the string name is used, <code>fnmatch()</code> compliant wildcard expressions can be used.

Comparison Expressions

The comparison expression is a general Python expression that must evaluate to *True* or *False*. For the cells for which it evaluates to *True*, the formatting expression is evaluated. If you want all the cells from the target expression to be evaluated, enter **True** as the comparison expression. Otherwise, standard Python expressions are allowed. For example, the following is a valid comparison entry:

```
(value > before()) and (value < 10.0)
```

The comparison expression uses the following functions/variables:

Function/Variable	Description
<code>value</code>	Numeric value of the cell. The value can be Not a Number (NaN). In the case of string inputs (trees or tables), this function can be used as a soft

Function/Variable	Description
	float conversion. It parses out the first floating point value it finds in the string. For example, the cell <code>273.0 [K]</code> has a svalue of 273.0 as the core parses the 273.0 and ignores the [K]. If the cell is <code>[K] 273.0</code> , the returned value is NaN as the first text ([K]) cannot be parsed as a floating point number.
svalue	String value of the cell if the cell contains a string element or the string representation of the cell numeric value.
min([name][, use_row=b])	Minimum value of the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
max([name][, use_row=b])	Maximum value of the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
avg([name][, use_row=b])	Average value of the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
var([name][, use_row=b])	Variance of the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
std([name][, use_row=b])	Standard deviation of the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
count([name][, use_row=b])	Number of non-NaN entries in the named row or column. If no name is specified, the row/column selected by the rule target expression is used.
before()	Value of the cell at the left of the current cell if the target is a column. The value of the cell on the row above the current row if the target is a row. If the source table is a string or a table, the value is a string.
after()	Value of the cell at the right of the current cell if the target is a column. The value of the cell on the row below the current row if the target is a row. If the source table is a string or a table, the value is a string.
neighbor(name)	For a row target, the value of the cell in the same column in the row selected by name. For a column target, the value of the cell in the same row in the column selected by name. If the source table is a string or a table, the value is a string.
clamp(value, lower, upper)	For a given value, clamp it to the range [lower, upper]. If value is less than lower, return lower, if value is greater than upper, return upper, otherwise return value.

All of the functions in the Python math module are available, as well as most of the functions in the str module. Any of the above functions can return Not a Number (NaN). The comparison expression `svalue.startswith("P")` is a valid comparison expression for a string as well as `isnan(value)` for checking if a value is a NaN.

The mathematical reduction operations (`min/max/avg/var/std/count`) operate on a selected row or column of the table/tree. By default, the array of data matches the row/column specification made by the target expression. For example if the target is `row(1)`, the `min()` function is performed on the row containing the current cell. The optional name parameter allows you to select a different row. For example, with a `row(1)` target and `min(3)`, the minimum is computed on row 3. The `use_row` keyword can be used to override the default row/column selection. For example, with a `row(1)` target and `min(use_row=False)`, the minimum of the column that contains the target cell is computed. When computing these functions, all NaN values (and strings that cannot be converted to valid floats) are ignored.

In the above table, name is a row or column specification, following the same rules as the `row()` and `col()` functions in the target expression. Names in square braces (`[name]`) are optional and if not specified, the row/column is the same as the current cell. For example, `var("Pressure*")` selects all of the rows or columns that start with *Pressure*. If multiple rows/columns are selected by these expressions, the first one is used.

Formatting Expressions

For target cells whose comparison evaluates to *True*, you can set the formatting. In the format expression, enter one or more of the following functions. Use a `+` sign in between objects if you want to set more than one.

Function	Description
<code>pal(name, min, max, num_levels=None, forecolor=False)</code>	Select a color based on interpolation into a color palette. The name argument is a string naming which palette to use (if the name has the '-' prefix, the palette is reversed). The palettes from the <code>plotly</code> library are available. The <code>min</code> and <code>max</code> arguments set the minimum and maximum values for the palette. The <code>num_levels</code> argument sets the number of levels to use. It accepts integer values. You can enter a Python expression for the <code>min</code>, <code>max</code> and <code>num_levels</code> arguments, such as <code>pal('-picnic', min()+10., max()-before(), num_levels=count())</code>. If the <code>forecolor</code> argument is set to <i>True</i>, then the coloration resulting from this palette is applied to the font of the cell, not its background. Only the name argument is needed, all other arguments are not mandatory and all arguments can be Python expressions building on the functions in the comparison expression table.
<code>rgb(r, g, b, forecolor=False)</code>	Sets the color for the cell. If <code>forecolor</code> is set to <i>False</i>, the RGB color that is set is applied to the background of the cell. If <code>forecolor</code> is set to <i>True</i>, it is applied to the cell font. The RGB values should be in the range <code>[0,1]</code> and are clamped if outside of that range.
<code>contrast()</code>	Sets the text color to <code>[0, 0, 0]</code> or <code>[1, 1, 1]</code> based on what best contrasts the background color specified by <code>rgb()</code> or <code>pal()</code>. This should be the last

Function	Description
	function in the format expression. If no background color is specified, the text color is [0, 0, 0].
<code>bold()</code>	This function changes the text style to bold .
<code>openparent()</code>	Only applies to tree (p. 120) items. If included in the expression, the immediate parent node to the node being processed is initially presented in expanded form.
<code>openparents()</code>	Only applies to tree (p. 120) items. If included in the expression, the immediate parent node to the node being processed and all of its parents, continuing up the tree to the root node, are initially presented in expanded form.

All of the functions in the Python math module are available as well as most of the functions in the `str` module. The functions/variables documented in the condition expressions can also be used in a formatting expression. For example:

```
pal("picnic", avg("Pressure") - 2.0*std("Pressure"), avg("Pressure") + 2.0*std("Pressure"), 5) + bold() + contra
```

Layout Templates

A layout template is used to organize the display of the HTML representation of items or child templates. It can filter, sort and otherwise organize data items, but not generate new data items and cannot modify existing data items (with the exception of being able to set display properties).

When a layout template generates content for inclusion in the current report, the template is passed a list of data items from its parent or an initial search. The template then performs a collection of operations common to all layout templates. Some specialized templates may handle these options a little differently than the default. Those differences are noted in the documentation for the individual layout template types. The execution of these common operations is sequential and follows these steps:

1. The template can contain an optional filter that can be used to refine the input list of data items. Alternatively, the filter can also be used to perform an independent search of the database and augment or replace the input list of data items.
2. The template can override the current collection of [report properties \(p. 83\)](#).
3. The data item list from the previous step can be sorted using a multi-key sorting system.
4. The data item list can be reduced to just the first or last items in the list (by default, the list is not modified in this step).

At this point, one of two steps is taken. If the template does not have any template children, it generates HTML blocks for each item in the processed list and place them in the report. If the template has template children, it iterates over those templates, passing the processed list to each of the templates in order. The process then repeats itself.

There are a number of predefined layout templates that can be combined to provide a flexible schema capable of laying out many different types of report pages:

- [Column Layout \(p. 140\)](#)

Places the children/items in multiple columns.

- [Panel Layout \(p. 141\)](#)

Extends the basic column layout to include captioned sections.

- [Table of Contents Layout \(p. 142\)](#)

Inserts a table of contents, list of figures or list of tables.

- [Tab Layout \(p. 144\)](#)

Places the children into different page tabs.

- [Box Layout \(p. 145\)](#)

Places the children/items into specific rectangles on a page.

- [Carousel Layout \(p. 149\)](#)

Displays items in a standard animated carousel.

- [Slider Layout \(p. 150\)](#)

Displays image items with interactive, multi-variable controls.

- [Iterator Layout \(p. 179\)](#)

Breaks items into collections and iterate the resulting sets into child layouts.

- [Linked Report \(p. 183\)](#)

Provides access to another report either via a link or inline frame.

- [Page Header \(p. 185\)](#)

Provides a header for a physically printed page.

- [Page Footer \(p. 185\)](#)

Provides a footer for a physically printed page.

- [Tag Properties \(p. 188\)](#)

Converts the tags of a collection of items into properties in the current context. This allows for data driven control over report layouts.

- [Data Filter \(p. 190\)](#)

Visualizes multiple data from tables or plots in a single widget and filter them using tag values.

- [PowerPoint \(PPTX\) \(p. 190\)](#)

Creates PowerPoint presentations with report templates and data items.

- [User Defined Content Block \(p. 190\)](#)

Exposes the Ansys Dynamic Reporting HTML document structure in a controlled fashion.

Common Layout Properties

All layout templates share a number of common options. These options can be configured using this portion of the **Template properties** pane:

The screenshot shows the 'Template properties' pane with the following sections and controls:

- Filtering**
 - Filter: `A|i_src|cont|build_imagery;A|i_tags|cont|timestep=10;`
 - Edit item filter...
 - Filter mode: Incoming Items (dropdown)
- Edit properties...
- Edit item sorting rules...
- Sort selection: All items (dropdown)
- Common Layout options**
 - Number of columns: 1 (spinner)
 - Column weights: 1.0 (text input)
 - ☐ Transpose columns/rows
 - Custom HTML header...
 - ☐ Skip if no input items

The filtering options control which data items are processed by this template and its children. Data items are passed from the parent template to this template. The filter is normally used to narrow down the incoming list of items. The filter mode also allows the filter to be applied directly to the raw database with the resulting items either replacing the incoming items or augmenting the data item list.

Filtering

See [Query Expressions \(p. 320\)](#) for more details.

Tags

All templates have a string of tags associated with them. This string defines one or more space-separated tags. These can be edited in the second column of the editor. These tags can be used to accelerate searches for specific layouts, but they can also be used to augment reports. [Generator templates \(p. 192\)](#) have the option of merging these tags into the items that they generate.

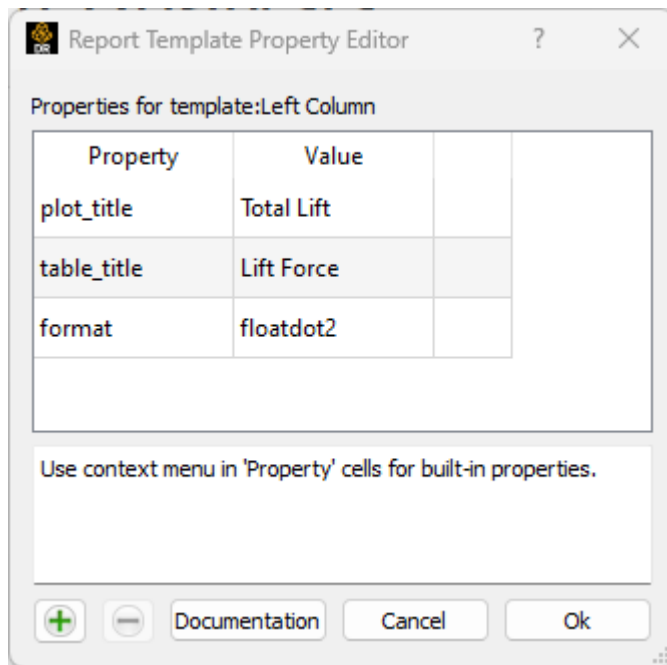
There is one special tag that may be useful when debugging a template. If any layout template has the substring `__INACTIVE__` in its tag string, it is skipped when the report is rendered. This can be helpful when experimenting with various template options or to simplify parts of a report while the settings are tweaked.

Properties

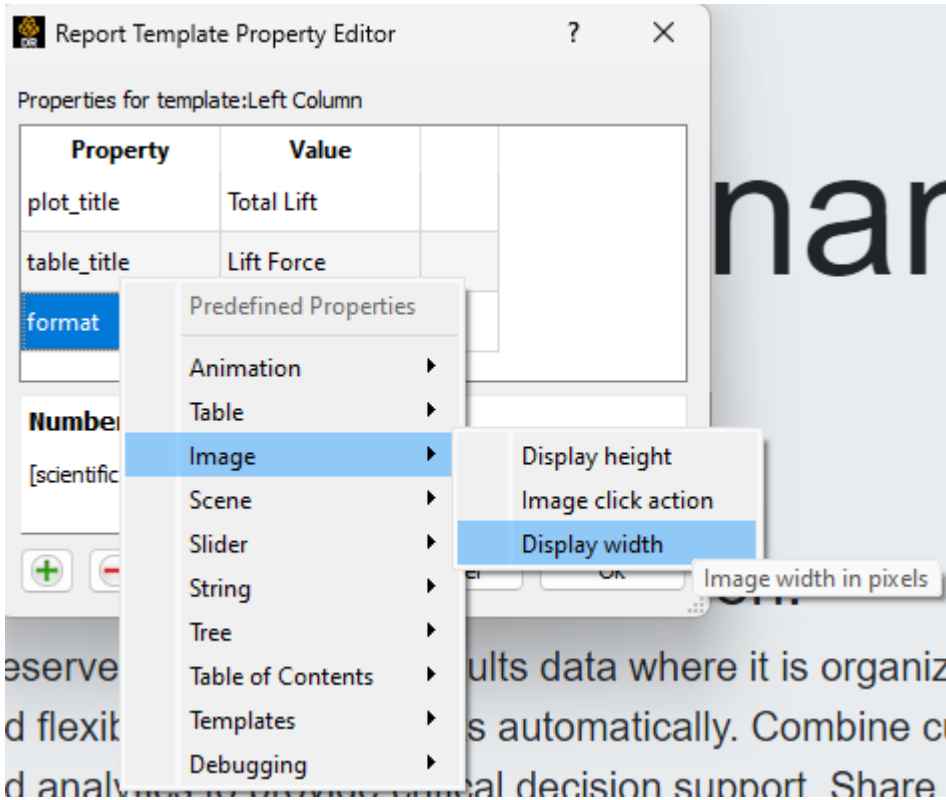
At each step in the report generation process, data items and layouts have access to a running collection of properties. This collection of properties is referred to as the rendering context. When performing various operations, a data item or layout template references the properties in the current rendering context to customize the output they generate. For example the, `plot_title` property

is used to generate the title for a plot, potentially overriding any title that might be stored for the plot source in the database. The contents of the context are hierarchical. If a template specifies a specific title for a plot, that title will be inherited by all of the children of the template. That property can be overridden by a child template at any time. Properties can also be used in [macros \(p. 83\)](#), in HTML text strings, or in other properties.

When processing a specific layout template, after the input data items have been filtered, the layout can override various properties which control specific display attributes. Open the **Report Template Property Editor** dialog by clicking **Edit properties**.

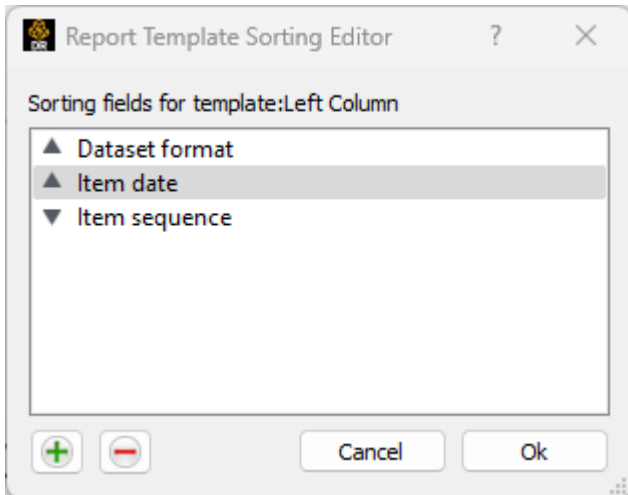


Use the dialog to add new properties and remove existing ones. There are no inherent limits on the names of the properties, other than they cannot include spaces or punctuation other than the underscore (`_`) character. Generic properties can be used in the [macro substitution \(p. 83\)](#) mechanism to customize various displays. Pre-defined properties also exist and are documented with the specific data items. Right-clicking on the property opens a list (organized by item type) of the various properties and in many cases, simple examples of the values that can be changed.



Sorting

The filtered list of items can also be sorted using a multi-key sort. Open the **Report Template Sorting Editor** dialog by clicking **Edit item sorting rules**.



The buttons in the lower left corner allow you to change the number of fields to use as keys. Right-clicking a key allows you to select specific item, session, or dataset keys. Sorting is performed top to bottom in the dialog, so items are primarily sorted by the dataset format field and in cases where they are the same, the item date field is used. The order can be changed by dragging the fields up and down within the dialog. Individual keys can be sorted in different directions by clicking on the triangles to the left of the key names. The triangles point toward the minimum value.

Once a specific sorting sequence has been established, the **Sort selection** list in the **Template properties** pane allows you to further filter the list of items to only the first or last item in the sorted sequence. This can be useful when designing a report that displays the most recent values for some piece of data.

Skip Layout

The **Skip if no input items** check box allows the layout to not be displayed if no items are contained in the template. This option is off by default. If turned on, the template and all its children are displayed only if there are items at the current level.

Common Layout Display Features

There are a number of additional layout properties that nearly all layout templates use (layout templates that re-define the display area like [Boxes \(p. 145\)](#) and [Carousel \(p. 149\)](#) may not support all of these features).

Property	Description
items_as_links	Controls how individual items are displayed. Values are: • 0: Items displayed normally (default) • 1: Uses link to the item display page
column_count_item_limit	Control the maximum number of columns to use in situations where the number of items is less than the column count. This allows for dynamic column adjustment. Values are: • 0: Ignore item count when determining the column count (default) • 1: If the number of items is less than the column count, the column count is limited to the number of items to be displayed. For example if the template has four columns specified and only two items are to be displayed, then the column count is reduced to two.
margin_left	Size of the margin between the layout content and the layout left boundary in points. The default value is -1 for the default HTML item margins.
margin_right	Size of the margin between the layout content and the layout right boundary in points. The default value is -1 for the default HTML item margins.
margin_top	Size of the margin between the layout content and the layout top boundary in points. The default value is -1 for the default HTML item margins.
margin_bottom	Size of the margin between the layout content and the layout bottom boundary in points. The default value is -1 for the default HTML item margins.

Property	Description
margin_line	By default, layout contents begin with a line break. Setting this property to 0 removes that line break. The default value is 1.

Margin Properties

By default, every layer of a template adds some spacing between the template border and the layout content region. This means that if you nest layouts, the content region becomes smaller and smaller with each level of nesting. The margin properties can be used to remove, or explicitly set, the spacing. This allows for templates to nest without content region reduction (for example, `margin_left=0`, `margin_right=0`, `margin_top=0`, `margin_bottom=0`, `margin_line=0`), or explicitly with one inch right and left margins (`margin_left=72`, `margin_right=72`), and so on. The margin properties are supported by most, but not all of the layout templates. They are supported for the basic [Columns \(p. 140\)](#), [Panel \(p. 141\)](#), [Page Header \(p. 185\)](#), and [Page Footer \(p. 185\)](#) layout templates.

Using Custom Web Components to Override the Report Style

A common use case for external web applications to leverage Ansys Dynamic Reporting is by embedding the report in an iframe as a widget. Using iframes can cause the following issues:

- Potentially expose security vulnerabilities to the hosting applications.
- Overriding the report style within iframes is difficult.

You can now use the new custom web component `<adr-report/>` through the PyDynamicReporting API instead of `<iframe/>`, in this case to help mitigate the security issues. For styling overwrite, using `<adr-report/>` can overwrite several report templates. These editable report templates are wrapped in the following web components to encapsulating the CSS styles. Restructuring the layout templates with custom web components provides more control over the report styling.

For a detailed tutorial of how to overwrite report styles, see the PyDynamicReporting [advanced example](#) (version 0.8 or later, or dev).

Available Report Layouts for Styles Overwrite

In the 2025 R1 release, the layout templates support the following web components:

Component	Description
<code><adr-report-root></adr-report-root></code>	The root container of the entire Ansys Dynamic Reporting report.
<code><adr-data-item></adr-data-item></code>	The container for data items (p. 91) .
<code><adr-tabs></adr-tabs></code>	The container for the Tabs (p. 144) layout template.

Component	Description
<code><adr-panel></adr-panel></code>	The container for the Panel (p. 141) layout template.

Note:

In this release, only the **Tabs** and **Panel** layout templates are supported. More templates will be supported in future releases.

Server Security

PyDynamicReporting provides a method to fetch and embed reports using the `<adr-report />` custom web component.

For documentation on how to use `<adr-report />` to fetch and embed reports, see the PyDynamicReporting [Report API](#) (version 0.8 or later, or dev).

Columns

The columns layout provides a simple mechanism for organizing the input data items into one or more columns with a header. It is unique in that it does not generate any border spacing around children, so can be stacked without introducing extraneous spacing.

Configuration Options

This layout is generally defined by the common layout template options:

Common Layout options

Number of columns: 2

Column weights: 1.0, 1.0

☐ Transpose columns/rows

Custom HTML header...

In the dialog, first select the **Number of columns** to subdivide the horizontal space occupied by this layout into. Once the number of columns have been set, then set the **Column weights**, one value per column. The ratio of the column widths is governed by the column weights. For example, three columns with weights: 1.0, 1.0, 1.0 (or 5., 5., 5.) evenly subdivide the widths into thirds. Column weights of 10., 5., 5. create three columns with the first being twice the width of the next two.

Ansys Dynamic Reporting currently rounds ratios to the nearest 1/12, so the actual widths are limited to that granularity. This limit is imposed in a hierarchical fashion, so a top level 1/12 width column can still be subdivided into 1/12ths by a child layout template.

If a layout template has no child templates, the columns are filled with the (filtered and sorted) list of data items. If the layout template does have child templates, the child templates themselves will be placed in the columns and the display disposition of data items passed to them is controlled by the child template, not the parent template.

When placing items into the columns, the default algorithm is to start in the left-most column with the first item and then place the next item in the column to the right. When the number of columns is reached, the next item is placed in the left-most column again, right under the previous entry in that column. The **Transpose columns/rows** option causes items to fill downward in the first column and then to fill downward in the next column to the right until all of the items are exhausted. The number of items placed in the first column is selected so that the number of items in the first column never exceeds the number in any other column by more than one item.

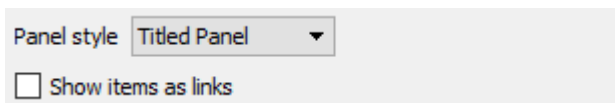
Many layouts display a header block before they display their item/child layout content. This header is expressed as a snippet of HTML code than can be edited using the **Custom HTML header** button. This header supports [macro substitution \(p. 83\)](#) from the current report context. This header is commonly used by the [Panel \(p. 141\)](#) template to provide descriptive decoration of its contents.

Panel

The panel layout is similar to the column layout, except that it draws a more formal boundary around its children that can be styled in different ways. It also allows you to specify a link to the item display page instead of including the HTML for an item inline.

Configuration Options

There are two options specific to this layout:

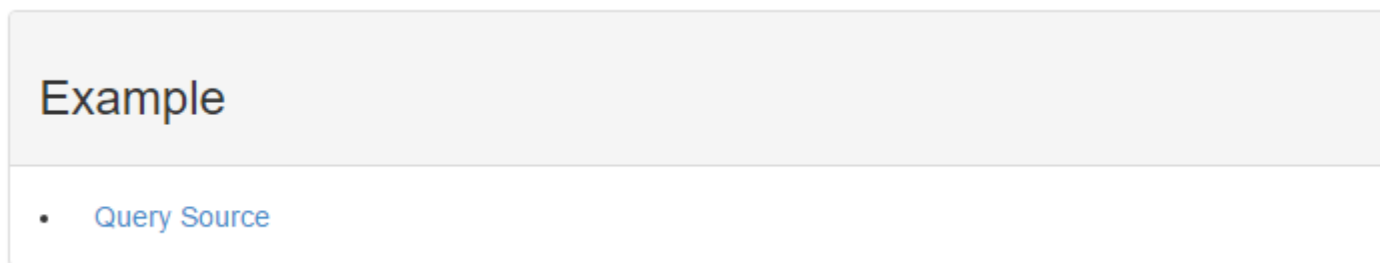
A configuration panel with a light gray background. It contains a 'Panel style' dropdown menu with 'Titled Panel' selected and a checkbox labeled 'Show items as links' which is currently unchecked.

Panel style Titled Panel ▼

☐ Show items as links

The panel styles are:

Titled Panel



Callout Panel

Example

- [Query Source](#)

Note:

In these examples, the **Show items as links** option has been selected and the custom HTML header has been set to `<h3>Example</h3>`.

The other callout panels differ in the color of the bar on the left side of the callout. The Error case is red, the Warning case is orange-yellow, the Success case is green, and the Info case is blue.

The **Show items as links** option places a URL link in the report that can be used to display the actual data item content instead of placing the content inline.

Properties

Property	Description
items_as_links	Controls how individual items are displayed. Values are: • 0: Items displayed normally (default) • 1: Uses link to the item display page
panel_template_toggle	Adds or hides an interactive icon in the rendered HTML that provides hide/show capabilities to the panel. Values are: • 0: Icon hidden (default) • 1: Icon displayed

Table of Contents

Items and layouts can be marked as table of contents items, figures, or tables. See [Indexing Options \(p. 88\)](#) for details of how to mark entities. Ansys Dynamic Reporting collects the entities and assign numbers to them as needed. Marking the entities does not automatically cause them to be displayed in the report. This template is used to place the various index tables in the report. They are inserted where the layout templates would appear.

The index tables do not include titles/headers, but the standard **Custom HTML Header** option can be used to add such a header. You can also use the table of contents properties in the HTML to place numbering in these headers as well. This layout is still a normal layout and displays all of the items that are passed to it if not filtered out. Ensure that you add an item filter to hide the incoming items.

There are three types of lists that can be inserted.

Table of Contents

An example table of contents might look like this:

- 1 TOC Example
- 2 Table of Contents
- 3 Data
 - 3.1 Plots
 - 3.1.1 Plane tables
 - 3.1.2 Cube tables
 - 3.2 Images
- 4 List of Tables
- 5 List of Figures

Each entity marked as `TOCItem` is displayed in this list, along with the auto-generated numbering. The numbering follows the level selections for the items. Each entry is an HTML link, and clicking it will move the browser to the appropriate location in the report. The default name for the link is the name of the report template or data item. The `TOCname` property can be used to override that name.

List of Figures

An example list of figures might look like this:

- 1. Image Source
- 2. Image Source
- 3. Image Source
- 4. Image Source
- 5. Image Source

Each entity marked as `TOCFigure` is displayed in this list, along with the auto-generated numbering. Each entry is an HTML link, and clicking it will move the browser to the appropriate location in the report. The default name for the link is the name of the report template or data item. The `TOCname` property can be used to override that name.

List of Tables

An example list of tables might look like this:

- 1. Plane tables
- 2. Query Source
- 3. Query Source
- 4. Query Source

Each entity marked as `TOCTable` is displayed in this list, along with the auto-generated numbering. Each entry is an HTML link, and clicking it will move the browser to the appropriate location in the

report. The default name for the link is the name of the report template or data item. The TOCname property can be used to override that name.

Configuration Options

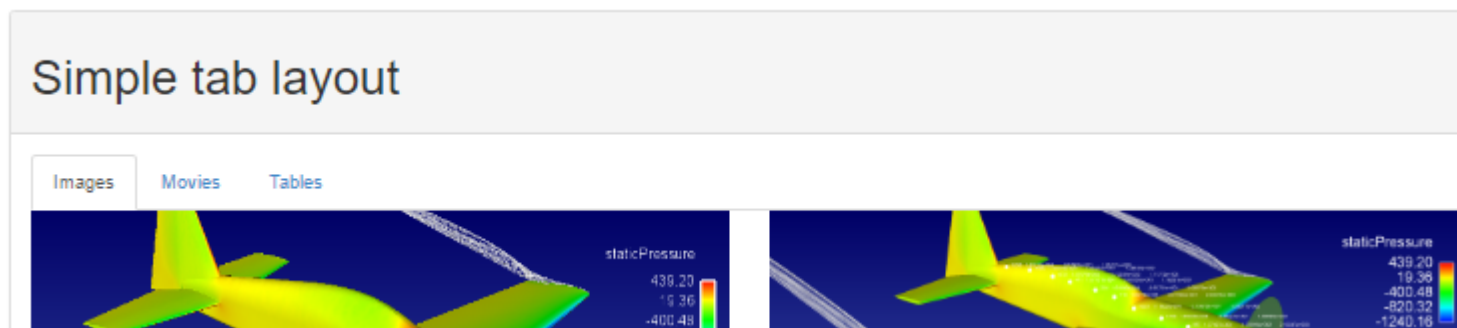
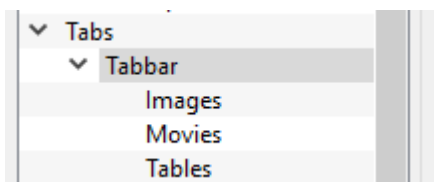
The only configuration option is to select what type of index list this layout should generate.

☒ Table of Contents
☐ Figure List
☐ Table List

Tabs

The tab layout is normally used with layout template children. It creates a tab for each child and then fills the tab with the resulting contents. You can click the tabs to switch between sub-page content. Each tab is named using the name of the child report template.

In this example, for the template tree **Tabs**, **Tabbar** is a tab layout and the child tabs are displayed as shown in the following image. The **Images** report template is a two column [Columns \(p. 140\)](#) layout template.



Ensure that you do not place too much content into a large number of tabs. All of the content for all of the tabs are sent to the browser when the page is generated, whether the content is displayed or not. The HTML payload can end up being several times the size of other layout options.

Configuration Options

None.

Properties

Property	Description
in-line_tabs	Controls how tabs are shown. Values are: • 0: Displays tabs inline (all displayed) • 1: Displays tabs in a HTML tab set (default)

Boxes

The boxes layout is a bit different than the other layouts. Most layouts are linear and their content flows down a page. Additionally, they adjust their size to react to changes in the browser window size. The box layout only supports template children. If it has no template children, it will act like a column layout. If it has template children, they are manually laid out relative to each other in the layout editor. Each child can be sized/placed with pixel precision. This can be used for precise layouts, at the expense of dynamic layout flexibility.

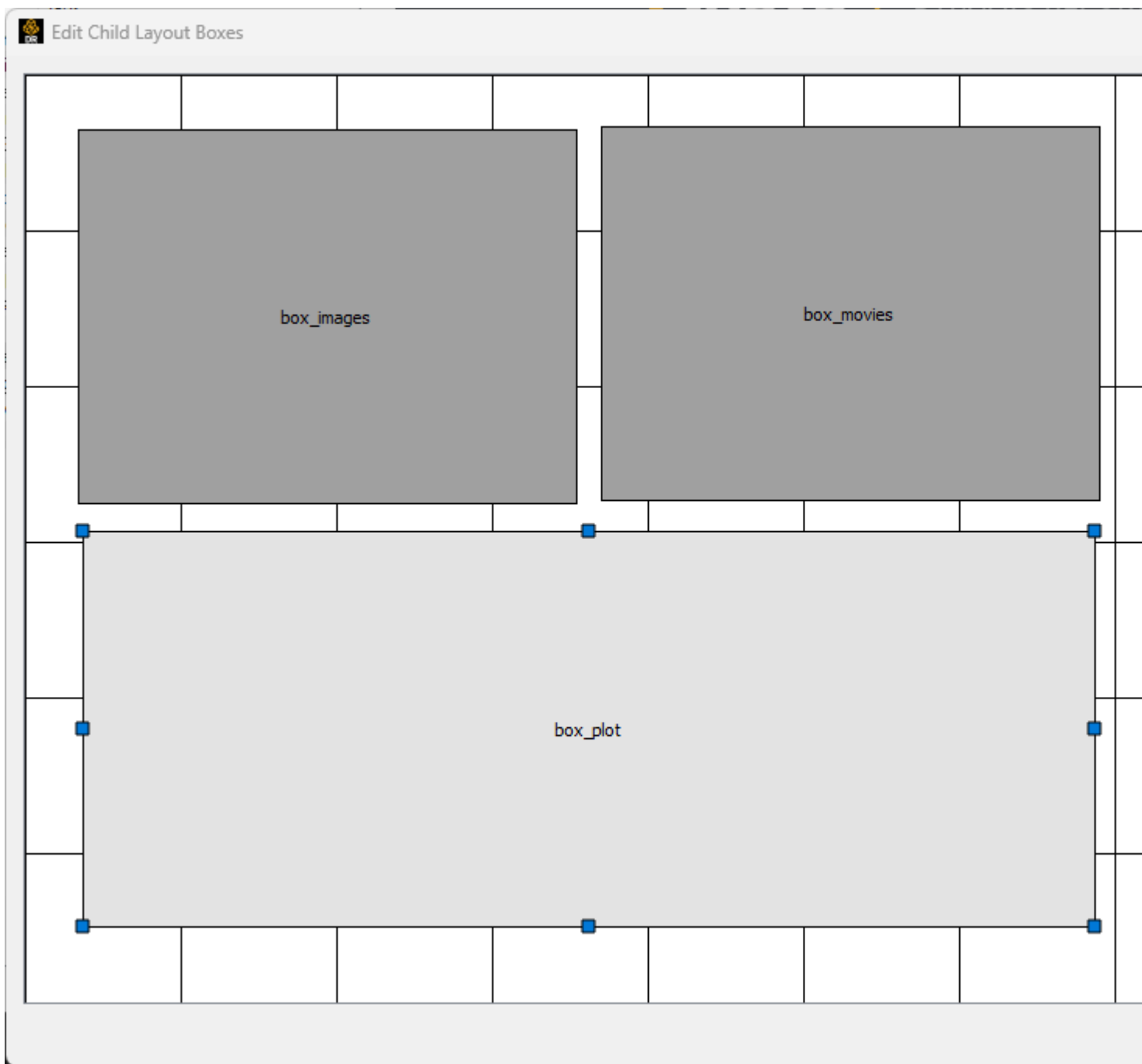
Configuration Options

Much of the layout configuration is provided by the child templates, but individual child rectangles and clipping options must be set for each child template. Click the **Edit child layout** button to open the **Edit Child Layout Boxes** dialog, allowing you to modify these settings.

For a template tree that looks like this:

▼ Box Template	boxes
box_images	images
box_movies	movies
box_plot	

The editor looks something like this:



Each of the child layouts is displayed as a rectangle, labeled by the name of the layout they represent. You can select individual layouts by clicking the rectangles, or on the list of children on the right-hand side of the dialog. The position and size of the child rectangle are shown for the selected template. You can left-click and drag boxes to change their positions, or use the blue handles on the box to resize it. You can also set the values explicitly using the controls on the right. The stacking order of the layouts is controlled by the order of the children under their parent. Boxes are drawn in that order and can overlap, making things like overlay possible. Note that in most cases, the last drawn child in an overlapped stack gets mouse input events over the templates it overlaps.

Each child template renders its contents in the fixed size rectangle as laid out by the editor. The contents of the rectangle can exceed the size of the rectangle. In which case, the **Clip** option decides how the contents are clipped. The options are:

- **To self**

The content is clipped to the boundary of the rectangle.

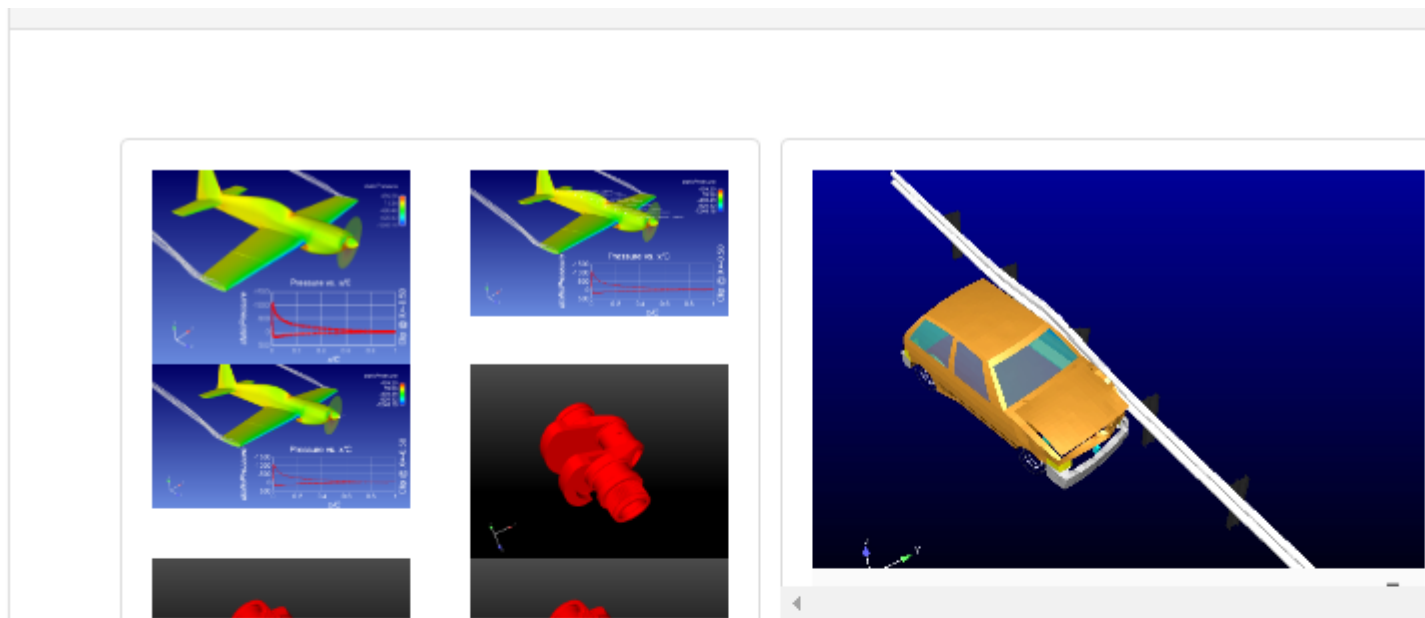
- **Scroll**

The rectangle has scrollbars and the contents can scroll.

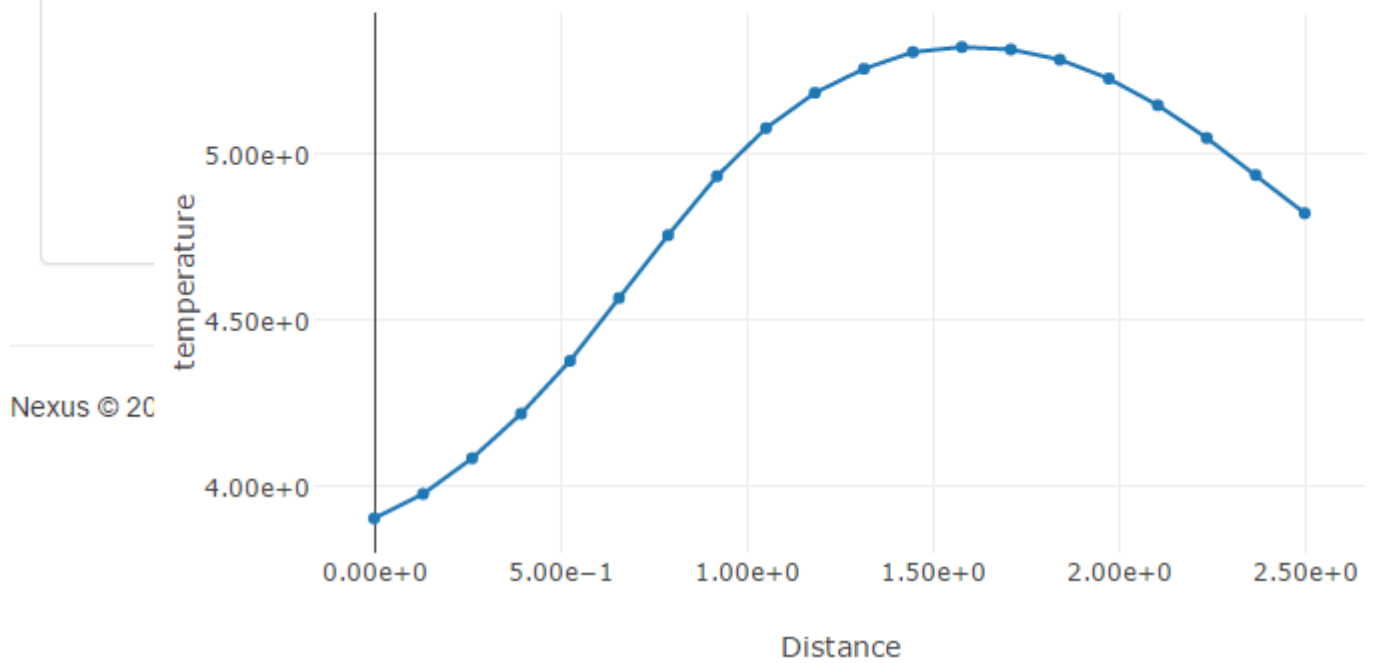
- **None**

The content is not clipped in any way and may overlap other content.

In the example, each of the child templates filters data objects of the same name as the template. The **box_images** template is set to **To self**, **box_movies** is set to **Scroll** and **box_plot** has clipping set to **None**. The resulting page appears like this:



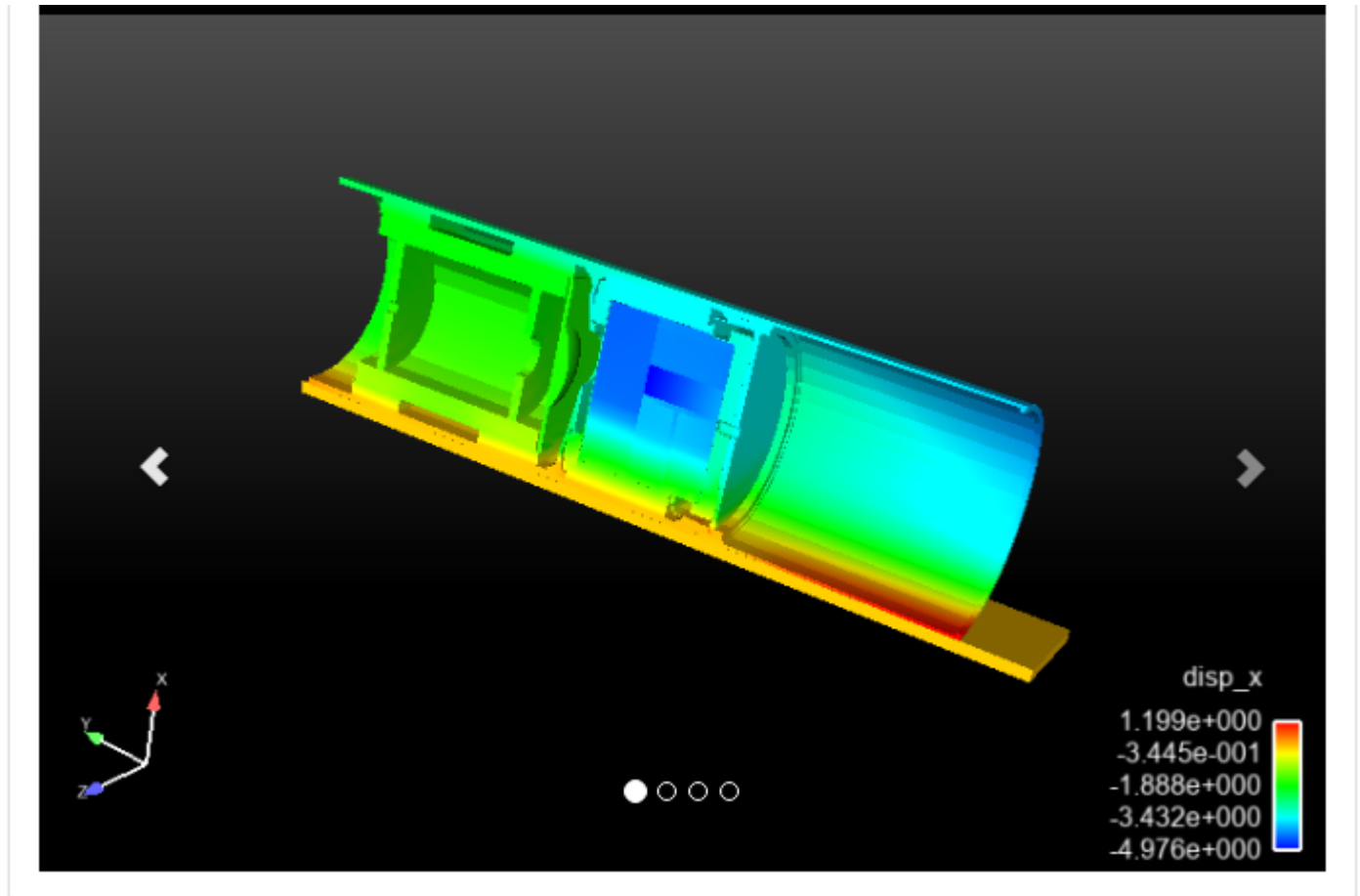
temperature vs. Distance for line tool



The panel of images is clipped to the rectangle laid out in the editor, the animation rectangle is scrollable and the plot at the bottom is not clipped at all, and in fact extends past the bottom of the parent panel.

Carousel

The carousel presents the data items passed in a panel with controls to pan from data item to data item. The most common usage is as an image carousel. You can move from item to item with arrow controls or directly with slide dots. The items can also be dynamically animated. Any data item type can be presented in the carousel layout. An example (that includes slide dots) might look like this:



Configuration Options

The carousel layout has two options. By default, the content can only be moved by clicking on the right and left arrows. If **Animated** is selected, the display advances without any user interaction. The other option is to include slide dots along the bottom of the item. Clicking these dots causes the display to jump to the specific data item. If there is not enough space for all of the slide dots, none are displayed, even if the option is selected.

- ☐ Animated
- ☐ Include slide dots

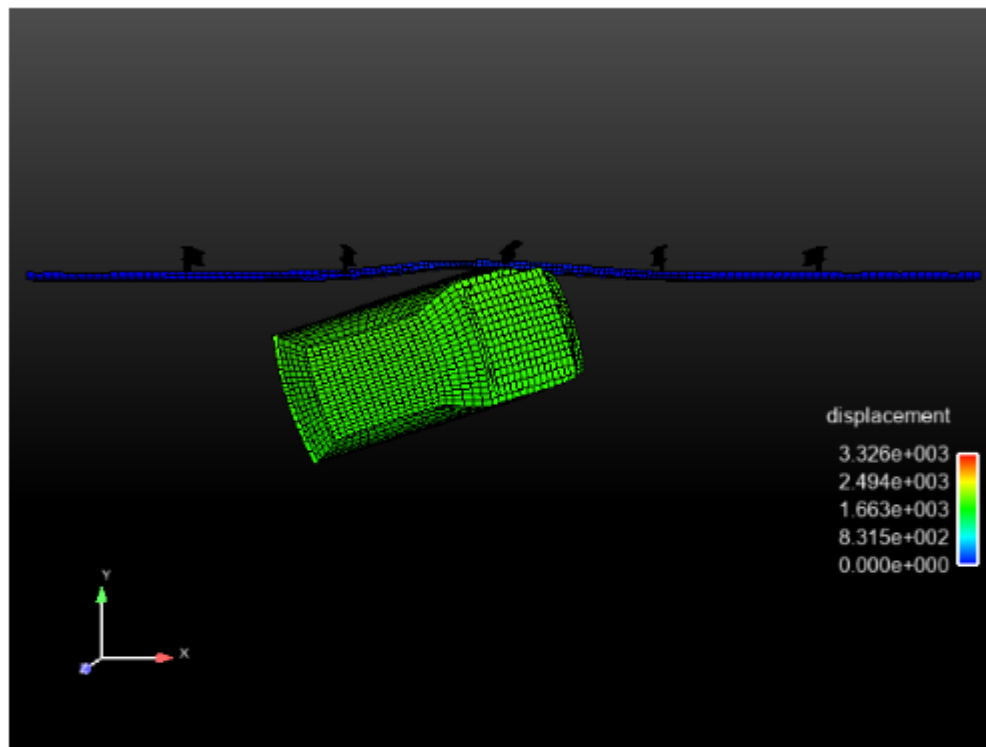
Slider

The slider layout is used to show a collection of image items under user control. It displays a single image item with an optional title under the image. Beneath that are one or more sliders that allow you to select between the various objects.

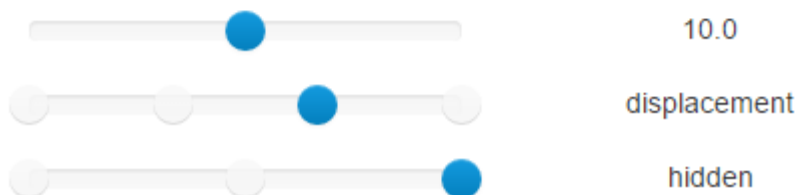
You can use the `type_filter` property (p. 160) to select image, scene, or animation data items for display. Normally, a single panel containing a data item is displayed. If `image_comparison` is set to a non-zero value, two panels are displayed, allowing for direct comparison of two items.

Image Item Mode

This is the default mode. The incoming layout items are filtered down to images. An example is shown below:



variable=displacement timestep=10.0



The slider template first selects the image items to be displayed. You do not need to filter in incoming data item list to the image type, that part of the filtering is implicit. It then uses tag values to sort the images into dimensions based on the tags. It generates one slider per dimension (tag) and the number of positions of that slider is equal to based on the number of values that tag can have. In

the example above, there are three tags: *timestep*, *variable*, and *mode*. The first is a number, the second the name of a variable, and the third a simple string. These names are set in the configuration options.

Each of the tags is represented using a slider. You select a value from each slider and the title (below the image) and the values to the right of the sliders update to reflect the new settings. The representative image is fetched from the ADR server and displayed. If a set of tags selects more than one image (in the example, if more than one image has the same timestep, variable, and mode tags), an extra slider is added automatically at the bottom of the sliders to allow you to view all of those images. The visibility of this slider is dynamic, based on the number of images at the current set of tag values. There is a property (`first_image_only`) that selects only the first image be displayed and no extra slider be displayed.

An image set in a slider can be made of images of two types:

- Simple PNG image

These are simple images with the `*.png` extension that can be generated from EnSight or uploaded.

- Deep/Enhanced image

These are TIFF images with the `*.tif` extension that can only be generated from EnSight by turning on enhanced mode during generation.

Image sets can only be simple or deep but cannot be a combination of both.

Enhanced images, also known as deep images, are a step beyond plain PNG images and are rich with information. They contain three layers of information:

- The color layer

Contains the RGB colors like a basic PNG image. This layer is what is initially visible.

- The part layer

Contains information on parts in each pixel of the image. These can be hidden parts.

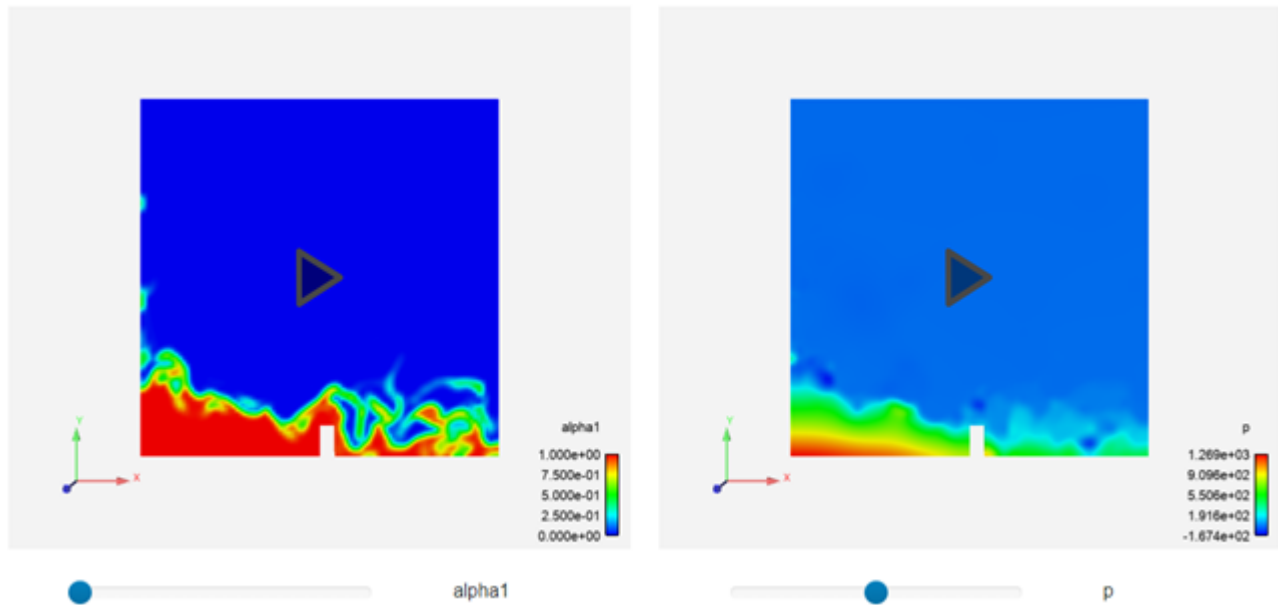
- The variable-value layer

Contains information on the variable and its corresponding value for each part. These can be values for hidden parts.

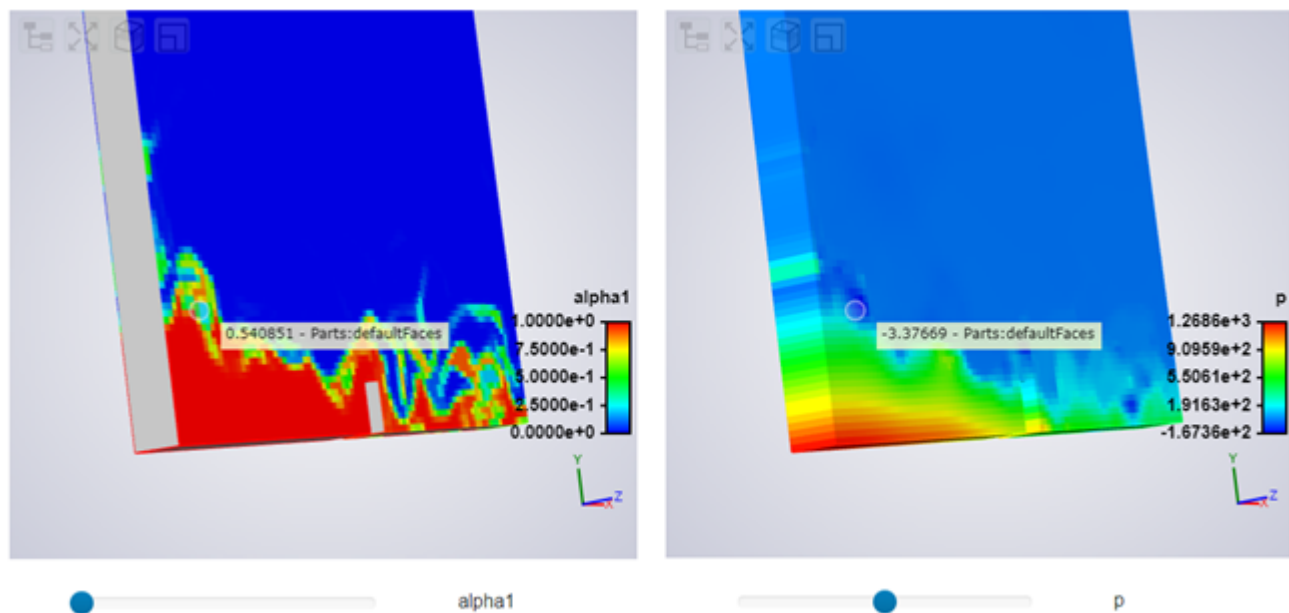
EnSight has the [option](#) to generate enhanced images that contain the above information. Once you have used EnSight to generate a collection of such images, you can then create a slider layout with them using the Template Editor. Ansys Dynamic Reporting then automatically recognizes the image's format and makes the appropriate features available in the slider layout based on the format.

Scene Item Mode

Setting the `type_filter` property to `scene` filters the incoming layout items down to 3D scenes. An example with the `image_comparison` property enabled is shown below:

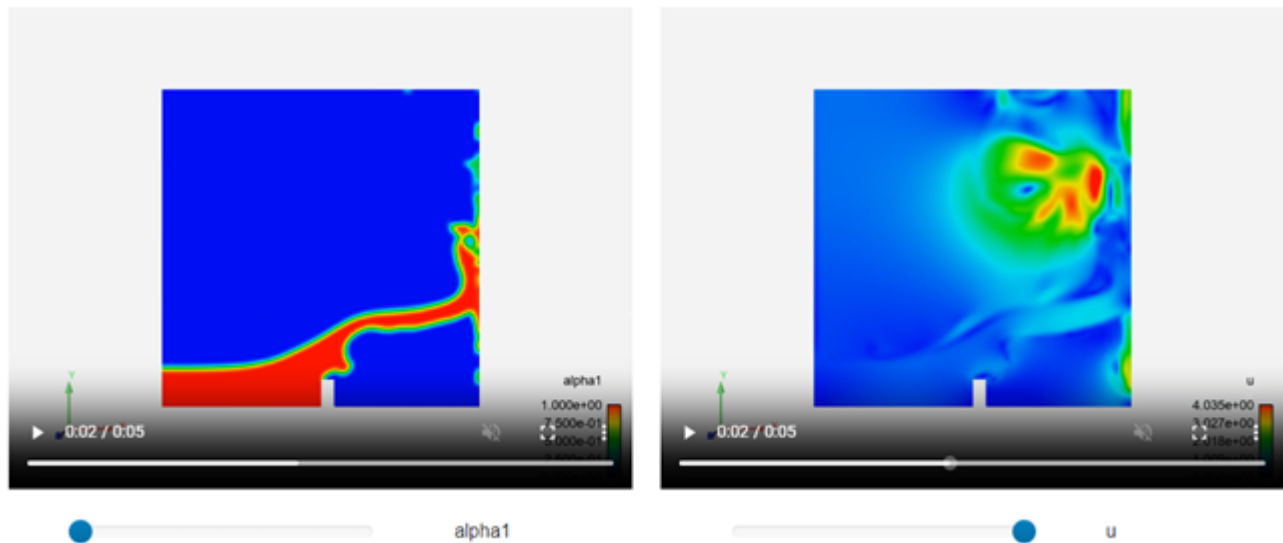


The 3D scenes can be activated for display by clicking on them. Camera controls can be linked using the `Linked_views` property. In this mode, parallel data probing is supported.



Animation Item Mode

Setting the `type_filter` property to `anim` filters the incoming layout items down to animations. An example with the `image_comparison` property enabled is shown below:



Animations can be synchronized using the `linked_views` property. Start, stop, and click-and-drag scrubbing can all be linked.

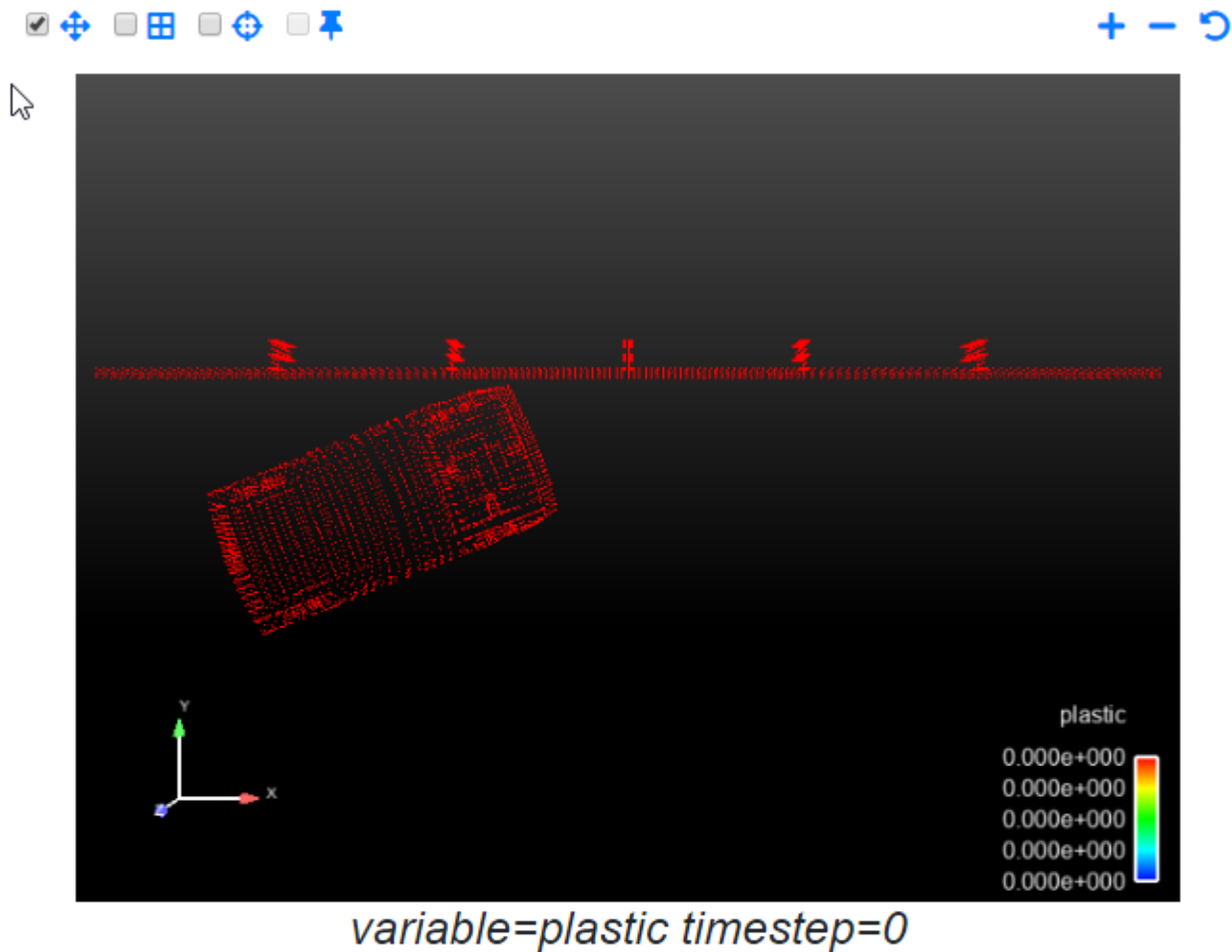
Related topics:

- [Image Controls](#)
- [Slider Configuration Options](#)
- [Slider Properties](#)
- [Slider Events](#)
- [Click and Drag Settings](#)
- [Comparison Sliders](#)

Image Controls

Image controls are a set of controls to enhance the interaction with slider images.

You may use them to control the display of single image sliders as well as both the images at the same time in a comparison slider.



The following controls are available:

Control	Icon	Description
Pan	☒	Pans a zoomed image.
Grid	☐	Displays a grid overlay over the image. This can be helpful when trying to zoom or focus on a particular point in the image.
Probe	☐	Probes for per-pixel information from the image. The information available from the displayed images are the RGB values of each

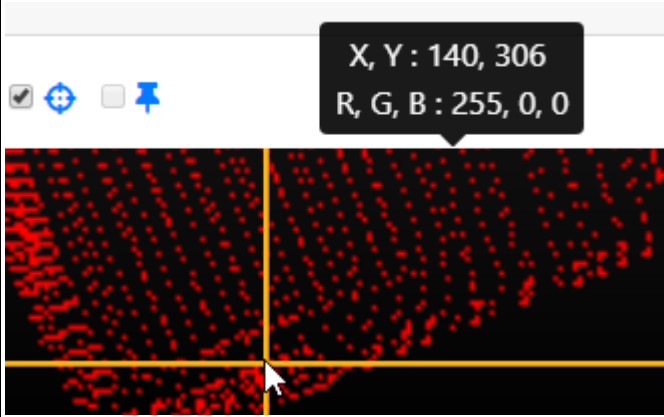
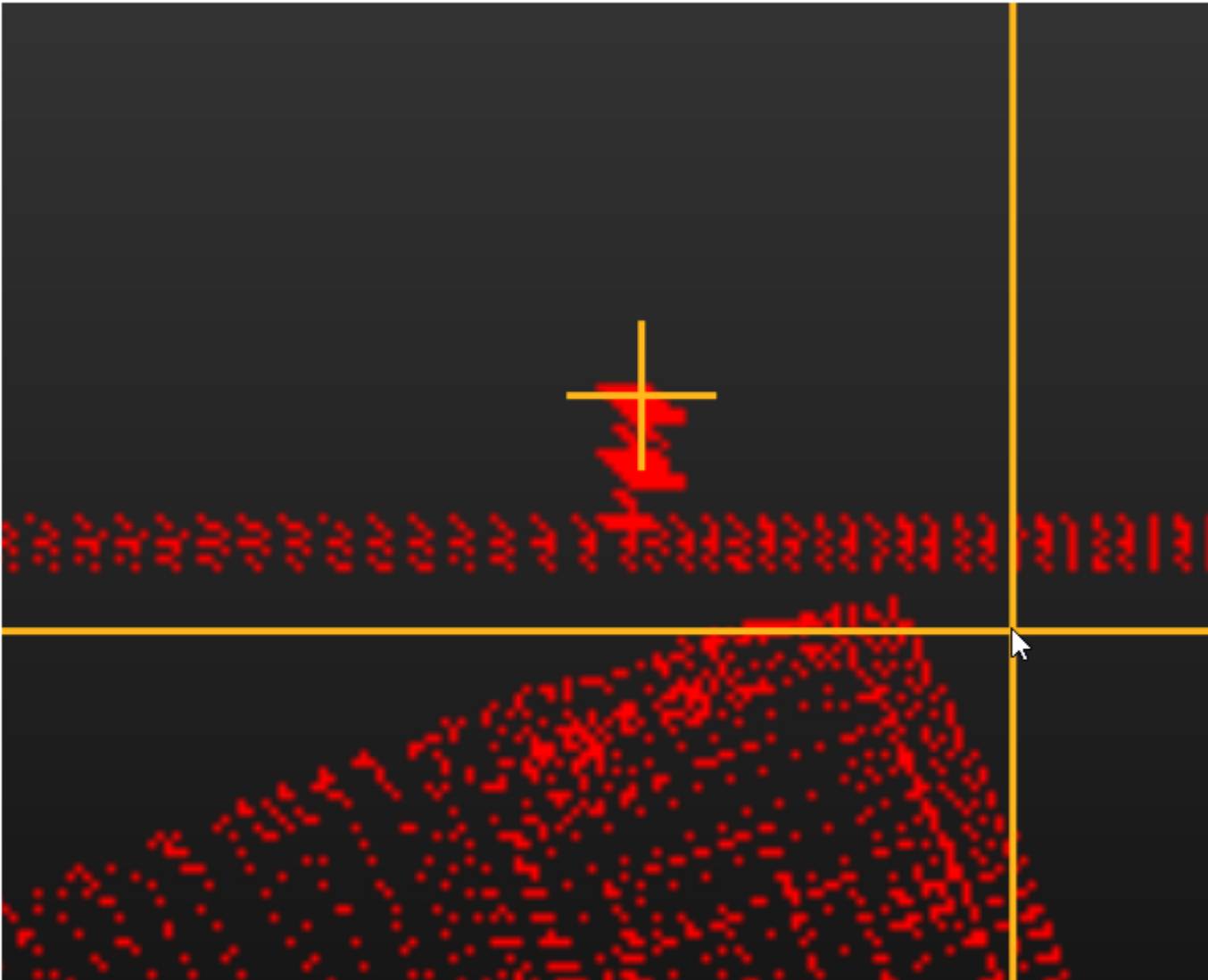
Control	Icon	Description
		pixel along with the X, Y coordinates of the image (origin beginning from the top-left corner). 
Pin		Marks a permanent point on an image.
Zoom In		Zooms into the image.
Zoom Out		Zooms out of the image.
Reset		Resets the zoom to the initial value.

Image Control Interactions

There are a number of ways the image controls interact:

- You cannot pan an image until it is zoomed in. Use the Zoom In control first, then select Pan, and drag across the image.
- When Probe is enabled, you cannot use the Pan or Grid option and vice versa.
- To place a pin, enable Probe first and then double-click the image. The clicked point is marked on the image and the values at that point are displayed. When you try to probe again, you can see that the probe's current value is displayed alongside the pin's value. You can use this to compare values at multiple points.



variable=plastic, timestep=0

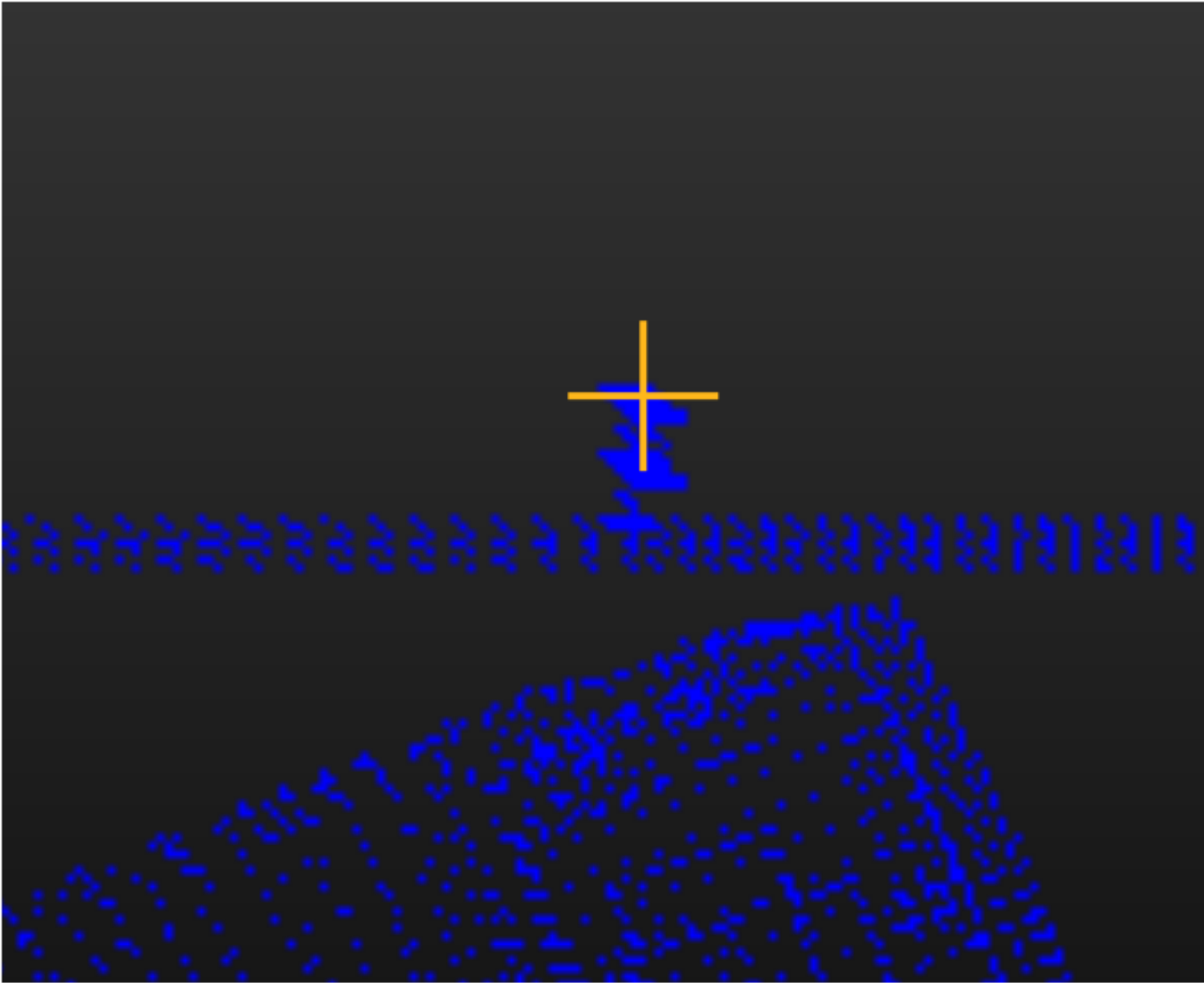
X, Y : 264, 183
R, G, B : 33, 33, 33

X, Y : 219, 155
R, G, B : 255, 0, 0



Times
Variable: p
Mode:

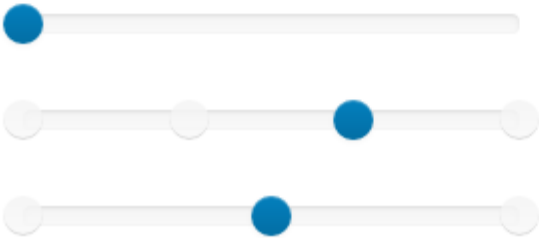
This pin persists through any action such as image changes, zooms and resets. It will, however, be lost when the window is resized. When the image changes, the value at the pinned point automatically updates.



variable=displacement timestep=0



X, Y : 219, 155
R, G, B : 0, 0, 255

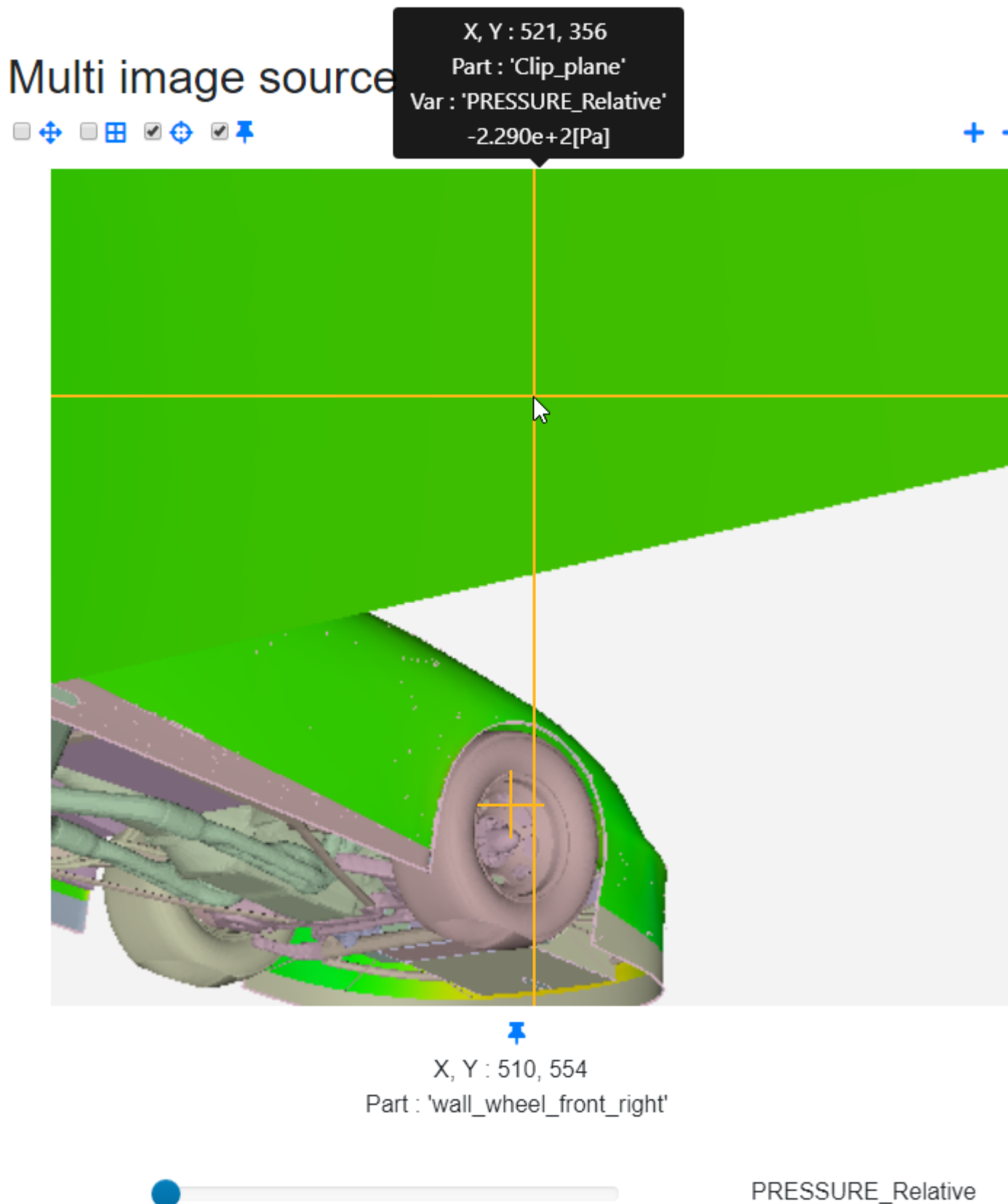


Time:

Variable: displac

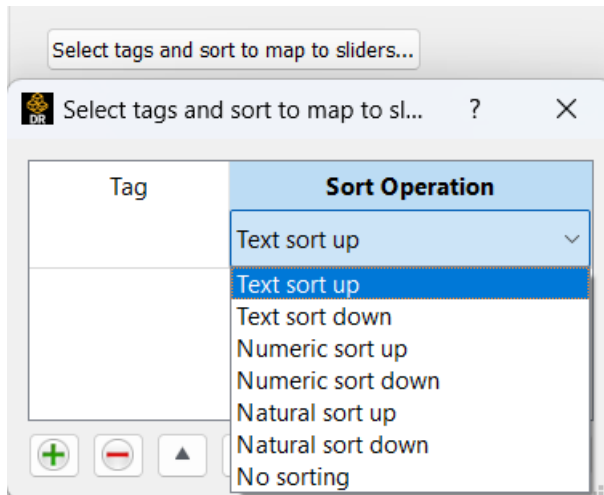
Mode:

Pin and probe may not be interesting with simple images, but are very useful when with enhanced images, as in the example below.



Slider Configuration Options

The **Select tags and sort to map to sliders** opens a dialog used to select the tags that become the sliders.



Type the name of a tag in the left column and pair it with a specific sorting option. The sort is applied to the tag values and determines the ordering of the values along the slider(s). You can sort up or down, interpreting the value as a number, text string, or applying natural sorting. An axis cannot be sorted at all, preserving the order of the data items themselves as set by the template sorting features (by date, sequence number, and so on).

Slider Properties

Several properties specifically control the display of the sliders and the titles.

Property	Description
type_filter	Sets the type of input objects that are displayed in the slider interface. Values are: • image (default) • anim • scene
common_slider_tags	A list of tag values to use for the common sliders controlling both image sections. Available when the image_comparison property is enabled. For example, [timestep, variable].
common_slider_title_width	Width of the common slider caption column in percent. Available when the image_comparison property is enabled. Default is 20. The value 0 removes the column.
common_slider_title_justification	Common slider caption justification. Available when the image_comparison property is enabled. Values are: • left (default)

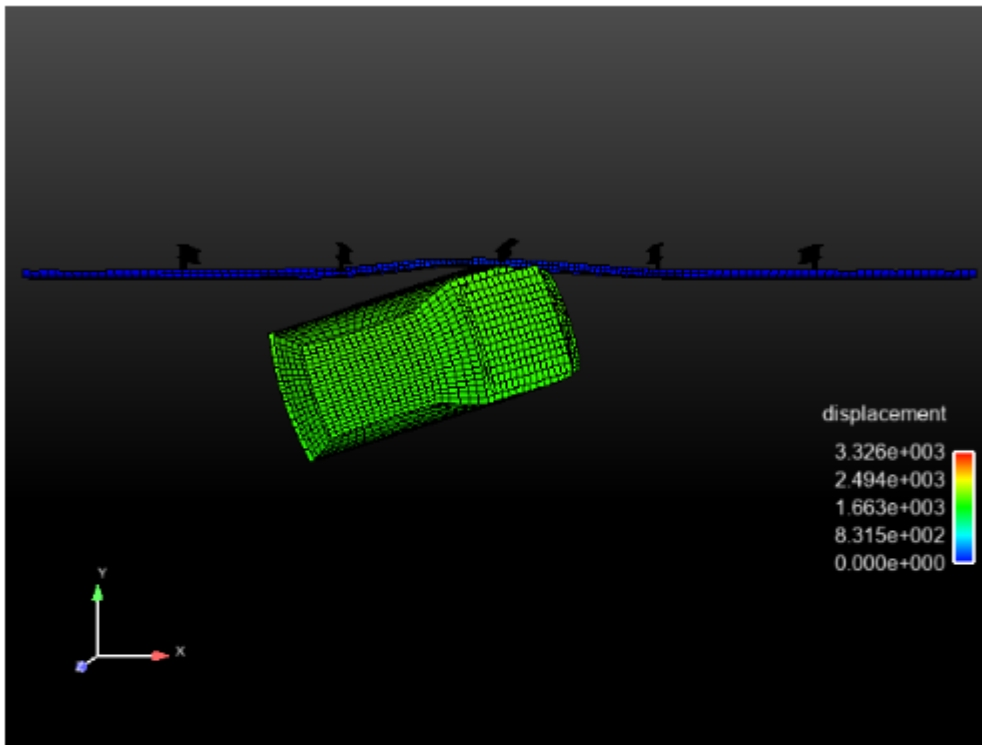
Property	Description
	• center • right • justify
first_image_only	If set to 1, the extra slider that allows you to walk through all of the images that match the current tag values is not displayed. Only the first image in the set is displayed.
height	The height of the image in pixels.
image_comparison	Flag to show/hide a copied section of the image with sliders for comparison. Values are: • 0: Comparison off (default) • 1: Comparison on
rgb_diff_comparison	Flag to enable RGB Differences feature. Values are: • 0: Disabled (default) • 1: Enabled
deep_image_comparison	Flag to enable comparison of enhanced images. Values are: • 0: Disabled (default) • 1: Enabled
comparison_initial_views	List of initial values to use as presets for the view drop-down in each section of a deep image comparison layout. Allowed values are: • this • this-other • other-this • abs For example, for the left image, these would correspond to Left, Left-Right image, Right-Left image, Abs(Right-Left image) respectively. Names must match exactly and are case sensitive, for example [this-other, this]. Default is empty. All values must be specified in the exact order as it appears in the template. Use a dash (-) to skip mentioning a value.

Property	Description
comparison_initial_vars	List of initial values to use as presets for the variable drop-down in each section of a deep image comparison layout. Variable names must match exactly and are case sensitive, for example [PRESSURE_Relative, PRESSURE_Relative]. Default is empty. All values must be specified in the exact order as it appears in the template. Use a dash (-) to skip mentioning a value.
comparison_initial_values	A list of initial values for each slider, for each section in an image comparison layout, for example [[3.8, PRESSURE_Relative], [1.4, PRESSURE_Relative]]. All values must be specified in the exact order as it appears in the template. Use a dash (-) to skip mentioning a value.
comparison_initial_palettes	List of initial values to define the palette preset which is applied after the image has loaded. One list for each section in a deep image comparison layout. Must be structured as: <pre>[[max,min,palette,invert,hide,bg],[. .section 2 here..]]</pre> where the following must be true: • max is a number (float/integer) • min is a number (float/integer) • palette name must match exactly • invert must be true or false • hide must be true or false • bg is a value between 0 and 1 An example is: <pre>[[30, -30, autumn, false, true, 0.4] , [50, -50, autumn, false, true, 0.7]]</pre> All values must be specified in the exact order as it appears in the template. Use a dash (-) to skip mentioning a value.
image_controls	Flag to show/hide image controls like pan, zoom, and so on. Available for both single image and image comparison sliders. Values are: • 0: Disabled (default) • 1: Enabled
image_link	Sets the option for image links. Values are: • 0: Image is not clickable (default) • 1: Click the image to go to a page with only the image

Property	Description
	• 2: Click the image to open it in a new tab • 3: Click the image to open the image data item details page in a new tab
image_title	String of HTML displayed beneath the image. Item and context properties can be used with the macro syntax <code>{{}}</code> . Default is "".
initial_values	List of the tag values that the sliders should be set to initially. This is an array of values, one per slider (tag). If the specific value cannot be found in the tags, the first value will be used. For example, <code>[10.0, displacement, hidden]</code> .
slider_title_justification	The justification of the slider title text in its column. Values are: • left (default) • center • right • justify
show_nodes	List of values that can be 1 or none. If 1, the steps of the corresponding slider are displayed as nodes. If none, nodes are not displayed.
slider_title	List of HTML strings in the same format as <code>image_title</code> . One for each slider. The macro expansion for each substring can include the values for any of the sliders. For example: <pre>[Timestep: {{timestep}}, Variable: {{variable}}, Mode: {{mode}}]</pre>
slider_title_width	The width of the slider title column in percent. If set to 0, slider titles are not displayed. The default is 30.
slider_xaxis_tag	Select the name of a tag that clicking and dragging (p. 167) on the image on the X axis will change the value for. No slider will be displayed for this tag.
slider_yaxis_tag	Select the name of a tag that clicking and dragging (p. 167) on the image in the Y axis will change the value for. No slider will be displayed for this tag.
slider_xaxis_scale	By default, dragging a distance the same as the width of the image in pixels causes all of the images for the specified tag to be iterated over. The default is 1.0. Increasing this value to 2.0 halves the pixel motion requirements to cause the same change in the image sequence.
slider_yaxis_scale	By default, dragging a distance the same as the height of the image in pixels causes all of the images for the specified tag to be iterated over. The default is 1.0. Increasing this value to 2.0 halves the pixel

Property	Description
	motion requirements to cause the same change in the image sequence.
slider_xaxis_clamp	By default, when mouse motion in the X axis causes the image sequence to progress beyond natural limits, the axis will wrap around to the opposite limit. If this property is set to 1, mouse motion beyond the natural limits will clamp to the minimum/maximum image in the sequence.
slider_yaxis_clamp	By default, when mouse motion in the Y axis causes the image sequence to progress beyond natural limits, the axis will wrap around to the opposite limit. If this property is set to 1, mouse motion beyond the natural limits will clamp to the minimum/maximum image in the sequence.
width	The width of the image in pixels.
linked_views	Flag to enable linked views. When set to 1: • If the <code>type_filter</code> property is set to <code>scene</code>, the 3D view cameras are connected. • If the <code>type_filter</code> property is set to <code>anim</code>, the animations are synchronized.
controls	Selects the controls for animation objects when the <code>type_filter</code> property is set to <code>anim</code> . Values are: • 0: No controls displayed (default) • 1: Display controls • 2: Autoplay animation, no controls displayed • 3: Display controls and autoplay videos

Example



variable=displacement timestep=10.0



In this example configuration, the `image_title` property has been set to:

```
<h4>variable={{variable}} timestep={{timestep}}</h4>
```

The `show_nodes` property has been set to:

```
[none, 1, 1]
```

Notice the slider value replacement in the title string and the inclusion of nodes for the latter two sliders.

Slider Events

The slider template reports status changes via JavaScript events. These events can be listened for using Ansys Dynamic Reporting features like the [User Defined Content Block \(p. 190\)](#) template.

To access the event data, add an event listener at the document or any element that is at a higher hierarchy level than the slider filter layout in the Data Object Model (DOM) tree. For example, listen and print out all event data on filtering:

```
document.addEventListener("adr.slidertemplate.update",
    function(e){console.log(e.detail) /* Other data manipulation */})
```

Supported Events

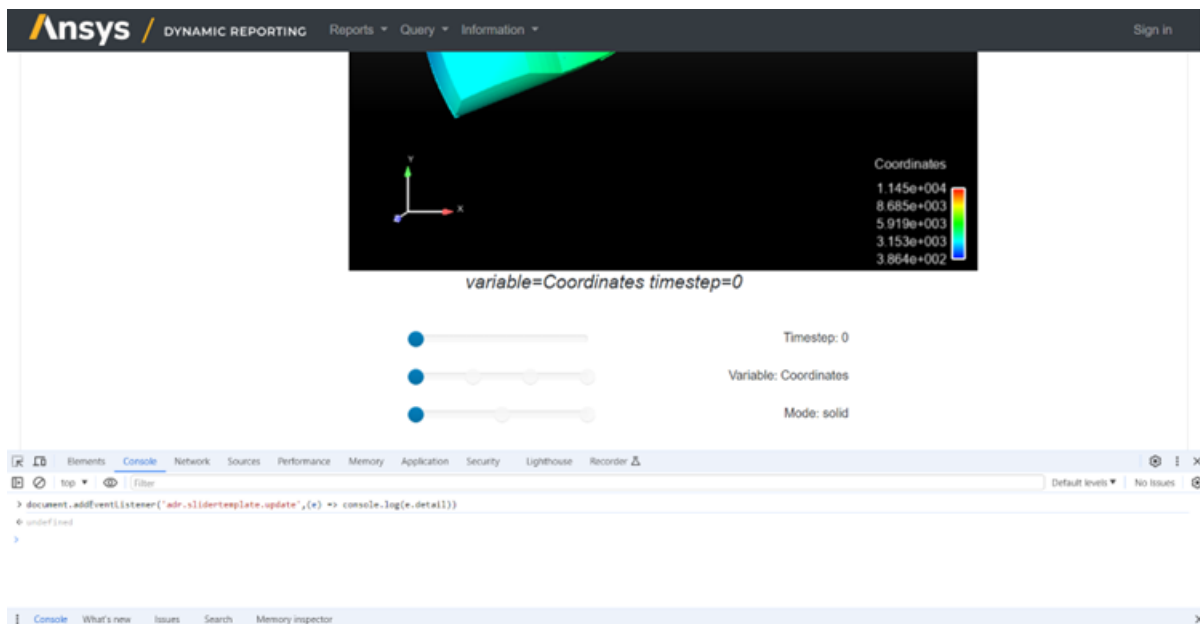
adr.slidertemplate.update

Emitted every time you make a change to the slider. The event records current status information that may be useful for external report generation manipulation. Currently supported data keys:

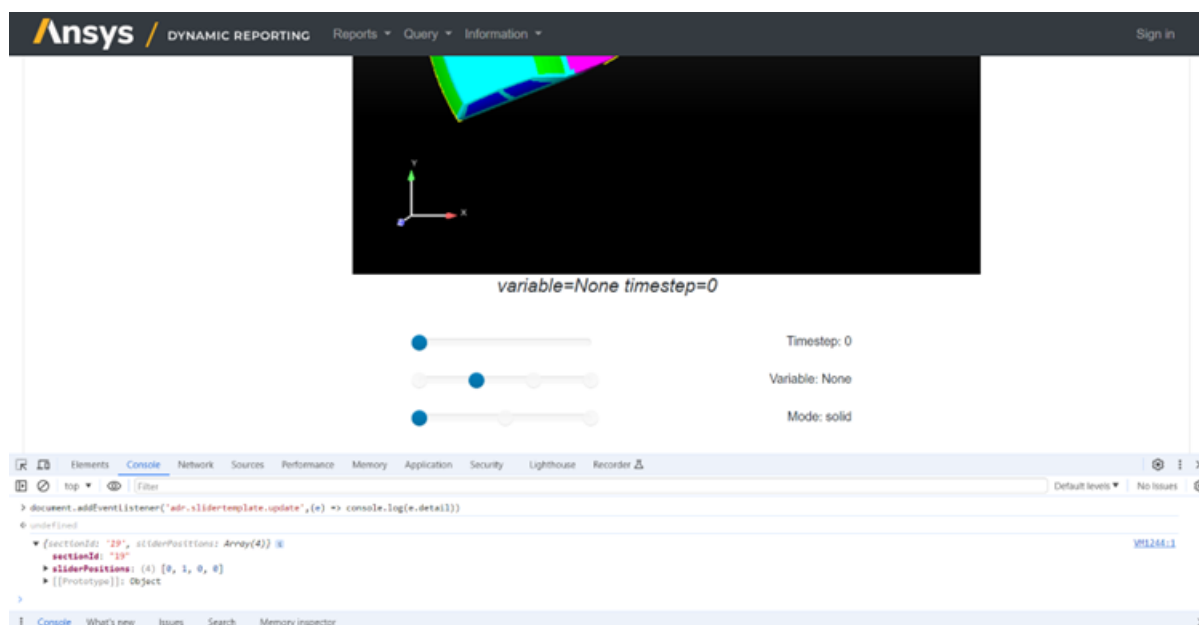
- **sectionId**: A unique ID for the slider section.
- **sliderPositions**: An array of indexes for each slider's current position index.

Example

Add an event listener to access and print the event data on the console.



The console prints out the event data on slider change.



Click and Drag Settings

To perform the same operation as dragging a tag slider, the X and Y axes can be independently linked to a specific tag and instead of displaying a slider for those tag(s), you can select the image by clicking and dragging. This can be used to simulate motion through time, clip plane position or even rotation over the screen axes.

To use this function, there are three [properties](#) (p. 160) that must be set up (they are duplicated for the x and y axes).

- `slider_xaxis_tag`

Set to the name of the tag that is mapped to the X axis. Note: if this property is set, no slider is displayed for that tag. By default, drag motion is set so that a drag over the entire image axis results in the entire domain of that tag. For example, if there are 10 different values for the tag and the image is 500 pixels wide, then dragging the mouse 50 pixels will increment/decrement the tag value one step, depending on the direction of the drag.

- `slider_xaxis_scale`

Changes the relative number of pixels of motion needed. Default is 1.0. In the previous example, with `slider_xaxis_scale` set to 2.0, dragging the mouse 25 pixels changes the tag one step. It is useful to set the scaling factors by taking into the aspect ratio of the image into account. This helps ensure that the same amount of motion in each axis has a similar effect, very useful when trying to simulate things like 3D rotation.

- `slider_xaxis_clamp`

Determines what happens when relative motion in the image window causes the tag value to increment/decrement beyond the number of items in the tag domain. By default (`slider_xaxis_clamp` defaults to 0), the tag value wraps around to the opposite end of the slider. This can be very useful for periodic sequences (for example 360 degree rotations). Setting

this property to 1 causes the tag value to clamp the values to the first/last positions. This can be useful for things like non-periodic time sequences.

Comparison Sliders

The Image Comparison layout is an extension of the Slider layout. In addition to displaying one image item, its title, and sliders, it also renders a copy of the same.

There are two available types of images that Ansys Dynamic Reporting supports, resulting in two different forms of comparison sliders.

- Image Comparison

Comparison of two basic PNG images.

- Deep Image Comparison

Comparison two of enhanced images.

Note:

- Image Comparison must be turned on for Deep Image Comparison to work.
- When Image Comparison and/or Deep Image Comparison is enabled, certain features present in a basic Slider layout like image dragging, image links, and custom image sizing are not available, to provide a non-intrusive experience.
- If for any combination of slider values an image is not available, a notification is displayed and the slider position automatically switches back to the previous position, if that position had an image.
- Due to security restrictions, RGB Differences and Deep Image Comparison are unavailable in exported HTML.
- The `comparison_initial_views` property is only supported in Deep Image Comparison and does not work for RGB Differences.
- If the `comparison_initial_views` is used, `common_slider_tags` is disabled and ignored to avoid collision.

Related topics:

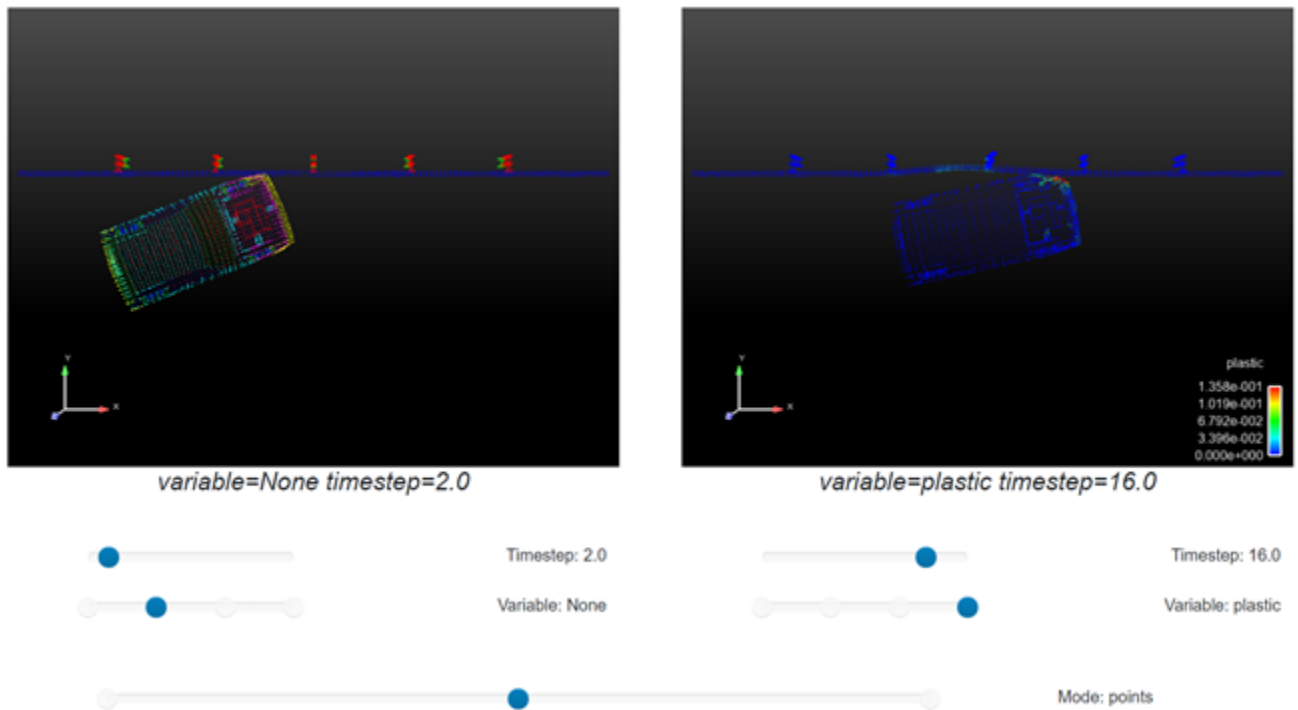
[Image Comparison](#)

[Deep Image Comparison](#)

[Image Comparison Keyboard Shortcuts](#)

Image Comparison

You can create a comparison involving two basic PNG images. For example:

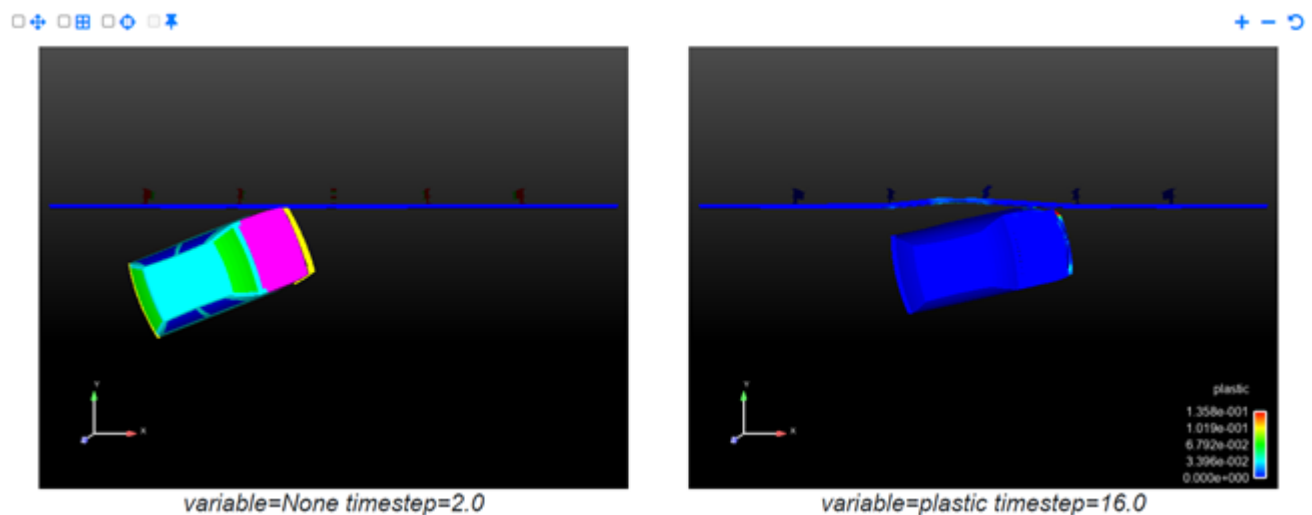


To set up an image comparison:

1. Set the `image_comparison` property (p. 160) to 1.
2. To enable optional common sliders that can control the images of both the sections at the same time, set the `common_slider_tags` property to a list of tag values For example, `[timestep, variable]`.

In the example above, the slider for **Mode** changes the displayed image for different values of the Mode variable of both the sections simultaneously.

3. To change the common slider title widths and justifications, adjust the `common_slider_title_width` and `common_slider_title_justification` properties.
4. To enable image controls, set the `image_controls` property to 1.

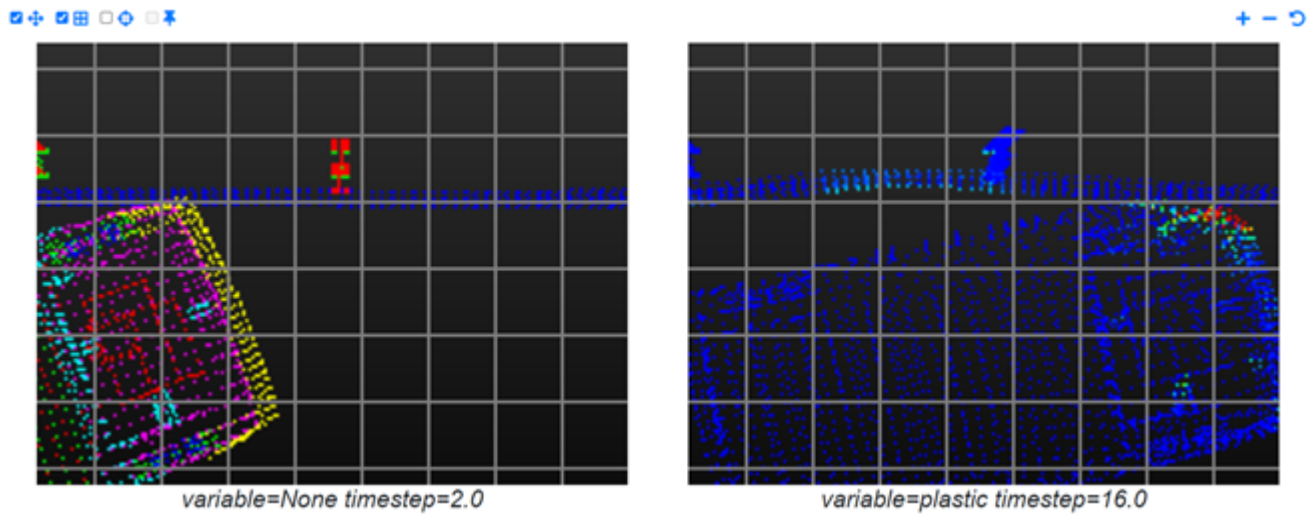


5. Use the [image controls](#) (p. 153) to compare both images.

The **Zoom In**, **Zoom Out**, and **Reset** actions are performed on both images at the same time.

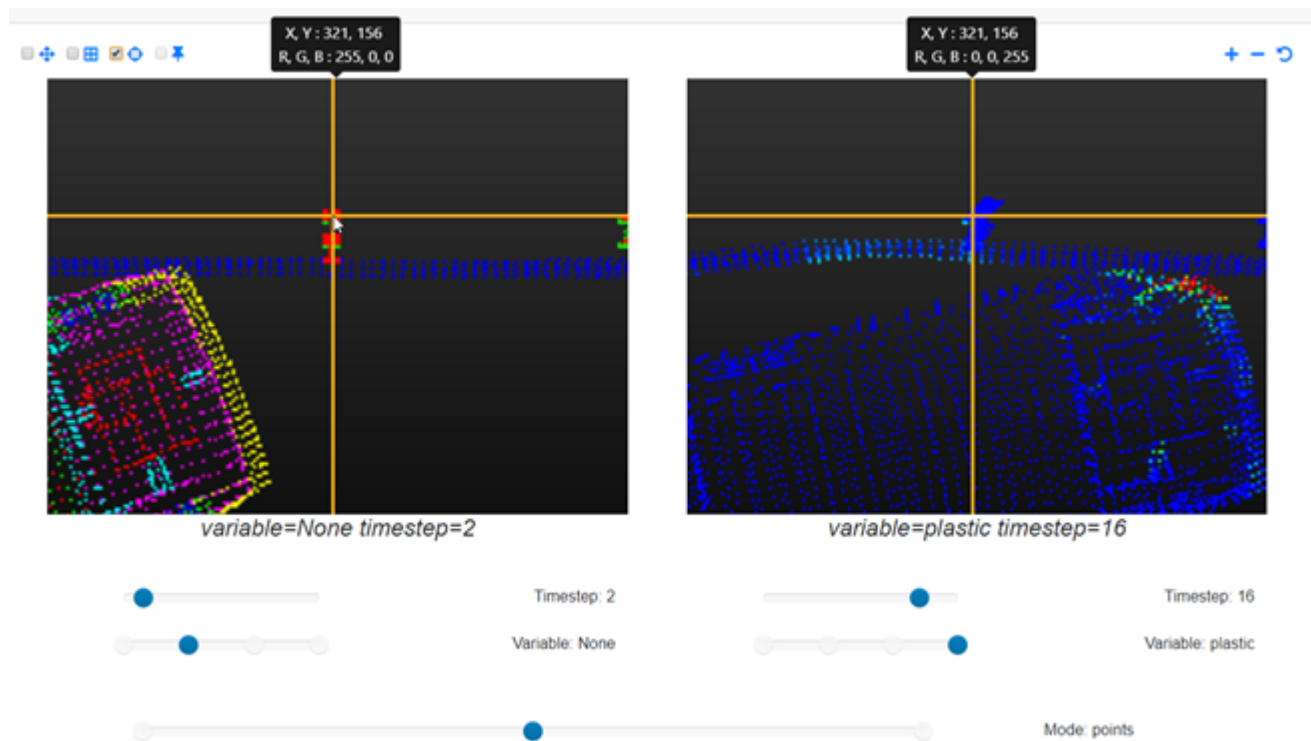
The **Pan** button is used to pan both the images at the same time. Note that panning only works if the images are zoomed in. To use, select the check box and drag across a zoomed image.

The **Grid** button is used to display a grid overlay over the images. This may be helpful when trying to zoom or focus into a particular point in the images. Here is an example of images which are zoomed, have a grid overlay and are also being panned.

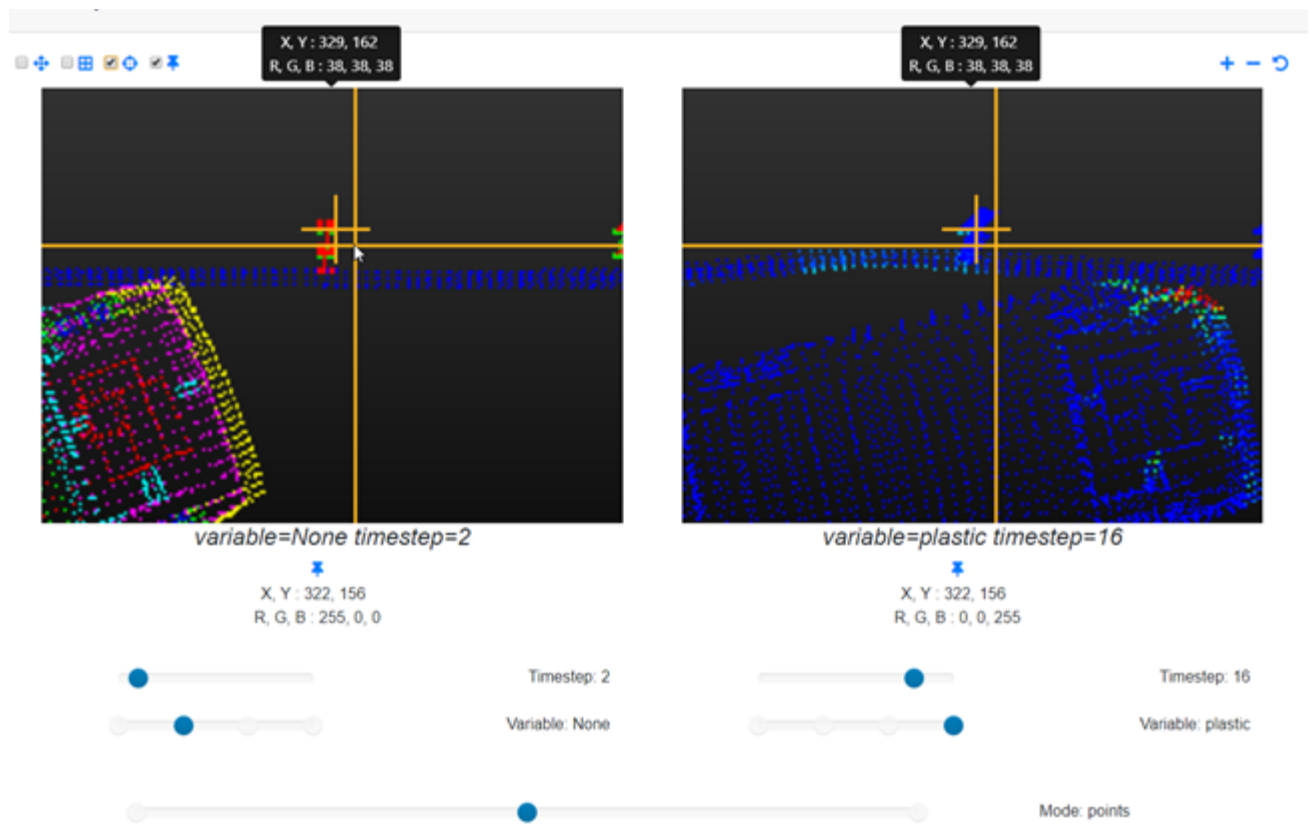


The **Probe** button is used to probe for per-pixel information from both the images at the same time.

When Probe is enabled, you cannot use the Pan or Grid option and vice versa.



The **Pin** button is used to mark a permanent point on both the images. To place a pin, enable Probe first and then double-click the image. The clicked point is marked on the image and the values at that point are displayed. When the image changes, the value at the pinned point automatically updates in the display.



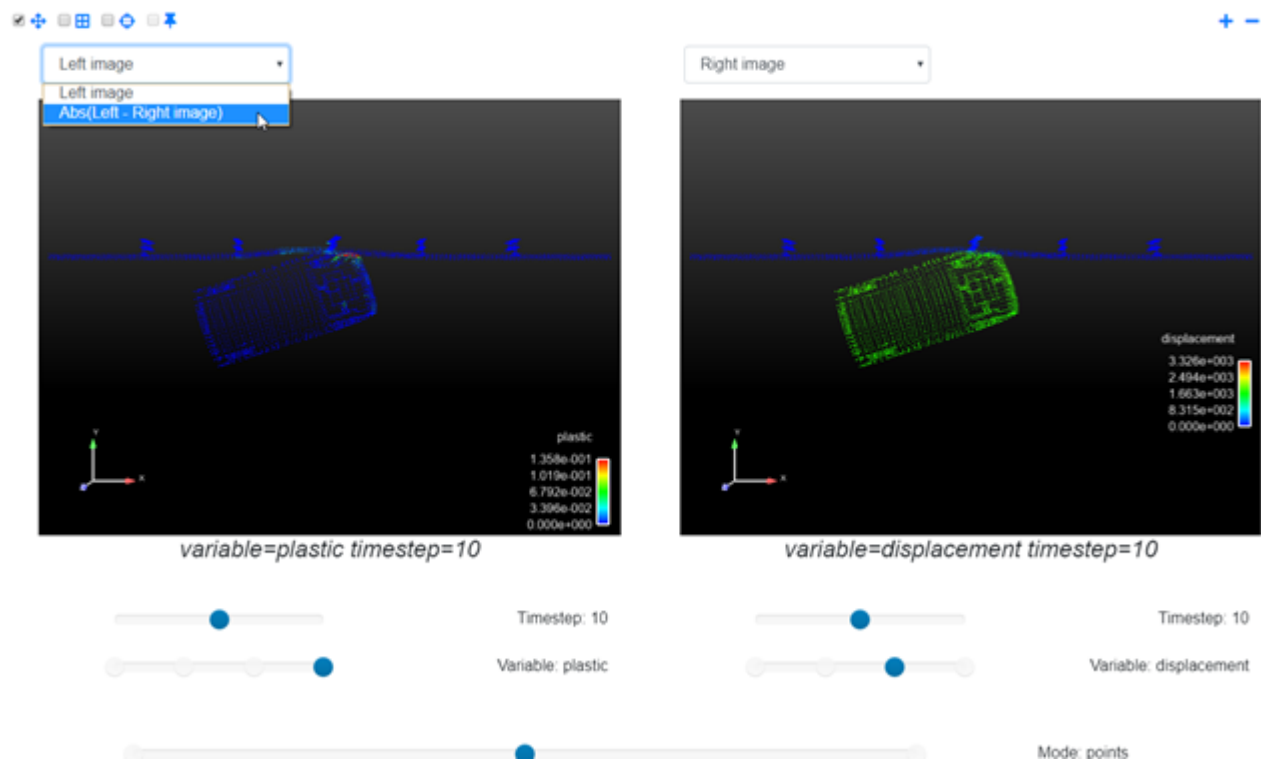
6. To use the RGB Differences feature:

- a. Set the `rgb_diff_comparison` property to 1.

Two drop-down menus are displayed, one for each image.

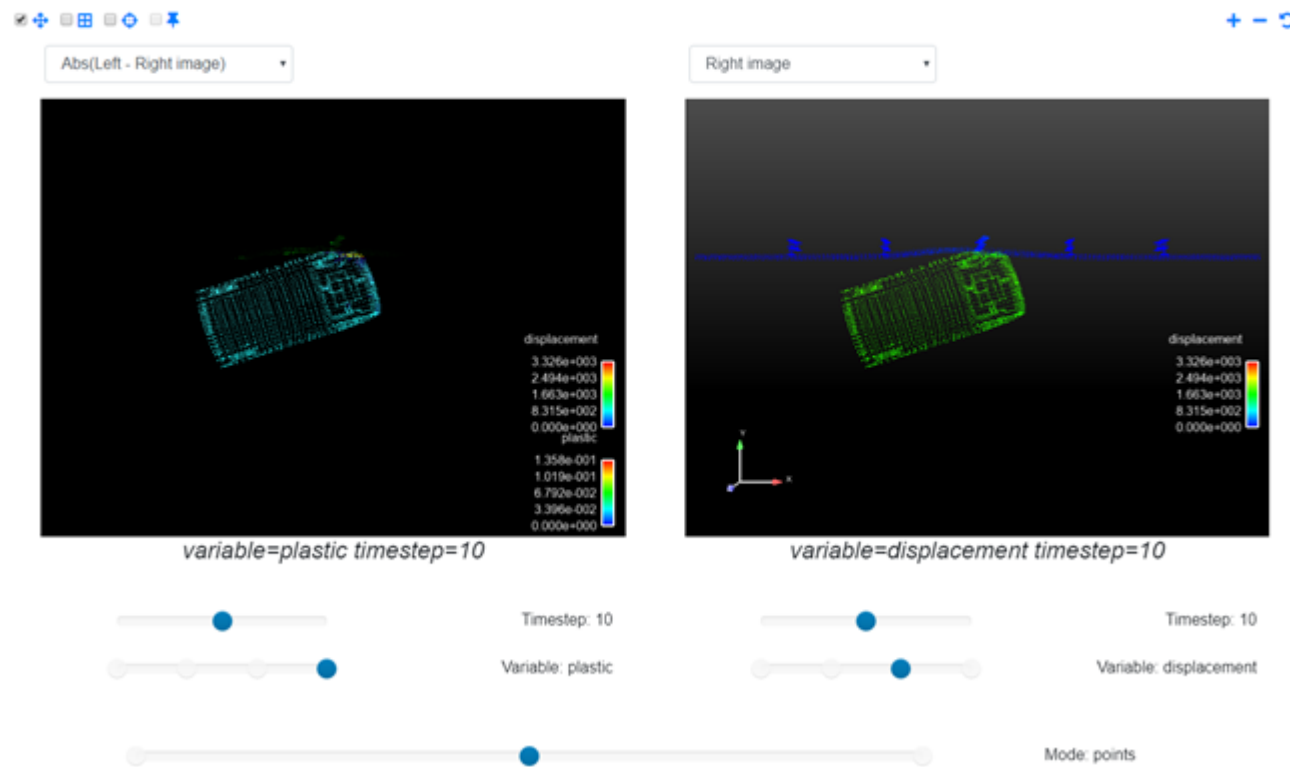
- b. Select different images for each section, so that the difference is visible.
- c. Find the difference between the two images by selecting options from the drop-down menu.

Example Slider Panels

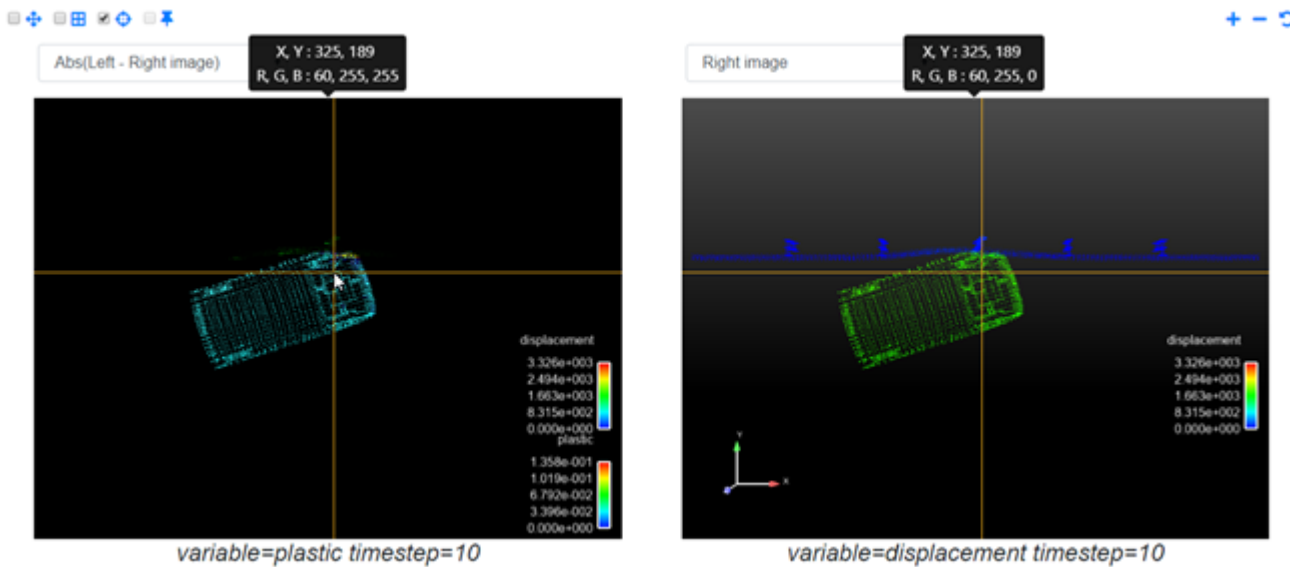


The result is a difference in the colors of the two images.

Example Slider Panels



- d. Verify the difference by probing the difference image's colors.



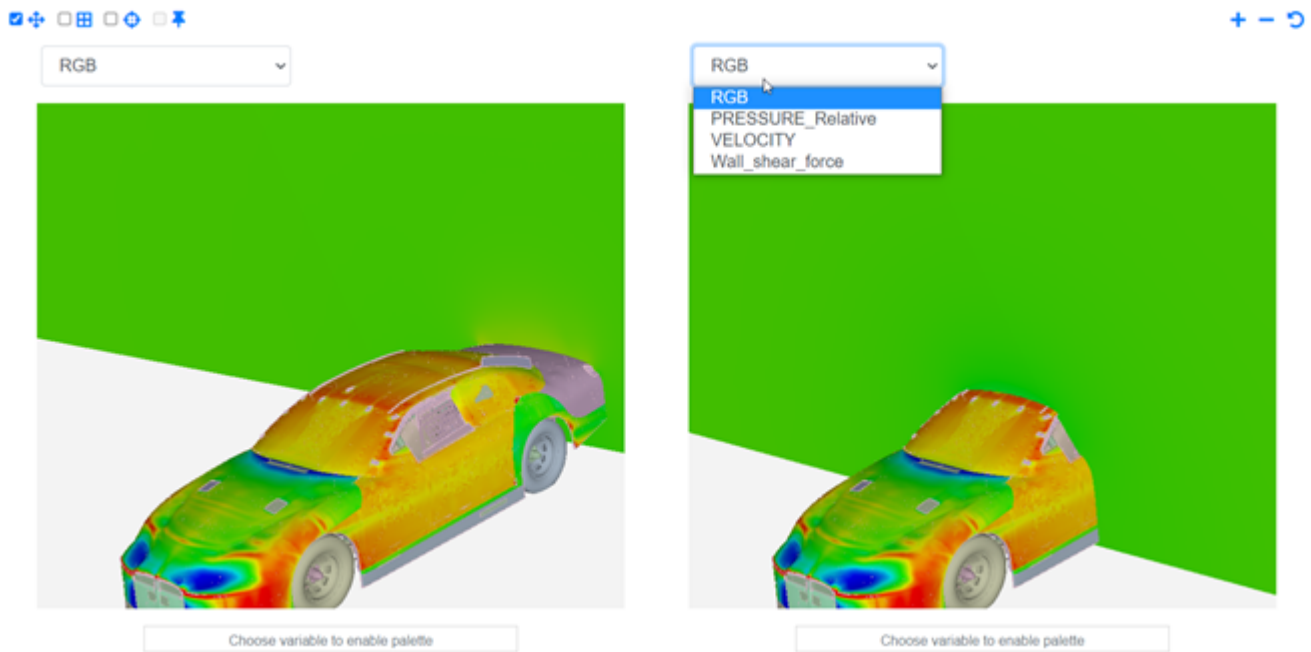
Deep Image Comparison

Deep images are TIFF images with the *.tif extension that can only be generated from EnSight by turning on enhanced mode during generation.

When a Slider Layout consists of deep images, you will have to explicitly turn on the `deep_image_comparison` property to enable this feature. If not, they are treated like normal images. Also, if any image in the set is not a deep image, this feature is automatically disabled.

To set up a deep image comparison:

1. Set the `image_comparison` property (p. 160) to 1.
2. Set the `deep_image_comparison` property to 1.
3. To enable image controls, set the `image_controls` property to 1.
4. Select a variable from the drop-down menu.

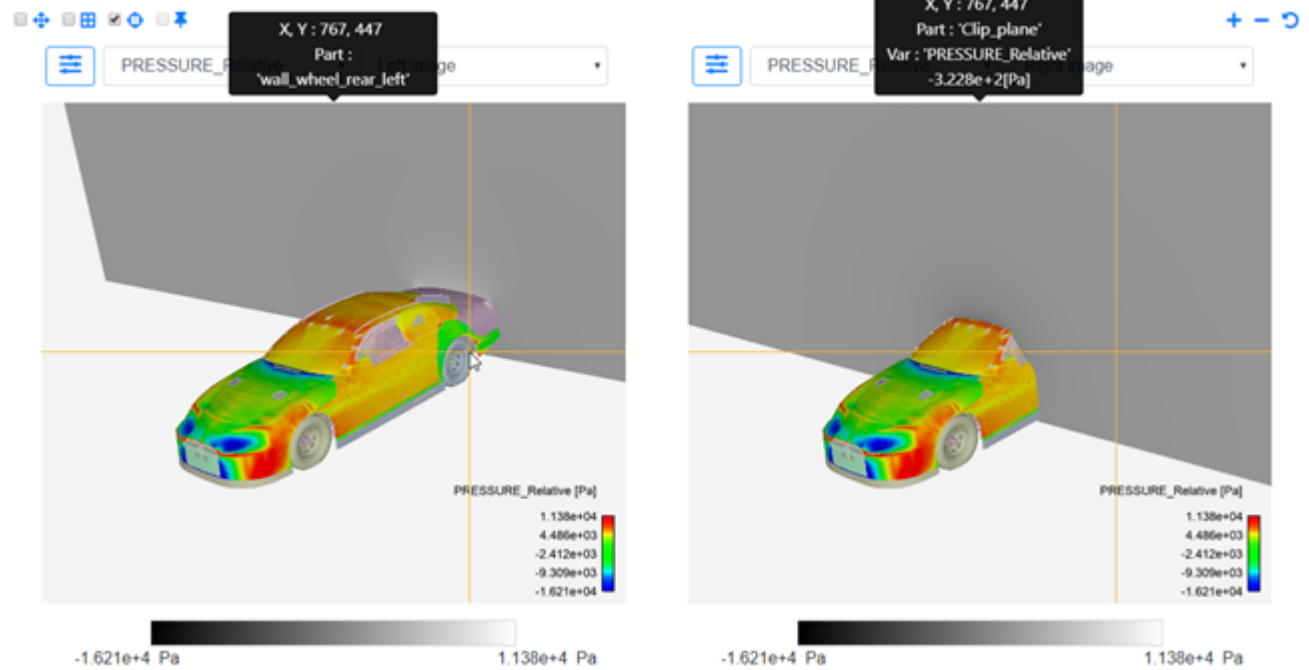


When a variable is selected, all parts that contain the selected variable are shaded. In the example below, the clip plane which contains **PRESSURE_Relative** is fully shaded in gray scale. The color or intensity of shading is based on the value of **PRESSURE_Relative** which may vary from region to region. The current mapping of the shading color to the value is displayed in the palette bar below the image. If a variable or any part that contains the variable is not displayed in the image, a warning is shown.

5. Use Probe () to view the value at each point.

Previously when the **RGB** option was selected in the drop-down menu, the RGB values were displayed. Now that a variable is selected, the variables are read from the enhanced image and then displayed, along with the values and the part associated with it as well. Each pixel's deep information can be read this way.

Sweep source



6. To adjust the image settings:

- a. Click .

The **Settings** dialog is displayed.

Settings for PRESSURE_Relative

-1.621e+4

1.138e+4

Pa

Pa

Min & max values

Max

11383.49902

Pa

Highest value in the current image is 2581.884033203125 Pa

Min

-16206.7001

Pa

Lowest value in the current image is -1204.674072265625 Pa

Colors

Palette

greys

☐ Invert color scale

☐ Hide remaining pixels

Reset

Apply

b. Adjust the settings as required.

Setting	Description
Max	Sets the range of colors or shading intensity that is used from the selected palette. Parts with values less than Min are colored the same as the minimum value. Parts with values greater than Max are colored the same as the maximum value. Parts with all values in between the Min and Max values are colored as shown in the preview scale at the top. The preview updates as the settings are changed.
Min	

Setting	Description
	The default Min and Max values are taken from the current palette range selected in EnSight for that variable. If not available, they are set to 0. Below the input fields, the lowest and highest values for that variable in the entire image, respectively, are displayed. Click Copy () next to it to populate the input fields with those values. You can also set your own values, but make sure the value is a valid number and Min is less than Max.
Palette	Sets the palette to use. The default is grayscale, which is used along with the default Min and Max values to initially generate the image. Changing the palette updates the preview scale..
Invert color scale	When selected, inverts the color scale. For example, inverting the grayscale palette changes Min from black to white and Max from white to black.
Hide remaining pixels	When selected, only displays the parts that contain the selected variable. The other parts that do not contain the selected variable are replaced with the selected background.
Background	Available when Hide remaining pixels is selected. Sets how the hidden area is displayed. This can be any value between 0 (full white) and 1 (full black).

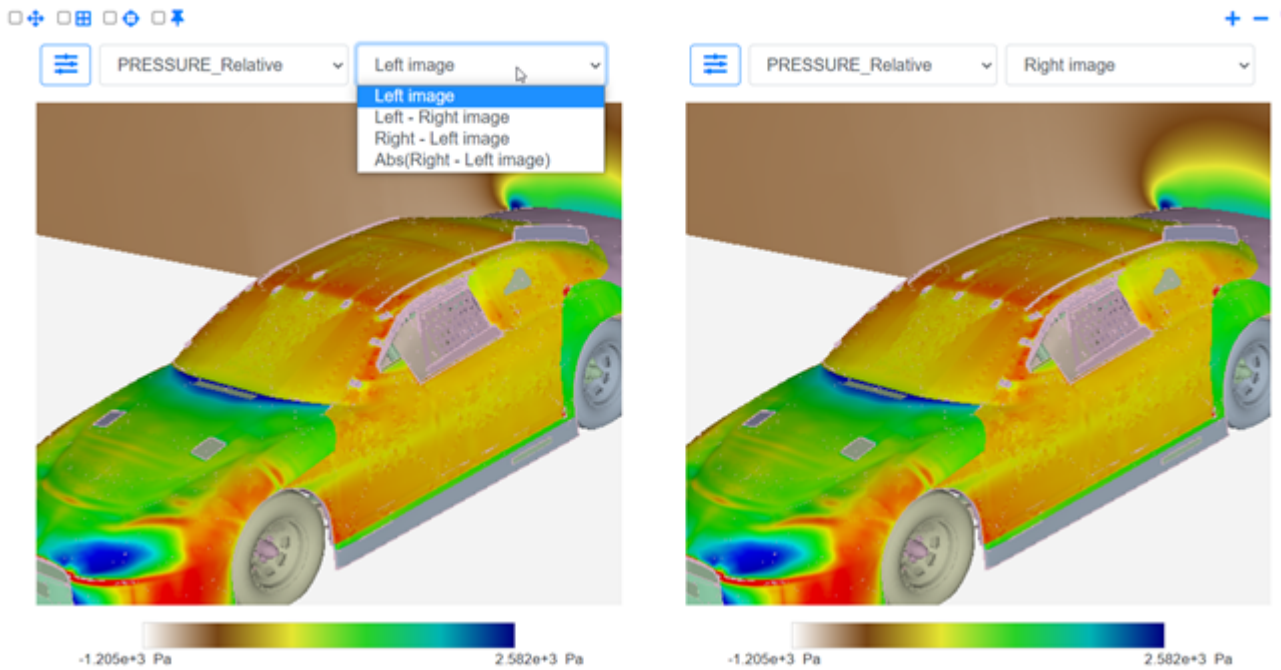
- c. Click **Apply**.

The settings are saved and the image is regenerated with the updated settings.

Settings are saved per variable, so that you do not have to set them again as you switch between variables. Click **Reset** to reset the settings back to defaults.

7. To use the Deep Image Differences feature:

- a. From the second drop-down menu, select a view.



The default view is the current image, which is **Left image** and **Right image** for images on the left and right respectively.

The available views are:

- **Left image:** The current image on the left.
 - **Right image:** The current image on the right.
 - **Left - Right image:** The computed image with values that are the difference between values on the left and values on the right image.
 - **Right - Left image:** The computed image with values that are the difference between values on the right and values on the left image.
 - **Abs(Right - Left image) :** The computed image with values that are the absolute value of the difference between values on the left and values on the right image.
- b. Use the combination of views, image settings, and [image controls \(p. 153\)](#) to compare both images.

Image Comparison Keyboard Shortcuts

Image controls support keyboard shortcuts. The cursor must hover on the image to activate. This is to keep controls contained to specific images if there are multiple comparisons in the same report. Shortcuts are not supported when the report is within an iframe.

Functionality	Shortcut
Zoom In	Shift + =
Zoom Out	Shift + -

Functionality	Shortcut
Reset	Shift + R
Probe	Shift + P
Pan	Shift + A
Pin	Shift + I
Grid	Shift + G

Iterator

The Iterator layout is used to distribute an incoming data item list into a collection of sub-lists. All of the items in a particular sub-list share a common values for a specific tag. The tag used to differentiate the items is specified in the Iterator layout configuration options. Once the sub-lists have been determined, the layout takes each sub-list in order and passes the sub-list as the data item input list to all of the child layouts. The iterator also makes the common tag value available as a property for utilization by the child layouts when customizing the generated report. This segregation of the input into sub-lists simplifies the display of data items in common layouts and reduces the need for the report designer to know specific tag values. A common use is to generate pages that use the same formatting that vary based on some filtering criteria represented using a tag (for example, variable name, design number, and so on).

Configuration Options

Iteration tag

☐ Sort items by tag

☐ Reverse the sort

Secondary sorting tag

Option	Description
Iteration tag	Enter the name of the data item tag. This tag should take the form tag=value. The value is the value of the named tag for all the items in this loop.
Sort items by tag	When enabled, orders items by tag. When disabled, the sort is inherited from the rules (p. 137) set on this template by one of its parents.
Reverse the sort	When enabled, reverses the order of the item sort.
Secondary sorting tag	Items that share the same tag value (from each iteration) can be sorted by the layout before they are rendered into the report using the values of another tag specified by the name entered here.

Example

Consider the report generated by the collection of layout templates configured like this:

▼ Iterator Example
Header
Content
Footer

The input to this example is a collection of three table items. These items have a month tag with values like month=Jan. The Iterator layout is configured as shown in the configuration options above. The header (Header layout template) custom HTML header is a little more complex than most:

```
<div style="background:#63E05A">
  <span style="float: left">Page Header</span>
  <span style="float:right"> Page:{{page_number}}</span>
  <br>
</div>
```

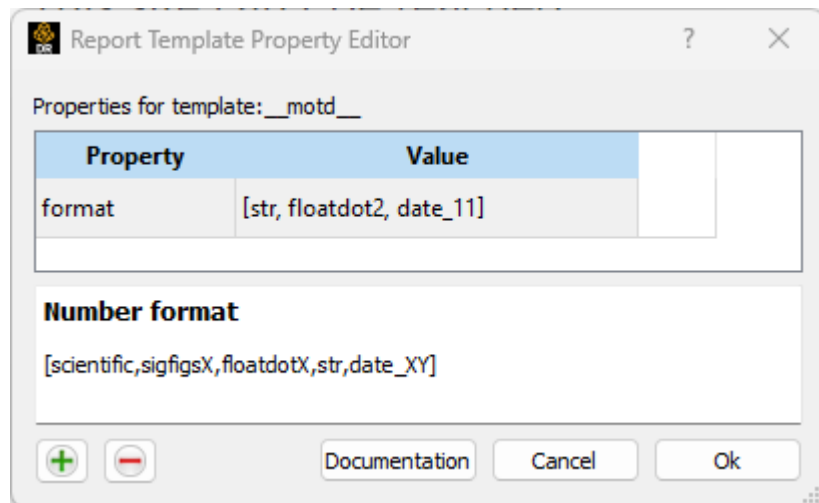
Similarly for the footer (Footer layout template):

```
<hr>
<div style="background:#D95AE0">
  <span style="float: left;">Page Footer {{date_date}}</span>
  <span style="float:right;"> Page:{{page_number}}</span>
  <br>
</div>
```

The Content layout is a Panel layout with a custom HTML header of:

```
<h4>Tables for the month: {{iterator_value}}</h4>
```

The format property set to re-format the table columns:



The report that results looks something like this:

Tables for the month: Jan

January

User	Version	Date
jill	1.10	Jan 24, 2017 06:56AM
mary	2.00	Jan 25, 2017 04:15PM
mary	1.00	Jan 17, 2017 08:07PM
jill	1.10	Jan 24, 2017 01:17PM
fred	1.10	Jan 31, 2017 06:27PM
jill	1.00	Jan 18, 2017 09:11PM
mary	1.30	Jan 20, 2017 03:27PM
mary	2.00	Jan 26, 2017 06:59AM
fred	2.00	Jan 04, 2017 07:45AM
fred	1.30	Jan 07, 2017 06:56AM
jill	1.00	Jan 05, 2017 10:56PM
bob	1.10	Jan 03, 2017 04:10PM

...

fred	1.30	Jan 14, 2017 05:01PM
jill	1.30	Jan 03, 2017 05:57AM

Tables for the month: Feb

February		
User	Version	Date
fred	1.30	Feb 23, 2017 06:23PM
bob	2.00	Feb 17, 2017 02:41AM
fred	1.10	Feb 23, 2017 09:11PM

...

jill	1.30	Feb 28, 2017 06:06AM
mary	2.00	Feb 10, 2017 09:59PM

Tables for the month: Mar

March		
User	Version	Date
bob	1.30	Mar 14, 2017 05:37AM
jill	2.00	Mar 15, 2017 10:00PM
marv	1.00	Mar 10, 2017 02:13PM

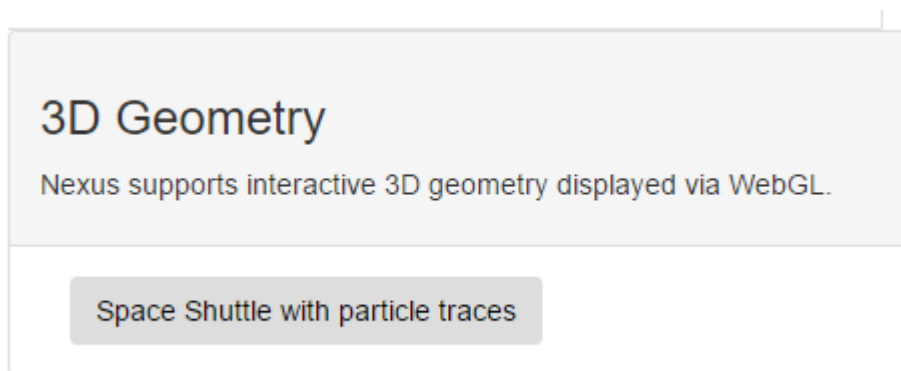
...

john	1.00	Mar 07, 2017 03:13PM
mary	1.30	Mar 23, 2017 12:20AM
mary	1.10	Mar 04, 2017 04:41PM

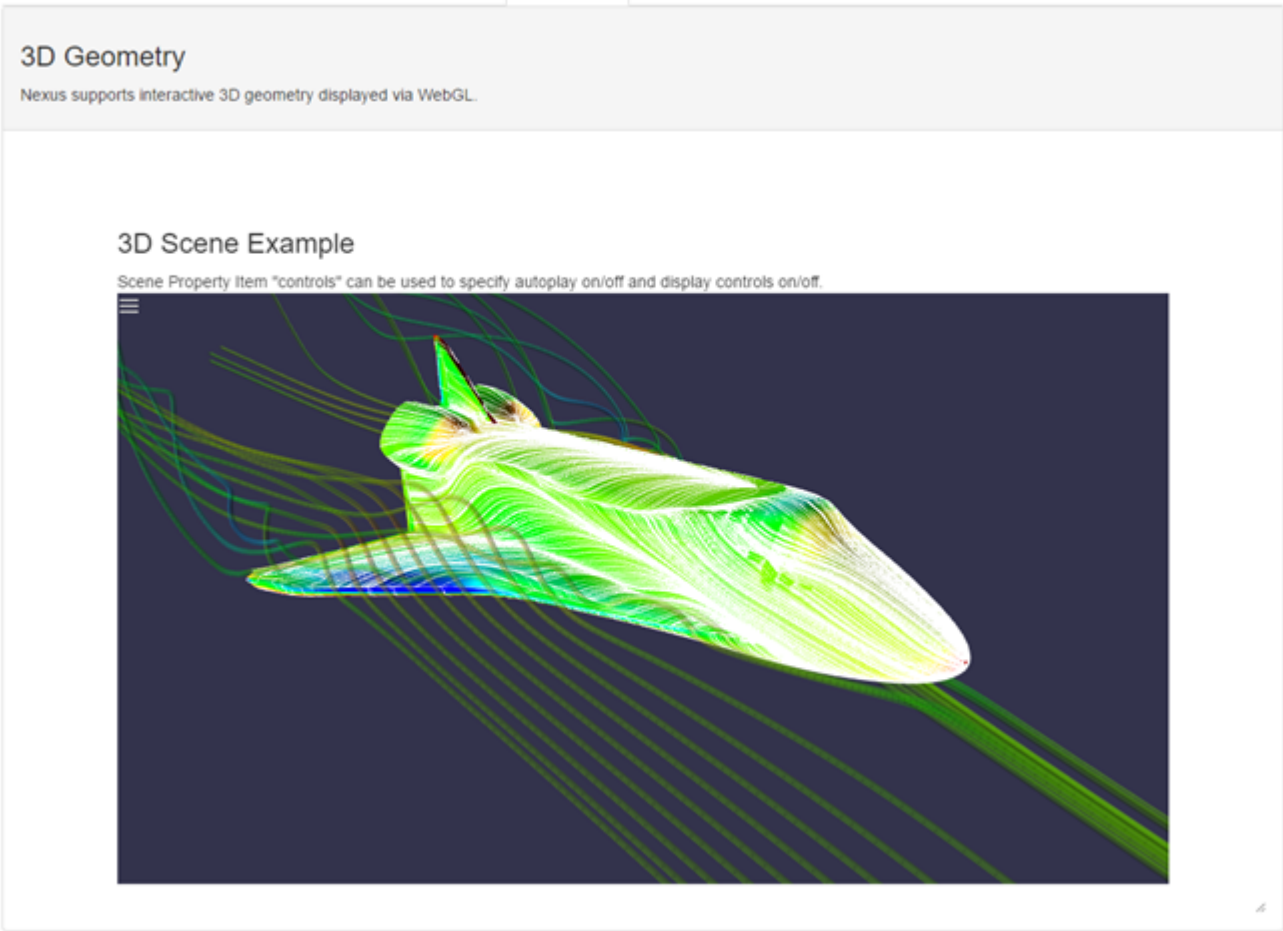
Notice that the value of the tag the iterator was working on changes from one instance of the three children to the next. Secondly, after each time the footer template is encountered, the `page_number` is incremented.

Linked Report

The linked report layout is used to provide a reference to another report from within an existing report. This layout renders a simple button or a link that when clicked causes the browser to navigate to the referenced report. In a simple example, a linked report template is set up as a child of a panel layout. When the report is rendered initially, the display looks like this:



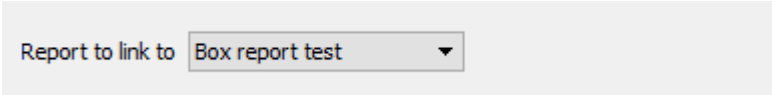
After clicking on the button, the button is replaced and the referenced report is displayed inside of the panel template that previously contained the button.



The linked report layout does not display any items. Instead, any specified item filter is used to pre-filter the items that will be selected by the referenced report.

Configuration Options

The linked report layout has a single option. This option allows you to select the specific report that the inserted link will refer to.



All other options are controlled using properties.

Properties

Several properties specifically control the display of the linked report.

Property	Description	
link_text	Sets the text that is displayed in the web link or in the button.	
link_type	Value	Operation

Property	Description	
	0	Display the report reference as a web link and clicking on the link causes the current page/tab to be replaced with the contents of the referenced report.
	1	Display the report reference as a web link and clicking on the link causes the current page/tab to be replaced with the contents of the referenced report.
	2	Display the report reference as a web link and clicking on the link causes a new page/tab to be created to display the referenced report.
	3	Display a button that when clicked causes the referenced report to display inline, in the location where the button was placed in the report. This is the default.
	4	This option is similar to 3 except that if the item list input to this template contains at least one Image Item, the image from that item is placed in the button to the left of the link text.

Page Header

When laying out a report that potentially include formal pagination, individual pages must be bracketed by instances of the Page Header and Page Foot layout templates. These templates have the ability to display children items and templates. When in use, they break up the HTML into pages and manage the `page_number` property. See [Page Footer \(p. 185\)](#) for an example. These layouts are most commonly used with the [Iterator \(p. 179\)](#) layout to present a multi-page report.

Configuration Options

None.

Page Footer

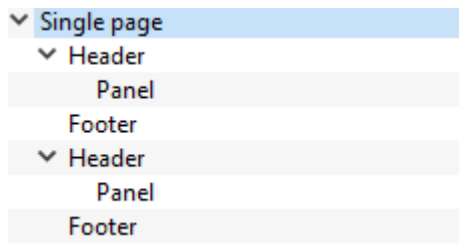
When laying out a report that potentially include formal pagination, individual pages must be bracketed by instances of the Page Header and Page Foot layout templates. These templates have the ability to display children items and templates. When in use, they break up the HTML into pages and manage the `page_number` property. These layouts are most commonly used with the [Iterator \(p. 179\)](#) layout to present a multi-page report.

Configuration Options

None.

Example

Consider a report template laid out using the following layouts:



Where **Single page** filters down to all of the scene items the database. **Header** is a header template layout with the custom HTML header of:

```
<b>This is the header for page: {{page_number}}</b>
```

Similarly, **Footer** is a footer template layout with the custom HTML header of:

```
<b>This is the footer for page: {{page_number}}</b>
```

and the item filter set up to not select any items. The **Panel** layout is a call-out panel layout with **Show items as links** selected. The similarly named items are identically configured layouts.

The resulting report looks like this:

This is the header for page: 1

Panel layout: Scene items as links

- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [ThePlane](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [The Cow](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)

This is the footer for page: 1

This is the header for page: 2

Panel layout: Scene items as links

- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [ThePlane](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [The Cow](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)
- [Scene Geometry Source](#)

This is the footer for page: 2

Notice that the `page_number` property is incremented after the footer layouts are rendered. If printed from the browser, two pages are printed.

Tag Properties

This layout inherits the basic features of the [Columns \(p. 140\)](#) layout, except that it performs an additional operation. The layout first performs the common item filtering, sorting and selection operations. It then takes the remaining items (in order) and converts the item tags into properties, placing the values in the template context. It then passes all of the original input items to the child templates or renders them (as the base column layout would). This makes it possible for a data item to set properties when the report is generated, so the report output specifics can be data driven.

Configuration Options

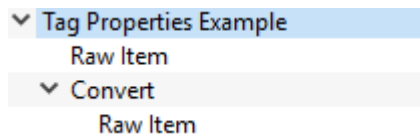
None.

Example

In this example, there are a collection of 12 thumbnail image items as the source data items. Each image has a tags string that looks like this:

```
'variable=plastic timestep=10.0 mode=points'
```

Three tags named **variable**, **timestep** and **mode** are defined on all Image items. Multiple tags are space-separated in a tag string.



The two **Raw Item** templates have their custom HTML header set to:

```
<h4>Raw Item Details Mode={{mode}}</h4>
```

The `item_context_debug` property is set to 1. The first **Raw Item** template has the sort selection option set to **First item**, so it displays a single image along with the default property context.

The **Convert** template is an instance of the Tag Properties layout. The only option it has set is the sort selection option is set to **First item**. It takes the tags from the first Image Item and turn them into properties in the rendering context. When you run this report, the following output is produced:

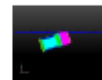
Example of tags to property conversion

Raw Item Details Mode=

Item: 9da768cf-f611-11e6-9104-ae3af682bb6a

Context: {'engine': , 'page_width': 12.03, u'item_context_debug': u'1', 'date_iso': '2017-03-08T22:14:18.3180', 'page_col_pixel_width': 96.24, 'date_year': 2017, 'year': 2017, 'base_title': 'Report', u'width': u'50', 'plotly': 0, 'page_dpi': 96.0, 'date_datetime': '03/08/17 22:14:18', 'message': , 'babylon': 0, 'hidemenus': False, 'UNIQUE_NUMBER': 20L}

Item tags: defaultdict(at 0x00000201035FA358>, {'variable': 'None', 'timestep': 10.0, 'mode': 'solid'})

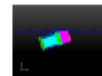


Raw Item Details Mode=solid

Item: 9da768cf-f611-11e6-9104-ae3af682bb6a

Context: {'engine': , 'page_width': 12.03, u'item_context_debug': u'1', 'date_iso': '2017-03-08T22:14:18.3180', 'page_col_pixel_width': 96.24, 'date_year': 2017, 'timestep': 10.0, 'year': 2017, 'base_title': 'Report', 'plotly': 0, 'solid': , 'date_date': '03/08/17', 'page_dpi': 96.0, 'variable': 'None', 'date_datetime': '03/08/17 22:14:18', 'message': , 'hidemenus': False, 'UNIQUE_NUMBER': 20L}

Item tags: defaultdict(at 0x00000201035FA358>, {'variable': 'None', 'timestep': 10.0, 'mode': 'solid'})



Item: 9eee19a1-f611-11e6-a189-ae3af682bb6a

Context: {'engine': , 'page_width': 12.03, u'item_context_debug': u'1', 'date_iso': '2017-03-08T22:14:18.3180', 'page_col_pixel_width': 96.24, 'date_year': 2017, 'timestep': 10.0, 'year': 2017, 'base_title': 'Report', 'plotly': 0, 'solid': , 'date_date': '03/08/17', 'page_dpi': 96.0, 'variable': 'None', 'date_datetime': '03/08/17 22:14:18', 'message': , 'hidemenus': False, 'UNIQUE_NUMBER': 20L}

Item tags: defaultdict(at 0x00000201035FACF8>, {'variable': 'None', 'timestep': 10.0, 'mode': 'points'})



The Item tags output (because `item_context_debug` is set) for the first item contains:

```
Item tags: defaultdict( at 0x00000201035FA358>, {'variable': 'None', 'timestep': 10.0, 'mode': 'solid'})
```

The mode tag is set to **solid**. When the second **Raw Item** template is rendered, the `{{mode}}` macro is expanded to the value of the mode tag that the **Convert** template placed in the rendering properties. The context status output for all of the items in the second **Raw Item** template is the same:

```
Context: {'engine': , 'page_width': 12.03, 'item_context_debug': u'1',
         'date_iso': '2017-03-08T22:14:18.318000+00:00',
         'page_col_pixel_width': 96.24, 'date_year': 2017, 'timestep': 10.0,
         'year': 2017, 'base_title': 'Report', 'plotly': 0, 'width': u'50',
         'mode': 'solid', 'date_date': '03/08/17', 'page_dpi': 96.0,
         'variable': 'None', 'date_datetime': '03/08/17 22:14:18',
         'message': '', 'babylon': 0,
         'hidemenus': False, 'UNIQUE_NUMBER': 20L}
```

It includes the item tags for variable, timestep, and mode that the **Convert** template turns into context properties from the selected image item.

The number of image items rendered by the second **Raw Item** template is not limited to a single item. Unlike most other templates, the filtering, sorting and selection options on the Tag Properties layout are used to select what items to grab the tags from. That filtering is not passed on to its children, they are presented with the same data items that the Tag Properties layout was presented with.

An additional example is included in the [Table Merge \(p. 197\)](#) generator example.

Data Filter

This layout template is currently a beta feature. See [Data Filter Layout Template](#) in the *Ansys Dynamic Reporting Beta Doc Guide*.

PowerPoint (PPTX)

This layout template is currently a beta feature. See [PowerPoint \(PPTX\) Layout Template](#) in the *Ansys Dynamic Reporting Beta Doc Guide*.

User Defined Content Block

The user-defined layout is an advanced feature. The layout directly exposes the Ansys Dynamic Reporting HTML document structure in a controlled fashion. You can use it to add custom HTML content into a report.

All templates render into the Document Object Model (DOM) within a `div` tag section that include the Globally Unique Identifier (GUID) of the template. For example:

```
<divnexus_template="bddf4640-4251-11e7-bd6a-f00faf56e564">
[layout content]
</div>
```

The user-defined template embeds an additional `div` tag that can be placed before or after any child template/data item generated HTML content. This includes an `adr_userdefined_template` attribute with a value specified by the template properties. The `div` tag is populated by the custom HTML header content. For example, if the `userdef_name` property is set to `Hello` and the custom HTML header is set to `<h1>Example</h1>` the generated `div` looks like this:

```
<divnexus_template="bddf4640-4251-11e7-bd6a-f00faf56e564">
[child layout/data item content]
<divadr_userdefined_template="Hello"><h1>Example</h1></div></div>
```

User-supplied JavaScript can find the `adr_userdefined_template` `div` tag and enhance the content externally. The `div` tag can be placed before or after any child item/layout content via a

property. By default, `adr_userdefined_template` is not included in PDF and offline HTML exporting, but a property can be used to control that as well.

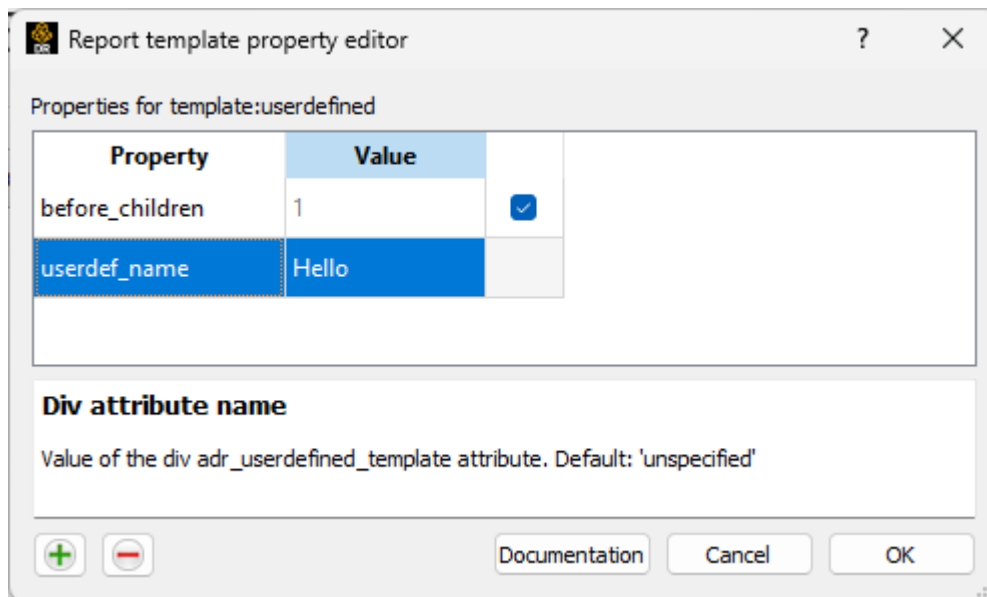
Properties

All template options are controlled using properties.

Property	Description	
<code>user-def_name</code>	This string property specifies the <code>adr_userdefined_template</code> attribute value. Macro expansion is applied to this string, allowing for use with tag/property driven or iterator layouts. The default value is unspecified.	
<code>inter-act-ive_only</code>	Value	Operation
	0	The <code>adr_userdefined_template</code> div is included in PDF or offline HTML content.
<code>be-fore_children</code>	1	The <code>adr_userdefined_template</code> div is not included in PDF or offline HTML content. This is the default.
	Value	Operation
	0	Display the <code>adr_userdefined_template</code> div before any child template and data item content. This is the default.
	1	Display the <code>adr_userdefined_template</code> div after any child template and data item content.

Example

A user-defined template with the following properties:

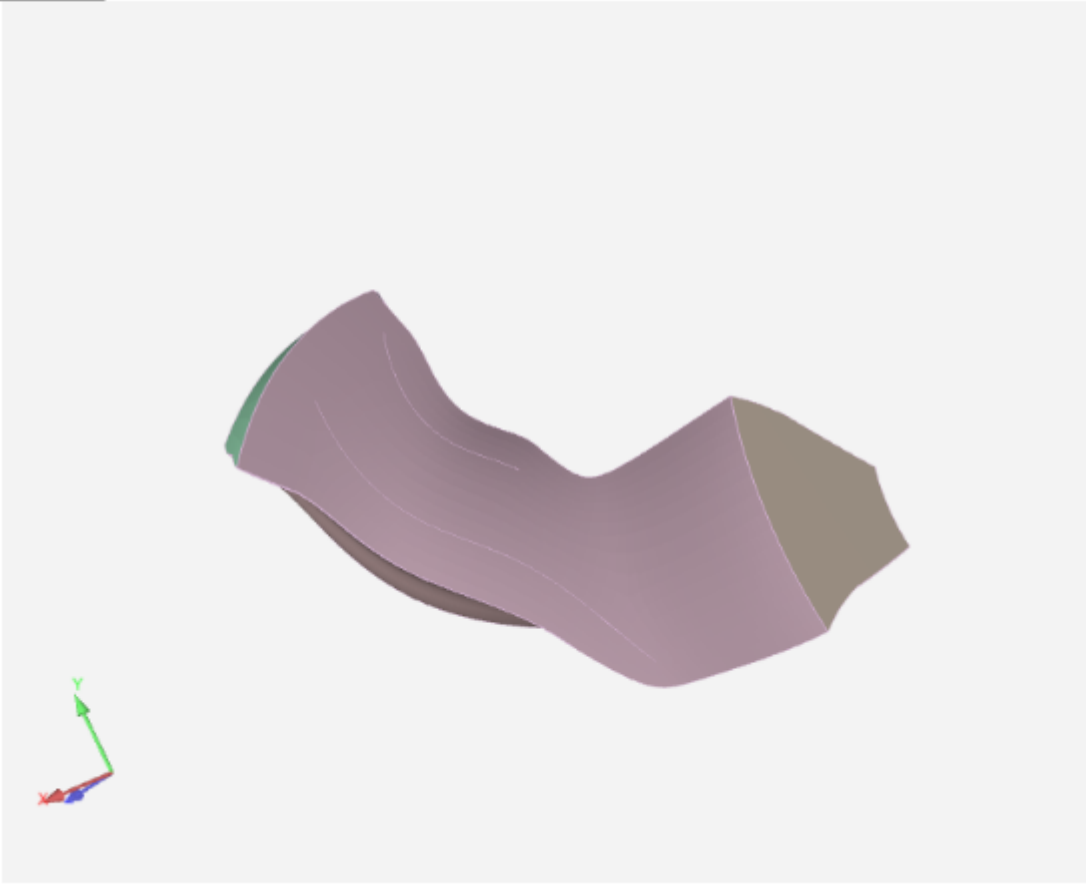


The custom HTML content is set to a button with an associated callback:

```
<button type="button"
onclick='document.querySelector("[adr_userdefined_template=\"Hello\"]").innerHTML += " Hello";'>
Click Me!
</button>
```

Generating the report and clicking on the button a couple of times results in:

Click Me! Hello Hello



JavaScript can easily find the `div` tag using the `document.querySelector` method, jQuery, or other DOM traversal mechanism. It is safe for user code to modify DOM content within `adr_user-defined_template` `div` tags.

Generator Templates

The generator converts a collection of items into other items. For example, it can convert an image item into an HTML representation of the item, a collection of vectors into a table, a table into HTML, perform a table pivot operation, a vector into a mean value, and so on. A generator has configurable options to control these operations and the generator parent template passes a list of data items to apply the configured operation to those items.

There are a number of different types of generators available:

- [Table Merge \(p. 197\)](#)

Takes a collection of tables and merges the rows and columns into a single output table.

- [Table Reduction \(p. 203\)](#)

Fold the rows and columns of a table into new rows/columns via various data reduction operations (for example, counting, maximum/minimum, averages, and so on).

- [Table Row/Column Filter \(p. 209\)](#)

Extracts or removes a collection of rows/columns from a table by selecting specific rows and/or columns.

- [Table Value Filter \(p. 214\)](#)

Extracts or removes rows/columns from a table using values in another row/column as a filter.

- [Table Row/Column Sort \(p. 218\)](#)

Sorts the rows/columns of a table using the values in a column/row or the column/row labels.

- [SQL Query \(p. 222\)](#)

Runs SQL queries against an SQL or PostgreSQL database, bringing the result into Ansys Dynamic Reporting as a table.

- [Tree Merge \(p. 225\)](#)

Combines multiple tree items into a single, multi-column tree item.

- [Items Comparison \(p. 230\)](#)

Finds all data items in the report database matching specific filters and returns them in the exact order specified.

- [Statistical Analysis \(p. 242\)](#)

Performs statistical analysis on an input data table.

Common Generator Properties

All generator templates share a number of common options. These options can be configured using this portion of the **Template properties** pane:

The screenshot shows a dialog box with the following sections:

- Filtering**
 - Filter: `A|i_type|cont|table;A|i_tags|cont|udtable;`
 - Edit item filter... Filter mode Incoming Items
- Edit properties...
- Edit item sorting rules... Sort selection All items
- Common Generator options**
 - Generated items Replace
 - ☒ Append template tags to new items

Data item filtering for a generator differs fundamentally from that for a layout template. The filtering options control which data items are processed by the generator. The filters do not filter the list of items passed to children of the generator (unlike the layout templates). Instead, items produced by the generator are merged into the input item list and the result is passed on to child templates of the generator template.

The item filter selects a subset of the incoming data items that the generator is applied to. The filter mode allows the filter to be applied directly to the raw database instead of to the input data item list, with the resulting items either replacing the incoming items or augmenting the data item list. The generator then produces some number of additional items based on the items selected by the filter.

The **Generated items** list controls how generated items are merged with the original data item list that was passed to the generator.

- **Append**

The generated items are added to the list.

- **Replace**

Item list is replaced with the items produced by the generator.

The resulting list of data items is passed on to any template children of the generator template (or if the template has no child templates, they are rendered sequentially into the output).

When a generator produces new items, the tags on the items are generally preserved from the input items. When selected, **Append template tags to new items** causes the generator to walk all of the tags on the generator itself and add/replace tags on all of the generated items to match those on the generator.

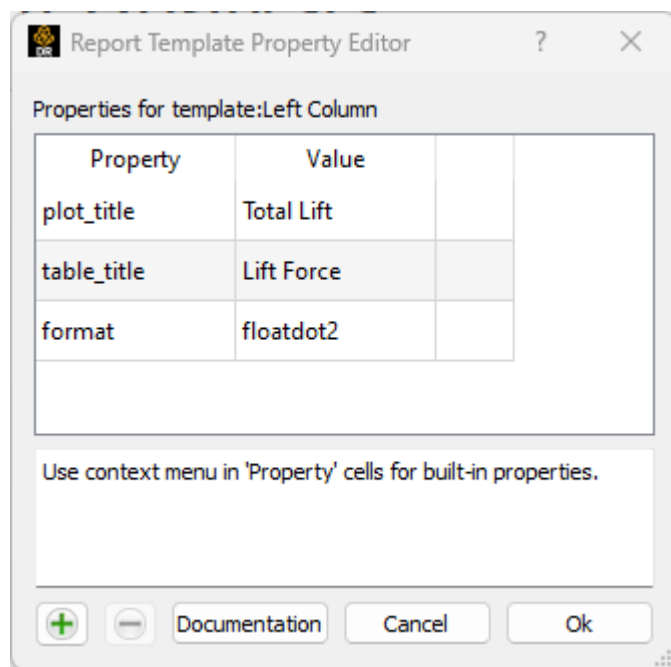
For example, consider a [Tree Merge \(p. 225\)](#) Generator where the specified item filter does not select any generator source items. The generator does not see any inputs to process and does not generate any new items. With the **Generated items, Append** option selected, while the generator does not generate any new tree items, the output displays all the items inherited from the parent template. This is because the option merges the (original) input item list and the (empty) output item list produced by the generator. Under these conditions, the generator is effectively a null operation. This configuration can be useful as it allows generator pipelines to be conditionally applied to a filtered list of items, informing child layout templates.

Filtering

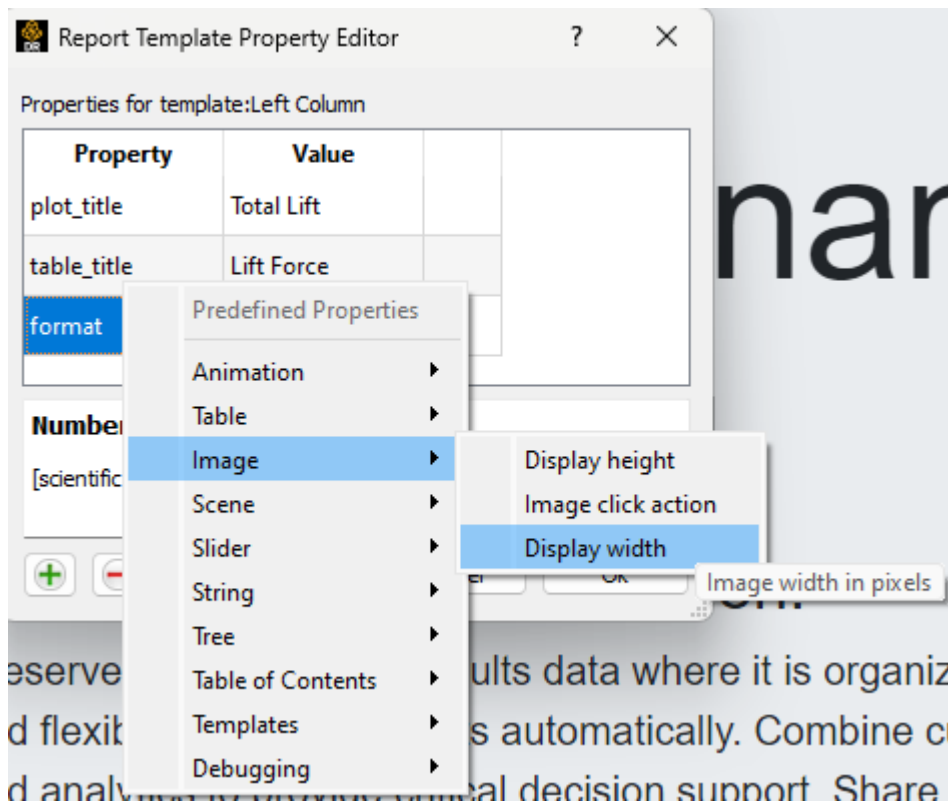
See [Query Expressions \(p. 320\)](#) for more details.

Properties

When processing a specific layout template, after the input data items have been filtered, the layout can override various properties which control specific display attributes. Open the **Report Template Property Editor** dialog by clicking **Edit properties**.

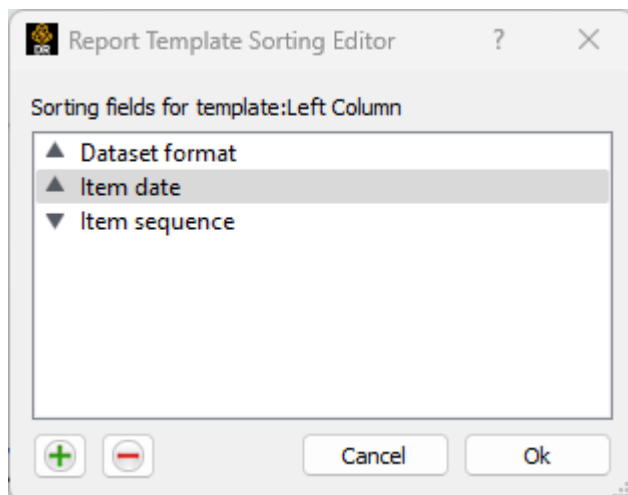


Use the dialog to add new properties and remove existing ones. There are no inherent limits on the names of the properties, other than they cannot include spaces or punctuation other than the underscore (_) character. Generic properties can be used in the [macro substitution \(p. 83\)](#) mechanism to customize various displays. Pre-defined properties also exist and are documented with the specific data items. Right-clicking on the property opens a list (organized by item type) of the various properties and in many cases, simple examples of the values that can be changed.



Sorting

The filtered list of items can also be sorted using a multi-key sort to change the order that the generator encounters the various items. This feature might be used to control the order of output rows in a table aggregation operation for example. Open the **Report Template Sorting Editor** dialog by clicking **Edit item sorting rules**.



The buttons in the lower left corner allow you to change the number of fields to use as keys. Right-clicking a key allows you to select specific item, session, or dataset keys. Sorting is performed top to bottom in the dialog, so items are primarily sorted by the dataset format field and in cases where they are the same, the item date field is used. The order can be changed by dragging the fields up

and down within the dialog. Individual keys can be sorted in different directions by clicking on the triangles to the left of the key names. The triangles point toward the minimum value.

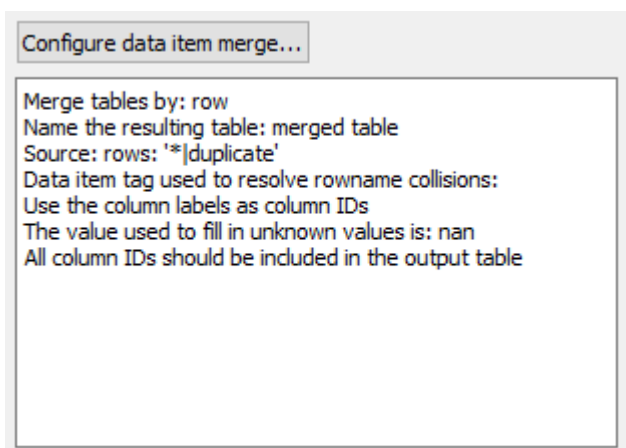
Once a specific sorting sequence has been established, the **Sort selection** list in the **Template properties** pane allows you to further filter the list of items to only the first or last item in the sorted sequence. This can be useful when designing a report that displays the most recent values for some piece of data.

Table Merge

The Table Merge generator takes multiple input data table items and combines them into a single table. The merging process can be customized to select specific rows to merge. The operation has a number of options that make it possible to fold data from multiple data items into individual rows. Unlike many other generators, the merge generator produces a single output table data item every time it is passed a list of input data items. If there are no table data items in the input list, no output table is generated.

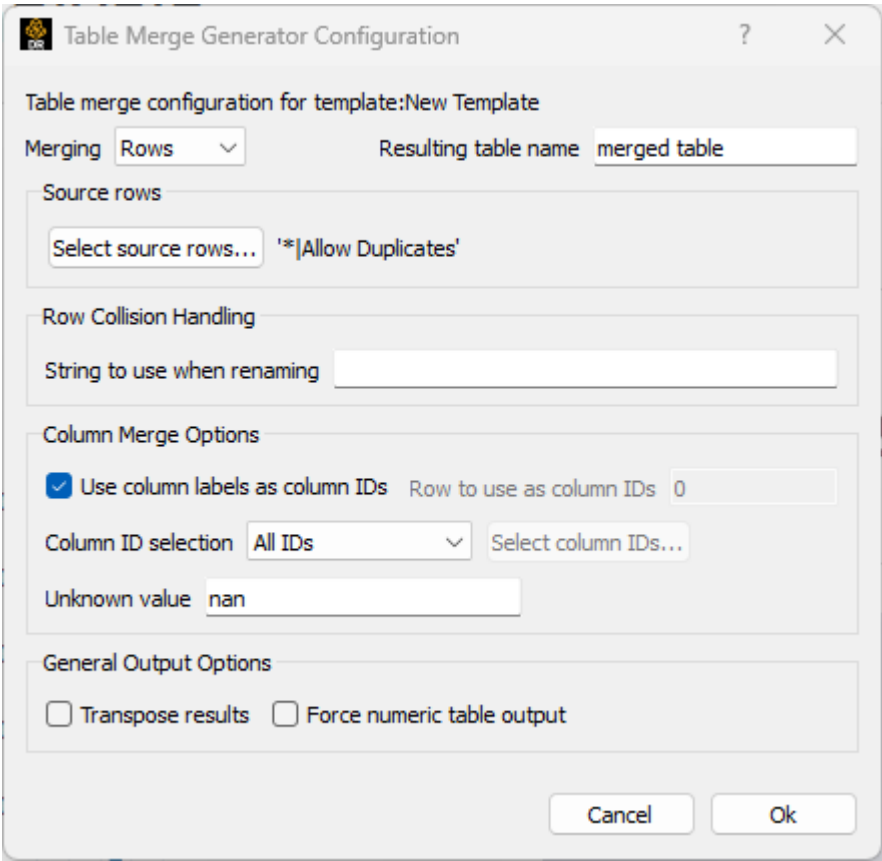
Configuration Options

The configuration options for this generator are relatively complex. A summary of the configured merge operation is presented in the template editor main window. For example:



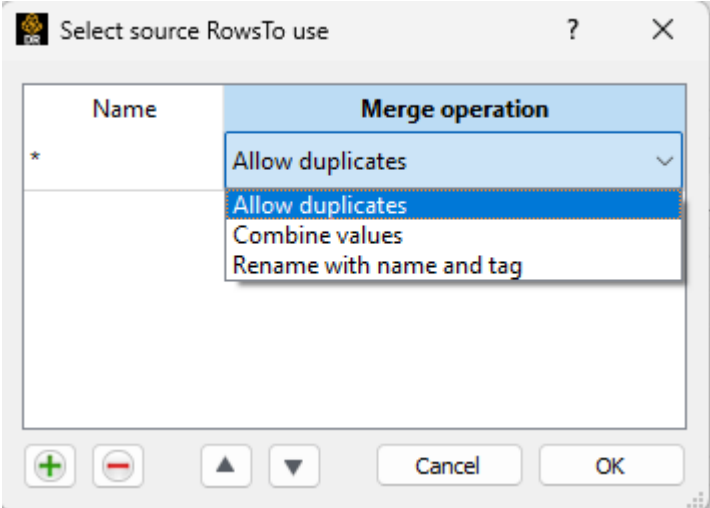
To open the settings editor, click **Configure data item merge**.

The **Table Merge Generator Configuration** dialog is displayed.



The default configuration is to simply concatenate all of the rows from all of the input tables into the output table. The resulting table will have the same number of rows as the sum of the numbers of rows in all of the input tables. The output columns will be the set of unique columns from the list of all of the columns in all of the input tables. When a row is selected for inclusion in the output (depending on the **Source rows** settings), the generator walks all of the columns in the selected row. The column itself may be selected or not depending on the **Column Merge Options** settings. If the column name already exists in the output table, the associated value in the selected row is placed in that column in the output. If the column name does not already exist, a new column is added to the output table. The undefined cells in the table are filled with the **Unknown Value** selection.

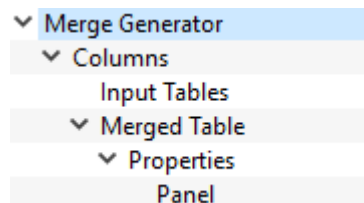
Option	Description
Mer- ging	The merge process can be performed by row or column. If Rows merging is selected, the selection of portions of each input table is performed row by row. If Columns is selected, the operation is performed column by column. A column merge is just a row merge applied to the transpose of the input table, followed by a final transpose before output.
Res- ult- ing table name	Sets the name for the generated table(s). The table name can include macro expansions and uses the current data item to access additional macro sources (p. 83) (for example, tag values). The generated tables include a generate_item_count=N tag that is the number of input table data items used to build the output table.
Se- lect	Opens a dialog, allowing you to define the row selection rules.

Option	Description												
source rows	 In the Name column, enter the name of the row(s) to be selected. The name can be the row index (a number), the row label name or a row label name that contains a glob style wildcard. In the Merge operation column, select a merge option that triggers when a row is selected for merging with the same name as an existing row: • Allow duplicates Adds the new row and the output table contains rows with identical row label names. • Combine values Combines the values of the new row with the row that already exists in the table. The final value is the last row merged into the output table. • Rename with name and tag Renames the new row then adds it to the output table. The final row name can be just the original row name (default) or a new string created starting from tag names and/or values.												
String to use when re-naming	Specifies the final row name. Enter any string and add the name of the tag to use in between the {{ }} characters. You can use wildcard (*) syntax to refer to tags when there is no tag value associated with a tag key. In addition to tags present on the source tables, the following tags can be used as well: <table border="1"> <thead> <tr> <th>Tag</th><th>Value</th></tr> </thead> <tbody> <tr> <td>_index_</td><td>Index number of the input table that produced the row (0 based)</td></tr> <tr> <td>_guid_</td><td>GUID of the input table item</td></tr> <tr> <td>_name_</td><td>Name field of the input table item</td></tr> <tr> <td>_source_</td><td>Source field of the input table item</td></tr> <tr> <td>_rowname_</td><td>Original name of the row</td></tr> </tbody> </table>	Tag	Value	_index_	Index number of the input table that produced the row (0 based)	_guid_	GUID of the input table item	_name_	Name field of the input table item	_source_	Source field of the input table item	_rowname_	Original name of the row
Tag	Value												
index	Index number of the input table that produced the row (0 based)												
guid	GUID of the input table item												
name	Name field of the input table item												
source	Source field of the input table item												
rowname	Original name of the row												

Option	Description
	So for example, if two rows come from items that have tags: <pre>Item 1: dp=dp0 SimulationA Item 2: dp=dp1 SimulationB</pre> the following value in the String to use when renaming field: <pre>New name is {{dp}} for {{Simulation*}}</pre> Results in merged rows named: <pre>New name is dp0 for SimulationA New name is dp1 for SimulationB</pre>
Use column labels as column IDs	When selected, input table column labels are used as column IDs. If the input table has no column labels, the column names are the index of the column. When cleared, the values in any row can be used as the column IDs. Enter the row number or row label name in the Row to use as column IDs field.
Column ID selection	Sets what column IDs are merged. Options are: • All IDs (default) Select all of the columns. • Only Common IDs Selects only those column IDs that are in common over all of the input table data items. • Select Specific IDs Click the Select column IDs button to select a collection of specific column IDs to be included in the output.
Unknown value	Fills in empty table cells that might be introduced by merging rows from two tables that do not have common columns. The Not a Number (NaN) value is mapped to floating point NaN in numeric tables. This can be useful when merging data for plots that do not have the same X axis values as NaN values are skipped when selected for plotting.
Transpose results	When selected, the final table is transposed before being placed in the output stream.
Force numeric table output	When selected, the output table is output as a table of numbers. The merge operation preserves the cell type of the input tables where possible. If any string tables are encountered, the output is a string table.

Example

A more complex, but common example is illustrated here. This example contains a collection of tables that represent template versus distance plots at three locations in a dataset. The aim is to put these three traces on a single plot. This is done by exploiting many of the core features of the table merge generator. The template layout used looks like this:



Merge Generator selects the three input tables to use. **Columns** is a simple two column layout so that the inputs and output can be displayed side by side. **Input Tables** is a simple Panel layout that is used to display the three input tables as plots. The **Merged Table** template is an instance of the Table Merge generator. It is configured as shown/discussed below. The **Properties** layout is an instance of the Tag Properties layout that makes the tags on the generated table available as properties for table layout/display. **Panel** is a simple panel layout with the custom HTML header set to:

```
<h2>Table row merged from {{generator_item_count|floatformat}} input tables.</h2>*
```

The generator layout adds a `generator_item_count=N` tag to all of the tables it outputs. This is the number of table data items used as the input to this output table. The Tag Properties layout made it available for use in the HTML header.

The generator configuration specifics look like this:

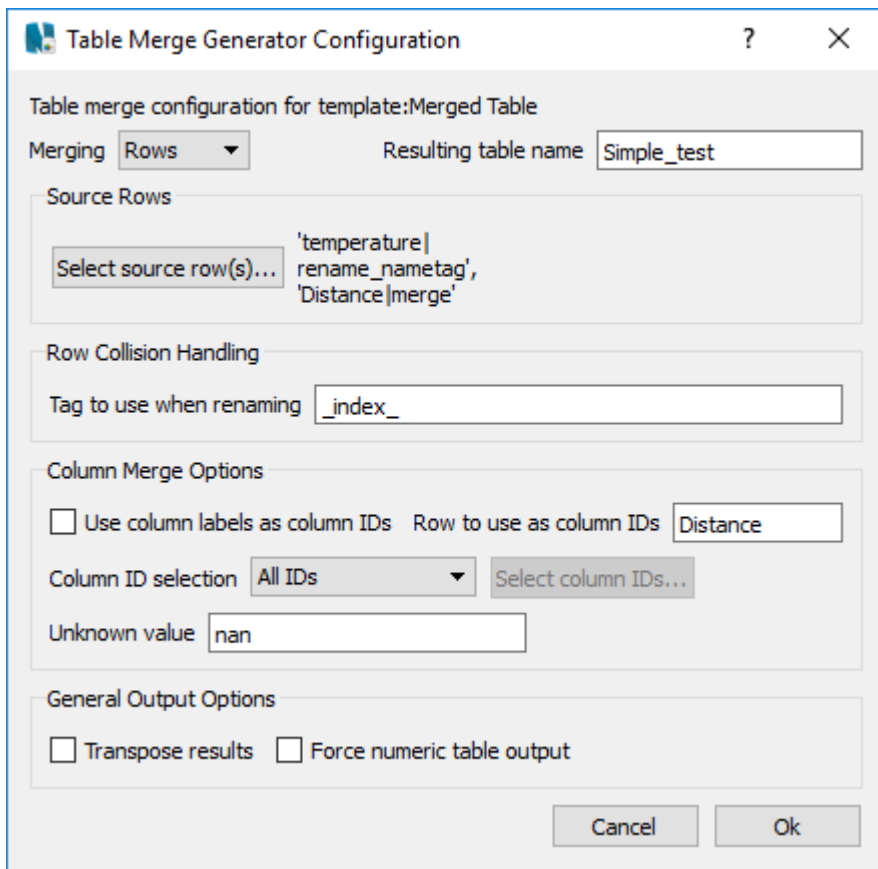


Table Merge Generator Configuration

Table merge configuration for template:Merged Table

Merging **Rows** Resulting table name **Simple_test**

Source Rows

Select source row(s)... 'temperature|
rename_nametag',
'Distance|merge'

Row Collision Handling

Tag to use when renaming **_index_**

Column Merge Options

☐ Use column labels as column IDs Row to use as column IDs **Distance**

Column ID selection **All IDs** Select column IDs...

Unknown value **nan**

General Output Options

☐ Transpose results ☐ Force numeric table output

Cancel Ok

Two sources are selected. All rows with the name **temperature** are to be merged and they should be renamed by adding the value of the tag to the input row name. In this case the tag is **_index_**. All the rows with the name **Distance** are also included and they should be merged into a single row. Assuming each input table has a single temperature and a single **Distance** row, the output will include three **temperature+_index_** named rows and a single **Distance** row. In the column options, the **Distance** row has been selected to be used as the column IDs, so the final **Distance** row has all of the unique values in all three **Distance** input rows. The implication here is that there will be empty cells in the output table, then the Distance values are not in common between the source tables. In this case, that is ideal as it preserves the desired X vs Y plots and allows for a common X axis named **Distance** to be used for all of the Y rows.

When run, the reports looks like this:

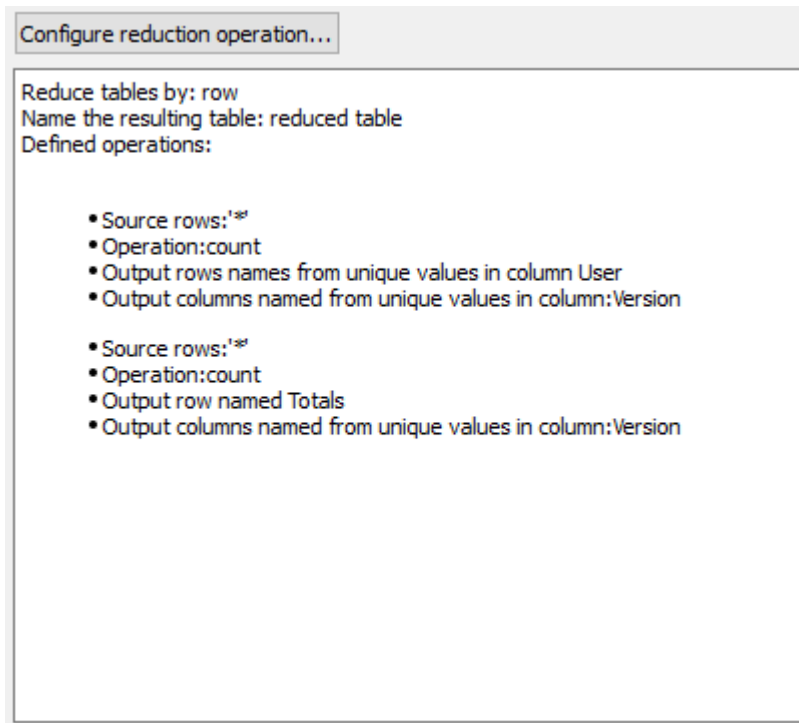


Table Reduction

The Table Reduction generator can be used to collapse a number of rows or columns into a single row/column. In this process, the values can be combined using common operations like count, sum, minimum or maximum. In the simplest case, the system can generate a *Total* row from a collection of rows. This operation is applied to each input table data item and a new output table data item is produced for each input table.

Configuration Options

The configuration options for this generator are relatively complex. A summary of the configured merge operation is presented in the template editor main window. For example:



To open the settings editor, click **Configure reduction operation**.

The **Table Reduce Generator Configuration** dialog is displayed.

Table Reduce Generator Configuration

Table reduction configuration for template:ASCII reduce

Reduce **Rows** Resulting table name **reduced table**

Reduce Operations

Select source rows...

Output rows

☒ Use unique values from a column as row names

Column name **User**

Output columns

☐ Use existing columns

Select columns *

☒ Use unique values from a column as new columns

Column name **Version**

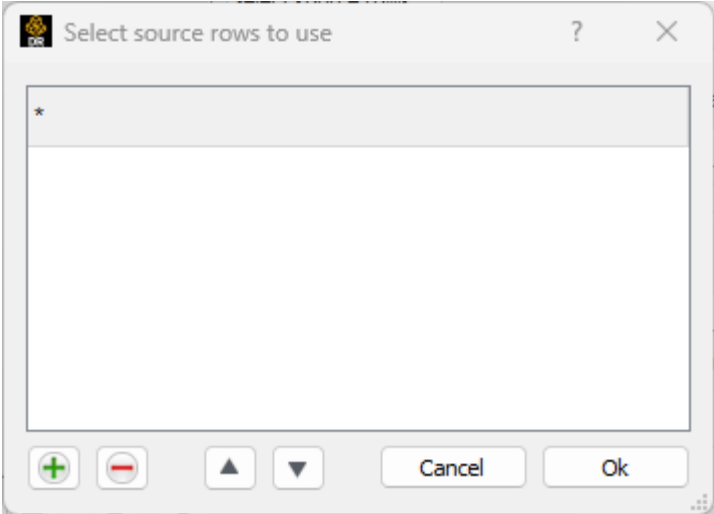
Operation **Count**

General Output Options

☐ Transpose results ☐ Force numeric table output

Cancel Ok

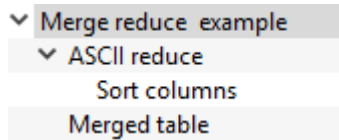
Option	Description
Re-duce	The process can be performed by reducing rows or columns. If Rows reducing is selected, the selection of portions of each input table is performed row by row. If Columns is selected, the operation is performed column by column. A column reduction is just a row reduction applied to the transpose of the input table, followed by a final transpose before output.
Resulting table name	Sets the name for the generated table(s). The table name can include macro expansions and uses the current data item to access additional macro sources (p. 83) (for example, tag values).
Re-duce Operations	Sets up one or more reduction operations. Use the  button to add an additional operation. Use the  button to remove the currently selected operation. Use the triangle buttons to select the order the operations are performed in. This ordering selects the order of the output rows as the output table rows are populated sequentially by each operation.
Select source rows	Opens a dialog, allowing you to define which input rows should be used as the source for the reduction operation.

Option	Description
	 Enter an expression for the row(s) to be selected. The expression can include row indices, row label names or row label names with glob style wildcards.
Use unique values from a column as row names	When selected, the output row label name is determined by the unique values found in a specific column. Enter the name in the Column name field. ☒ Use unique values from a column as row names Column name output row The reduction operation works a bit differently in this mode. First, all of the unique values in the specified column are collected. The input rows are separated into separate groups based on what unique value they have for that column. At that point, each group of input rows has the specified reduction operation applied to them, independently, producing one output row for each group. The output row labels are the unique value from the specified column. When cleared, the output row label name is specified explicitly and the reduction operation is performed on this specific row. Enter the name in the Output row name field. ☐ Use unique values from a column as row names Output row name output row
Use existing columns	Select to copy the column structure from the input table to the output table. Enter a glob style wildcards expression into the Select columns field to select the column label names. If the input table has no column labels, the index number of the column becomes the column label for this selection process. The default is to include all of the input columns (*). Note: The output column specification can be different for each reduction operation. This is handled by first collecting all of the output columns generated by each of the reduction operations. If two operations create the same column labels,

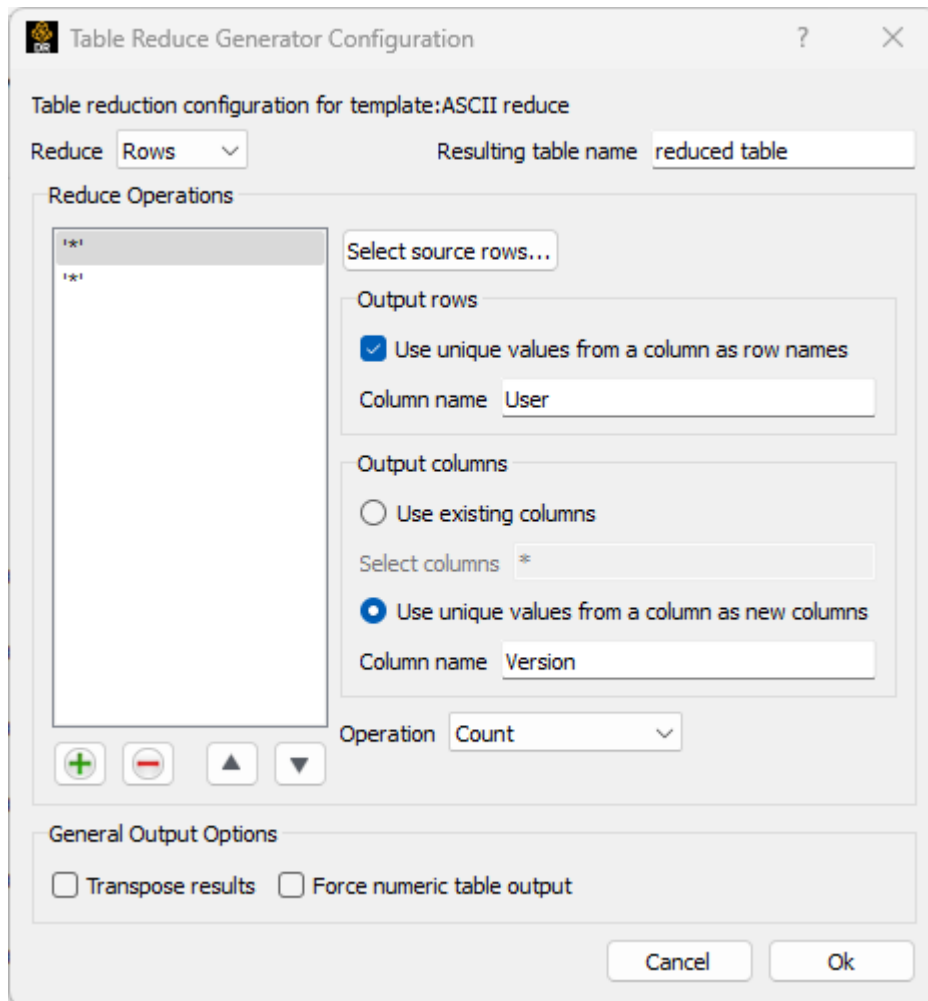
Option	Description																				
	that label is only added once into the output table. The order of the columns (like the rows) depends on the ordering of the reduction operations as they are added left to right.																				
Use unique values from a column as new columns	Select to potentially create multiple output columns using the unique values in an existing column for all of the selected rows. For example if the <i>Version</i> column includes the values 1.0, 1.1, 1.3 and 2.0 (in the rows selected by the reduction operation), the output table has columns with the labels <i>1.0</i>, <i>1.1</i>, <i>1.3</i>, and <i>2.0</i>. In this case, the selected input rows are separated into four separate groups based on those values. The reduction operation for the output column only includes input values from the rows in the associated groups. Enter the column name to use into the Column name field																				
Operation	The generator collects all of the values for that target and performs the operation selected by this setting. Note that NaN values are not used when computing these values.																				
	<table><tr><th>Operation</th><th>Result</th></tr><tr><td>Minimum</td><td>The minimum of all of the values is returned.</td></tr><tr><td>Maximum</td><td>The maximum of all of the values is returned.</td></tr><tr><td>Count</td><td>The number of values is returned.</td></tr><tr><td>Sum</td><td>The result of adding all of the values together is returned.</td></tr><tr><td>Difference</td><td>The difference between the values is returned. The difference is taken in order of the rows so generally it is the first value minus all of the other values. This option may only make sense when two rows are being collapsed together.</td></tr><tr><td>Mean</td><td>The average of all of the values is returned.</td></tr><tr><td>Standard deviation</td><td>The standard deviation of all of the values is returned.</td></tr><tr><td>Skewness</td><td>The skewness of all of the values is returned.</td></tr><tr><td>Kurtosis</td><td>The kurtosis of all of the values is returned.</td></tr></table>	Operation	Result	Minimum	The minimum of all of the values is returned.	Maximum	The maximum of all of the values is returned.	Count	The number of values is returned.	Sum	The result of adding all of the values together is returned.	Difference	The difference between the values is returned. The difference is taken in order of the rows so generally it is the first value minus all of the other values. This option may only make sense when two rows are being collapsed together.	Mean	The average of all of the values is returned.	Standard deviation	The standard deviation of all of the values is returned.	Skewness	The skewness of all of the values is returned.	Kurtosis	The kurtosis of all of the values is returned.
	Operation	Result																			
	Minimum	The minimum of all of the values is returned.																			
	Maximum	The maximum of all of the values is returned.																			
	Count	The number of values is returned.																			
	Sum	The result of adding all of the values together is returned.																			
	Difference	The difference between the values is returned. The difference is taken in order of the rows so generally it is the first value minus all of the other values. This option may only make sense when two rows are being collapsed together.																			
	Mean	The average of all of the values is returned.																			
	Standard deviation	The standard deviation of all of the values is returned.																			
Skewness	The skewness of all of the values is returned.																				
Kurtosis	The kurtosis of all of the values is returned.																				
Trans- pose res- ults	When selected, the final table is transposed before being placed in the output stream.																				
Force nu- meric table out- put	When selected, the output table is output as a table of numbers. By default, the reduction operation can be performed on a table containing strings, only interpreting the necessary cells as numbers for the operations. The generator tries to preserve the string format in the output table unless this check box is selected.																				

Example

This example consists of some input tables that include **User**, **Version** and **Date** columns, using a report template layout that looks like this:



The configuration options are set as:



The **Merge reduce example** template is a [Table Merge generator \(p. 197\)](#) that is used to select three input tables and concatenate them into a single table. That single, merged table is passed to the **ASCII reduce** Table Reduction generator instance, configured with two reduction operations. **Merged table** is a simple column layout to show the input table. **Sort columns** is a [Table Row/Column Sort \(p. 218\)](#) generator used to sort the columns by their label.

The reduction generator creates a new table consisting of rows aggregated by the **User** names and columns aggregated by the **Version** values in the input table. Each cell in the output is the number of rows in the input data item with those specific user and version values. Finally, the second reduction

operation includes a **Total** row that collapses all of the user values together into a single row. The final report looks like this:

Number of Runs by User and Version

	1.0	1.1	1.3	2.0
jill	23	22	25	20
mary	25	27	29	24
fred	18	18	15	18
bob	25	27	22	22
Totals	91	94	91	84

Raw Program Use by Version and Date

User	Version	Date
jill	1.1	24 Jan 2017 06:56AM
mary	2.0	25 Jan 2017 04:15PM
mary	1.0	17 Jan 2017 08:07PM
jill	1.1	24 Jan 2017 01:17PM
fred	1.1	31 Jan 2017 06:27PM
jill	1.0	18 Jan 2017 09:11PM
mary	1.3	20 Jan 2017 03:27PM
mary	2.0	26 Jan 2017 06:59AM
fred	2.0	04 Jan 2017 07:45AM
fred	1.3	07 Jan 2017 06:56AM
jill	1.0	05 Jan 2017 10:56PM
bob	1.1	07 Jan 2017 04:40PM
bob	2.0	30 Jan 2017 05:39AM

Table Row/Column Filter

The Table Row/Column Filter generator looks at all of the input table Items and for each one, removes selected rows or columns from the table item creating a new table item. There is one output table item for every input table item. There are separate filters for the rows and columns. The generator

may either select the rows/columns to keep in the output table or select what rows/columns to remove from the output table. Finally, it includes an option to output the transpose of the input table. The default configuration is to essentially clone the input table, that is select all rows and columns to be placed into the output table and not to transpose the table.

Configuration Options

There are a few configuration options for this generator.

Select rows & columns to preserve

New table name

row/column filtered table

Rows...

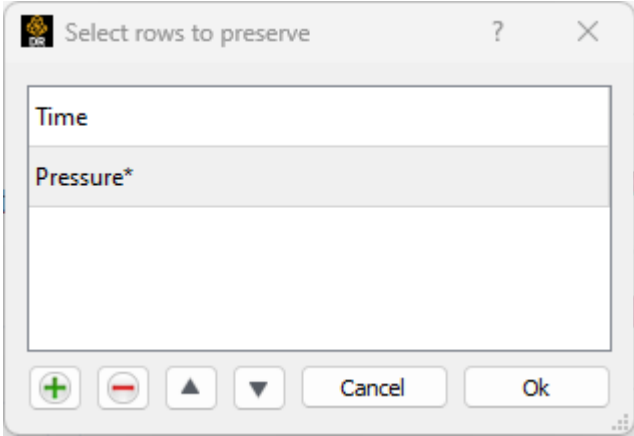
Columns...

☐ Select the rows/columns to remove

☐ Sort rows/columns by selection order

☐ Transpose the output table

Option	Description
New table name	Sets the name for the generated table(s). The table name can include macro expansions and uses the current data item to access additional macro sources (p. 83) (for example, tag values).
Rows	Opens a dialog that allows you to select a collection of rows or columns. By default, the selected rows/columns are included in the output and other rows/columns are removed.
Columns	
Select the rows/columns to remove	When selected, the collection of rows and columns are removed instead of retained.
Sort rows/columns by selection order	By default, the output rows and columns retain relative positions they occupied in the original table. If the generation has been configured to preserve rows/columns (instead of removing them), select this option to change their positions. The "*" row or column selection copies all rows/columns in their original order, even if this option is selected. The order of the row/column specification is used to select the new order. The sorting is performed both on the rows and columns, independently. For example, if the Select rows to preserve dialog is configured like this:

Option	Description
	 Then the row with the name Time is the first row in the output table. The rows selected by Pressure* follow in the same relative order that those rows had in the original table.
Transpose the output table	When selected, specifies that the new, filtered table should be transposed (rows and columns swapped) before being output.

Example

This example focuses on a single input table of element counts by the names of parts. The report template layout looks like this:

▼ RC Filter Generator

▼ Filter

▼ Tab layout

Raw Table

Filtered Table

The top level layout selects the specific table to process. **Filter** is a Table Row/Column generator template instance. The configuration of this generator looks like this:

Template properties

guid:f6027a0f-cfb0-11e6-9729-8b07041e08e6

Modified:2017-03-09 20:03:46

Template type: Table Row/Column Filter Generator ▼

Filtering

Filter:

Edit item filter... Filter mode Incoming Items ▼

Edit properties...

Edit item sorting rules... Sort selection All items ▼

Common Generator options

Generated items Append ▼

☒ Append template tags to new items

Select rows & columns to preserve

New table name RC_Filtered_Table

Rows... '0', 'fuselage', '*wing*'

Columns... '1', 'Tria3', 'Tetra*'

☐ Select the rows/columns to remove

☐ Transpose the output table

The **Generated items** option is set to **Append**, so that after running, both the input table and the generated table are both passed to the **Tab layout** layout. The generated table is named **RC_Filtered_Table** (as per the configuration). The **Filtered Table** layout selects only input tables with that name while the **Raw Table** layout selects only tables with names that do not have that name. This is a common theme with generators. They can augment the data item lists with new items with specially constructed names or tags that child layouts may filter down to.

The row and column filter dialogs in this example look like this:

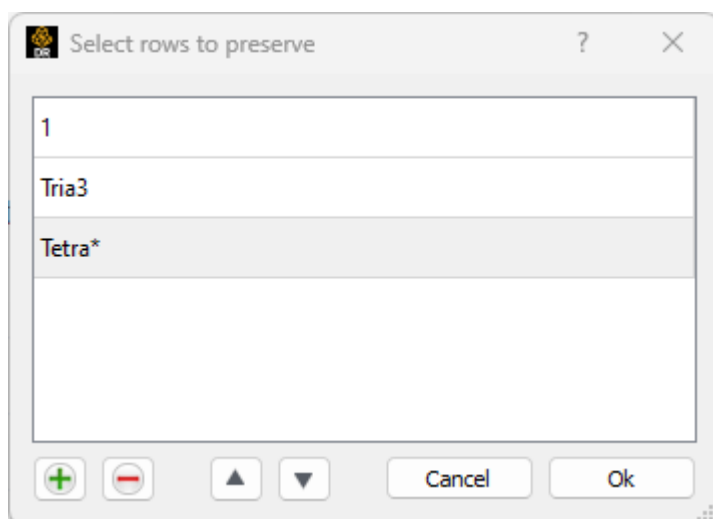
Select rows to preserve

0

fuselage

wing

+ - ▲ ▼ Cancel Ok



Three rows and three column names are selected for inclusion in the output. The dialogs illustrate the various methods for selecting rows/columns. First, you can use the row or column number. In the example, row **0** and column **1** will be preserved. This can be useful if there are no row/columns labels on a table item. The second case involves the use of a specific row/column name. The row labeled **fuselage** and the column labeled **Tria3** will be preserved in the output. Finally, you can use 'glob' style wildcards together with the label filters. So, any row with **wing** in its label will be kept and any column whose label starts with **Tetra** will be kept.

The output of this report looks like this:

Raw Table		Filtered Table															
		Server Element Distribution															
	Point element	Bar2	Bar3	Tria3	Tria6	Quad4	Quad8	Tetra4	Tetra10	Pyra5	Pyra13	Penta6	Penta15	Hexa8	Hexa20	N-Sided	N-faced
internalMesh	0	0	0	0	0	0	0	1595826	0	0	0	0	0	0	0	0	0
canopy	0	0	0	734	0	0	0	0	0	0	0	0	0	0	0	0	0
fuselage	0	0	0	10576	0	0	0	0	0	0	0	0	0	0	0	0	0
horizontal_stabilizer	0	0	0	7985	0	0	0	0	0	0	0	0	0	0	0	0	0
inlet	0	0	0	3010	0	0	0	0	0	0	0	0	0	0	0	0	0
nose	0	0	0	1408	0	0	0	0	0	0	0	0	0	0	0	0	0
outlet	0	0	0	3010	0	0	0	0	0	0	0	0	0	0	0	0	0
prop-downstream	0	0	0	2232	0	0	0	0	0	0	0	0	0	0	0	0	0
prop-upstream	0	0	0	2232	0	0	0	0	0	0	0	0	0	0	0	0	0
symmetry	0	0	0	21518	0	0	0	0	0	0	0	0	0	0	0	0	0
vertical_stabilizer	0	0	0	4036	0	0	0	0	0	0	0	0	0	0	0	0	0
wing_lower_surface	0	0	0	5180	0	0	0	0	0	0	0	0	0	0	0	0	0
wing_te	0	0	0	140	0	0	0	0	0	0	0	0	0	0	0	0	0
wing_tip	0	0	0	247	0	0	0	0	0	0	0	0	0	0	0	0	0
wing_upper_surface	0	0	0	5180	0	0	0	0	0	0	0	0	0	0	0	0	0
Clip through wing	0	158	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Particle trace part	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Raw Table Filtered Table

	Bar2	Tria3	Tetra4	Tetra10
internalMesh	0	0	1595826	0
fuselage	0	10576	0	0
wing_lower_surface	0	5180	0	0
wing_te	0	140	0	0
wing_tip	0	247	0	0
wing_upper_surface	0	5180	0	0
Clip through wing	158	0	0	0

Notice that the selected rows and columns were extracted from the original table and used to make a new table with a different name.

Table Value Filter

The Table Value generator allows you to remove rows/columns from a table based on a filter applied to the values in a specific column/row. The generator is applied to every incoming table and produces a new output table for every input table.

There are a number of value filters that can be applied. These include:

- Range based min < value < max form tests where individual min or max tests can be skipped
- Keep the row/column if the value is in the upper/lower N values
- Keep the row/column if the value is in the upper/lower X percent of values
- Keep the row/column if the value is in a list of specific, valid values

The test can be inverted, so keep becomes reject.

Configuration Options

There are a few configuration options for this generator.

Filter table by values

New table name ValueFilteredTable

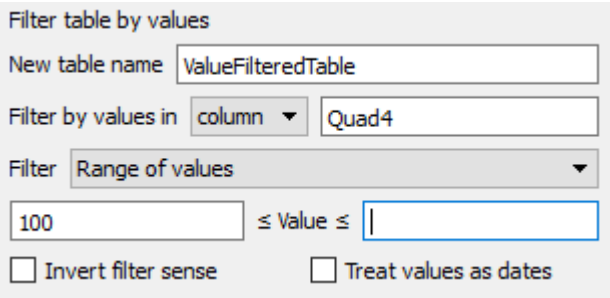
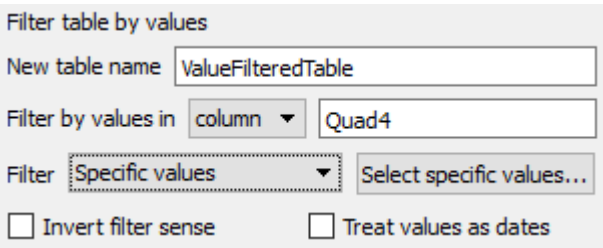
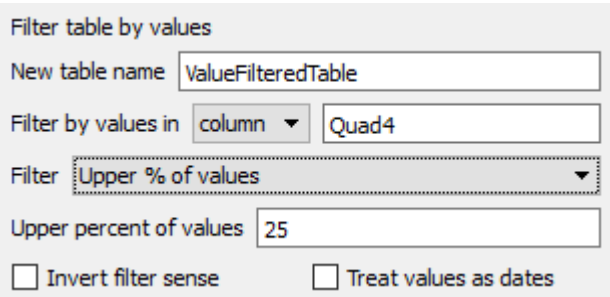
Filter by values in column Quad4

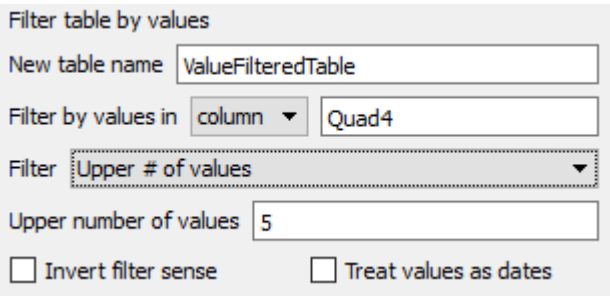
Filter Range of values

100 ≤ Value ≤

☐ Invert filter sense ☐ Treat values as dates

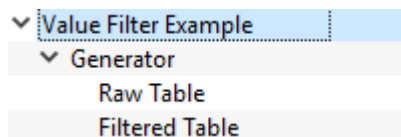
Option	Description
New table name	Sets the name for the generated table(s). The table name can include macro expansions and uses the current data item to access additional macro sources (p. 83) (for example, tag values).

Option	Description
Filter by values in	Sets the column or row to scan for values to filter by. In the example above, the generator will filter rows, using values in the column with the label Quad4 as the criteria. The name of the column (or row) can be the name of the label or a number which is the column or row index into the table (0-based numbering).
Filter	Sets the filter to use. Select from the following options: Range of values  Enter a lower bound and/or an upper bound. If the value is between or equal to the bounds, it will be kept. If one of the bounds is not specified (left empty), that specific test is not made. In the example, no comparison is made to an upper bound and the row is kept if the value is greater than or equal to 100. Specific values  Click Select specific values to enter a list of specific values. If the Filter by values in value matches any of the ones in the list, the row or column will be preserved. Upper/Lower % of values 

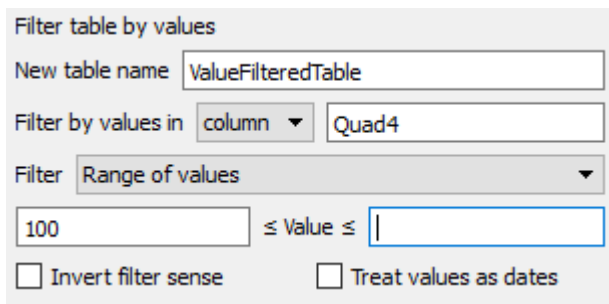
Option	Description
	Enter a value in the Upper percent of values or Lower percent of values field. This filter will keep the row or column if the value in Filter by values in is in the upper/lower percent of the values (over all rows or columns). • Upper/Lower # of values  Enter a value in the Upper number of values or Lower number of values field. This filter will keep the row or column if the value in Filter by values in is in the upper/lower number of the values (over all rows or columns). If there is at least the same number of rows or columns in the table as the value entered in the field, than the filter returns that number of items in all cases. If there are ties (for example, when ranked, rows 3 through 10 all have the same value), the specific rows selected from the list of tied rows is random.
Invert filter sense	When selected, the filter selects the rows or columns to remove instead of keep.
Treat values as dates	In Ansys Dynamic Reporting, it is common to put dates in tables as a double. The double is the number of seconds since midnight, Jan 1, 1970. When filtering a column that consists of such numbers, you can enter the limits as date/time values. When selected, this option enables this mode and makes it possible to easily filter a column of dates to a given range.

Example

In this example, the Table Value Filter generator is used to reduce the number of rows in a table. The template layout looks like:



Value Filter Example is a simple layout that selects the table to process. The actual generator instance is **Generator**. It is configured as:



Filter table by values

New table name

Filter by values in column ▾

Filter Range of values ▾

≤ Value ≤

☐ Invert filter sense ☐ Treat values as dates

The filter selects the rows to keep using the values in the **Quad4** column. It only keeps rows with values greater than or equal to 100. The generator merges the new table into the data items list and gives it a specific name, **ValueFilteredTable**. The **Raw Table** and **Filtered Table** templates are panel layouts that filter to the original and generator table items respectively. The resulting report looks like this:

Original table

Server Element Distribution																	
	Point element	Bar2	Bar3	Tria3	Tria6	Quad4	Quad8	Tetra4	Tetra10	Pyra5	Pyra13	Penta6	Penta15	Hexa8	Hexa20	N- Sided	N- faced
engine		0	0	0	0	0	0	0	0	0	0	0	0	338	0	0	0
tires		0	0	0	0	0	0	0	0	0	0	0	0	192	0	0	0
wheels		0	0	0	0	0	0	0	0	0	0	0	0	128	0	0	0
lights		0	0	0	0	0	32	0	0	0	0	0	0	0	0	0	0
front body		0	0	0	0	0	276	0	0	0	0	0	0	0	0	0	0
rear body		0	0	0	0	0	1244	0	0	0	0	0	0	0	0	0	0
floor		0	0	0	0	0	344	0	0	0	0	0	0	0	0	0	0
windshields		0	0	0	0	0	144	0	0	0	0	0	0	0	0	0	0
windows		0	0	0	0	0	104	0	0	0	0	0	0	0	0	0	0
bumpers		0	0	0	0	0	494	0	0	0	0	0	0	0	0	0	0
hood		0	0	0	0	0	182	0	0	0	0	0	0	0	0	0	0
mounts		0	0	0	0	0	86	0	0	0	0	0	0	0	0	0	0
guardrail supports		0	0	0	0	0	980	0	0	0	0	0	0	0	0	0	0
guardrail supports -> 0001		0	0	0	0	0	385	0	0	0	0	0	0	0	0	0	0
guardrail		0	0	0	0	0	1668	0	0	0	0	0	0	0	0	0	0

Filtered table

	Point element	Bar2	Bar3	Tria3	Tria6	Quad4	Quad8	Tetra4	Tetra10	Pyra5	Pyra13	Penta6	Penta15	Hexa8	Hexa20	N- Sided	N- faced
front body		0	0	0	0	0	276	0	0	0	0	0	0	0	0	0	0
rear body		0	0	0	0	0	1244	0	0	0	0	0	0	0	0	0	0
floor		0	0	0	0	0	344	0	0	0	0	0	0	0	0	0	0
windshields		0	0	0	0	0	144	0	0	0	0	0	0	0	0	0	0
windows		0	0	0	0	0	104	0	0	0	0	0	0	0	0	0	0
bumpers		0	0	0	0	0	494	0	0	0	0	0	0	0	0	0	0
hood		0	0	0	0	0	182	0	0	0	0	0	0	0	0	0	0
guardrail supports		0	0	0	0	0	980	0	0	0	0	0	0	0	0	0	0
guardrail supports -> 0001		0	0	0	0	0	385	0	0	0	0	0	0	0	0	0	0
guardrail		0	0	0	0	0	1668	0	0	0	0	0	0	0	0	0	0

Table Row/Column Sort

The Table Row/Column Sort generator looks at all of the input table items and for each one, applies a hierarchical sort to the rows and/or columns of the table. There is one output table for every input table.

The template always attempts to sort numerically first, regardless of the type of data in the table item. If this sort is not possible, then alphabetic sorting is performed.

Configuration Options

There are a few configuration options for this generator.

New table name

sorted table

Select columns to sort rows by...

Select rows to sort columns by...

Option	Description
New table name	Sets the name for the generated table(s). The table name can include macro expansions and uses the current data item to access additional macro sources (p. 83) (for example, tag values).
Select columns to sort rows by	Opens a row or column sorting editor. This editor is the same dialog as used for sorting data items (p. 137) in layouts. There are separate buttons used to set the sorting rules for the rows and columns. These rules are applied independently. Rows or columns can be sorted using values in specific columns or rows respectively. You can also select Labels to specify that the values of the column or row labels be used to sort the rows or columns.
Select rows to sort columns by	

Example

This example targets a single table and uses a template layout that looks like this:

▼ RC Sort Filter Example

Unsorted

▼ Sorted

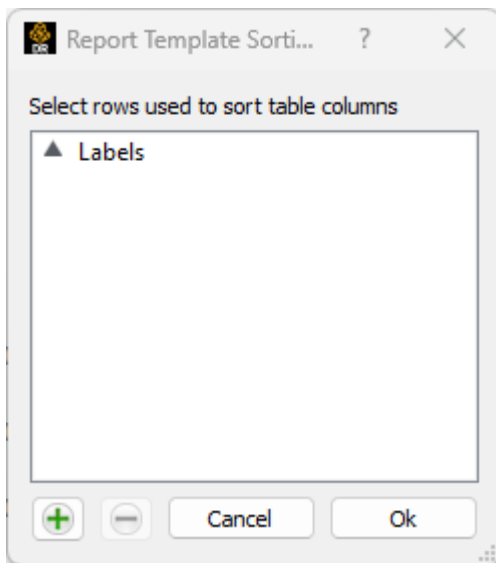
Panel

The top level template simply filters the input down to one table to process. The raw table is displayed by the **Unsorted** panel template. The start of the output looks like this (there are many more rows not displayed here):

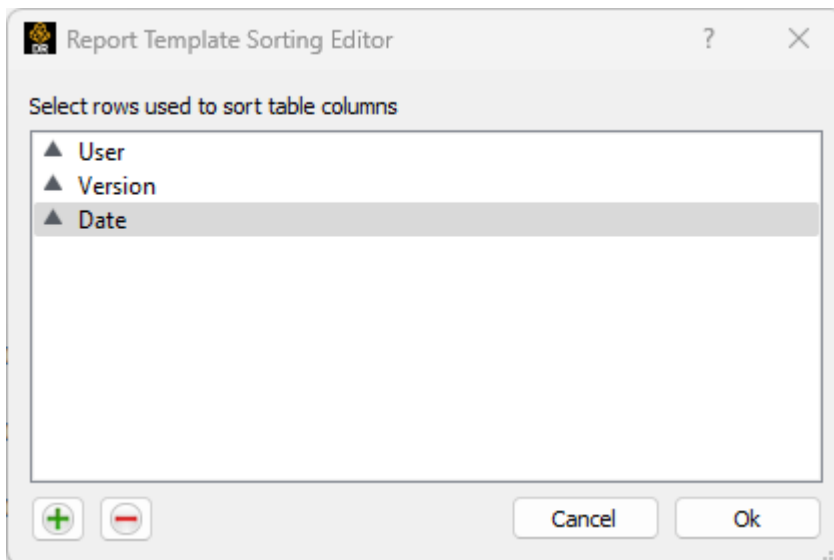
Unsorted table

January			
User	Version	Date	
jill	1.10	01/24/17 06:56:11	
mary	2.00	01/25/17 16:15:37	
mary	1.00	01/17/17 20:07:50	
jill	1.10	01/24/17 13:17:27	
fred	1.10	01/31/17 18:27:24	
jill	1.00	01/18/17 21:11:44	
mary	1.30	01/20/17 15:27:23	
mary	2.00	01/26/17 06:59:07	
fred	2.00	01/04/17 07:45:55	
fred	1.30	01/07/17 06:56:17	
jill	1.00	01/05/17 22:56:03	
bob	1.10	01/07/17 16:40:49	
bob	2.00	01/30/17 05:39:10	
bob	1.30	01/02/17 22:26:29	
jill	1.10	01/02/17 06:46:41	
fred	1.00	01/22/17 14:23:39	
bob	1.30	01/14/17 16:26:33	
mary	1.00	01/21/17 02:39:20	
bob	1.00	01/22/17 00:22:21	
jill	1.00	01/15/17 08:24:49	
jill	1.00	01/17/17 03:02:10	
bob	2.00	01/31/17 05:07:31	

The **Sorted** template is an instance of the Table Row/Column Sort generator. It sorts the columns using the labels for the columns sorting in an increasing fashion left to right. The **Select rows to sort columns by** configuration looks like this:



The **Select columns to sort rows by** configuration looks like this:



When considering the proper order of the rows, this configuration specifies that the **User** column value is the primary sort key. If there is a tie, the **Version** column value is used, and if there is still a tie, the **Date** column value is used. In all cases, the sort is performed from smallest to largest value.

The resulting sorted table is displayed like this:

Sorted table

Date	User	Version
01/22/17 00:22:21	bob	1.00
01/26/17 12:10:34	bob	1.00
01/07/17 16:40:49	bob	1.10
01/02/17 22:26:29	bob	1.30
01/14/17 16:26:33	bob	1.30
01/23/17 17:36:13	bob	1.30
01/30/17 05:39:10	bob	2.00
01/14/17 21:36:16	fred	1.00
01/22/17 14:23:39	fred	1.00
01/31/17 18:27:24	fred	1.10
01/07/17 06:56:17	fred	1.30
01/14/17 17:01:16	fred	1.30
01/04/17 07:45:55	fred	2.00
01/17/17 11:06:21	fred	2.00
01/05/17 22:56:03	jill	1.00
01/15/17 08:24:49	jill	1.00
01/17/17 03:02:10	jill	1.00
01/18/17 21:11:44	jill	1.00
01/02/17 06:46:41	jill	1.10
01/24/17 06:56:11	jill	1.10
01/24/17 13:17:27	jill	1.10
01/03/17 05:57:06	jill	1.20

SQL Query

The SQL Query generator allows you to type in an SQL query to run against an SQL or a PostgreSQL database. The result of this query is reported in Ansys Dynamic Reporting as a table item. The items in the table are always returned as strings, even if they are numeric in the source database.

Configuration Options

There are a few configuration options for this generator.

Option	Description
Database type	Sets the type of database to use. Select one of the following options: • Sqlite • Postgresql

Option	Description
Sqlite file-name	Is shown when Database type is set to Sqlite . Enter the name of the SQLite database file, including the absolute path.
Database name	Is shown when Database type is set to Postgresql . Enter the name of the database, including the absolute path.
Hostname	Is shown when Database type is set to Postgresql . Enter the name of the machine where the PostgreSQL instance is running.
Port	Is shown when Database type is set to Postgresql . Enter the port to be used for the connection to the PostgreSQL instance. If empty, the default port number 5432 is used.
Username	Is shown when Database type is set to Postgresql . Enter the username to access the database.
Password	Is shown when Database type is set to Postgresql . Enter the password to access the database.
Verify database connection	Click this button to have Ansys Dynamic Reporting try to connect with the database using the information provided. If Ansys Dynamic Reporting is able to establish a connection, a successful message is displayed. If Ansys Dynamic Reporting is not able to establish a connection, an error message is displayed.
SQL query text	Enter the SQL query to run against the database, as a string. There is no check in Ansys Dynamic Reporting about the validity or what exactly the SQL query does. It is up to you to make sure that the query syntax is correct.

Example

In this example, assume you want to get some information from an external SQLite database as part of the data to use in the current report. The database is stored in D:\tmp\report_db. The only information you must enter is the database type, location of the database, and the query text.

Database type: Sqlite

Sqlite filename: D:\tmp\report_db\db.sqlite3

Verify database connection...

SQL query text:

```
SELECT name, date, source, guid FROM data_item
```

The result of the SQL query is reported as a table (all items are strings):

name	date	source	guid
Logo Image	2019-03-28 21:04:32.303000	Session View	15722bf0519d11e982b5a44cc8bbc693
Part Element Details Source	2019-03-28 21:04:32.902000	part_element_source	15cd925e519d11e99642a44cc8bbc693
Scene Geometry Source	2019-03-28 21:04:33.246000	scene_geometry_source	16020fe1519d11e9b843a44cc8bbc693
Image Source	2019-03-28 21:04:34.088000	image_source	16828a80519d11e9b714a44cc8bbc693
Query Source	2019-03-28 21:04:34.366000	query_source	16acf5de519d11e982e9a44cc8bbc693
Query Source	2019-03-28 21:04:34.366000	query_source	16acf5df519d11e99bb8a44cc8bbc693
Query Source	2019-03-28 21:04:34.366000	query_source	16acf5e0519d11e9a6b1a44cc8bbc693

Being a table, you can apply all the properties that tables support.

The SQL Query template passes the table as an item to all its children templates. This table item is not persistent in the Ansys Dynamic Reporting database: it is not stored as an individual item. If you delete the SQL Query template, the resulting table will disappear as well.

Tree Merge

The Tree Merge generator takes multiple input data tree items and combines them into a single tree item. It produces a single output tree every time it is passed a list of input data items. If there are no tree data items in the input list, no output tree is generated. The tree merging operation works by matching the tree row names and concatenating the values from each tree. The resulting tree has at least one column of values added for each input tree. The merging process can be customized to select specific rows to merge.

The generator starts by copying the first tree item in the list. It then builds a table from the tree with one row for each leaf in the tree. Each row consists of a name and the value columns from the tree leaves. For example, the tree:

— top 1	Numbers	strings
leaf 1 1	1.00	
leaf 1 2	2.00	
— top 2	Numbers	strings
leaf 2 1		Hello
leaf 2 2		Bye
top 3	100	200

is represented by the generator as the table:

Row name	name	key	value[0]	value[1]
top 1/top	top 1	top	Numbers	strings
top 1/top leaf 1 1/leaf	leaf 1 1	leaf	1.0	N/A
top 1/top leaf 1 2/leaf	leaf 1 2	leaf	2.0	N/A
top 2/top	top 2	top	Numbers	strings
top 2/top leaf 2 1/leaf	leaf 2 1	leaf		Hello
top 2/top leaf 2 2/leaf	leaf 2 2	leaf		Bye
top 3/top	top 3	top	100	200

The generator compares values in the **Row name** column. If there is a match, the value columns from the additional tables are added to the working table.

If the value in a table cell is unspecified, the **Fill value** specified in the configuration options is used as the value of the cell. In the example, two rows had no values specified for value[1] and the fill value **N/A** is supplied by the generator.

The generated tree has the tags copied from the generator itself and has the item name specified by the **Output tree name** option (so the item can be filtered in child templates).

The **Header tag name** option can be used to generate a new header tree item at the top of the output tree. If the value is not empty, the values for the tree item is taken from the tag named in that field.

Tree row hints are copied from the first tree where a given **Row name** is encountered. There is no requirement that each tree has the number of values.

Configuration Options

There are a few configuration options for this generator.

Row merge

All rows

Match rows by

Name and key

Fill value

N/A

Output tree name

treemerge

Header tag name

header

Option	Description
Row merge	Sets what happens when a row name match or non-match happens. Select from the following options: • All rows If two trees have the same row, the value columns are appended. • Common rows Only those row names that are present in all of the input trees are included in the output tree. • Rows from first tree The first input tree defines all of the rows to include in the output. If a row name is not present in another tree, the fill value is output for the value columns for that tree.

Option	Description
Match rows by	Controls how the Row name column is computed. The row name is formed by {parentname} {rowitemname}. The {rowitemname} can be one of three forms based on this option. Select from the following options: • Name and key {name}/{key} • Name {name} • Key {key} This allows the generator to decide to merge rows using the hidden tree item key values independently of the item name.
Fill Value	Enter a value to use when the value in a table cell is unspecified.
Output tree name	Enter a name to use for the output tree.
Header tag name	Enter a tag name to use to generate a new header tree item at the top of the output tree.

Example

This example illustrates a more complex but common setup. There are two input trees:

Tags	header='Design Point 10'
Item ID	a3b0930a-502d-11eb-9a60-28f10e13ffe6
Session ID	a19c44cb-502d-11eb-a875-28f10e13ffe6
Dataset ID	a19c6bd8-502d-11eb-a128-28f10e13ffe6
Type	tree

Details

— Configuration	
Solver	Fluent
Tolerance	0.02
Order	Quadratic
Max Iterations	10000
— Output Values	
Output pressure	136.52
Output temperature	295.73
— Input Values	
Inlet pressure	310.64
Inlet diameter	3.02
Viscosity	1.09

Tags	header='Design Point 20'
Item ID	a3cb9510-502d-11eb-949c-28f10e13ffe6
Session ID	a19c44cb-502d-11eb-a875-28f10e13ffe6
Dataset ID	a19c6bd8-502d-11eb-a128-28f10e13ffe6
Type	tree

Details

— Configuration	
Solver	Fluent
Tolerance	0.02
Max Iterations	10000
— Output Values	
Output pressure	131.14
Output temperature	267.41
— Input Values	
Inlet pressure	317.01
Inlet diameter	2.58
Viscosity	0.94

Note the header tag value and the fact that the **Design Point 20** tree lacks the **Order** row. Using the options noted above, the resulting tree looks like this:

	Design Point 10	Design Point 20
— Configuration		
Solver	Fluent	Fluent
Tolerance	0.02	0.02
Order	Quadratic	N/A
Max Iterations	10000	10000
— Output Values		
Output pressure	136.52	131.14
Output temperature	295.73	267.41
— Input Values		
Inlet pressure	310.64	317.01
Inlet diameter	3.02	2.58
Viscosity	1.09	0.94

Notice the introduction of a header row at the top of the tree with values pulled from the header tags. Also, the introduction of the **N/A** value for the missing row in the **Design Point 20** tree.

Items Comparison

The Items Comparison generator allows you to define a list of items to compare. The generator finds all data items in the report database matching the set filters and returns them in the exact order specified. You can also specify a chunk size to define how many data items to group together in each comparison group. Each comparison group renders child templates of the generator independently.

Related topics:

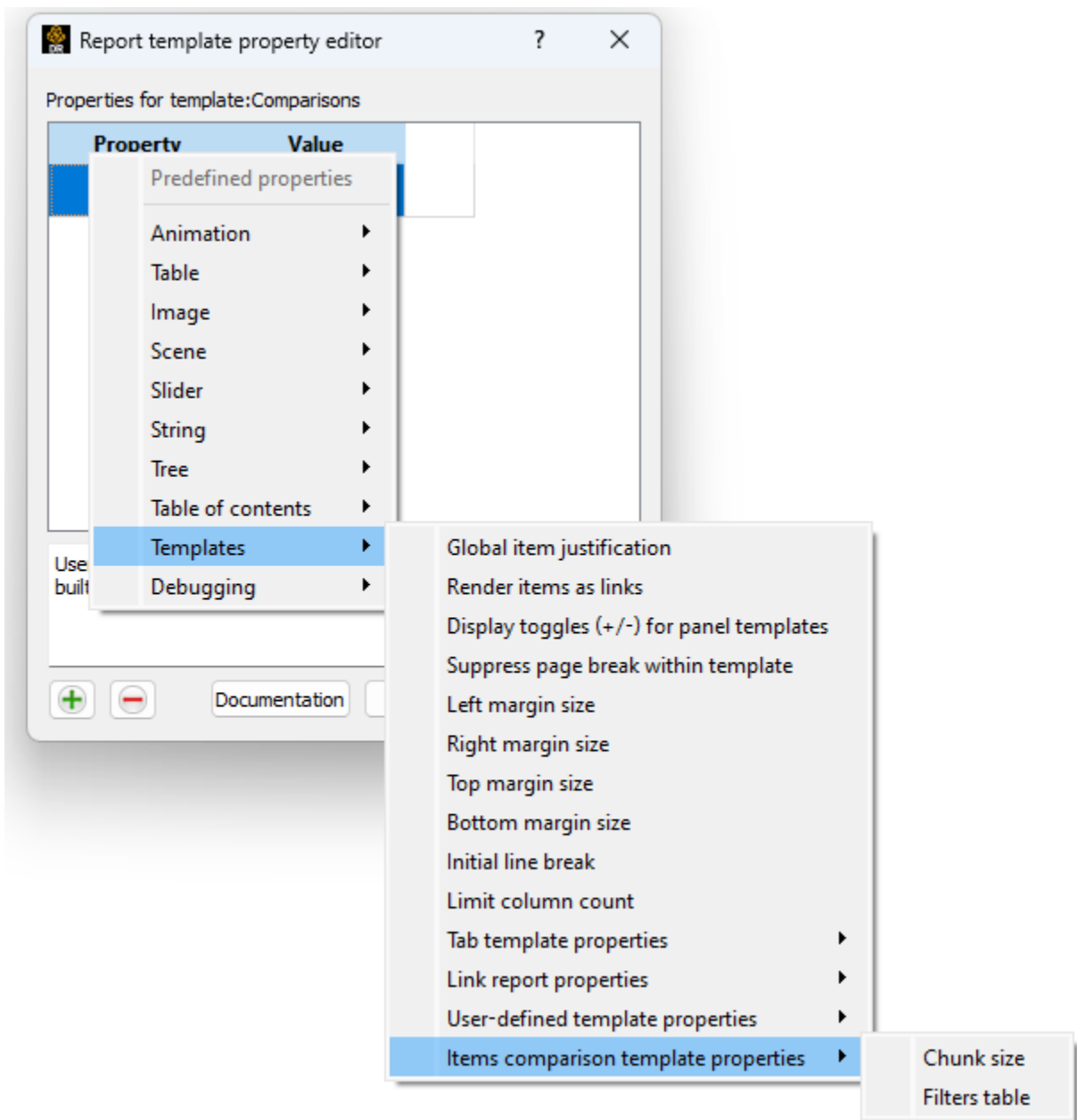
[Items Comparison Configuration Options](#)

Items Comparison Examples

Items Comparison Configuration Options

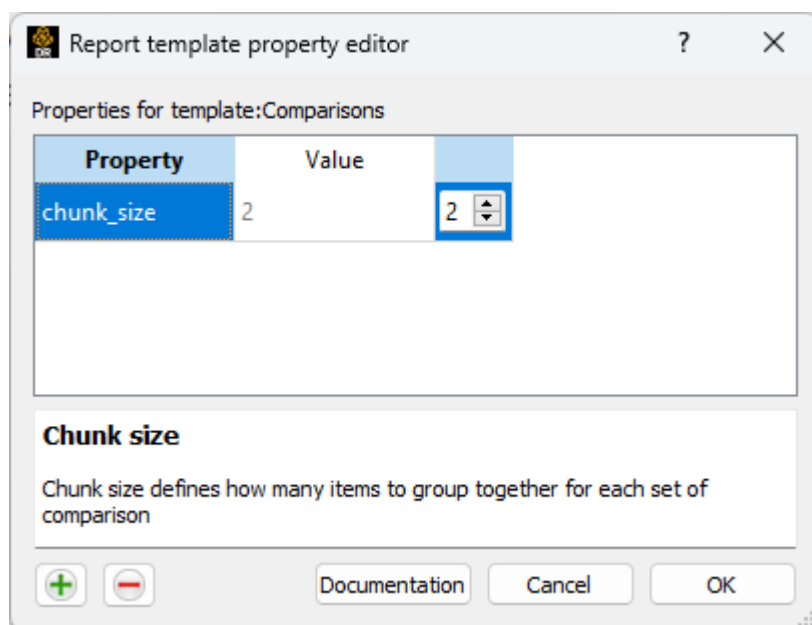
To set up this generator, set the `filters_table` and `chunk_size` properties:

1. Select a template or [create a new one](#). (p. 76)
2. In the **Template properties** pane, set the template type to **Item comparison generator**.
3. Click **Edit properties**.
4. In the dialog, right-click an empty **Property** cell and select **Templates** → **Items Comparison template properties** → **Chunk Size** or **Templates** → **Items Comparison template properties** → **Filters Table** from the context menu.



Chunk Size

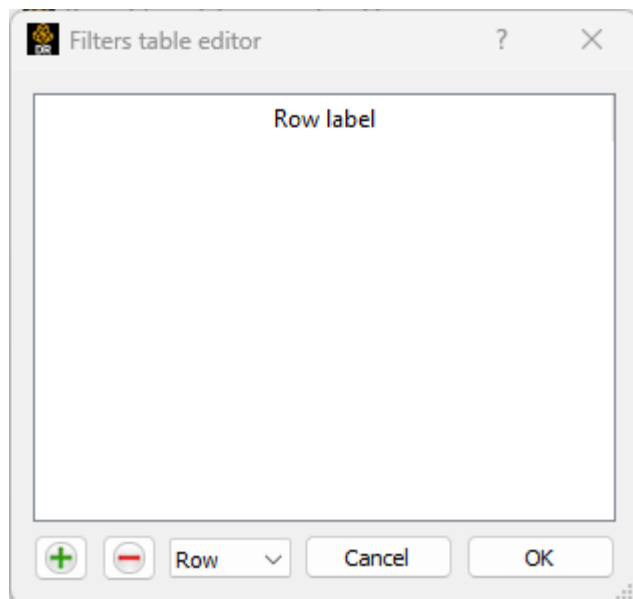
This property is an integer representing how many items to group together for each comparison group. Make sure your chunk size is less than or equal to the final number of items produced by the generator. If a chunk size is not specified, then the chunk size is set to the total number of items returned by the generator by default, so all resulting items are in the same group.



Filters Table

This property is a table where you can enter a list of filters to query a list of items to compare.

When you open the table editor, the first column by default is **Row label**.



The input format in this column is `{type} | {operator}`, where `{type}` is the filter type and `{operator}` is the filter's operator.

For `{type}`, the following entries are supported:

- `i_name`
- `i_src`

- i_date
- i_tags
- i_type
- i_seq
- meta

Special output value that is used to set a field in the context presented into child templates.

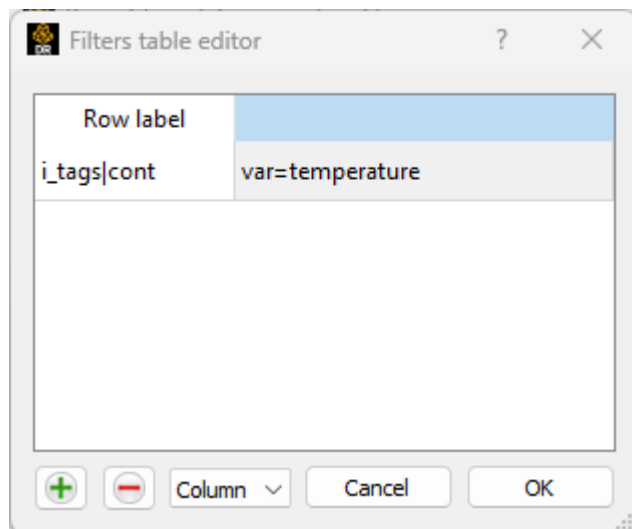
For {operator}, the following entries are supported:

- cont
- ncont
- eq
- neq
- meta

The name of the context key to be set to the value (if any) in the table.

For more details on the query syntax, see [Query Expressions \(p. 320\)](#).

Each cell represents an individual query stanza formed by appending the row label of the cell and the user input string captured by the cell. For example, if you enter **var=temperature**, and the cell has a row label of **i_tags|cont**, then that cell represents the query stanza **i_tags|cont|var=temperature**.



Each column is a complete query expression formed by chaining together all its cells using the *and* relationship.

To add a new filter stanza, select **Row**, from the drop-down list and click .

To add a new column, select **Column**, from the drop-down list and click .

To define the list of items to query, add the query field and operator in the rows and define the items by adding new columns and filling in the cells. The generator goes through each column, producing the corresponding query, and queries for matching items. The items corresponding to the first column are provided first in the returned items, followed by the items returned by query of the second column, and so on. The resulting items are passed down into child templates for further rendering.

The table accepts valid strings (just the string itself with no extraneous formatting such as double quotes) and blank (a blank cell gets translated into a blank query stanza and has no effect).

Items Comparison Examples

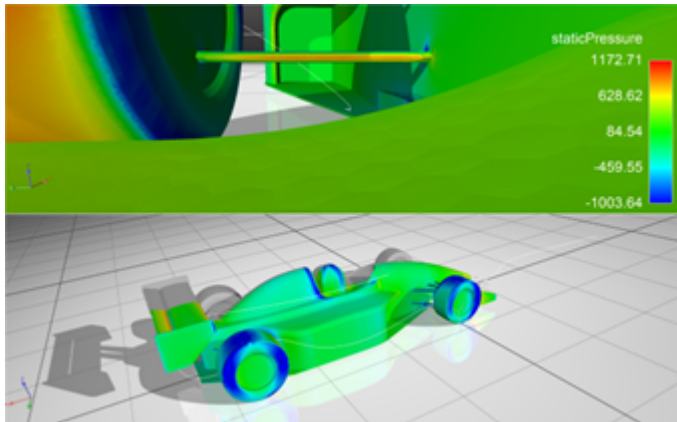
For more details on how to use this generator, see the following examples.

Example 1

In this example, there are three items in the database each with different tags:

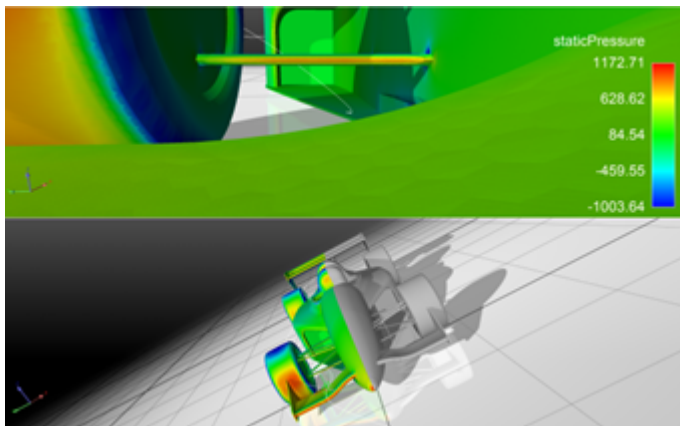
- **Image one**

Tags: dp=dp0, var=temperature



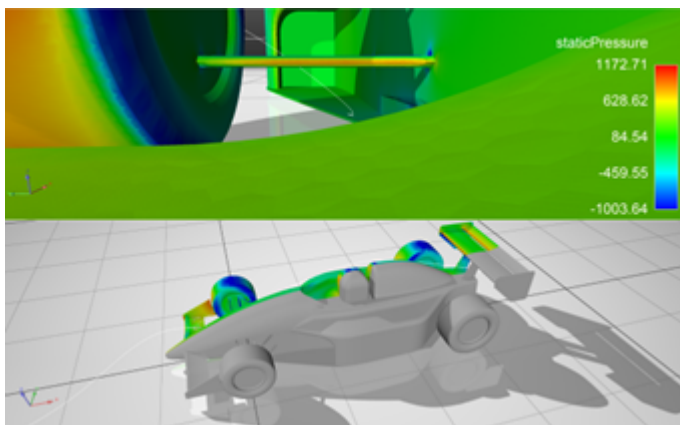
- **Image two**

Tags: dp=dp1, var=temperature

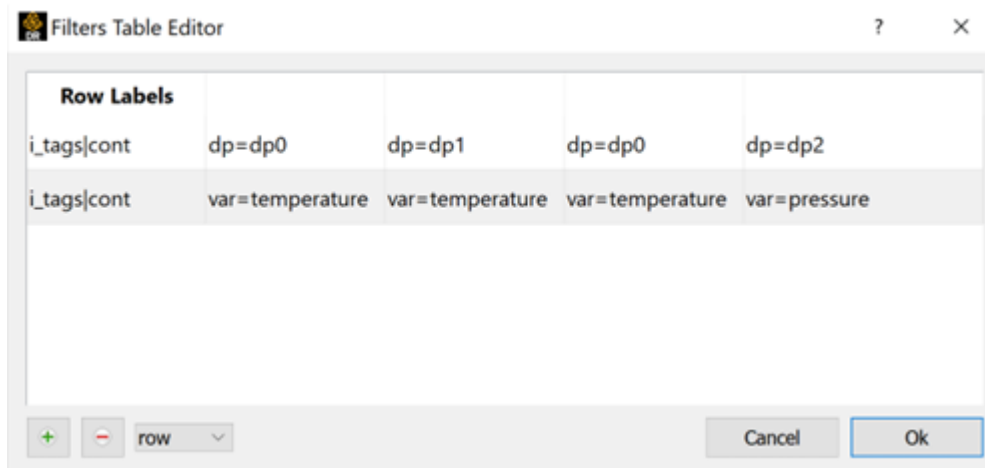


- **Image three**

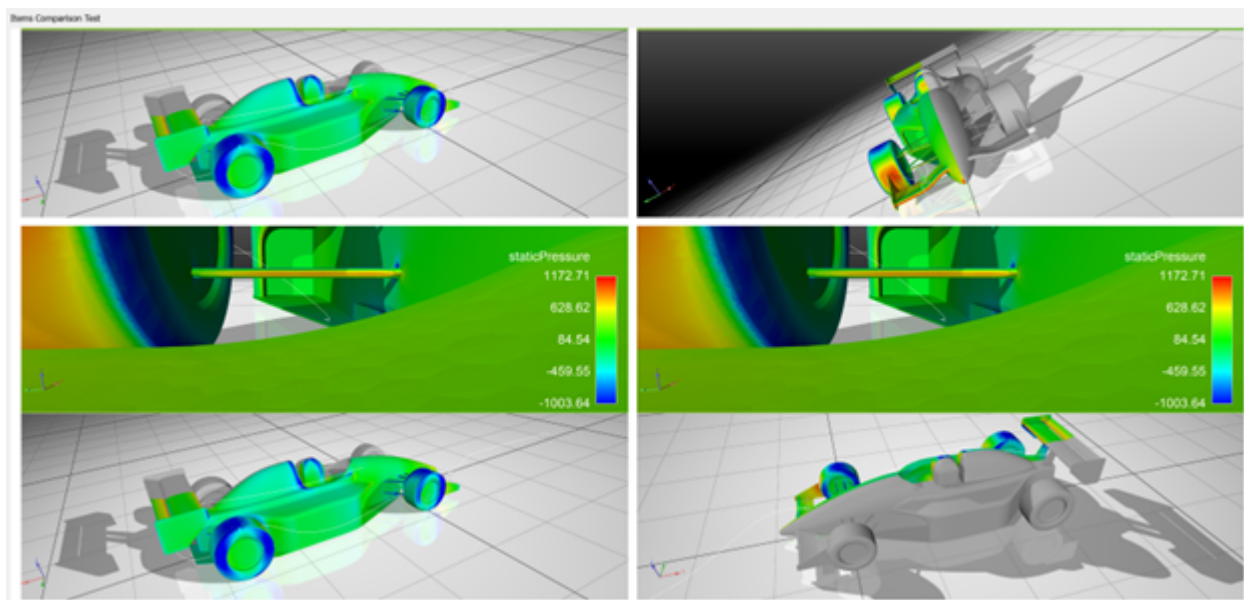
Tags: dp=dp2, var=pressure



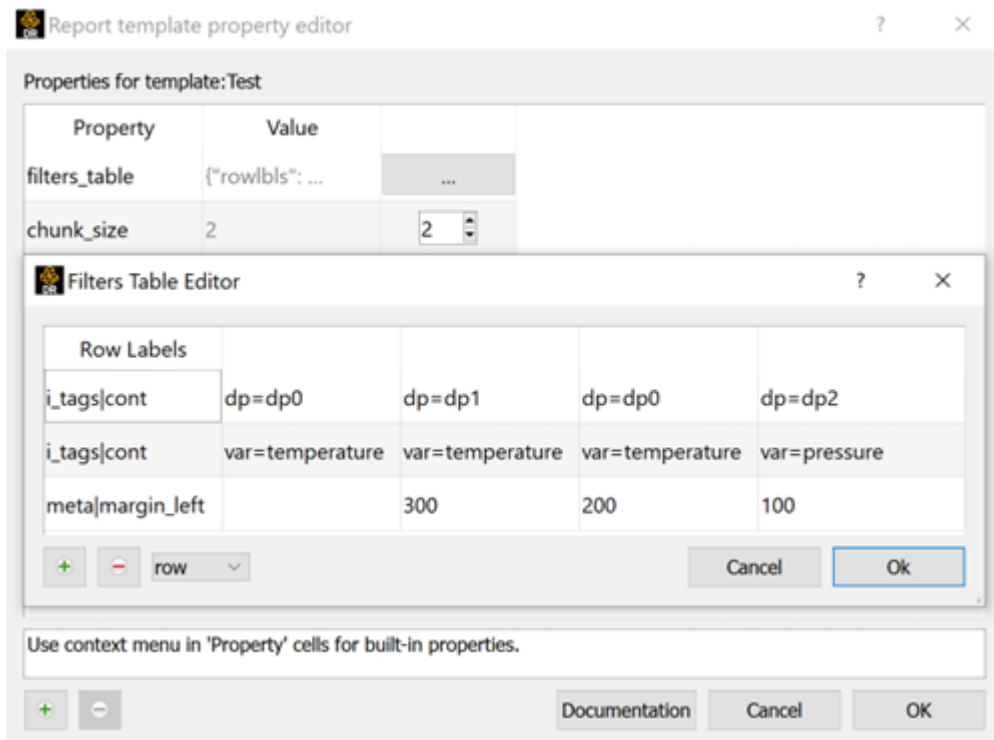
Suppose you want to compare image one and image two side by side, and then image one and image three side by side. To do so, you create a new items comparison generator template and enter these values into the filters table:



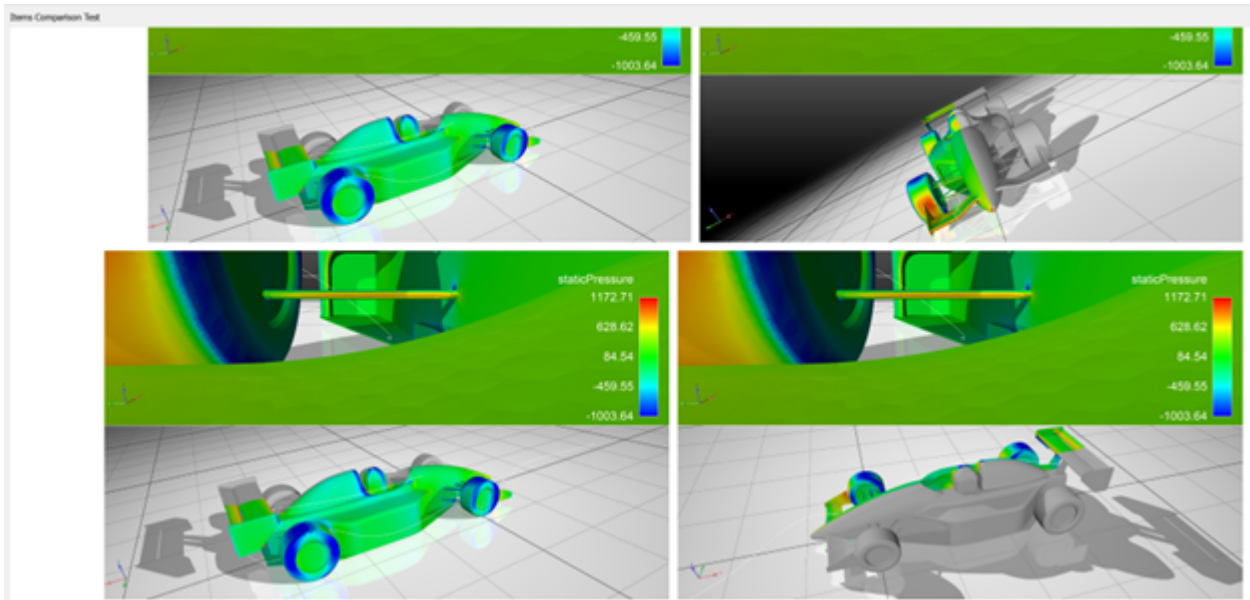
This produces image one, image two, image one, and image three in order. To make the images appear side by side, you add a Columns layout as a child template to the items comparison generator template and set the **Number of columns** field to **2**. If you save everything and view the resulting report, you get the following result:



You can also customize the report by applying meta tags to the filters table. In this example, you set the `chunk_size` to be **2** and enter the following values for `meta|margin_left`:



This produces the following:



Notice how the first two images are pushed 300 pixels to the right and the next two images are pushed 200 pixels to the right. This is because each two images are chunked together into the same comparison group, so they share the same context which can be modified by applying the meta tags. Where there is only one unique meta tag value supplied in the entire group, the meta tag value is applied. When there are multiple meta tag values, the first meta tag value within the group is used. In the example above, in the second group meta|margin_left takes the value of 200 since it comes first.

Besides columns, the items comparison template can also have other templates as its child templates, some examples include the slider template and the panel template. Each comparison group renders each child template independently.

Example Two

The Items Comparison generator can filter any type of items in the database. It lets you easily combine and compare different types of items. For example, you have the following items in the report database:

Data Item List, Input to Generator			
Image A Pressure=1.0 Design=Base	Image B Pressure=2.0 Design=DP1	Image C Pressure=2.0 Design=DP3	Image D Pressure=1.4 Design=DP4
Table E Design=Base	Table F Design=DP1	Table G Design=DP3	Table H Design=DP4

Report Item List

[Back to query](#)

9 item(s) found

10 entries per page Search:

Name	Type	Date	Source	Tags
_Logo_Image_	image	2024-06-20 10:57 am	Session View	stateset=80382f76-229c-11ef-a8c1-d7d02973da4f
Image.D	image	2024-06-20 10:57 am	image_source	Pressure=1.4, Design=DP4 ctmpl=7acf9bee-2f15-11ef-a2c6-cc483a666e49 stateset=80382f76-229c-11ef-a8c1-d7d02973da4f
Image.C	image	2024-06-20 10:58 am	image_source	Pressure=2.0, Design=DP3 ctmpl=7f610cc5-2f15-11ef-937e-cc483a666e49 stateset=80382f76-229c-11ef-a8c1-d7d02973da4f
Image.B	image	2024-06-20 10:58 am	image_source	Pressure=2.0, Design=DP1 ctmpl=83a4fa2b-2f15-11ef-4bb9-cc483a666e49 stateset=80382f76-229c-11ef-a8c1-d7d02973da4f
Image.A	image	2024-06-20 10:58 am	image_source	Pressure=1.0, Design=Base ctmpl=87efc402-2f15-11ef-8dfe-cc483a666e49 stateset=80382f76-229c-11ef-a8c1-d7d02973da4f
Table.E	table	2024-06-20 11:03 am	test_table_1.csv	Design=Base
Table.F	table	2024-06-20 11:07 am	test_table_2.csv	Design=DP1
Table.G	table	2024-06-20 11:07 am	test_table_3.csv	Design=DP3
Table.H	table	2024-06-20 11:07 am	test_table_4.csv	Design=DP4

Each design point (Base, DP1, DP3, DP4) has an image and a table with a specific parameter, so you can compare each design point easily with the Items Comparison generator.

For example, to compare the base design point with DP1, and DP1 with DP4, enter the following in the filters table:

Filters Table Editor

Row Labels								
i_tags cont	Pressure=1.0	Pressure=2.0			Pressure=2.0	Pressure=1.4		
i_tags cont	Design=Base	Design=DP1	Design=Base	Design=DP1	Design=DP1	Design=DP4	Design=DP1	Design=DP4
i_type eq	image	image	table	table	image	image	table	table
meta title	Test				Optimal			

row

Set chunk_size to 4, so that each comparison group has two images and two tables as specified above.

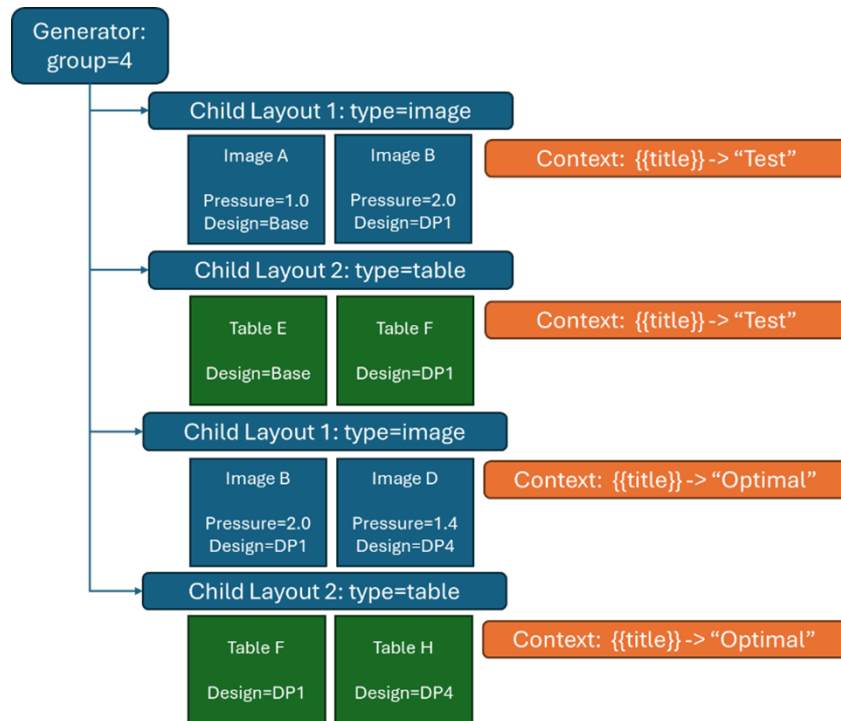
Report template property editor

Properties for template: Comparison Base

Property	Value
filters_table	{"rowlbls": ...
chunk_size	4

Use context menu in 'Property' cells for built-in properties.

These parameters return the items in order and in groups of four. Then for the child layout template, use a Columns layout with two layouts so two consecutive items are rendered side by side in each comparison group. The overall template structure looks like the following:



In the Template Editor, the template structure is:

Items Comparison Base

Images Column Layout

Table Column Layout

Note how there is two column layouts for the items comparison base, the two child templates are rendered once for each comparison group. In **Images Column Layout**, you filter for image type data items only:

Template type: Column layout

Filtering

Filter: `A|i_type|cont|image;`

Edit item filter... Filter mode Incoming items

In **Table Column Layout**, you filter for table type data items only:

Template type: Column layout

Filtering

Filter: A|i_type|cont|image;

Edit item filter...

Filter mode Incoming items

You set the number of columns to **2** in both child templates:

Common layout options

Number of columns 2

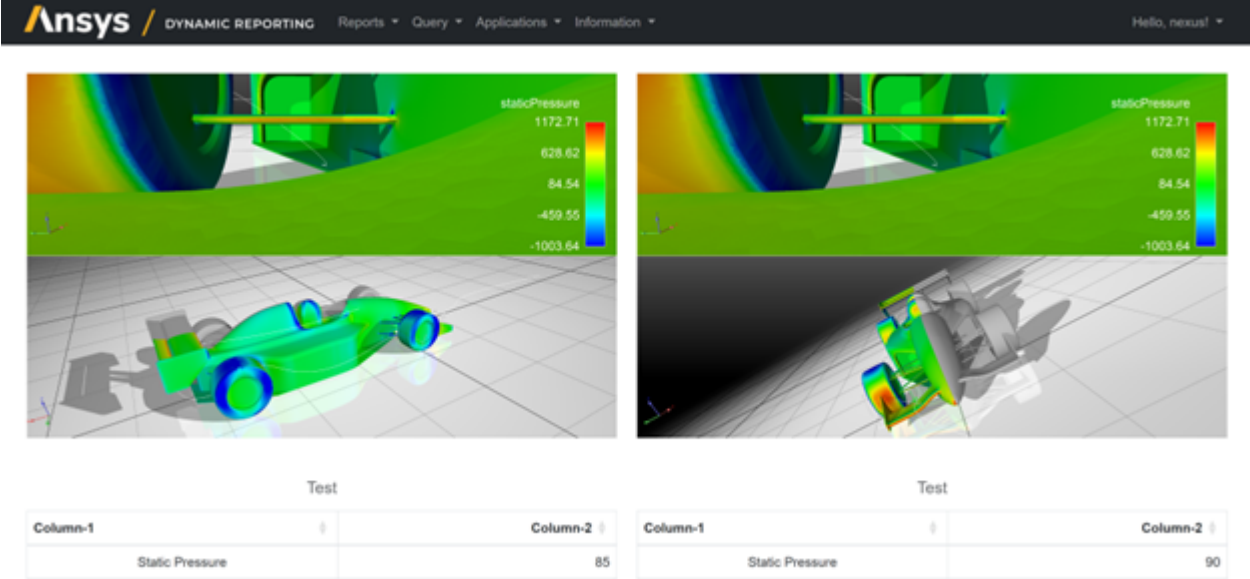
Column weights 1.0, 1.0

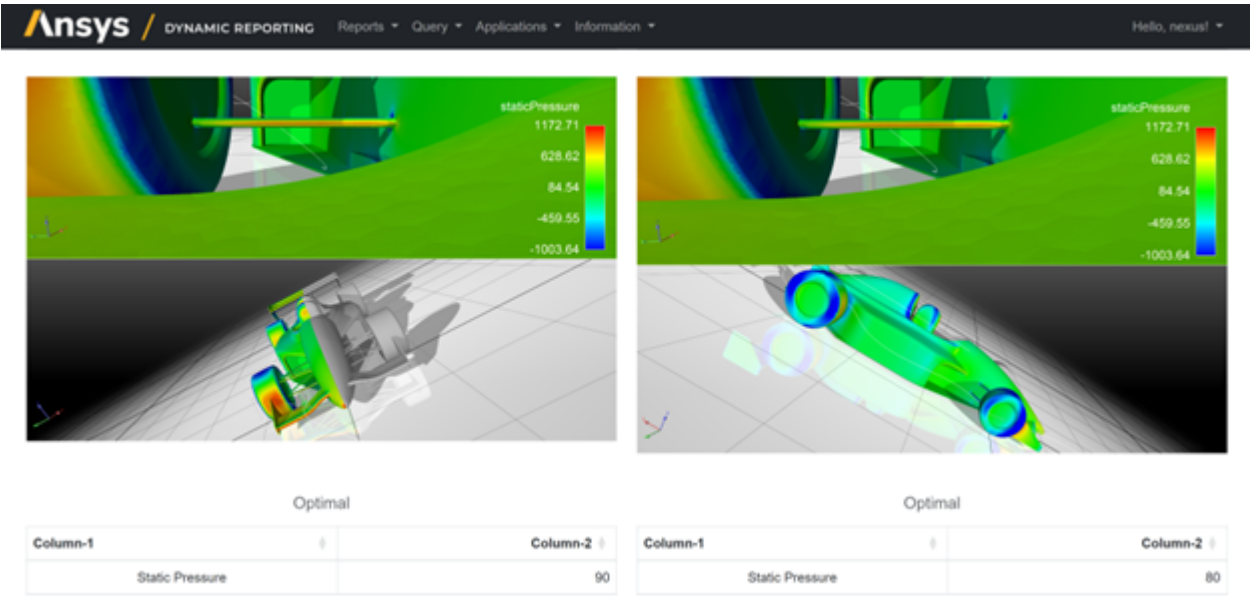
☐ Transpose columns/rows

Custom HTML header...

Skip Never

In the resulting report, comparison across the specified design points is rendered accordingly:





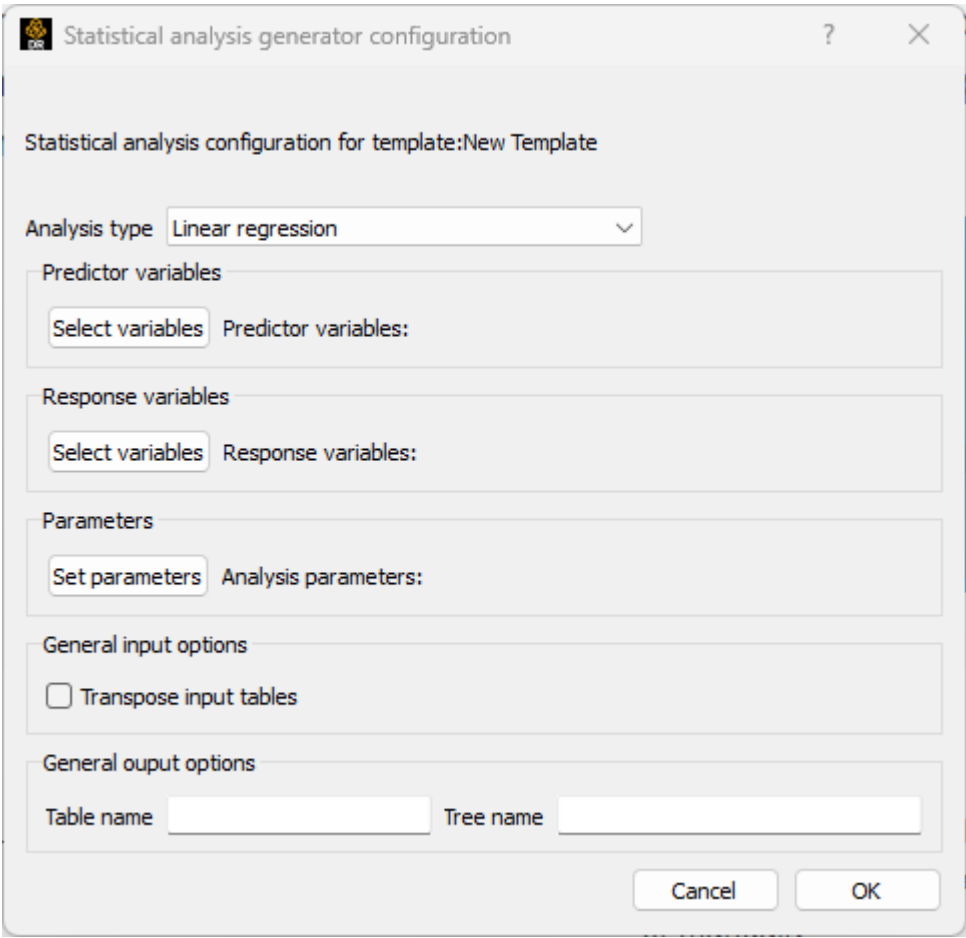
Statistical Analysis

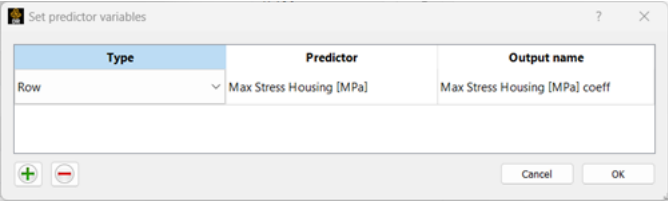
The Statistical Analysis generator allows you to perform statistical analysis on an input data table. In 2025 R1, only linear regression is supported.

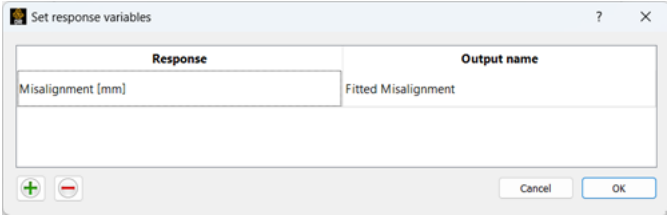
The generator uses the numerical values in a table as input. One or more rows can be used as the predictors (input parameter). For each response variable (output parameter), the generator creates a new set of values corresponding to the fitted value and a tree with the linear regression parameters (R^2 , intercept, coefficient) and the corresponding standard errors.

Configuration Options

In the **Template properties** pane, clicking **Configure analysis parameters** opens the configuration options.



Option	Description
Analysis type	Sets the type of statistical analysis used. Currently, only linear regression is supported.
Predictor variables	Opens a dialog to set the details of the predictor variables. A predictor variable is a variable used as input for the linear regression calculation.  Enter the following: • Type: Currently, the only supported type is <i>Row</i>. This means that only a row from the input table can be used as predictor variable. The row must contain only numerical values. • Predictor: Specifies which table row to use. • Output name: Sets the name used for the coefficient related to this predictor.

Option	Description
	You can set multiple predictor variables. Use the add () and remove () buttons to adjust the list.
Re- sponse vari- ables	Opens a dialog to set the details of the response variables. For each row, a new linear regression model is calculated.  Enter the following: • Response: Specifies the name or index of the row to use. • Output name: Sets the name of the row that displays the fitted values. You can set multiple response variables. Use the add () and remove () buttons to adjust the list.
Para- meters	Adds extra parameters for the statistical analysis. Currently, parameters are not supported for linear regression.
Trans- pose in- put tables	When selected, the input table is transposed before the generator is executed. The generator assumes that the inputs are in row formats.
Table name	Sets a name for the table and the tree that will be created by the generator. The new table will contain the initial rows, plus an extra row for each response entry, with the fitted value. This can be used to create a plot that contains the original data and the newly fitted line, to better display the linear regression result. As only 2D plots are currently supported, this visualization is valuable for linear regression models with only one predictor. Multiple predictors make for a harder 2D visualization. The generator also creates a tree for each response variable with the values of the linear regression coefficients and their standard deviation (truncated according to the laws of the propagation of error). An example can be seen here: 
Tree name	

Example

The following example shows a table that collects the results from a set of a few hundred design points. Some columns contain design point input parameters, some contain output values.

run #	LA [mm]	RA [mm]	RB [mm]	LB [mm]	Material	Displ. Ymin [mm]	Displ. Ymax [mm]	Misalignment [mm]	Ma
0.000000e+00	1.000000e+01	1.000000e+01	1.500000e+01	5.000000e+01	1.000000e+00	8.915470e-02	6.960045e-02	1.587552e-01	
1.000000e+00	1.000000e+01	1.000000e+01	1.500000e+01	6.750000e+01	1.000000e+00	8.413366e-02	5.072762e-02	1.348613e-01	
2.000000e+00	1.000000e+01	1.000000e+01	1.500000e+01	8.500000e+01	1.000000e+00	8.004975e-02	3.575963e-02	1.158094e-01	
3.000000e+00	1.000000e+01	1.000000e+01	2.300000e+01	5.000000e+01	1.000000e+00	8.830745e-02	6.745224e-02	1.557597e-01	
4.000000e+00	1.000000e+01	1.000000e+01	2.300000e+01	6.750000e+01	1.000000e+00	8.301588e-02	4.699215e-02	1.300080e-01	
5.000000e+00	1.000000e+01	1.000000e+01	2.300000e+01	8.500000e+01	1.000000e+00	7.874621e-02	3.213080e-02	1.108770e-01	
6.000000e+00	1.000000e+01	1.000000e+01	3.000000e+01	5.000000e+01	1.000000e+00	8.828459e-02	6.729554e-02	1.555801e-01	
7.000000e+00	1.000000e+01	1.000000e+01	3.000000e+01	6.750000e+01	1.000000e+00	8.295453e-02	4.659285e-02	1.295474e-01	
8.000000e+00	1.000000e+01	1.000000e+01	3.000000e+01	8.500000e+01	1.000000e+00	7.864368e-02	3.172297e-02	1.103667e-01	

It is hard to understand the correlation between different parameters such large table of values. One thing that you can do is plot the table as 2D plots. This can provide a visual feeling for a correlation.

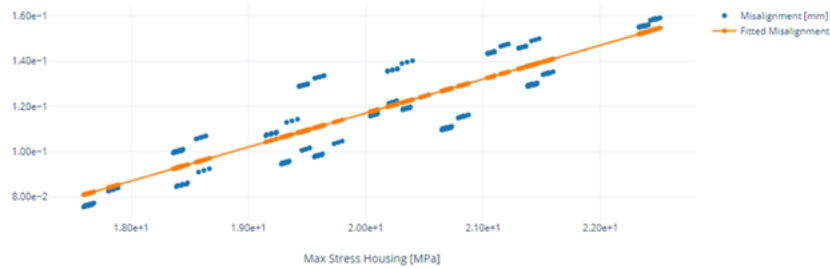


Here it looks evident that there is some correlation between **Misalignment** and **Max Stress Housing**. You can now use the Statistical Analysis generator to quantify the correlation.

In the configuration:

1. Select the **Transpose input tables** option, to make sure the data is stored in rows.
2. In the **Predictor variables** table, set the **Max Stress Housing [MPa]** variable. Make sure the text corresponds exactly to the row label.
3. In the **Response variables** table, enter the **Misalignment [mm]** row. Set a name for the output name of the response.

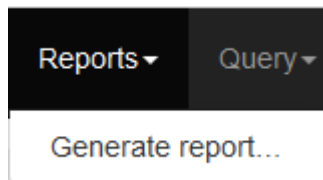
Running the generator will return the original table with an extra row, with the row label corresponding to what you entered in the response output name. You can now display the same plot as before, with an extra Y axis row corresponding to the new data entry. The result is the fitted line overlaid on top of the original plot:



Generating Reports Using the Web Interface

Once you have created or edited report templates, you can use them to generate a report.

1. [Open the web interface \(p. 23\).](#)
2. From the menu bar, select **Reports** → **Generate report**.



The **Report Generation** page is displayed, containing the list of all report templates available in the database.

Item Search Filters

Field

and

Item name

Comparison

Contains any of

Value(s)

+

-

+

-

Display Options

Select a report template:

Show

10

 entries

Search:

Name	Date	Tags
Reduction Generator	March 9, 2017, 4:49 p.m.	
Session=0491db68-2f28-11ef-b7d5-87c54b3f626d	June 20, 2024, 1:12 p.m.	stateset=0491db68-2f28-11ef-b7d5-87c54b3f626d
Tag Properties Example	March 8, 2017, 5:08 p.m.	
Value Filter Example	March 9, 2017, 4:50 p.m.	
Merge Generator	March 12, 2017, 12:20 p.m.	
Merge reduce example	March 10, 2017, 1:06 p.m.	
Lift Analysis	Oct. 22, 2021, 9:27 a.m.	
RC Sort Filter Example	March 9, 2017, 1:11 p.m.	
RC Filter Generator	March 9, 2017, 3:04 p.m.	

- Item Search Filters

	Field	Comparison	Value(s)	
and	Item types	Include	File HTML Image Movie None	+ - + -

- Release 2025 R1 - © ANSYS, Inc. All rights reserved. - Contains proprietary and confidential information of ANSYS, Inc. and its subsidiaries and affiliates.

Navigation menus:

Show

Hide

Display DPI:

96

Page width (in):

20.00

6. Click **Generate Report**.

The generated report results are displayed in the web interface.

Configuring a Remote Session

Configuring a remote session relies on the resources and requirements of a specific site. Ansys Dynamic Reporting provides some core functionality that simplifies the delivery of VNC-based rendering solutions. These features include:

- An integrated VNC client run in a web-browser. This allows for a true zero-footprint install solution that only relies on Ansys Dynamic Reporting and VNC server technologies installed on the remote systems. Ansys Dynamic Reporting generates a web page with an embedded URL containing a private cookie.

Note:

Care should be taken to ensure that the transmission of the web page be protected by encryption. For the system to be secure, it is strongly recommended that SSL be used to access the NGINX server for all browser connections. The ADR server honors this encryption.

- Resource tracking and simple configuration. All users of the remote sessions allocate the sessions using web pages generated by the ADR server. ADR can track session allocations and exert control over the remote rendering resources via a common database.
- Proxy authentication and authorization. The remote session system exploits the [authentication and authorization \(p. 10\)](#) system in the ADR server. From that information, it generates a private cookie for the allocated session and serves it up via REST from the WebSocket server when the JavaScript VNC server is realized.
- WebSocket interface for data tunneling. The system implements a WebSocket-based transport system that utilizes existing socket system used by the NGINX instance. It also supports SSL for encrypted connections.

There are two types of remote sessions.

1. Built-in local EnSight and EnVision sessions. These are associated with file data item filename extensions. For example, an uploaded .evsn file will automatically be associated with EnVision. These sessions will run on the same system as the ADR server and require no specialized configuration.
2. Externally specified configurations. These are specified in the `external_services.json` file that is placed in the database directory (the same directory as the `view_report.nexdb` file).

Note:

All of the files used in examples here can be found in the Ansys Dynamic Reporting distribution in:

v251/CEI/nexus251/django/utils/remote/config

Related topics:

[Creating the Configuration File](#)

[Creating the Session Lifecycle Python Script](#)

Creating the Configuration File

The first step in the configuration is to consider what types of remote rendering solutions you want to provide access to.

This example focuses on providing access to a software rendering VNC server-based configuration. Included in the distribution is a simple example that uses a start/stop script running on remote nodes.

The example uses an instance of the `x11vnc` application to provide VNC access to a hardware accelerated X11 server. See the `start_x11_server.sh`, `stop_x11_server.sh`, and `x11vnc_desktop_session.py` scripts for an example of how this type of configuration can be realized.

To use the remote rendering system, you must create the `external_services.json` file, a configuration file in JSON format, and place it in the database directory.

The file describes a Python dictionary that contains descriptions of all of the available session types. An example might be:

```
{
  "Local Software Session": {"script": "desktop_session.py", "count": 2,
    "options": {"sizes": ["12080x1024", "1600x1200"], "display_base": 100}},
  "Cluster Node Session": {"script": "x11vnc_desktop_session.py", "count": 1,
    "options": {"sizes": ["1600x1200"], "group": "ClusterRendering", "host": "n8"}}
}
```

This configuration file defines a single Python dictionary with multiple keys (for example, *Local Software Session*). Each key is the name of a type of session. The value of each session key is a Python dictionary. There are three required keys in each session dictionary. These include:

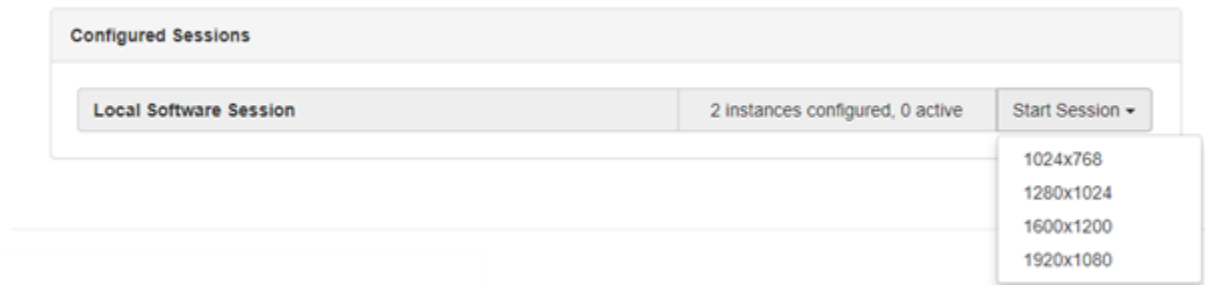
- **script**: the name of a Python script file responsible for creating an object that can start and stop a session.
- **count**: the maximum number of simultaneous instances of this session type.
- **options**: a dictionary of optional data items that can be used to customize the web interface and any additional data items that can be used by the session instance.

Each type of session must provide the name of a Python script file. This file must create an instance of a Python object (a subclass of `RemoteServiceScript`) that is responsible for starting and ending a session. There can be multiple instances of each session type active at the same time. This is limited with the **count** key which specifies the maximum number of simultaneously allowed sessions. When a session of a given type is started, an index number running from zero to this number minus one will be passed to the Python code. The code should use this number to ensure that each session uses non-conflicting resources (for example, port numbers, node allocations, graphics cards, and so on).

Additional pieces of data can be passed via the `options` object. The option dictionary is passed to the `RemoteServiceScript` creation method as `common_options` and by default is stored as the `_common_options` attribute on the `RemoteServiceScript` subclass instance. You can add any customization options to this dictionary.

The ADR system uses two keys to configure the HTML page it presents for selecting sessions. The first is `sizes`. This should be a list of strings that are displayed in the pull-down menu used to start the session. It defaults to a list of common window sizes.

Remote sessions



The value selected for `sizes` is passed in the `options` dictionary to the `RemoteServiceScript.start()` method.

The ADR system will also look for the `group` key in the `options` dictionary. In the example above, this key has been set to the value `ClusterRendering` for instances of the `Cluster Node Session` session type. If the `group` key is set to the name of a ADR administration group, the session type will only be made available to users who are members of that specific group. In the example, if you are not a member of the `ClusterRendering` group, you will not be allowed to create `Cluster Node Session` sessions.

Creating the Session Lifecycle Python Script

The script file is responsible for actually creating and deleting a session. It takes the form of a Python script that creates instances of the core session lifecycle class.

The following is an example `desktop_session.py` file (which should be located in the same directory as the configuration JSON formatted file):

```
# Simple example remote service. The loader will call a method to
# realize the services associated with a specific service name.
# All services must subclass RemoteServiceScript.

from nexus_remote_service import RemoteServiceScript
import subprocess
import time

# Launch a vnc server (tightvnc) and run an optional command in the token json map file

class SimpleVNC(RemoteServiceScript):

    def __init__(self, *args, **kwargs):
        super(SimpleVNC, self).__init__(*args, **kwargs)

    def display_base(self):
```

```

        return self._common_options.get("display_base", 100)

    def start(self, token, timestamp=time.time(), options=dict()):
        username = options.get('username', '')
        size = options.get('size', '1600x1200')
        cmd = ["vncserver", "-depth", "24", "-geometry", size, ":"+str(self.display_base()+self._index)]
        try:
            print("SimpleVNC start: {} {}".format(self, cmd))
            code = subprocess.call(cmd)
            if code != 0:
                return False
        except:
            return False
        self.set_host_port("localhost", 5900+self._index+self.display_base())
        return super(SimpleVNC, self).start(token, timestamp=timestamp, options=options)

    def stop(self):
        cmd = ["vncserver", "-kill", ":"+str(self.display_base()+self._index)]
        try:
            print("SimpleVNC stop: {} {}".format(self, cmd))
            code = subprocess.call(cmd)
        except:
            pass
        return super(SimpleVNC, self).stop()

    def realize_service(name="", count=1, index=0, common_options=dict()):
        print("Realizing SimpleVNC service: {} {} {} {}".format(name, count, index, common_options))
        return SimpleVNC(name, count=count, index=index, common_options=common_options)

```

Realize Service Startup Function

The ADR core will import this file and call the function `realize_service()` with information gleaned from the JSON configuration file. The name will be the session type name (the top level key in the JSON file). The count and common_options keyword values will also have come directly from the JSON file count and option keys respectively. The `realize_service()` function will be called once for each of the instances that may be legal. For count=3, it will be called with index=0, index=1 and index=2. The function must respond with an instance of a subclass of the `RemoteServiceScript` class. This subclass must at least implement the `start()` and `stop()` methods. All other methods are optional.

Remote Service Script Start Method

The `RemoteServiceScript.start()` method is called when a session has been authorized, authenticated and is ready to begin (for example, an upstream entity has requested data over a WebSocket). In the example, the `start()` method creates an instance of `vncserver` using the size option selected in the ADR web page. It also selects a port number using a base port number specified in the JSON configuration file and the index of this session instance. In a production environment, it can use the authenticated/authorized username from ADR to select a user target for the session. The `start()` method must call `set_host_port()` to set the hostname and port over which the processes it created (`vncserver` in this example) will be accepting VNC connections. The ADR system will automatically tunnel this port over the WebSocket interface and apply any selected SSL encryption to the stream. If the operation fails, the function must return `False`.

Remote Service Script Stop Method

The `RemoteServiceScript.stop()` method is called after all active connections to a specific session instance have closed their connections (for example, the browser or the specific tab is closed). It can

also timeout. It will only be called if `start()` had previously been called. In our example, it simply calls `vncserver -kill` with the proper port specification for the instance.

Access Control Lists

Access Control Lists, or ACLs, are an extra layer of authentication in the ADR server. ACLs allow you to organize users into groups and assign specific permissions to each group. Access to an object through a user-group is controlled by these predefined list of permissions.

ACLs allows data objects to be grouped into as many categories as you want, with each category linked to user-groups through a variety of permissions.

The access on an object through a user-group is controlled by multiple permissions. With the help of these permissions, data privacy can be consistently maintained across objects that users upload in the database.

Only data items are supported for now and support for other objects (templates, data sets, and so on) will be rolled out incrementally in future updates.

Related topics:

[Understanding Access Control Lists](#)

[Enabling Access Control Lists](#)

[Creating Access Control Lists in Ansys Dynamic Reporting](#)

Understanding Access Control Lists

An Access Control List (ACL) is an extra authentication back-end on the ADR server. It has a predefined set of permissions that can be reused across user-groups and object categories. These permissions and their association with object categories are stored in the database, making them persistent. ACLs are backwards compatible, older versions of external tools like template editor and EnSight can still communicate without changing anything.

Users/Groups

- A standard user in Ansys Dynamic Reporting can be assigned to a particular group, or more than one group.
- A group can also contain multiple users. (A many-to-many relationship between user and group)
- A user may or may not belong to a group.
- You must add users to the appropriate groups depending on their roles.
- In order to make use of ACLs, a user **must** belong to at least one group.
- The ADR server includes a default *nexus* group with a default *nexus* user. This user is a superuser (admin).

Superuser

- A superuser or admin is a user who has uncontrolled and unrestricted access to almost every operation and object in Ansys Dynamic Reporting, regardless of which group he/she belongs to. Objects available to a superuser are unfiltered.
- Almost any operation that is requested by a superuser will not undergo any permission checks through ACLs. So, create superusers carefully.
- Unlike users, a superuser only sometimes may need to belong to a group in order to make use of ACLs and categories, depending upon the operation.

Categories

- Categories, in addition to being a way of grouping similar kinds of data objects, are also a way of defining access of a user-group over a set of objects.
- The permissions of a user-group over an object category applies to all objects assigned that specific category. For example, if a user X belongs to group A and group A has *change* permissions on an item category C, then the user has *change* permissions (can edit all items) over all objects that are assigned the category C.
- For backwards compatibility, all existing items (that have no categories assigned) from old databases will be visible to any user without restrictions. When filtering occurs, a user is shown all items they have permissions on and also all items that do not have any categories assigned (do not have any permissions assigned through the categories).

Note:

A any point, a user can only perform operations with/on categories that the user owns, (has *own* permissions on through any of the groups the user is a member of). The only exception to this is the superuser who has unrestricted access and can perform any operation on all available objects and categories.

Permissions

- Access of user-groups over objects are controlled through several predefined permissions.
- Even though users may not belong to a group at all, to make use of the ACL, a user **must** belong to at least one group. This is because users cannot have permissions of their own.
- There are four basic permissions:
 - **add**
 - **change**
 - **view**
 - **delete**
- There is one super permission, **own**, which overrides all of the basic permissions.

- A group can be assigned one or multiple permissions. Similarly, one permission can be assigned to one or multiple groups. (A many-to-many relationship between group and permissions)
- If a user has *change* permissions, they are automatically assumed to have *view* permissions as well.
- There are two types of permissions, **per-category** and **global**.

Per-category Permissions

- A per-category permission is permission of a group tied to one specific object category. For example: with an item-category, a permission assigned for a group G to a particular category A is specific to that category (and inherently, the items assigned that category) and is called a per-category permission.
- An object-category can have multiple groups/users with multiple permissions assigned on itself. Example: item-category A can be owned by group A and group B, only viewed by group C, only changed by group D, only changed or deleted by group E, and so on. It is very flexible.

Global Permissions

- Global permissions exist on all objects of a class/type. These are the permissions available through the admin interface while creating/editing a group. This will override any per-category permissions. Example: If the *Can delete item category* global permission is assigned to a group G through the admin interface, group G (and in turn any user belonging to G) will have *delete* permissions over all item categories (and in turn all items assigned any category).
- A global permission always takes priority over per-category permissions.
- Even though the implementation exists, per-category *add* permissions do not make sense because there will not be any category to check the add permission on, unless you add it. Use global *add* permissions for these cases.
- A user's group must be assigned the global *Can add item* and *Can add item category* permissions to create an item and an item-category respectively.

Usage

- When you create a category, it is saved in the database along with the permissions and the user groups that were specified for the category.
- A relation is created in the database between that category and the list of groups through the permissions that were specified, respectively.
- The available category may then be assigned to any object: in our case, items.
- Whenever you deal with a category, it must be owned by at least one of your groups in order for you to edit, delete or assign it to an object.
- Once an object is assigned a category, the permissions specified for the groups initially, will apply to the object as well.
- Any access to that object must go through a layer of permission checks which check if any of the user's groups have the required permissions on at least one of the categories assigned to that object.

If not, access to that object will be denied. This check makes use of the relation that was stored in the database initially.

- The list of objects that a user has permissions on is determined by the categories that the user has permissions on, through the various groups the user belongs to. For example, when querying for items to delete in Ansys Dynamic Reporting, only those items with categories on which the user's group(s) has *delete* permissions, are returned.
- At any point, if a user is not authenticated/logged in, (is *Anonymous*), and whatever object they are trying to access has no owner-group assigned to any of its categories (the object has no owner), the anonymous user will be given access to it. Objects with categories that have owners need authentication and membership in the corresponding group that owns any of the categories.

Enabling Access Control Lists

Since Access Control Lists are an additional layer of authentication and are also stored in the database, using them has performance trade-offs, so they are turned off (set to 0) by default.

To enable them, you must set the environment variable `CEI_NEXUS_ENABLE_ACLS` to 1. For example:

```
export CEI_NEXUS_ENABLE_ACLS=1
```

It can also be enabled using the ADR Launcher [command line](#) (p. 304) and [user interface](#) (p. 13).

Creating Access Control Lists in Ansys Dynamic Reporting

You can create Access Control lists in the Ansys Dynamic Reporting web interface using either of the following options:

- [The admin interface](#) (p. 258)
- [The user interface](#) (p. 267)

Creating ACLs in the Admin Interface

To use the admin interface, you must be logged into Ansys Dynamic Reporting as a superuser.

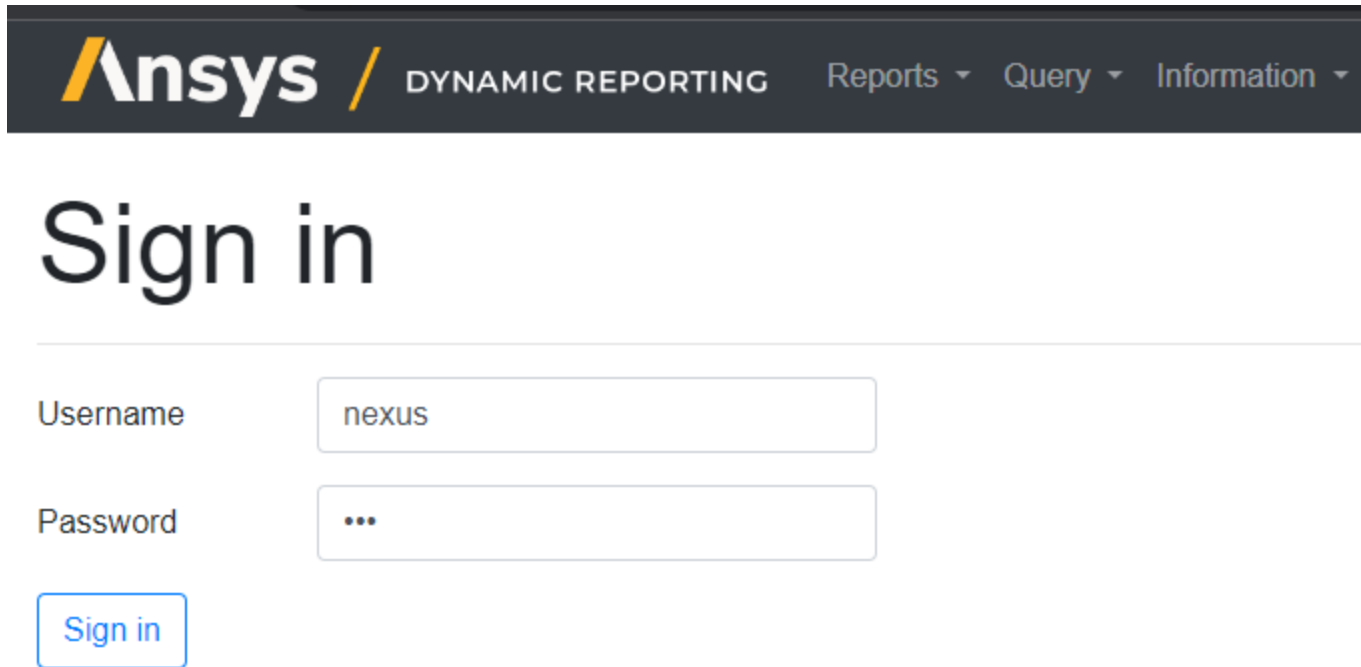
Related topics:

- [Accessing the Admin Interface](#)
- [Adding a Group](#)
- [Changing Permissions in a Group](#)
- [Deleting Groups](#)
- [Adding a User](#)
- [Changing a User](#)
- [Deleting Users](#)

Accessing the Admin Interface

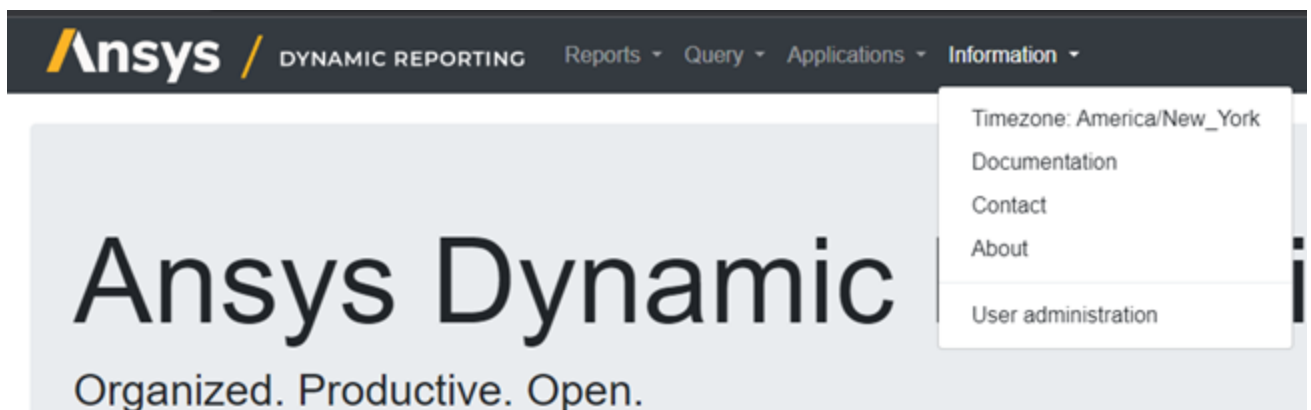
To open the Admin interface:

1. On the menu bar of the Ansys Dynamic Reporting web interface, click **Sign in**.
2. Enter the admin/superuser **Username** and **Password**.



The screenshot shows the top navigation bar of the Ansys Dynamic Reporting web interface. The bar is dark gray with the Ansys logo on the left, followed by 'DYNAMIC REPORTING'. To the right are three dropdown menus: 'Reports', 'Query', and 'Information'. Below the navigation bar is a large 'Sign in' heading. Underneath the heading are two input fields: 'Username' with the text 'nexus' and 'Password' with three dots indicating a masked password. Below these fields is a blue 'Sign in' button.

3. Click **Sign in**.
4. From the menu bar, select **Information** → **User administration**.



The following page opens. The interface may vary a bit, depending on which version of Ansys Dynamic Reporting you are running.

Site Administration

AUTHENTICATION AND AUTHORIZATION		
Groups	+ Add	Change
Users	+ Add	Change

Adding a Group

To add a group:

- 1. On the **Site Administration** page, next to **Groups**, click **Add**.
- 2. In the **Name** field, enter the new group name.

Add group

Name:

Newgroup

Permissions:

Available permissions ?

Q

Filter

data | dataset | Can add dataset

data | dataset | Can delete dataset

data | item | Can add item

data | item category | Can add item category

data | item category | Can change item category

data | item category | Can delete item category

data | item category | Owns the item category

data | item category | Can view item category

data | session | Can add session

data | session | Can delete session

reports | template | Can add template

- 3. Move one or more permissions from the **Available permissions** list into the **Chosen permissions** list using the arrow buttons or by double-clicking a permission.

Name:

Permissions:

Available permissions ⓘ

Filter

- data | dataset | Can delete dataset
- data | item | Can add item
- data | item category | Can add item category
- data | item category | Can change item category
- data | item category | Can delete item category
- data | item category | Owns the item category
- data | item category | Can view item category
- data | session | Can delete session
- reports | template | Can add template
- reports | template | Can change template

Choose all ⓘ

Chosen permissions ⓘ

Filter

- data | dataset | Can add dataset
- data | session | Can add session

Remove all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

- Click **Save** if you are done, **Save and continue editing** if you want to continue editing, or **Save and add another** if you want to save and then add another group.

Changing Permissions in a Group

To change permissions in a group:

- On the **Site Administration** page, next to **Groups**, click **Change**.
- From the list, click a group name.

Select group to change

Q Search

Action: Go 0 of 2 selected

☐	GROUP
☐	Mygroup
☐	cei

- Add and remove permissions as appropriate.

Change group

Mygroup

Name:

Permissions:

Available permissions ⓘ

Q Filter

- data | dataset | Can delete dataset
- data | item | Can add item
- data | item category | Can add item category
- data | item category | Can change item category
- data | item category | Can delete item category
- data | item category | Owns the item category
- data | item category | Can view item category
- data | session | Can delete session
- reports | template | Can add template
- reports | template | Can change template

Choose all ⓘ

Chosen permissions ⓘ

Q Filter

- data | dataset | Can add dataset
- data | session | Can add session

Remove all ⓘ

Hold down "Control", or "Command" on a Mac, to select more than one.

- Click **Save** if you are done, **Save and continue editing** if you want to continue editing, or **Save and add another** if you want to save and then add another group.

Deleting Groups

There are multiple ways to delete groups. To delete a single group.

- On the **Site Administration** page, next to **Groups**, click **Change**.
- From the list, click a group name.

Select group to change

Q Search

Action: Go 0 of 2 selected

☐	GROUP
☐	Mygroup
☐	cei

- On the **Change group** page, click **Delete**.

4. To confirm the change, click **Yes, I'm sure**.

To delete multiple groups:

1. On the **Site Administration** page, next to **Groups**, click **Change**.
2. Select the check boxes for the groups to delete.
3. From the **Action** list, select **Delete selected groups**.

Select group to change

Search

Action: **Delete selected groups** Go 2 of 2 selected

☒	GROUP
☒	Mygroup
☒	cei

4. Click **Go**.
5. To confirm the change, click **Yes, I'm sure**.

Adding a User

To add a user:

1. On the **Site Administration** page, next to **Users**, click **Add**.
2. Enter a username and password for the user.

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:	testuser
	Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Password:	...
Password confirmation:	...
	Enter the same password as before, for verification.

3. Click **Save** if you are done, **Save and continue editing** if you want to continue editing, or **Save and add another** if you want to save and then add another user.

Note:

Clicking **Save and continue editing** brings you to the **Change user** page.

Changing a User

To change user information:

1. On the **Site Administration** page, next to **Users**, click **Change**.
2. From the list, click a user name.

Select user to change

Search Search

Action: Go 0 of 3 selected

☐	USERNAME
☐	ensight
☐	nexus
☐	testuser

3. Add or change the user's login and/or personal information as required.

Change user

testuser

Username:
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password: **algorithm:** pbkdf2_sha256 **iterations:** 600000 **salt:** pvJ6Mv***** **hash:** Q0q3+S*****
Raw passwords are not stored, so there is no way to see this user's password, but you can change the password using this form.

Personal info

First name:

Last name:

Email address:

4. To activate/deactivate the user, toggle the **Active** check box.
5. To assign/remove administrative privileges, toggle the **Superuser status** check box.
6. To assign groups to a user, move one or more groups from the **Available groups** list into the **Chosen groups** list using the arrow buttons or by double-clicking a group. You can also remove groups as appropriate.

Permissions

☒ Active
Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Superuser status
Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups ⓘ
Filter
cei
Choose all ⓘ

Chosen groups ⓘ
Filter
Mygroup
Remove all ⓘ

7. Click **Save** if you are done, **Save and continue editing** if you want to continue editing, or **Save and add another** if you want to save and then add another user.

Deleting Users

There are multiple ways to delete users. To delete a single user.

1. On the **Site Administration** page, next to **Users**, click **Change**.
2. From the list, click a user name.

Select user to change

Search

Action: ----- Go 0 of 3 selected

☐	USERNAME
☐	ensight
☐	nexus
☐	testuser

3. On the **Change user** page, click **Delete**.
4. To confirm the change, click **Yes, I'm sure**.

To delete multiple users:

1. On the **Site Administration** page, next to **Users**, click **Change**.
2. Select the check boxes for the users to delete.
3. From the **Action** list, select **Delete selected users**.
4. Click **Go**.
5. To confirm the change, click **Yes, I'm sure**.

Creating ACLs in the User Interface

This interface can be used by both general users and administrators. Users have the ability to add/edit/delete categories and assign/remove per-category permissions to/from user groups in this interface. They can also bulk-assign/remove categories to/from multiple items at once.

To use this interface, you must be logged into Ansys Dynamic Reporting as either a standard user or a superuser.

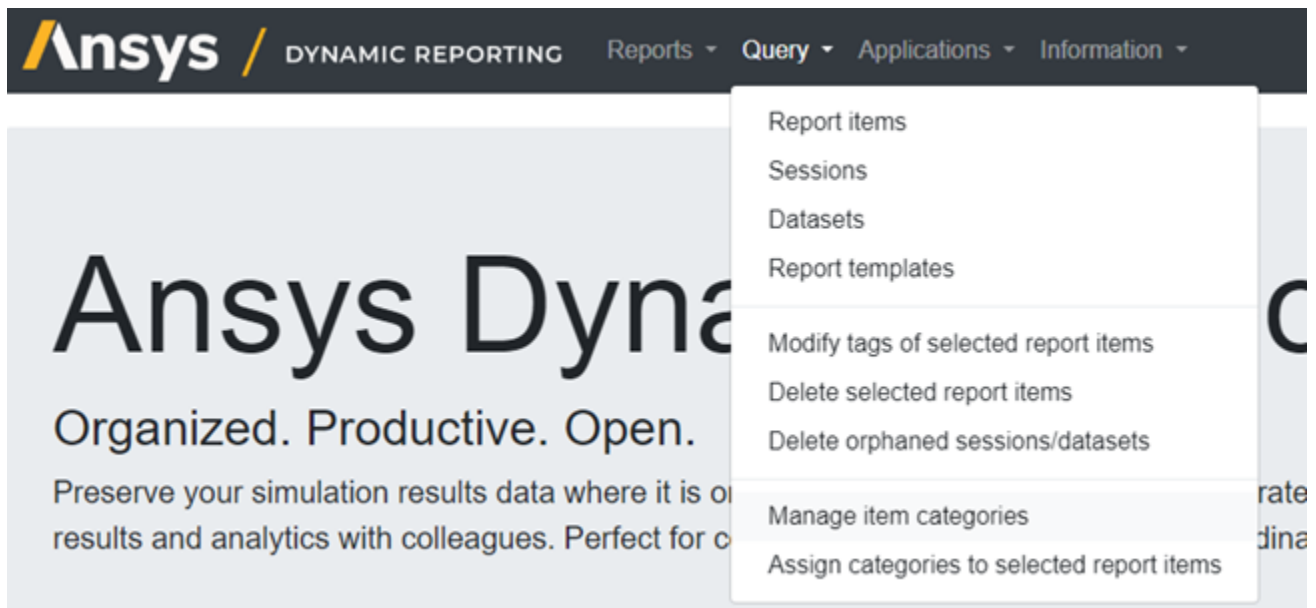
Related topics:

- [Managing Item Categories](#)
- [Assigning and Removing Categories for Multiple Items](#)
- [Assigning Categories During Upload](#)
- [Editing Categories in the Item Detail Page](#)
- [Sharing an Item from the Item Detail Page](#)

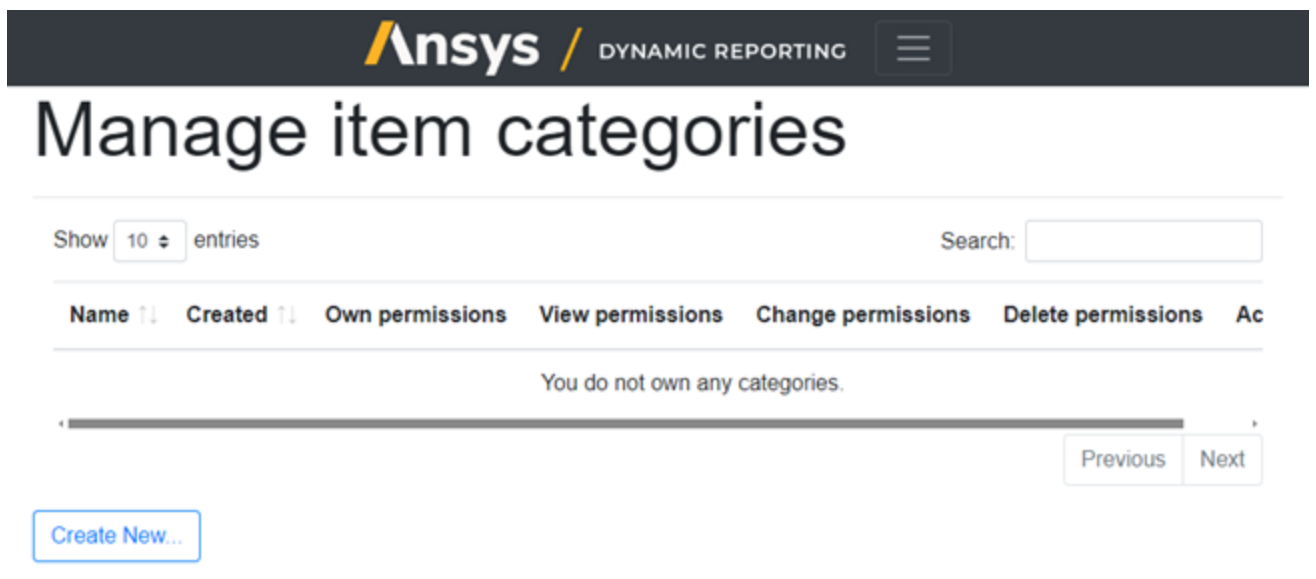
Managing Item Categories

You can create/delete/edit item-categories and also modify permissions on them.

1. From the menu bar, select **Query** → **Manage item categories**.



The **Manage item categories** page opens, where you can create/delete/edit item-categories and also modify permissions on them. You will only see item categories that you have *own* permissions on, through the groups you belong to. The only exception to this is the superuser who has unrestricted access.



2. To create a new item category, click **Create New**.

Note:

You must have the global permission *Can add item category* to create an item category and for the **Create New** button to be displayed. The only exception to this is the superuser who has unrestricted access.

The **Create category** dialog opens.

Create category



Category name

Permissions

Own

cei

View

Nothing selected

Change

Nothing selected

Delete

Nothing selected

Save

3. Enter the **Category name**.

Create category ×

Category name

mycategory

Permissions

Own

cei

View

Nothing selected

Change

Nothing selected

Delete

Nothing selected

Save

- Under **Permissions**, select one or more groups for the applicable permission.

Note:

You must select at least one owner group, otherwise you will lose access to the category since you are no longer the owner (unless you are the superuser).

Create category ×

Category name

mycategory

Permissions

Own

cei

View

Change

Delete

Select All

Deselect All

cei ✓

Mygroup

TestGroup

Save

- Click **Save**.

The new category is added to the list.

Ansys / DYNAMIC REPORTING Reports ▾ Query ▾ Applications ▾ Information ▾ Hello, myadminuser! ▾

Manage item categories

Show 10 entries

Search:

Name	Created	Own permissions	View permissions	Change permissions	Delete permissions	Actions
mycategory	2024-06-14 10:37 am	cei				 

Create New...

Previous 1 Next

6. To edit a category:

Note:

In the permissions list, you may see other groups that have permissions on that category. You cannot perform changes on these groups, since you do not belong to them. They are for viewing only.

- a. Under **Actions** click 
- b. In the **Edit category** dialog, adjust permissions as appropriate.

Edit mycategory

×

Category name

mycategory

Permissions

Own

cei

▼

View

TestGroup

▼

Change

Nothing selected

▼

Delete

Nothing selected

▼

Save

- c. When done, click **Save**.

7. To remove a category

- a. Under **Actions** click .
- b. In the **Confirmation** dialog, click **Continue**.

Confirmation ×

Are you sure you want to proceed?

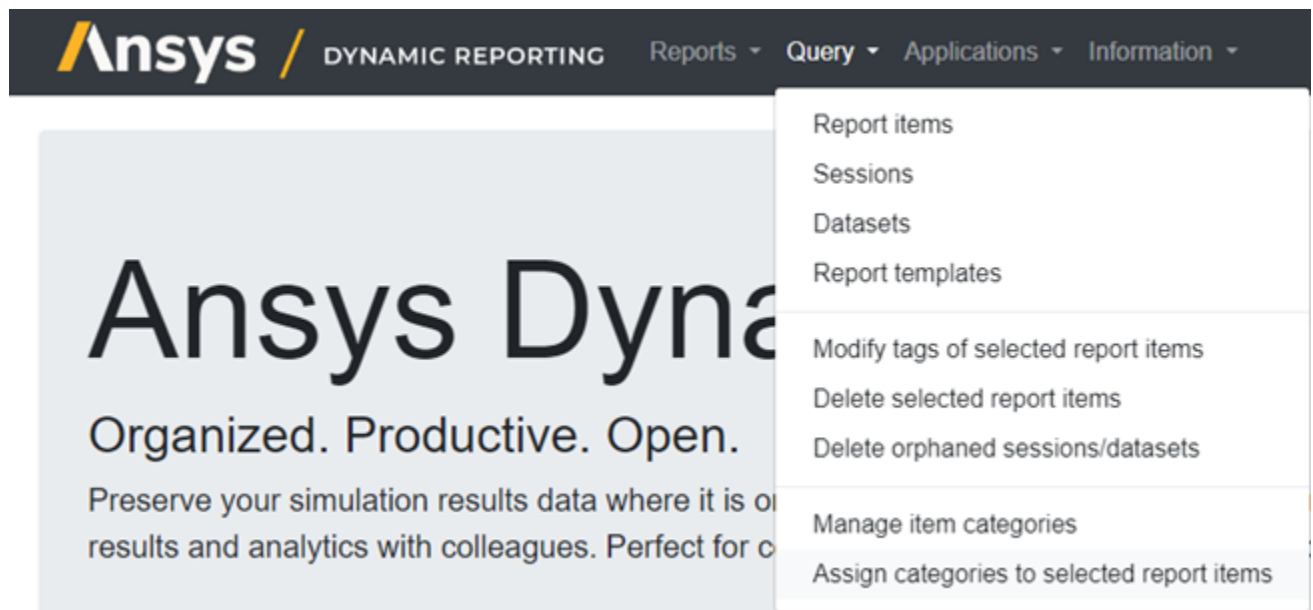
Continue

The category is removed from the list.

Assigning and Removing Categories for Multiple Items

You can assign/remove categories to/from multiple items at once.

1. From the menu bar, select **Query** → **Assign categories to selected report items**.



2. Set the query for appropriate list of items you want to change.

Ansys / DYNAMIC REPORTING

Assign Categories to Report Items:
Search

Search Filters

	Field	Comparison	Value(s)	
and	Item types	Include	MOVIE None Scene String Table	+ - + -

Run Query

3. Click **Run Query**.
- The search results are displayed.

 / DYNAMIC REPORTING 

Select categories to assign

Categories

Nothing selected

Back to query

Reset selection

Assign to selected...

12 item(s) found
0 item(s) selected

Show

10

entries

Search:

☐	Name	Type	Date	Source	Categories
☐	Text Source	string	2016-08-08 9:40 am	text_source	
☐	Text Source	string	2016-10-05 5:26 pm	text_source	
☐	Text Source	string	2016-10-05 5:38 pm	text_source	
☐	Text Source	string	2016-10-05 5:43 pm	text_source	
☐	Text Source	string	2016-10-06 5:08 pm	text_source	

- Next to **Categories**, click the drop-down list and select one or more options.

Anslys / DYNAM

Select categories to

Categories

Anothercategory, Firstcateg ▾

Back to quer

Select All

Deselect All

Anothercategory ✓

Firstcategory ✓

to selected...

Show

10 ▾

entries

☐	Name	↑↓	Type	↑↓	Date
☐	Text Source		string		2016-08-08 9:40 am

Note:

You will only see categories you own, those categories that any of your groups have *own* permissions on. The only exception to this is the superuser who has unrestricted access.

5. Select the check boxes next to the items you want to assign the categories to.


DYNAMIC REPORTING


Select categories to assign

Categories
Anothercategory, Firstcateg ▼

Back to query
Reset selection
Assign to selected...

Show 10 entries

	Name	Type	Date	Source
☒	Text Source	string	2016-08-08 9:40 am	text_source
☒	Text Source	string	2016-10-05 5:26 pm	text_source
☒	Text Source	string	2016-10-05 5:38 pm	text_source

- Click **Assign to selected**.
- In the confirmation dialog, click **Continue**.

Confirmation


Are you sure you want to proceed with assignment of the selected categories?

Continue

The assigned categories are applied to the items.

Ansys / DYNAMIC REPORTING

Select categories to assign

Categories

Anothercategory, Firstcateg ▾

Back to query

Reset selection

Assign to selected...

12 item(s) found
0 item(s) selected

Show 10 ▾ entries

Search:

☐	Name	Type	Date	Source	Categories
☐	Text Source	string	2016-08-08 9:40 am	text_source	Firstcategory, Anothercategory
☐	Text Source	string	2016-10-05 5:26 pm	text_source	Firstcategory, Anothercategory
☐	Text Source	string	2016-10-05 5:38 pm	text_source	Firstcategory, Anothercategory
☐	Text Source	string	2016-10-05 5:43 pm	text_source	
☐	Text Source	string	2016-10-06 5:08 pm	text_source	
☐	Text Source	string	2016-10-12 3:14 pm	text_source	

8. To remove categories, click the **Categories** drop-down list and click **Deselect All**.

Categories

Nothing selected ▾

Back to query

Remove from selected...

Select All

Deselect All

Anothercategory

Firstcategory

Show 10 ▾ entries

Now, your category selection is an empty set. You remove the categories by assigning this empty set to the item's category set. Assigning an empty set of categories will effectively remove all categories the item(s) were previously assigned.

9. Select the check boxes next to the items you want to remove the categories from.

Categories: Nothing selected

Back to query Reset selection Remove from selected...

12 item(s) found
3 item(s) selected

Show 10 entries Search:

Name	Type	Date	Source	Categories
☒ Text Source	string	2016-08-08 9:40 am	text_source	Firstcategory, Anothercategory
☒ Text Source	string	2016-10-05 5:26 pm	text_source	Firstcategory, Anothercategory
☒ Text Source	string	2016-10-05 5:38 pm	text_source	Firstcategory, Anothercategory
☐ Text Source	string	2016-10-05 5:43 pm	text_source	Firstcategory, Anothercategory

10. Click **Remove from selected**.
11. In the confirmation dialog, click **Contin-**

Confirmation

Are you sure you want to proceed with **removal** of all categories from the selected items?

Continue

ue.

The assigned categories are removed from the items.

Ansys / DYNAMIC REPORTING

Select categories to assign

Categories

Nothing selected

Back to query

Reset selection

Remove from selected...

12 item(s) found
0 item(s) selected

Show 10 entries

Search:

☐	Name	Type	Date	Source	Categories
☐	Text Source	string	2016-08-08 9:40 am	text_source	
☐	Text Source	string	2016-10-05 5:26 pm	text_source	
☐	Text Source	string	2016-10-05 5:38 pm	text_source	
☐	Text Source	string	2016-10-05 5:43 pm	text_source	

Assigning Categories During Upload

You can set categories when uploading items.

- 1. From the menu bar, select **Applications** → **Upload files into data items**.

Ansys / DYNAMIC REPORTING

Reports

Query

Applications

Information

Upload files into data items

Media browser

Ansys Dynamic Rep

The **Upload desktop files into data items** page is displayed.

Ansys / DYNAMIC REPORTING Reports • Query • Applications • Information •

Upload desktop files into data items

Select Files To Upload As Data Items

Choose file(s) ...

Warning: To avoid data loss, please make sure any files with text content (csv, txt, html, etc.) are **UTF-8** encoded before uploading. To disable an implicit guess of the encoding and a check to force UTF-8, use the checkbox below (not guaranteed to work). Note that it is ok to disable the check if you know your file is valid and correctly encoded.

☐ Disable file encoding check
☐ Import unmatched file types as 'file' data items
☐ Force all uploaded files to 'file' data items
☐ Tables include column labels
☐ Tables include row labels
☐ Force tables to be numeric

• Images : png, jpg, tif, tiff
 • Movies : mp4
 • Tables : csv
 • Scenes : stl, ply, cst, avz, scdoc, dsoo, glib
 • Strings : txt
 • HTML : htm, html
 • File : ens, enc, evsn

Data Item Details

Name:

Date:

Tags:

Source:

Sequence:

Category:

Session Details

Dataset Details

- Click **Browse** and choose something to upload. Supported file formats are displayed on the right.
- Under **Data item Details**, click the **Category** drop-down list and select one or more options.

Data Item Details

Name:

Date:

Tags:

Source:

Sequence:

Category:

Anothercategory ✓
 Firstcategory ✓

Session Details

Dataset Details

Note:

You will only see categories you own, those categories that any of your groups have *own* permissions on. The only exception to this is the superuser who has unrestricted access.

- Click **Upload**.

A notification is displayed.

Ansys

DYNAMIC REPORTING

Reports ▾Query ▾Applications ▾Information ▾

Upload desktop files into data items

Files uploaded successfully

• 2020-02-20_12-51-45.png

Clicking on the file name in the notification takes you to the uploaded item's detail page. Here you can see the categories that were applied.

Ansys

DYNAMIC REPORTING

Reports ▾Query ▾Applications ▾Information ▾

Report Item   

Metadata

Click to edit fields

Name

Date

Source

Sequence number

Tags

Item ID

Session ID

Dataset ID

Type

Categories

2020-02-20_12-51-45.png

June 20, 2024, 11:50 a.m.

2020-02-20_12-51-45.png

0

e61f3b4b-2f1d-11ef-9e90-381428ee5561

e61f6269-2f1d-11ef-879f-381428ee5561

e61f626b-2f1d-11ef-abd2-381428ee5561

image

Anothercategory, Firstcategory ▾

You can only change categories you own.

Details



PYTHON

Editing Categories in the Item Detail Page

The item detail page shows metadata that includes attributes of an item and also its contents, all in one page. In addition, the page also lets you edit and save, delete or share an item. This topic covers how to edit categories from this page.

1. Access a list of report items by running a query, using one of the following menu options:

- **Query** → **Report Items**.
- **Query** → **Modify tags of selected report items**.
- **Query** → **Delete selected report items**.
- **Query** → **Assign categories to selected report items**.

Note:

You can also open the item detail page by clicking on the file name in the notification after you upload an item, as shown in [Assigning Categories During Upload \(p. 280\)](#).

2. Run a search for the report item, then click the item's name in the returned list.
3. Next to **Categories**, click the drop-down list and add or remove categories as required.

Ansys

DYNAMIC REPORTING

Reports ▾Query ▾Applications ▾Information ▾

Report Item



Metadata

Click to edit fields

Name	2020-02-20_12-51-45.png
Date	June 20, 2024, 11:50 a.m.
Source	2020-02-20_12-51-45.png
Sequence number	0
Tags	
Item ID	e61f3b4b-2f1d-11ef-9e90-381428ee5561
Session ID	e61f6269-2f1d-11ef-879f-381428ee5561
Dataset ID	e61f626b-2f1d-11ef-abd2-381428ee5561
Type	image
Categories	Anothercategory ▾ Select All Deselect All Anothercategory ✓ Firstcategory

Details



Note:

You will only see categories you own, those categories that any of your groups have *own* permissions on. The only exception to this is the superuser who has unrestricted access.

4. To save the changes, click **Save** ().

Sharing an Item from the Item Detail Page

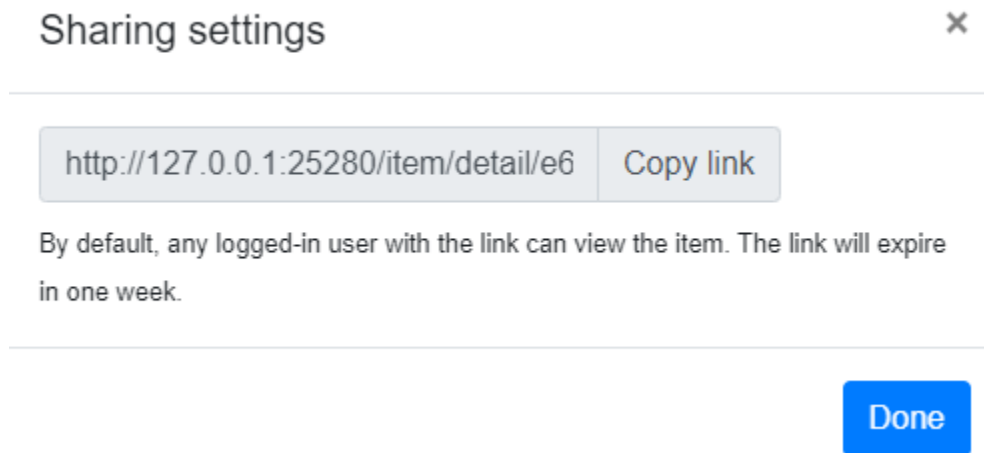
The item detail page, in addition to displaying item data, also lets you edit and save, delete or share an item.

Item sharing is a feature that is a part of the ACL infrastructure. It lets you share an item to another person who may not have access to that item. The shared item is only temporarily accessible through the shared URL, for a period of one week.

The shared URL only provides temporary viewing permissions, nothing else. This means any viewing permission checks as a part of the ACL infrastructure are skipped if a shared URL is used, until the URL expires.

Any logged-in user with the URL can view the item. You may need this option to share an item with a colleague temporarily without the need for permanently changing any permissions on the item or its category.

1. From the item detail page of any item, click **Share** ().
2. In the **Sharing settings** dialog, click **Copy link**.



The link is copied and a notification is displayed.

3. To close the dialog, click **Done**.

Creating Access Control Lists in EnSight

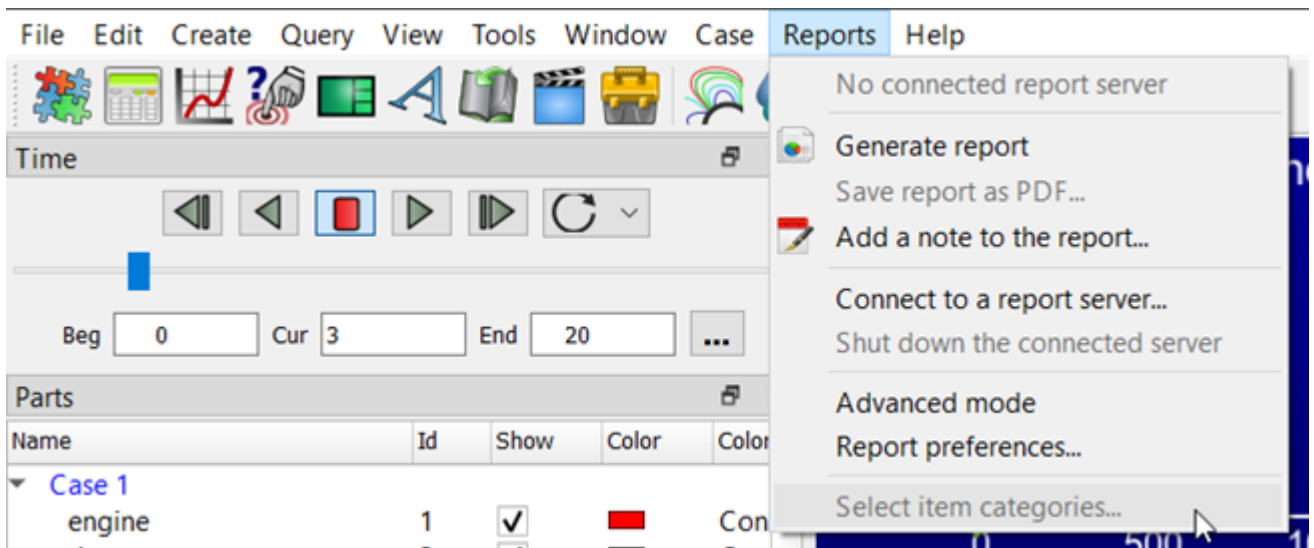
Access Control Lists (ACLs) can be used inside EnSight. You can set a list of item categories to be used while pushing items to Ansys Dynamic Reporting.

These categories, once chosen from within Ansys Dynamic Reporting, are used commonly for all items that are pushed in an EnSight session.

1. Start the EnSight user interface.
2. From the Main Menu Bar, select **Reports** → **Select item categories**.

This option is disabled until you:

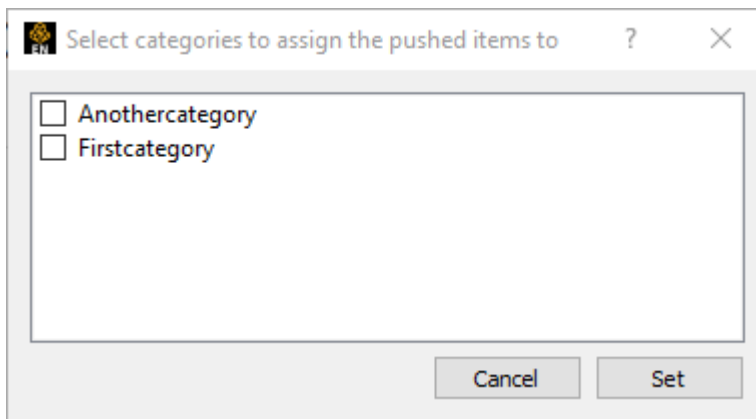
- Have a [working connection \(p. 21\)](#) to an ADR Server
- Have [enabled \(p. 258\)](#) ACLs in the target ADR Server.



If you are creating a new server and starting it within EnSight, set the environment in the shell used to start EnSight, before running it. This variable will be propagated from EnSight to the newly started ADR Server.

When you provide connection details while connecting to a report server, the login information you provide will decide the ACLs options (item-categories and so on) that will be available to you within EnSight.

3. In the dialog, select one or more item categories obtained from the connected ADR server.



4. Click **Set**. This will set the selected categories for the entire session.

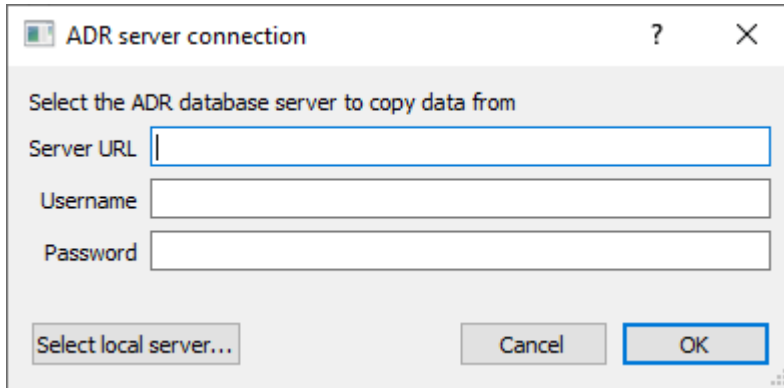
Any items pushed during this session by sending them to the report will be saved as well as assigned to the selected categories. You may change the selected categories at any point and re-generate the report.

Creating Access Control Lists in Ansys Dynamic Reporting Template Editor

When you use the template editor to copy items from one server to another, you can set a list of item categories to be used for all the items copied to the destination server.

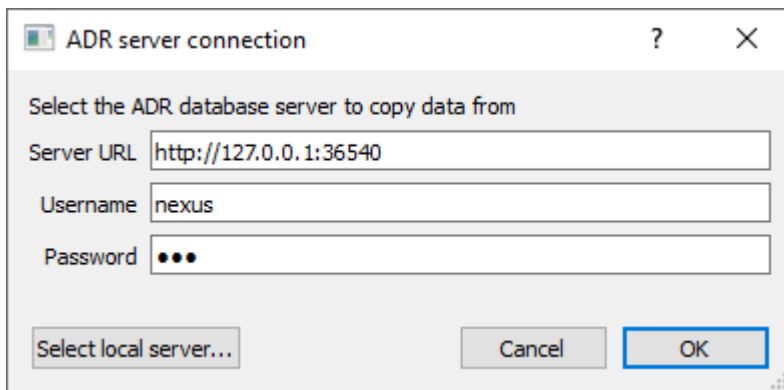
1. Open the template editor and either [launch a local server \(p. 18\)](#), or [connect to a running one \(p. 27\)](#). This is the destination server to copy items to.
2. From the menu bar, select **Tools** → **Import data from another nexus server**

The following dialog is displayed:



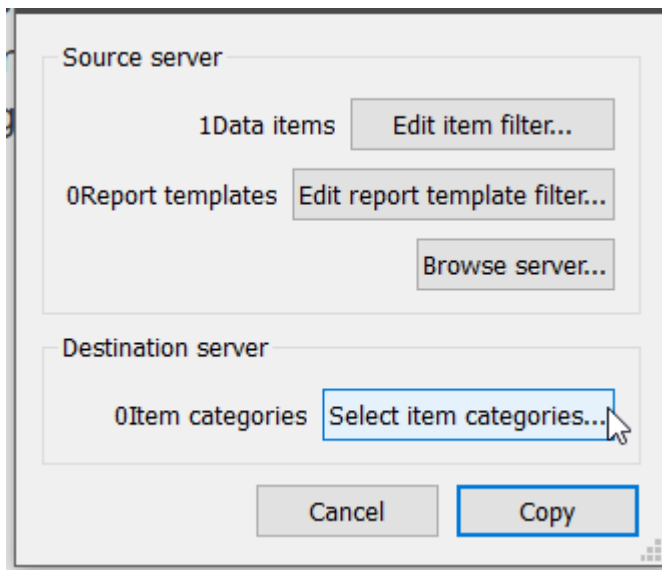
3. Enter the information manually to connect to a remote server, or click **Select local server**.
4. Set the port number to the number used for the source server and click **OK**.

Username and password will be filled automatically with the default *nexus* and *cei*.



5. Click **OK**.

The following dialog is displayed. It lets you select what to copy over from the source server. After you select the items and templates, you can choose the item categories in the destination server.

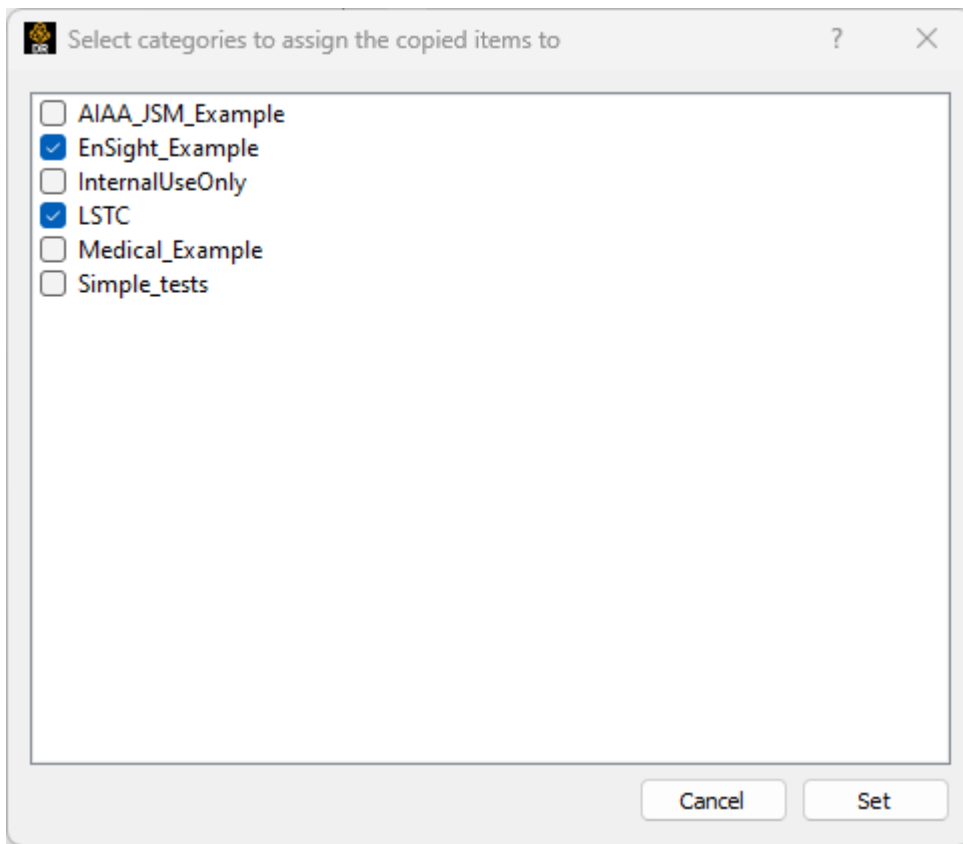


6. Click **Select item categories**.

The following dialog is displayed. It contains a checklist of item categories obtained from the destination server. Any categories selected here will be used to assign the copied items to.

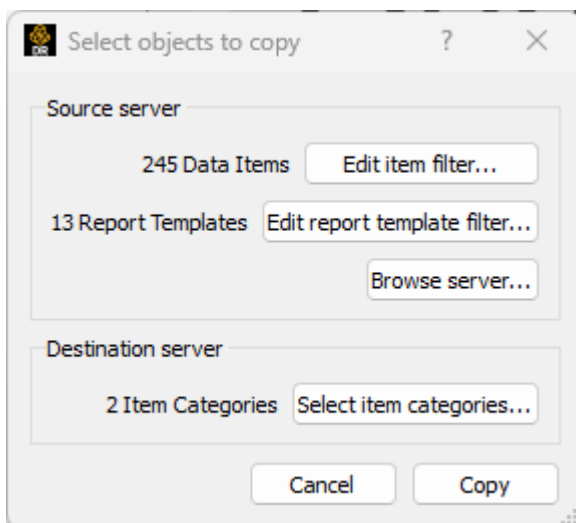
Note:

You will only see categories you own, those categories that any of your groups have *own* permissions on. The only exception to this is the superuser who has unrestricted access.



7. Select one or more categories and click **Set**.

This sets the selected categories for all the items that are being copied. The **Select objects to copy** dialog is updated.



8. Click **Copy**.

After the copy is complete, you should see the items show up in your destination server with the chosen categories assigned to them.

Creating Access Control Lists in Ansys Dynamic Reporting Data Import

The Ansys Dynamic Reporting [Data Import tool \(p. 309\)](#) is used to push files on your local disk into a running ADR server and create data items from the file contents.

You can optionally use the `--data_categories [categoryA [categoryB ...]]` option to specify a list of space-separated categories to assign the item to. The default is empty (no item categories). You can only set categories that the authenticating user has *own* permissions on, through any of the user's groups. The superuser has no restrictions.

Scripting Tools

There are multiple APIs and command line tools available to use in Ansys Dynamic Reporting.

Related topics:

[Scripting APIs](#)

[Command Line Tools](#)

[Query Expressions](#)

Scripting APIs

The Ansys Dynamic Reporting scripting APIs enable EnSight and other developers to access an Ansys Dynamic Reporting server through scripts written in Python or EnSight's command language. The API includes a large number of functions:

Database Operations

- Create a new, local database
- Launch a local server on a local database
- Stop a local server
- Attach to a server
- Query the server version

Data Operations

- Create a new data item
- Push data items into the server
- Get/set the current session information
- Get/set the current dataset information

There are two primary Ansys Dynamic Reporting Python APIs.

The [EnSight Ansys Dynamic Reporting API \(p. 292\)](#) is built into EnSight and can be called from Python or EnSight command language. It provides a semi-automated interface for exporting data from an EnSight session. It is primarily designed to support EnSight running in an interactive fashion. It supports dynamic triggers, user interface triggering and semi-automatic generation of useful data items.

There is a higher-level API in EnSight related to the use of EnSight state objects and their use in the semi-automated generation of reports. This API is covered in the EnSight [Python API website](#), in the **Object API** section under 'States: ENS_STATE' and 'Sources: ENS_SOURCE'.

The [External Python API](#) is designed to provide more of a lower-level, batch-oriented Ansys Dynamic Reporting API. It can be used from within EnSight or from the EnSight Python system. It is a bit more flexible and lower-level, more complete Ansys Dynamic Reporting interface than the EnSight API. For example, it requires you to perform tasks like image/movie generation and so on. The external python API does not require EnSight to use. It is better suited for EnSight batch mode data item capture.

EnSight Python Ansys Dynamic Reporting API

This API is integrated into the EnSight Python interpreter. It can be accessed via the **nexus** module and can only be accessed from within the Python interpreter embedded in EnSight. The alternative [External Python API](#) can be used from within EnSight or from an EnSight cpython interpreter running outside of EnSight.

To access the module from inside of a running EnSight session, run the following:

```
from ensight.core import nexus
```

Related topics:

[API Modules](#)

[API Objects](#)

API Modules

The following modules are available.

nexus module

This module is the root of the EnSight Ansys Dynamic Reporting API.

Method	Description
<code>version_float = nexus.version()</code>	Return the version of the Ansys Dynamic Reporting framework. 0.0 means the Ansys Dynamic Reporting framework could not be found.

The nexus module provides several sub modules.

nexus.ReportServer module

This module controls the connection to an Ansys Dynamic Reporting server. Most operations in the nexus module use the server connection specified by this module. The server instance is a singleton for the current EnSight session.

Method	Description
<code>nexus.ReportServer.set_URL("http://localhost:8000")</code> <code>nexus.ReportServer.set_username("ensight")</code> <code>nexus.ReportServer.set_password("cei")</code>	Specifies what server to connect

Method	Description
	the current session to.
<pre>url = nexus.ReportServer.get_URL() username = nexus.ReportServer.get_username() password = nexus.ReportServer.get_password()</pre>	Retrieves the current session configuration.
<pre>server_version_float = nexus.ReportServer.validate()</pre>	Attempts to connect to the database and verifies that the server connection specifications are valid. It returns the version of the server API that the server is using. Note: This method throws an exception when encountering an error.
<pre>has_database_url = nexus.ReportServer.valid_database()</pre>	Checks to see if a database URL has been set. It returns <i>True</i> if a URL has been set. It does not verify that the connection, username,

Method	Description
	and password is valid.
<pre>success = nexus.ReportServer.launch_local_server(directory, port=8000, connect=False, terminate_on_python_exit=False)</pre>	Launches an instance of the report server using the SQLite database and media directory found in the directory included in the arguments. The listening port may be set via the port keyword. You can also specify the port as <i>None</i>. In this case, an unused port is found and assigned automatically. If the connect keyword is <i>True</i>, once the server is launched, the nexus.ReportServer server connection is connected to the new server. If termin-

Method	Description
	ate_on_py- thon_exit is set to <i>True</i> and EnSight terminates for any reason before stop_loc- al_serv- er() is called, the server is stopped automatically.
nexus.ReportServer.stop_local_server()	Stops a local instance of a report server. The specific server is the one currently specified by the ReportServer URL, username and password. This method will have no effect on a server that is using a database other than SQLite.
success = nexus.ReportServer.create_local_database(directory)	Creates a new, empty database in the directory passed as an argument.

Method	Description
	The call generates an SQLite database and a media subdirectory in the specified directory. Note: The directory must be empty. You cannot place two databases in the same directory.

nexus.ReportItemSourceInterface module

This module provides the methods needed to create data items and push them into the database.

Method	Description
<pre>session = nexus.ReportItemSourceInterface.getSession()</pre>	Returns the current Re-portsSession object for this EnSight session. Note: If the current

Method	Description
	session has been modified since the last save, calling this method causes the session to be saved to the database.
<pre>dataset = nexus.ReportItemSourceInterface.getDataset()</pre>	Returns the current Re-reportDataSet object for this EnSight session. Note: If the current dataset has been modified since the last save, calling this method causes the

Method	Description
	dataset to be saved to the database.
item = nexus.ReportItemSourceInterface.createItem(name, source="", sequencenum=0)	Creates a new Re- portItem object. The name of the object should be a string and the source should be a string that describes the source of the item (it is often a Python namespace). This object has not yet been saved in the database. The sequence number allows multiple objects with the same name and namespace to have a sequential index number.

Method	Description
nexus.ReportItemSourceInterface.saveItem(item)	Saves an item to the database. Once an item has been created using createItem(), it can be saved to the database using this method. Note: This method saves the associated ReportDataSet and/or ReportSession if they have not yet been saved to the database.

nexus.ReportTriggerInterface module

This module provides the methods required to operate with triggers that have been defined in this session.

Method	Description
<pre>triggers = nexus.ReportTriggerInterface.readTriggerFile(filename, server=None)</pre>	Reads a report trigger file (*.rptt file written by the report trigger editor) and returns a list of trigger instances (class ReportTriggerInterface) from the file. If the server is specified and the file contains server information, the server instance is updated accordingly.

API Objects

The core API exposes a number of instances of other object classes. Several of these classes are documented here.

ReportDataset object

This object represents the information stored in the database for the current dataset. By default, this object tracks the current case automatically. It is re-evaluated when a new item is created.

The data members are:

- GUID

By default, this string is set from `ensight.objs.core.CURRENTCASE[0].UUID`

- filename

The filename of the current case.

- dirname

the directory name of the current case.

- reader

The reader used to load the current case.

- `numparts`

The number of ENS_LPART objects in the current case.

- `numelements`

The number of potential elements in the current case.

Method	Description
<code>item.set_tags(tagstring)</code>	Set the tags for the dataset to the passed string. Multiple tags are space-separated.

ReportSession object

This object represents the information stored in the database for the current session. It is initialized at startup to a singleton for the session.

The data members are:

- `GUID`

By default, this string is set from `ensight.objs.core.UUID`.

- `date`

Date/time when the session started (`datetime.datetime.now(pytz.utc)`).

- `hostname`

The hostname for the current session.

- `version`

The version of EnSight for this session (for example, '10.2.0(b)').

- `platform`

The platform the client is running on (for example, 'win64').

- `application`

The name of the application running this session ('EnSight' by default).

Method	Description
<code>item.set_tags(tagstring)</code>	Set the tags for the session to the passed string. Multiple tags are space-separated.

ReportItem object

The report item contains all of the information for a given data item. This class should not be created directly, use `nexus.ReportItemSourceInterface.createItem()` to create these objects.

The data members are:

- `GUID`

By default, this string is set via `ensight.objs.core.generate_uuid()`.

- `session`

A reference to the current session.

- `dataset`

A reference to the current dataset.

- `sequencenum`

The sequence number for this object.

- `date`

The date/time of this object creation.

- `sourcename`

The namespace of the class that generated this object.

- `name`

The name of the object.

Method	Description
<code>item.set_payload_string(string)</code>	Set the item type to a string and the value to the string argument.
<code>item.set_payload_table_values(array, rowlbls=None, collbls=None, title=None)</code>	Set the item type to HTML and the value to the string argument.
<code>item.set_payload_table_values(array, rowlbls=None, collbls=None, title=None)</code>	Set the item type to table. Array should be a numpy 2D array of floats. Set <code>rowlbls</code> to a list of strings that act as labels for the rows. Set <code>collbls</code> to a list of strings that act as labels for the columns. Set <code>title</code> to a string that is used as a title for the array (using when in plotted mode).
<code>d = {'array': a, 'rowlbls': rowlbls, 'collbls': collbls, 'title': title}</code> <code>item.set_payload_table(d)</code>	Similar to <code>set_payload_table_val-</code>

Method	Description
	ues, but passes a dictionary that includes all of the parameters.
<code>item.set_payload_image(img)</code>	Set the payload to an image. The <code>img</code> value can be an instance of a <code>QImage()</code> object, an <code>enve.image</code> object, or the in-memory representation of a PNG or JPEG file as a string (for example, <code>f = open("file.png", "rb")</code> <code>img = f.read()</code>).
<code>item.set_payload_animation(filename.mp4)</code>	Set the payload to a movie file in <code>*.mp4</code> format. The filename is passed to the method.
<code>item.set_payload_scene(filename.csf)</code>	Set the payload to a geometry scene file in <code>*.csf</code> format. The filename is passed to the method.
<code>item.set_payload_scene_capture(alltimesteps=False)</code>	Set the payload to the current EnSight geometry. By default, only the current timestep is saved. Setting <code>alltimesteps</code> to <code>True</code> will save all timesteps into the geometry item.
<code>item.set_payload_file(filename)</code>	Set the payload to the context of a general file. The filename is passed to the method.
<code>item.set_tags(tagstring)</code>	Set the tags for the item to the passed string. Multiple tags are space-separated.
<code>item.get_tags()</code>	Returns the tags string for this object. Multiple tags are space-separated.
<code>item.add_tag(tag, value=None)</code>	Adds a tag to the current tag string. If no

Method	Description
	value is passed, the simple tag string is added to the tags string. If a value is specified, a string of the form tag=value is added to the tag string.
<code>item.rem_tag(tag)</code>	Remove the tag (and any potential associated value) from the current tag string.

EnSight Command Language Ansys Dynamic Reporting API

The command language API is the same as the [EnSight Python Ansys Dynamic Reporting API \(p. 292\)](#). The only difference is that you must start lines with the fully qualified name of the nexus module. For example:

```
ensight.core.nexus...
```

Similarly, return values must be placed in the `enscl` module. A simple example that adds a HTML string to the current database:

```
enscl.item = ensight.core.nexus.ReportItemSourceInterface.createItem("test item", source="cmdlang")
enscl.item.set_payload_html("<h1>Example</h1><p>Some data</p>")
ensight.core.nexus.ReportItemSourceInterface.saveItem(enscl.item)
```

This example can be included in a command language (*.enc) file.

Command Line Tools

Using the [External Python API](#), you can write your own command line tools to manipulate data in an Ansys Dynamic Reporting database. These tools can read and write data items and even modify items that already exist in the server.

Related topics:

[Ansys Dynamic Reporting Launcher Command Line](#)

[Ansys Dynamic Reporting Command Line Utility](#)

Ansys Dynamic Reporting Launcher Command Line

The Ansys Dynamic Reporting launcher application has a complete command line interface. This interface can be used to create a new database as well as configure and launch a dedicated ADR server instance. The command line simplifies the creation and maintenance of ADR instances located on remote servers with no native user interfaces (for example, AWS or Azure servers).

The command line closely follows the user interface. The entire interface can be viewed using the `-h` option:

```
C:> adr_launcher.bat -h
```

Operations

There are two required command line arguments, the location of the database and the operation to be performed. Additional command line arguments can be used to specify specific server configuration options. All commands require you to specify the `--db_directory` command line option. This must be the name of the directory where the Ansys Dynamic Reporting database was (or is to be) created.

Every operation (except create) begins by reading the configuration settings from the `nexus.conf` file in the database directory. Next, the tool uses command line options to override various settings. For example, `--instance_count 2` changes the number of server instances (for parallel report generator) to 2, from whatever had been set in the `nexus.conf` file. At that point, the tool performs one of several operations using the augmented configuration options.

Operation	Description
create	Creates a new database. This is the only operation that does not read the <code>nexus.conf</code> file. It requires that the target directory is empty. The operation creates a new, empty Ansys Dynamic Reporting database in the directory. For example: <pre>\$ adr_launcher create --db_directory /tmp/nexus_db</pre> creates a new Ansys Dynamic Reporting database in <code>/tmp/nexus_db</code>.
save_config	Saves the <code>nexus.conf</code> file, applying any additional command line option changes. This operation is used to change configuration settings. After reading the <code>nexus.conf</code> file, the options specified on the command line are used to override options that were read and the result is then written into the <code>nexus.conf</code> file. For example: <pre>\$ adr_launcher save_config --db_directory /tmp/nexus_db --instance_count 3</pre> changes the instance count setting to 3 and updates the <code>nexus.conf</code> file.
migrate	Updates the database to the most recent version of Ansys Dynamic Reporting. When run, the database is updated to the latest version. This operation can take some time, particularly if 3D scenes are included in the database as they may need to be reprocessed. This command is not generally needed as the operation is performed automatically with the start command.
status	Provides status output for the server. This operation checks to see what the status of any ADR server associated with this database might be. For example, if no running server is associated with this database, the output might be:

Operation	Description
	<pre>\$ adr_launcher status --db_directory /tmp/nexus_db server_status: stopped</pre> or if a server is running: <pre>\$ adr_launcher status --db_directory /tmp/nexus_db server_status: running server_port: 8000 server_pid: 43984</pre>
start	Starts an ADR server for the database using the contents of the nexus.conf file along with any additional command line arguments. For example: <pre>\$ adr_launcher start --db_directory /tmp/nexus_db --tray 1</pre> starts a new server and installs a control user interface in the system tray to allow the server to be monitored, launch the template editor, stop the server, and so on. Note: This command does not return until the server stops running. The server may be stopped using the ADR launcher stop command run in another shell or via the desktop system tray menu.
stop	Shuts down any running ADR server associated with the Ansys Dynamic Reporting database.
gui	Runs the ADR user interface launcher (p. 13). The launcher user interface can be used to interactively modify the various database options. This operation is the default, so if no other operation is specified, the user interface is launched.

Server Options

Various server options control how ports are assigned to nginx and the ADR servers as well as the number of servers and the database specifics.

For more details on server architecture, see [Ansys Dynamic Reporting Server Architecture](#) (p. 5).

Option	User Interface Setting	Description
--in-stance_count	Number of parallel servers to launch	Specifies the number of ADR server instances to run.
--serv-er_port	Port	Specifies the port number that the nginx instance should listen on for incoming requests. If the port number is set to 0, a random, open port is used. This port number can be queried using the command line status option (it is also include in the

Option	User Interface Setting	Description
		URL reported by the user interface). If setting this to a non-zero value, ensure that the port does not conflict with other processes.
--server_host-name	Hostname	Specifies the external hostname that the server responds as. For example, if the server is configured to be accessed externally via the URL: <code>http://my.nexusserver.com:80</code>, the server hostname should be set to <code>my.nexusserver.com</code>. The nginx server only accepts incoming requests that resolve to this hostname. A common mistake is setting the hostname to <code>localhost</code> or <code>127.0.0.1</code> and enabling remote access (the user interface presents a warning in this situation). Some features of the server do not work in such a setup, unless it has been hosted behind an appropriately configured proxy server. The server hostname should be set to the name of the system as seen by other machines on the same network.
--internal_base_port	Base port for dynamic allocation	Specifies the port number where Ansys Dynamic Reporting starts looking for unused ports when allocating ports for the ADR servers and other components. For example, if there are two ADR servers and nginx and <code>server_port</code> is set to 0, Ansys Dynamic Reporting starts allocating random ports for these components. If all of the ports starting at 8000 are available, it selects 8000, 8001 and 8002 for the components.
--ssl_certificate	N/A	Specifies the name of the SSL certificate file that nginx uses to serve up this connection. It switches the ADR server to use https instead of http. The <code>--ssl_key</code> option must also be specified.
--ssl_key	N/A	Specifies the name of the SSL key file that nginx uses to serve up this connection. It switches the ADR server to use https instead of http. The <code>--ssl_certificate</code> option must also be specified.
--remote_session	Embedded sessions tab	Enables embedded remote sessions (p. 249). Ansys Dynamic Reporting has the ability to embed applications that are capable of rendering (remotely) via the VNC protocol into reports. There are built-in interfaces for local EnSight and EnVision sessions as well as custom sessions defined in the <code>external_services.json</code> file. For embedded remote sessions to work, this option must be enabled. The built-in EnSight and EnVision local sessions can be enabled by setting the <code>--local_sessions_ensight</code> and <code>--local_session_envision</code> options.
--local_sessions_ensight	Number of local EnSight sessions	Selects the maximum number of local instances of the EnSight application that can be running in embedded form simultaneously. These sessions are available for *.ens file data

Option	User Interface Setting	Description
		items. The <code>--remote_session</code> option must be enabled in addition to setting this to a non-zero value.
<code>--local_sessions_envision</code>	Number of local EnVision sessions	Selects the maximum number of local instances of the EnVision application that can be running in embedded form simultaneously. These sessions are available for <code>*.evsn</code> file data items. The <code>--remote_session</code> option must be enabled in addition to setting this to a non-zero value.
<code>--server_name</code>	Name	Specifies a string that appears in the server About page or the system tray menu to describe this server.
<code>--postgresql_url</code>	Use postgresql Postgresql server host-name Postgresql server port Postgresql database name User-name Password	Specifies that this ADR server should use a PostgreSQL server as the database instead of an SQLite database. From the command line, all of these options are passed as a single string: <pre>postgresql://user:password@host:port/database</pre> In the user interface, the individual fields have been broken out for easier editing. There are also buttons to test the database connection and initialize the PostgreSQL database that store the database tables. Note: All Ansys Dynamic Reporting databases begin as SQLite databases. Once created, they can be converted over to PostgreSQL.
<code>--acIs</code>	Enable per-item ACL support	Enables Access Control Lists (p. 258) .
<code>--allow_remote_access</code>	Allow remote (non-localhost) access	ADR servers are only accessible from the same system the server is running on (for example, a local desktop session). When this option is selected, the server can be accessed from machines other than the one the ADR server is running on.
<code>--allow_iframe_embedding</code>	Allow ADR webpage iframe embedding	By default, Ansys Dynamic Reporting prevents the HTML it generates to be displayed in iframes for security. Select this option to allow views of generated reports only from inside of an embedded iframe. When enabled, the content is still subject to other iframe limits. For example, if you use an SSL enabled server to serve up an iframe containing an Ansys Dynamic Reporting report, and that report is not accessed via localhost or an SSL enabled Ansys Dynamic Reporting session, the content

Option	User Interface Setting	Description
		will still be blocked as you cannot transition from SSL to non-SSL via the iframe. The admin functions (including login) are not supported in iframes because of cookie limitations.
<code>--max_upload_size</code>	Maximum item upload size	There is a limit on the size of a file/data item that can be uploaded to an ADR server. By default, that limit is 5000 megabytes. Use this option to adjust the size (from 1 to 20000 megabytes) for a single data item.
<code>--tray</code>	N/A	This option is not saved in the <code>nexus.conf</code> file and is only used if <code>--start</code> is the action. If enabled, the launcher application attempts to create a system tray menu for controlling the server being started. The host running the launcher instance must be capable of displaying user interfaces for this option to work.
<code>--verbose</code>	Verbose logging	When enabled, additional diagnostic information is logged when the server is run, and the specifics of the configuration is displayed on standard output. The logging information can be viewed from log files created in the Ansys Dynamic Reporting database directory or in the system tray logging dialog.
<code>--debug</code>	N/A	When enabled, the server provides additional debugging information. This option should only be used when directed to by Ansys support.
<code>--stdout</code>	N/A	When enabled, the server logs all output to stdout. This can be useful for debugging.

Ansys Dynamic Reporting Command Line Utility

The command line utility allows many Ansys Dynamic Reporting operations to be run from the command line. These operations include listing entities on the server, importing entities from files on disk, deleting entities, copy entities between servers, and modifying the tags on entities. These functions are implemented as a single command line tool, leveraging sub-commands to perform the individual operations.

The command is structured as follows:

```
adr [-h] {cp,htmllexport,pdflexport,import,ls,rm,tag} ...
```

The sub-commands are:

- [cp \(p. 310\)](#)
- [import \(p. 311\)](#)
- [ls \(p. 315\)](#)
- [rm \(p. 316\)](#)

- [htmlexport](#) (p. 317)
- [pdfexport](#) (p. 318)
- [tag](#) (p. 319)

You can access help on individual tags by combining `-h` with the sub-command. For example,

```
adr cp -h
```

displays help related to the `cp` sub-command.

Note:

Most commands require the URL of a server to be provided. This is usually in the form of `http://{username}:{password}@hostname:{port}`. For example: `http://nexus:cei@127.0.0.1:8000`. This URL syntax exposes the username and password of the database in cleartext. Command lines are often visible to all users running on the same system. If no username or password is supplied (for example, `http://127.0.0.1:8000`), the command line tool uses the values of the `ADR_TOOLS_USERNAME` and `ADR_TOOLS_PASSWORD` environment variables for the username and password to use, avoiding command line exposure.

For templates, generally only top level templates can be manipulated for operations like deleting and copying, and those operations are applied to the entire tree of templates.

Copy (cp)

```
usage: adr.bat cp [-h] [--test] [--verbose] [--type {item,template}] [--query query_string]
                source_server_url destination_server_url
```

Copy items, sessions, datasets and templates from one ADR server to another.

positional arguments:

`source_server_url` ADR server URL. Example: `http://nexus:cei@127.0.0.1:8000`

`destination_server_url` Target ADR server URL. Example: `http://nexus:cei@127.0.0.1:8000`

options:

`-h, --help` show this help message and exit

`--test` Validate the operation, but do not perform it

`--verbose` Include progress and other information

`--type {item,template}` Select the object type to copy. Default: `item`

`--query query_string` Specify a query string using URI syntax. Default: no filter

This tool copies a selected collection of objects (data items or templates) from one server to another. Globally Unique Identifiers (GUIDs) are preserved and any referenced session and dataset objects are copied as well.

Argument/Option	Description
Arguments	
<code>source_server_url</code>	Specifies the hostname, port, username and password of the ADR server to read items from. For example: <code>http://nexus:cei@127.0.0.1:8000</code> specifies a server running on the local

Argument/Option	Description
	machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
destination_server_url	Specifies the hostname, port, username and password of the ADR server to copy items into. For example: http://nexus:cei@127.0.0.1:8000 specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
Options	
-h, --help	Displays the usage message and exits.
--verbose	Displays additional status and progress information during the operation.
--test	Simulates the operation without actually performing it. This is helpful when debugging command lines, especially when used in conjunction with the --verbose option.
--type {item,template}	Specifies the type of objects to copy. This command can copy data items or templates.
--query query_string	Uses the query string to select the specific objects of the specified type (--type {option}) to perform the operation on. See Query Expressions (p. 320) for the syntax.

Import (import)

```
usage: adr.bat import [-h] [--disable_encoding_check] [--session_guid [session_guid]] [--session_date [date]]
                    [--session_hostname [hostname]] [--session_platform [platform]]
                    [--session_application [application]] [--session_version [version]] [--session_tags [tag_string]]
                    [--dataset_guid [dataset_guid]] [--dataset_filename [filename]] [--dataset_dirname [dirname]]
                    [--dataset_format [format]] [--dataset_parts [count]] [--dataset_elements [count]]
                    [--dataset_tags [tag_string]] [--data_guid [item_guid]] [--data_date [date]]
                    [--data_sequence [number]] [--data_name [name]] [--data_source [name]] [--data_tags [tag_string]]
                    [--data_categories [categoryA [categoryB ...]]] [--ext_image [ext ...]] [--ext_movie [ext ...]]
                    [--ext_table [ext ...]] [--ext_scene [ext ...]] [--ext_file [ext ...]] [--ext_text [ext ...]]
                    [--ext_html [ext ...]] [--table_col_labels] [--table_row_labels] [--table_numeric]
                    [--table_delimiter [char]] [--table_quote_char [char]] [--verbose] [--test] [--print_op]
                    adr_server_url [file|directory ...]
```

Import data items from disk into the ADR server.

positional arguments:

adr_server_url ADR server URL. Example: http://nexus:cei@127.0.0.1:8000
file|directory File and directory names to import

options:

-h, --help show this help message and exit
--disable_encoding_check Disable the implicit check for UTF-8 encoding of uploaded files
--session_guid [session_guid] GUID of an existing Ansys Dynamic Reporting session to use for new data items
--session_date [date] Creation date for the created session
--session_hostname [hostname] Hostname for the created session
--session_platform [platform] Platform name for the created session
--session_application [application] Application name for the created session
--session_version [version] Version for the created session
--session_tags [tag_string]

```

                                Tags for the created session
--dataset_guid [dataset_guid]  GUID of an existing Ansys Dynamic Reporting dataset to use for new data items
--dataset_filename [filename]  Filename for the created dataset
--dataset_dirname [directory]  Directory name for the created dataset
--dataset_format [format]      Format for the created dataset
--dataset_parts [count]        Number of parts for the created dataset
--dataset_elements [count]     Number of elements for the created dataset
--dataset_tags [tag_string]     Tags for the created dataset
--data_guid [item_guid]         GUID for newly imported items (note only useful when importing a single data item, use
--data_date [date]              Creation date for created data items (__timestamp__ is the file timestamp)
--data_sequence [number]       Sequence number for the created data items
--data_name [name]              Name for created data items (__file__ is the file basename)
--data_source [name]           Source for created data items (__file__ is the file basename)
--data_tags [tag_string]       Tags for created data items
--data_categories [categoryA [categoryB ...]] List of space-separated categories to assign the item to.
--ext_image [ext ...]           Filename extensions to be imported as image. Default: ['png', 'jpg']
--ext_movie [ext ...]          Filename extensions to be imported as movie. Default: ['mp4']
--ext_table [ext ...]          Filename extensions to be imported as table. Default: ['csv']
--ext_scene [ext ...]          Filename extensions to be imported as scene. Default: ['stl', 'ply', 'csf', 'avz', 'scd']
--ext_file [ext ...]           Filename extensions to be imported as file. Default: []
--ext_text [ext ...]           Filename extensions to be imported as text. Default: ['txt']
--ext_html [ext ...]           Filename extensions to be imported as html. Default: ['htm', 'html']
--table_col_labels             Input csv files will have a column label row
--table_row_labels             Input csv files will have a row label column
--table_numeric                Table items generated from .csv files will have strictly numeric content
--table_delimiter [char]       The delimiter in csv files
--table_quote_char [char]      The quote character in csv files
--verbose                      Enable verbose mode
--test                         Validate the operation, but do not perform it
--print_options                 Print import configuration option values

```

The tool allows you to upload data from files into an Ansys Dynamic Reporting database. This is similar to [uploading files into data items \(p. 57\)](#) in the Ansys Dynamic Reporting web interface. The tool provides options for specifying files or directories to upload and defining exactly how the data should be incorporated into the existing data on the ADR server. These include specification of session, dataset, tags, what data item type-specific file extensions should be uploaded as, and how tables should be handled.

Item file uploads for csv, txt and html files must be strictly UTF-8 encoded. On upload, there is a check for UTF-8 encoding which is used to force this requirement. This check is enabled by default and uploads fail if this check fails. You can disable this check using `--disable_encoding_check` if you feel that your file is correctly encoded.

The tool takes a list of files and/or directories as arguments. When a directory is specified, the tool recursively visits its contents, uploading all files that match the file extensions specified with the `--ext_*` options.

Argument/Option	Description
Arguments	
<code>adr_server_url</code>	Specifies the hostname, port, username and password of the ADR server to import the data items into. For example: <code>http://nexus:cei@127.0.0.1:8000</code> specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
<code>file directory ...</code>	Specifies the name of a specific file to import or the name of a directory whose files should be imported. This argument may be repeated as needed. The tool recurses into the directory, looking for files with the matching extensions to import.
Options	
<code>-h, --help</code>	Displays the usage message and exits.
<code>--session_guid [GUID]</code>	The Globally Unique Identifier (GUID) of the session to associated the data with. If omitted, a new session is created and assigned a new GUID. This new session GUID is then associated with the data. The additional <code>--session_*</code> command-line options are provided to specify attributes of the new session.
<code>--session_date [date]</code>	The creation date for the created session. This option is ignored if an existing session GUID is used instead of creating a new session. If omitted, the current time stamp is used. The format of the date is that parsed by the Python <code>dateutil.parser.parse</code> method, which is able to parse most known formats.
<code>--session_hostname [hostname]</code>	The host name to associate with the created session. This option is ignored if an existing session GUID is specified. The default is the host name of the system the tool is run on.
<code>--session_platform [platform]</code>	The platform (operating system) name to associate with the created session. This option is ignored if an existing session GUID is specified. The default is the platform of the system the tool is run on.
<code>--session_application [application]</code>	The application name to associate with the created session. This option is ignored if an existing session GUID is specified. The default is <i>adr_data_import</i> .
<code>--session_version [version]</code>	The version to associate with the created session. This option is ignored if an existing session GUID is specified. The default is the version number of the tool.
<code>--session_tags [tag string]</code>	A space-separated list of tags to associate with the created session. This option is ignored if an existing session GUID is specified. The default is empty (no session tags). Session tags, dataset tags, and data item tags are distinct and separate. It is a common mistake to specify <code>--session_tags</code> when <code>--data_tags</code> is intended.
<code>--dataset_guid [GUID]</code>	The GUID of the dataset to associate the data with. If omitted, a new dataset is created and assigned a new GUID. This new dataset GUID is then associated with the data. The additional <code>--dataset_*</code> command-line options are provided to specify attributes of the new dataset.

Argument/Option	Description
<code>--dataset_filename</code> [file name]	The file name for the created dataset. This option is ignored if an existing dataset GUID is specified. The default file name is <i>unspecified</i> .
<code>--dataset_dirname</code> [directory name]	The directory name for the created dataset. This option is ignored if an existing dataset GUID is specified. The default directory name is the working directory used when the ADR data import tool was run.
<code>--dataset_format</code> [format]	The format for the created dataset as a string. This option is ignored if an existing dataset GUID is specified. The default format string is <i>unspecified</i> .
<code>--dataset_parts</code> [count]	The number of parts in the created dataset. This is only useful when 3D scene data is imported. This option is ignored if an existing dataset GUID is specified. The default is 0.
<code>--dataset_elements</code> [count]	The number of elements in the created dataset. This is only useful when 3D scene data is imported. This option is ignored if an existing dataset GUID is specified. The default is 0.
<code>--dataset_tags</code> [tag string]	A space-separated list of tags to associate with the created dataset. This option is ignored if an existing dataset GUID is specified. The default is empty (no dataset tags). Session tags, dataset tags, and data item tags are distinct and separate. It is a common mistake to specify <code>--dataset_tags</code> when <code>--data_tags</code> is intended.
<code>--data_date</code> [date]	The creation date for the created data items. The format of the date is that parsed by the <code>Python dateutil.parser.parse</code> method, which is able to parse most known formats. If the string <code>"__timestamp__"</code> is specified for the date, the creation date used is each data file. The default is now, the time when the tool was run.
<code>--data_sequence</code> [sequence number]	The sequence number for the created data items. This is used if data items can be considered in a specific ordered sequence. The default sequence number is 0.
<code>--data_name</code> [name]	The name for the created data items as a string. If <code>"__file__"</code> is specified, the name is of each data file. The default is <code>"__file__"</code> .
<code>--data_source</code> [source]	The source of the data as a string. The default is <code>"data_import"</code> . Set this to track where data comes from.
<code>--data_tags</code> [tag string]	A space-separated list of tags to associate with the data items. The default is empty (no data item tags). Session tags, dataset tags, and data item tags are distinct and separate. It is a common mistake to specify <code>--session_tags</code> or <code>--dataset_tags</code> when <code>--data_tags</code> is intended. See Tagging (tag) (p. 319) .
<code>--disable_encoding_check</code>	Item file uploads for csv, txt and html files must be strictly UTF-8 encoded. On upload, there is a check for UTF-8 encoding which is used to force this requirement. This check is enabled by default and uploads fail if this check fails. You can disable this check using this option if you feel that your file is correctly encoded.
<code>--ext_files</code> [ext [ext ...]] <code>--ext_tables</code> [ext [ext ...]]	The <code>--ext_*</code> options are used to map file extensions to a data item (p. 91) type. Each option corresponds to a different data item type and specifies the filename extensions that should be identified as such. For example, using <code>--ext_html html htm</code> specifies

Argument/Option	Description
• <code>--ext_html</code> [ext [ext ...]] • <code>--ext_movies</code> [ext [ext ...]] • <code>--ext_images</code> [ext [ext ...]] • <code>--ext_text</code> [ext [ext ...]] • <code>--ext_scenes</code> [ext [ext ...]]	that *.html and *.htm files should be imported as HTML data items. By default, *.png and *.jpg map to image (p. 94) items, *.mp4 maps to animation (p. 92) items, *.csv maps to table (p. 100) items, *.stl, *.ply, *.csf, and *.avz map to scene (p. 95) items. File items, string items, tree items, and HTML items have no explicit mapping. If you do not provide arguments after an option, file extensions are not mapped to that data item type.
• <code>--table_col_labels</code> • <code>--table_row_labels</code>	Specifies that imported tables should consider the first column and first row, respectively, as labels for the table. If either of these options are omitted, it indicates that the column (or row) has no labels in the data file.
<code>--table_numeric</code>	Indicates that table data should be changed into numeric data types instead of strings (the default).
<code>--table_delimiter</code> [char]	The character that separates cell items from each other in an table input file. By default, the separator is a comma. Specifying nothing for this option causes each line to be imported as a single cell item.
<code>--table_quote_char</code> [char]	The character that delimits quoted text in cell items in a table input file. By default, the quote character is a single quote ('). This allows the table delimiter to be used within a single cell. Specifying nothing for this option causes no special parsing for quoted strings.
<code>--verbose</code>	Displays additional status and progress information during the operation.
<code>--print_options</code>	Prints out the selected command line options.
<code>--test</code>	Simulates the operation without actually performing it. This is helpful when debugging command lines, especially when used in conjunction with the <code>--verbose</code> option.

List (ls)

```
usage: adr.bat ls [-h] [--full] [--long] [--guid] [--header] [--no-header]
               [--type {item,session,dataset,template,category,group}] [--query query_string]
               adr_server_url
```

List items, sessions, datasets and templates in the ADR server.

positional arguments:
 adr_server_url ADR server URL. Example: http://nexus:cei@127.0.0.1:8000

options:
 -h, --help show this help message and exit
 --full Increase the level of output detail
 --long One item per line
 --guid Include the GUID in the output

```
--header           Include headers in the output
--no-header        Do not include headers in the output
--type {item,session,dataset,template,category,group}
                   Select the object type to list. Default: item
--query query_string Specify a query string using URI syntax. Default: no filter
```

This tool selects a collection of objects and prints the details of the objects to stdout.

Argument/Option	Description
Arguments	
adr_server_url	Specifies the hostname, port, username and password of the ADR server to list items from. For example: <code>http://nexus:cei@127.0.0.1:8000</code> specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
Options	
-h, --help	Displays the usage message and exits.
--full	Displays additional information not included in a simple listing. This option implies --long.
--long	Displays a single item on each line.
• --header • --no-header	If --header is specified, some query details are printed at the start of the operation. The --no-header option suppresses the query details (the default).
--guid	Displays the GUID for each object, as well as other selected information.
--verbose	Displays additional status and progress information during the operation.
--type {item,template,dataset,category,group}	Specifies the type of objects to list.
--query query_string	Uses the query string to select the specific objects of the specified type (--type {option}) to perform the operation on. See Query Expressions (p. 320) for the syntax.

Delete (rm)

```
usage: adr.bat rm [-h] [--type {item,template}] [--query query_string] [--verbose] [--test] adr_server_url
```

Delete selected items or report templates from the ADR server. Note: if the query type is 'template', the query automatically prefixed to restrict the query to top level templates. Only top level templates may be deleted and are, all of their children will be deleted as well.

positional arguments:

adr_server_url ADR server URL. Example: `http://nexus:cei@127.0.0.1:8000`

options:

-h, --help show this help message and exit

--type {item,template}

Select the object type to remove. Default: item

--query query_string Specify a query string using URI syntax. Default: no filter

--verbose Enable verbose mode

--test Validate the operation, but do not perform it

This tool deletes a selected collection of objects (data items or templates) from a server.

Argument/Option	Description
Arguments	
<code>adr_server_url</code>	Specifies the hostname, port, username and password of the ADR server to delete items from. For example: <code>http://nexus:cei@127.0.0.1:8000</code> specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
Options	
<code>-h, --help</code>	Displays the usage message and exits.
<code>--verbose</code>	Displays additional status and progress information during the operation.
<code>--test</code>	Simulates the operation without actually performing it. This is helpful when debugging command lines, especially when used in conjunction with the <code>--verbose</code> option.
<code>--type {item,template}</code>	Specifies the type of objects to delete. This command can delete data items or templates. It may delete sessions/datasets implicitly if they are no longer referenced.
<code>--query query_string</code>	Uses the query string to select the specific objects of the specified type (<code>--type {option}</code>) to perform the operation on. See Query Expressions (p. 320) for the syntax.

HTML Export (htmlexport)

```
usage: adr.bat htmlexport [-h] [--query query_string] [--filename top_level_filename] [--no_inline_files] [--verbose]
                        adr_server_url destination_directory
```

Export a selected report as offline html from the ADR server. Note: the template query will always include a to check. The report will be exported into the named destination directory, which must not exist. The default top filename will be `index.html` stored in the destination directory. The top level filename can be specified on the command line and should include both filename and extension.

positional arguments:
`adr_server_url` ADR server URL. Example: `http://nexus:cei@127.0.0.1:8000`
`destination_directory` Directory to save HTML report into. Must not exist.

options:
`-h, --help` show this help message and exit
`--query query_string` Query string used to select the report template to export. Default: no filter
`--filename top_level_filename`
 The filename of the top-level file. Default: `index.html`
`--no_inline_files` Disable the use of data URIs in the output
`--verbose` Enable verbose mode

This tool generates a report on the ADR server and downloads the resulting HTML into a local directory structure. Some item types are converted into forms that do not require a running ADR server.

Argument/Option	Description
Arguments	
<code>adr_server_url</code>	Specifies the hostname, port, username and password of the ADR server to modify items on. For example: <code>http://nexus:cei@127.0.0.1:8000</code>

Argument/Option	Description
	us:cei@127.0.0.1:8000 specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
destination_directory	The downloaded HTML/CSS/image and other files are downloaded into this directory. This directory cannot exist before the operation is performed.
Options	
-h, --help	Displays the usage message and exits.
--query query_string	This query string is used to select the report to be exported. The result of the query must be a single top level report template.
--filename top_level_filename	By default, the top level HTML file is named <i>index.html</i> in the destination directory. Use this option to specify a different file name.
--no_inline_files	By default, the export operation uses data URI strings embedded in the HTML to represent some files. If this option is specified, separate disk files are used instead, but the local HTML cannot be displayed in a browser without a server or special browser flags.
--verbose	Displays additional status and progress information during the operation.

PDF Export (pdfexport)

```
usage: adr.bat pdfexport [-h] [--query query_string] [--verbose] adr_server_url pdf_filename
```

Export a selected report in pdf format from the ADR server. Note: the template query will always include a top check. The report will be written into the named file, which must not exist.

positional arguments:

adr_server_url ADR server URL. Example: http://nexus:cei@127.0.0.1:8000
pdf_filename Filename to save the PDF file as. Must not exist.

options:

-h, --help show this help message and exit
--query query_string Query string used to select the report template to export. Default: no filter
--verbose Enable verbose mode

This tool generates a report on the ADR server and renders the selected report into a local PDF format file.

Argument/Option	Description
Arguments	
adr_server_url	Specifies the hostname, port, username and password of the ADR server to modify items on. For example: http://nexus:cei@127.0.0.1:8000 specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
pdf_filename	The name to save the PDF file as. This file cannot exist before the operation is performed.
Options	

Argument/Option	Description
-h, --help	Displays the usage message and exits.
--query query_string	This query string is used to select the report to be exported. The result of the query must be a single top level report template.
--verbose	Displays additional status and progress information during the operation.

Tagging (tag)

```
usage: adr.bat tag [-h] [--value VALUE] [--type {item,session,dataset,template}] [--query query_string] [--verbose]
                  [--test]
                  adr_server_url {add,remove,replace} tagname
```

Modify item, template, session, and dataset object tags in the ADR server.

positional arguments:

```
adr_server_url  ADR server URL. Example: http://nexus:cei@127.0.0.1:8000
{add,remove,replace}  Tag operation
tagname         The tag to modify
```

options:

```
-h, --help          show this help message and exit
--value VALUE       Tag value if needed
--type {item,session,dataset,template}
                    Select the object type to list. Default: item
--query query_string Specify a query string using URI syntax. Default: no filter
--verbose           Enable verbose mode
--test             Validate the operation, but do not perform it
```

This tool gathers a collection of objects (data items, sessions, datasets or templates) selected via a query string, and manipulates the tag strings.

The tag form is `tag_name=value`, where `=value` is optional. The `tag_name` is specified using an argument and `value` may be specified using the `--value` option. The resulting tag can be added to, removed from, or modified, on the selected objects. This tool cannot add a tag multiple times to the same object.

Tags in Ansys Dynamic Reporting are simple strings and can be applied to data items, sessions, and datasets. Tags are not required, but allow a wide variety of expression to define a data model. Without them, queries can be done only with the default data that Ansys Dynamic Reporting provides (dates, names, connections). Best practice involves creating a data model through the use of tags. Depending on the data model, the presence or absence of a tag is significant.

For example, one data model might attach significance to the `verified` tag, applied to a data item. Such a tag might indicate that the data item has gone through a review process, and the absence of such tag indicates that the process has not been performed. Other tags take the form `key=value`.

Using this format, tags are more fine-grained, allowing them to take on different values. For example, it is common to have `variable=pressure` or `variable=density` tags applied to image data that comes from physical simulations. Any number of tags may be specified and are separated by spaces when defined. So the tag string `verified variable=viscosity` is a valid list of two tags, the second one of which has a value.

Argument/Option	Description
Arguments	

Argument/Option	Description
<code>adr_server_url</code>	Specifies the hostname, port, username and password of the ADR server to modify items on. For example: <code>http://nexus:cei@127.0.0.1:8000</code> specifies a server running on the local machine (127.0.0.1) on port 8000. The username is <i>nexus</i> and the password is <i>cei</i> .
<code>tag_operation</code> {add,remove,replace}	Selects the type of modification to apply to the named tag. Options are: • <code>add</code>: Add the tag to the items. • <code>remove</code>: Remove the tag from the items. • <code>replace</code>: Replace the tag. Replace will add the tag if it does not exist. If it does exist, the value of the tag is changed or removed, depending on the <code>--value</code> option.
<code>tag_name</code>	The name of the tag to modify.
Options	
<code>-h, --help</code>	Displays the usage message and exits.
<code>--value [value]</code>	Specifies the tag's value, forming the string: <code>{tag_name}={value}</code> to be used as the tag. If this option is not specified, the tag is simply <code>{tag_name}</code> .
<code>--verbose</code>	Displays additional status and progress information during the operation.
<code>--test</code>	Simulates the operation without actually performing it. This is helpful when debugging command lines, especially when used in conjunction with the <code>--verbose</code> option.
<code>--type {item,template}</code>	Specifies the type of objects to delete. This command can delete data items or templates. It may delete sessions/datasets implicitly if they are no longer referenced.
<code>--query query_string</code>	Uses the query string to select the specific objects of the specified type (<code>--type {option}</code>) to perform the operation on. See Query Expressions (p. 320) for the syntax.

Query Expressions

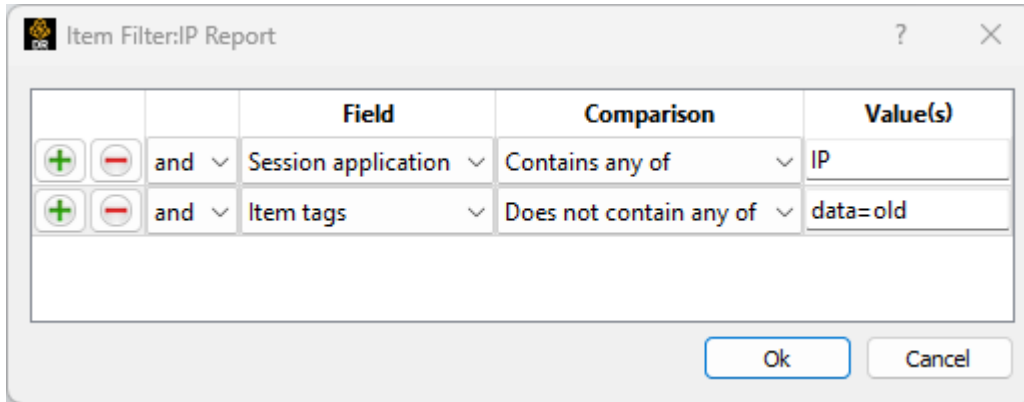
Several of the server methods and templates support query expression strings to help select and filter database objects. These strings have a common format, namely a collection of stanzas that specify a field, a comparison operation, a value and a chaining code to link to other stanzas. Stanzas are delineated by the semicolon (;) character and the fields in the stanza are separated by the pipe (|) character.

The following is an example query expression:

```
A|i_tags|cont|chart=pie;A|i_name|ncont|Total;
```

This query consists of two stanzas. The A is the chaining code and stands for *and*. The first stanza looks at the `i_tags` field (the Item object tags field) and the second looks at the `i_name` field (the Item object name field). The first stanza uses `cont` to specify that the item tags field must contain the value `chart=pie` and the second stanza specifies that the item name field must not contain the string `Total`.

In the Ansys Dynamic Reporting Template Editor, this query is set up as:



Query Stanza Chaining Commands

Command	Description
A	Chain from previous stanza with <i>and</i> logic
O	Chain from previous stanza with <i>or</i> logic

Query Fields

All fields may be used for Item object type queries. The query is performed on the associated session or dataset objects if a session or dataset field is specified in an Item query string. For a session object type query, only session fields are allowed and for a dataset object type query, only dataset fields are allowed.

Field	Description
Item Fields	
<code>i_name</code>	Item name
<code>i_src</code>	Item source
<code>i_date</code>	Item date and time
<code>i_tags</code>	Item tag string
<code>i_type</code>	Item type: • <code>html</code> • <code>string</code> • <code>table</code>

Field	Description
	• image • anim • scene • file • none
i_seq	Item sequence number
Session Fields	
s_app	Session application name
s_ver	Session application version string
s_date	Session date and time
s_tags	Session tag string
s_host	Session hostname
s_plat	Session platform
s_guid	Session GUID
Dataset Fields	
d_name	Dataset file name
d_dir	Original dataset directory
d_fmt	Format of the dataset (for EnSight, this is the reader name)
d_tags	Dataset tag string
d_guid	Dataset GUID
Report Template Fields	
t_name	Report template file name
t_filt	Report template item filter
t_types	Report template type (for example, Layout:basic)
t_mstr_chk	If the report template is a root level template (instead of a child template), contains <i>true</i>
t_date	Report template date and time
t_tags	Report template tag string
t_guid	Report template GUID

Comparison Operators

Some of these operators have mixed meanings, based on their target fields.

Operator	Description
Date Field Comparisons	

Operator	Description
eq	On the specified date
neq	Not on the specified date
gte	On or after the specified date
gt	After the specified date
lte	On or before the specified date
lt	Before the specified date
Item Type Field Comparisons	
cont	Include only these types
ncont	Do not include these types
Item Sequence Field Comparisons	
cont	Contains one of the specified sequence numbers
ncont	Does not contain any of the specified sequence numbers
eq	Equal to the specified sequence number
neq	Not equal to the specified sequence number
gte	Greater than or equal to the specified sequence number
gt	Greater than the specified sequence number
lte	Less than or equal to the specified sequence number
lt	Less than the specified sequence number
Other Field Comparisons	
cont	Contains one of the specified values
ncont	Does not contain any of the specified values
eq	Equal to the specified value
neq	Not equal to the specified value
sw	Starts with the specified value
ew	Ends with the specified value
gte	Greater than or equal to the specified value
gt	Greater than the specified value
guid	Equal to the value of the object GUID
lte	Less than or equal to the specified value
lt	Less than the specified value

Query URI Syntax

When using queries as parameters in command line tools, you must use the URI syntax. All the rules previously explained still apply, but some characters must be replaced by special strings as the command prompt/shell can interpret them incorrectly. The following table provides a list of the replacement strings that are required for query URI syntax.

Original Character	URI Syntax Replacement
'?'	"%3F"
'#'	"%23"
'['	"%5B"
']'	"%5D"
'@'	"%40"
'!'	"%21"
'\$'	"%24"
'&'	"%26"
'\"'	"%27"
'('	"%28"
')'	"%29"
'*'	"%2A"
'+'	"%2B"
','	"%2C"
':'	"%3B"
'='	"%3D"
'%'	"%25"
' '	"%20"

For example, the following query:

```
A|i_tags|cont|chart=pie;A|i_name|ncont|Total;
```

Must be replaced with:

```
A|i_tags|cont|chart=pie%3BA|i_name|ncont|Total%3B
```

Troubleshooting

The following topics list common issues that you may encounter while running Ansys Dynamic Reporting (both as part as the EnSight workflow and as independent software).

Related topics:

[Multiple Reports in Parallel](#)

[Compatibility Between Versions](#)

[Known Issues and Limitations](#)

Multiple Reports in Parallel

For this scenario, you want to automatize your process of report generation in EnSight. You perform the analysis operations interactively, generate a report, and export the commands that correspond to these steps using the [EnSight Command Language Ansys Dynamic Reporting API \(p. 304\)](#). For the report generation section, you create something like the following:

```
ensight.core.nexus.ReportServer.create_local_database('D:/tmp')
ensight.core.nexus.ReportServer.launch_local_server('D:/tmp', port=None, connect=ensight.core.nexus.ReportServer.get_
ensight.core.nexus.ReportServer.get_server().set_username('nexus')
ensight.core.nexus.ReportServer.get_server().set_password('cei')
ensight.objs.core.STATES[0].update_tags('', new_session=False, random_session=False)
ensight.objs.core.STATES[0].generate_report()
```

In this particular example, it is assumed that you are setting a new database in the D:/tmp directory. This code performs the following steps:

1. Creates a new database in D:/tmp
2. Starts an ADR server
3. Sets the default username and password
4. Generates the report by pushing the items created by the sources (in the states) onto the database

If you now want to repeat the same operation for multiple datasets, you run the same script against all your datasets. You should change the D:/tmp directory for each dataset, so that the report items do not overwrite each other.

As long as you run the analysis one after the other (first dataset A, then once completed run dataset B, then dataset C, and so on), this will work fine. But if you want to parallelize your postprocessing, you might want to run the script on all the datasets at the same time. You will therefore have dataset A being analyzed at the same time as dataset B and dataset C, with the reports being created at the same time.

This can create some problems. If you try this approach, you might see that the items generated for the report end up in the wrong database. This is due to the fact that you have left in the command to launch the report server the default:

```
port=None
```

With this setting, EnSight queries the operating system for an open free port, starting with port 8000. If the port is open, EnSight launches the ADR server needed for the report on that port. However, there is a time delay between when EnSight first asks the operating system for the status of the port, and when the ADR instance is actually launched. During this time window, the port is not locked down, so another EnSight instance can query the operating system for the status of the same port before the first instance manages to launch. Therefore, it is possible that you end up having multiple ADR instances running on the same port.

When this occurs, and EnSight generates report items, they end up in a random database chosen from all the ADR servers on the same port.

This will happen only if you have multiple sessions of EnSight trying to generate a report at the same time, each one on a different report server. This is not the most efficient way of running the report structure of EnSight (instead of one centralized ADR instance, where all the reports would go to), but it still is a possible configuration.

To avoid the conflict, you can manually change the port number to hard-code a specific port, making sure it is different for each EnSight session you run. So, your new script for dataset A will contain something like:

```
ensight.core.nexus.ReportServer.launch_local_server('D:/datasetA',
                                                    port=8000,
                                                    connect=ensight.core.nexus.ReportServer.get_server(),
                                                    terminate_on_python_exit=True)
```

to force the usage of port 8000, while for dataset B you might have the following:

```
ensight.core.nexus.ReportServer.launch_local_server('D:/datasetB',
                                                    port=8001,
                                                    connect=ensight.core.nexus.ReportServer.get_server(),
                                                    terminate_on_python_exit=True)
```

that will force the report to run on port 8001.

Compatibility Between Versions

Every new release of Ansys Dynamic Reporting is designed to be as compatible as possible with previous versions of the software. This is not always possible, though, so it is always best practice to use the same version of Ansys Dynamic Reporting to run the server, to generate the database, and for the API that is used to push/read data into/from the database.

Upgrades

Upgrading an ADR server permanently upgrades the database that is associated with the server. If the ADR server were to be downgraded, the database would no longer be readable by the server.

ADR servers support upgrading databases for a two-year release window, per Ansys policy. If you need to upgrade an Ansys Dynamic Reporting database that is older than two years old, you must upgrade it in stages (or contact Ansys support).

To upgrade a database, launch the server from within the template editor. You can also run the server migration script. It automatically upgrades (migrates) the database it is associated with.

```
% cpython manage.py migrate
```

Template Editor

The version of the [Template Editor \(p. 18\)](#) must be identical to the version of the ADR server it communicates with for the purposes of editing templates.

The Template Editor is always able to read templates and data from older servers because it provides the capability to copy data between servers.

Python Compatibility

All versions of Ansys Dynamic Reporting from 2019 R3 and earlier were implemented using Python 2, while those from 2020 R1 and later use Python 3. The implementation of text strings changed dramatically between the two versions, so compatibility between pre-2020 R1 and 2020 R1 onward is not guaranteed.

The 2019 R3 Server

If you are running an ADR 2019 R3 server and you want to connect to it with a Template Editor from the 2020 R1 version or above, you can read the templates and modify them.

If you are running an ADR 2019 R3 server and you want to push new items to that server using an API from the 2020 R1 release (for example, using EnSight 2020 R1), this is not allowed and results in an error.

Known Issues and Limitations

The following are known issues at the time of release, as well as system and software limitations.

CSRF Detection Issues

Changes were made in 2024 R1 which require more explicit remote host specification in some configurations. For example, if an ADR server is accessed via a proxy web server, the hostname and protocol of the proxy server must be made available to Ansys Dynamic Reporting. These kinds of problems show up in various situations, but two are fairly common. First, if you try to log into the ADR server, you can be presented with a page reporting a CSRF problem (403 error). Second, it may not be possible to create/update any content in the server (for example, upload new data, create a new template, and so on).

This can be fixed by explicitly telling ADR what hostnames to trust for CSRF purposes. For example, if the ADR server is running behind a proxy server which is used for handling SSL. In cases like this, set the `CEI_NEXUS_TRUSTED_ORIGINS` environment variable to the domain name(s) of the trusted proxy. For example:

```
export CEI_NEXUS_TRUSTED_ORIGINS="https://*.example.com"
adr_launcher start --db_directory /my/database/directory
```

The CEI_NEXUS_TRUSTED_ORIGINS environment variable is a list of trusted origins for unsafe requests (like, POST). It can be in one or more of the following forms:

- https://subdomain.example.com
- http://... if requests originate from an insecure page
- https://*.example.com to allow access from all subdomains of the website

More than one trusted server can be included by separating them by commas in the environment variable.

Internet Explorer Support

The Ansys Dynamic Reporting web interface **does not support** Internet Explorer.

EnSight and Template Editor

Logging in and out of the Ansys Dynamic Reporting web interface when embedded inside the Ansys Dynamic Reporting Template Editor or EnSight is not supported. Disconnect from the server using the **File** menu and reconnect with the new credentials to switch users.

UNC Paths

Launching an ADR server from a UNC location on a Windows system is not supported.

Network Delay Ansys Dynamic Reporting Session Limitations

Integrated operation of Ansys Dynamic Reporting with EnSight and EnVision is reliant upon high performance network connections between the components, display environments (. frames in a browser) and distribution installation location (for example, network file system installs). There are a variety of implied timeouts within and between these components which can disrupt successful launching of these applications within specific environments.

Some Ansys Dynamic Reporting configurations involving slower virtual machines and the Firefox web browser have been observed running into problems displaying remote, embedded EnVision sessions. In such cases, switching to browsers with longer default web acknowledgment timeouts (like Chrome) or using higher performance virtual machines can help. Problems have also been observed when running Ansys Dynamic Reporting from software installations on remote (NFS) file systems with slower input/output response or limited remote file locking mechanisms. In these cases, moving the Ansys Dynamic Reporting database and/or the software installation to local disk storage can avoid excessive operational timeouts.

Inability to Create New/Start Databases On Linux

Ansys Dynamic Reporting relies on the platform-specific time zone specification being correct. On some Linux systems, it can be configured in an ambiguous fashion (different environment variables pointing to differently named time zones). On these systems, the Ansys Dynamic Reporting APIs for creating a

new database or launching a server may fail. The following test can be used to check to see if the system is configured incorrectly (use the v251/CEI/bin/cpython Python executable).

```
import tzlocal
print(tzlocal.get_localzone_name())
```

If this code raises an exception, the system has not been configured correctly. Setting the TZ environment variable to a valid time zone specification like US/Eastern will work around the issue.

